

TREINAMENTOS

Desenvolvimento Web com JSF2 e JPA2

Desenvolvimento Web com JSF 2.2 e JPA 2.1

2 de maio de 2016
As apostilas atualizadas estão disponíveis em **www.k19.com.br**

Esta apostila contém:

- 282 exercícios de fixação.
- 36 exercícios complementares.
- 1 desafios.
- 0 questões de prova.

Sumário i

Sobre a K19 1

Seguro Treinamento 2

Termo de Uso 3

Cursos 4

1 Banco de dados 1
- 1.1 Sistemas Gerenciadores de Banco de Dados . . . 1
- 1.2 MySQL Server . . . 2
- 1.3 Bases de dados (*Databases*) . . . 2
- 1.4 Criando uma base de dados no MySQL Server . . . 2
- 1.5 Tabelas . . . 3
- 1.6 Criando tabelas no MySQL Server . . . 3
- 1.7 CRUD . . . 4
- 1.8 Chaves Primária e Estrangeira . . . 5
- 1.9 Exercícios de Fixação . . . 5
- 1.10 Exercícios Complementares . . . 9

2 JDBC 27
- 2.1 Driver . . . 28
- 2.2 JDBC . . . 29

2.3 Instalando o Driver JDBC do MySQL Server . . . 30
2.4 Criando uma conexão . . . 30
2.5 Apagando uma base de dados . . . 31
2.6 Criando uma base de dados . . . 31
2.7 Criando uma tabela . . . 32
2.8 Inserindo dados . . . 32
2.9 Exercícios de Fixação . . . 33
2.10 Exercícios Complementares . . . 37
2.11 SQL Injection . . . 37
2.12 Exercícios de Fixação . . . 38
2.13 Exercícios Complementares . . . 39
2.14 Listando registros . . . 39
2.15 Exercícios de Fixação . . . 40
2.16 Exercícios Complementares . . . 41
2.17 Connection Factory . . . 41
2.18 Exercícios de Fixação . . . 42
2.19 Exercícios Complementares . . . 46

3 JPA 2.1 e Hibernate 47
3.1 Múltiplas sintaxes da linguagem SQL . . . 47
3.2 Orientação a Objetos VS Modelo Relacional . . . 47
3.3 Ferramentas ORM . . . 48
3.4 O que é JPA e Hibernate? . . . 49
3.5 Bibliotecas . . . 49
3.6 Configuração . . . 50
3.7 Mapeamento . . . 51
3.8 Gerando Tabelas . . . 54
3.9 Exercícios de Fixação . . . 54
3.10 Manipulando entidades . . . 59
3.11 Exercícios de Fixação . . . 61
3.12 Objetos Embutidos . . . 64
3.13 Exercícios de Fixação . . . 67
3.14 Relacionamentos . . . 70
3.15 One to One . . . 71
3.16 Exercícios de Fixação . . . 73
3.17 One to Many . . . 74
3.18 Exercícios de Fixação . . . 76
3.19 Many to One . . . 78
3.20 Exercícios de Fixação . . . 80
3.21 Many to Many . . . 81
3.22 Exercícios de Fixação . . . 83
3.23 Relacionamentos Bidirecionais . . . 85
3.24 Exercícios de Fixação . . . 87
3.25 Consultas . . . 89
3.26 Exercícios de Fixação . . . 92
3.27 Sistema de Controle de Finanças Pessoais . . . 95
3.28 Exercícios de Fixação . . . 95
3.29 Desafios . . . 119

4 Web Container 121

4.1 Necessidades de uma aplicação web . . . 121
4.2 Web Container . . . 122
4.3 Servlet e Java EE . . . 123
4.4 Exercícios de Fixação . . . 123
4.5 Aplicação Web Java . . . 137
4.6 Exercícios de Fixação . . . 138
4.7 Processando requisições . . . 140
4.8 Servlet . . . 140
4.9 Exercícios de Fixação . . . 141
4.10 Frameworks . . . 143

5 Visão Geral do JSF 2.2 145
5.1 MVC e Front Controller . . . 145
5.2 Configurando uma aplicação JSF . . . 145
5.3 Managed Beans . . . 146
5.4 Processamento de uma requisição . . . 150
5.5 Exemplo Prático . . . 153
5.6 Exercícios de Fixação . . . 155

6 Componentes Visuais 161
6.1 Estrutura Básica de uma Página JSF . . . 161
6.2 Formulários . . . 162
6.3 Caixas de Texto . . . 162
6.4 Campos Ocultos . . . 165
6.5 Caixas de Seleção . . . 165
6.6 Botões e Links . . . 172
6.7 Exercícios de Fixação . . . 174
6.8 Exercícios Complementares . . . 177
6.9 Textos e Imagens . . . 178
6.10 Exercícios de Fixação . . . 179
6.11 Componentes de Organização . . . 180
6.12 Tabelas . . . 182
6.13 Exercícios de Fixação . . . 184
6.14 Exercícios Complementares . . . 187
6.15 Mensagens . . . 187
6.16 Adicionando JavaScript e CSS . . . 188
6.17 Outros Componentes . . . 188
6.18 Exercícios de Fixação . . . 189
6.19 Exercícios Complementares . . . 190

7 Templates e Modularização 193
7.1 Templates . . . 193
7.2 Exercícios de Fixação . . . 196
7.3 Modularização . . . 197
7.4 Exercícios de Fixação . . . 199
7.5 Exercícios Complementares . . . 201

8 Navegação 203
8.1 Navegação Implícita . . . 203
8.2 Navegação Explícita . . . 205

8.3 Exercícios de Fixação . . . 206
8.4 Navegações Estática e Dinâmica . . . 210
8.5 Exercícios de Fixação . . . 211
8.6 Exercícios Complementares . . . 213

9 Escopos 215
9.1 Request . . . 215
9.2 View . . . 216
9.3 Session . . . 217
9.4 Application . . . 219
9.5 Exercícios de Fixação . . . 219

10 Conversão e Validação 225
10.1 Conversão . . . 225
10.2 Conversores Padrão . . . 225
10.3 Exercícios de Fixação . . . 227
10.4 Exercícios Complementares . . . 229
10.5 Mensagens de Erro . . . 229
10.6 Exercícios de Fixação . . . 232
10.7 Validação . . . 233
10.8 Validadores Padrão . . . 233
10.9 Exercícios de Fixação . . . 234
10.10 Exercícios Complementares . . . 236
10.11 Bean Validation . . . 236
10.12 Exercícios de Fixação . . . 239
10.13 Criando o seu Próprio Conversor . . . 240
10.14 Exercícios de Fixação . . . 242
10.15 Exercícios Complementares . . . 244
10.16 Criando o seu Próprio Validador . . . 245
10.17 Exercícios de Fixação . . . 247
10.18 Exercícios Complementares . . . 249
10.19 Criando o seu Próprio Bean Validator . . . 249
10.20 Exercícios de Fixação . . . 251

11 Eventos 255
11.1 FacesEvent . . . 255
11.2 Exercícios de Fixação . . . 258
11.3 PhaseEvent . . . 260
11.4 Exercícios de Fixação . . . 261
11.5 SystemEvent . . . 262
11.6 Exercícios de Fixação . . . 264
11.7 Immediate . . . 265
11.8 Exercícios de Fixação . . . 266

12 Ajax 271
12.1 Fazendo requisições AJAX . . . 271
12.2 Processando uma parte específica da tela . . . 272
12.3 Recarregando parte da tela . . . 273
12.4 Associando um procedimento a uma requisição AJAX . . . 273
12.5 Palavras especiais . . . 273

12.6 Exercícios de Fixação . . . 274

13 Integração JSF e JPA **277**
13.1 Bibliotecas . . . 277
13.2 Configuração . . . 277
13.3 Mapeamento . . . 278
13.4 Inicialização e Finalização . . . 278
13.5 Transações . . . 279
13.6 Recuperando o EntityManager da Requisição . . . 280
13.7 Exercícios de Fixação . . . 280
13.8 Otimizando o número de consultas ao SGDB . . . 285
13.9 Exercícios de Fixação . . . 286

A Autenticação **289**
A.1 Exercícios de Fixação . . . 289

B Páginas de Erro **295**
B.1 Exercícios de Fixação . . . 295

C PrimeFaces **299**
C.1 Instalação . . . 299
C.2 Exercícios de Fixação . . . 299
C.3 AutoComplete . . . 301
C.4 Exercícios de Fixação . . . 301
C.5 Poll . . . 302
C.6 Exercícios de Fixação . . . 302
C.7 Calendar . . . 304
C.8 Exercícios de Fixação . . . 304
C.9 InputMask . . . 305
C.10 Exercícios de Fixação . . . 306
C.11 DataTable . . . 306
C.12 Exercícios de Fixação . . . 306
C.13 DataExporter . . . 310
C.14 Exercícios de Fixação . . . 310
C.15 PickList . . . 312
C.16 Exercícios de Fixação . . . 312
C.17 MegaMenu . . . 314
C.18 Exercícios de Fixação . . . 314
C.19 Exercícios de Fixação . . . 314
C.20 Chart . . . 317
C.21 Exercícios de Fixação . . . 317

D Respostas **321**



Sobre a K19

A K19 é uma empresa especializada na capacitação de desenvolvedores de software. Sua equipe é composta por profissionais formados em Ciência da Computação pela Universidade de São Paulo (USP) e que possuem vasta experiência em treinamento de profissionais para área de TI.

O principal objetivo da K19 é oferecer treinamentos de máxima qualidade e relacionados às principais tecnologias utilizadas pelas empresas. Através desses treinamentos, seus alunos tornam-se capacitados para atuar no mercado de trabalho.

Visando a máxima qualidade, a K19 mantém as suas apostilas em constante renovação e melhoria, oferece instalações físicas apropriadas para o ensino e seus instrutores estão sempre atualizados didática e tecnicamente.



Seguro Treinamento

Na K19 o aluno faz o curso quantas vezes quiser!

Comprometida com o aprendizado e com a satisfação dos seus alunos, a K19 é a única que possui o Seguro Treinamento. Ao contratar um curso, o aluno poderá refazê-lo quantas vezes desejar mediante a disponibilidade de vagas e pagamento da franquia do Seguro Treinamento.

As vagas não preenchidas até um dia antes do início de uma turma da K19 serão destinadas ao alunos que desejam utilizar o Seguro Treinamento. O valor da franquia para utilizar o Seguro Treinamento é 10% do valor total do curso.



Termo de Uso

Termo de Uso

Todo o conteúdo desta apostila é propriedade da K19 Treinamentos. A apostila pode ser utilizada livremente para estudo pessoal . Além disso, este material didático pode ser utilizado como material de apoio em cursos de ensino superior desde que a instituição correspondente seja reconhecida pelo MEC (Ministério da Educação) e que a K19 seja citada explicitamente como proprietária do material.

É proibida qualquer utilização desse material que não se enquadre nas condições acima sem o prévio consentimento formal, por escrito, da K19 Treinamentos. O uso indevido está sujeito às medidas legais cabíveis.



Conheça os nossos cursos



K01- Lógica de Programação



K02 - Desenvolvimento Web com HTML, CSS e JavaScript



K03 - SQL e Modelo Relacional



K11 - Orientação a Objetos em Java



K12 - Desenvolvimento Web com JSF2 e JPA2



K21 - Persistência com JPA2 e Hibernate



K22 - Desenvolvimento Web Avançado com JFS2, EJB3.1 e CDI



K23 - Integração de Sistemas com Webservices, JMS e EJB



K41 - Desenvolvimento Mobile com Android



K51 - Design Patterns em Java



K52 - Desenvolvimento Web com Struts



K31 - C# e Orientação a Objetos



K32 - Desenvolvimento Web com ASP.NET MVC

www.k19.com.br/cursos

CAPÍTULO 1

BANCO DE DADOS

Em geral, as aplicações necessitam armazenar dados de forma persistente para consultá-los posteriormente. Por exemplo, a aplicação de uma livraria precisa armazenar os dados dos livros e dos autores de forma persistente.

Suponha que esses dados sejam armazenados em arquivos do sistema operacional. Vários fatores importantes nos levam a descartar tal opção. A seguir, apresentamos as principais dificuldades a serem consideradas na persistência de dados.

Segurança: O acesso às informações potencialmente confidenciais deve ser controlado de forma que apenas usuários e sistemas autorizados possam manipulá-las.

Integridade: Restrições relacionadas aos dados armazenados devem ser respeitadas para que as informações estejam sempre consistentes.

Consulta: O tempo gasto para realizar as consultas aos dados armazenados deve ser o menor possível.

Concorrência: Em geral, diversos sistemas e usuários acessarão concorrentemente as informações armazenadas. Apesar disso, a integridade dos dados deve ser preservada.

Considerando todos esses aspectos, concluímos que um sistema complexo seria necessário para persistir as informações de uma aplicação de maneira adequada. Felizmente, tal tipo de sistema já existe e é conhecido como **Sistema Gerenciador de Banco de Dados** (SGBD).

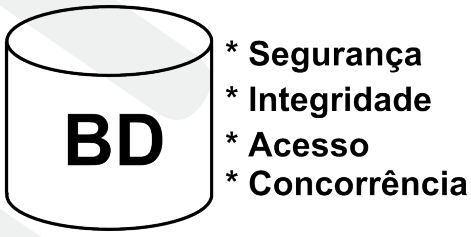


Figura 1.1: Sistema Gerenciador de Banco de Dados

Sistemas Gerenciadores de Banco de Dados

No mercado, há diversas opções de sistemas gerenciadores de banco de dados. Os mais populares são:

- Oracle Database

- SQL Server
- MySQL Server
- PostgreSQL



MySQL Server

Neste treinamento, utilizaremos o MySQL Server, que é mantido pela Oracle e amplamente utilizado em aplicações comerciais. Para instalar o MySQL Server, você pode utilizar o artigo disponível em nosso site: http://www.k19.com.br/artigos/instalando-mysql/



Bases de dados (*Databases*)

Um sistema gerenciador de banco de dados é capaz de gerenciar informações de diversos sistemas ao mesmo tempo. Por exemplo, as informações dos clientes de um banco, além dos produtos de uma loja virtual ou dos livros de uma livraria.

Suponha que os dados fossem mantidos sem nenhuma separação lógica. Implementar regras de segurança específicas seria extremamente complexo. Tais regras criam restrições quanto ao conteúdo que pode ser acessado por cada usuário. Por exemplo, determinado usuário poderia ter permissão de acesso aos dados dos clientes do banco, mas não às informações dos produtos da loja virtual, ou dos livros da livraria.

Para obter uma organização melhor, os dados são armazenados separadamente em um SGDB. Daí surge o conceito de **base de dados** (database). Uma base de dados é um agrupamento lógico das informações de um determinado domínio.



Criando uma base de dados no MySQL Server

Para criar uma base de dados no MySQL Server, podemos utilizar o comando **CREATE DATABASE**.

```
mysql> CREATE DATABASE livraria;
Query OK, 1 row affected (0.02 sec)
```

Terminal 1.1: Criando uma base de dados.

Podemos utilizar o comando **SHOW DATABASES** para listar as bases de dados existentes.

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| livraria           |
| mysql              |
| test               |
+--------------------+
4 rows in set (0.03 sec)
```

Terminal 1.2: Listando as bases de dados existentes.

Repare que, além da base de dados **livraria**, há outras três bases. Essas bases foram criadas automaticamente pelo próprio MySQL Server para teste ou para armazenar configurações.

Quando uma base de dados não é mais necessária, ela pode ser removida através do comando **DROP DATABASE**.

```
mysql> DROP DATABASE livraria;
Query OK, 0 rows affected (0.08 sec)
```

Terminal 1.3: Destruindo uma base de dados.



Tabelas

Um servidor de banco de dados é dividido em bases de dados com o intuito de separar as informações de domínios diferentes. Nessa mesma linha de raciocínio, podemos dividir os dados de uma base a fim de agrupá-los segundo as suas correlações. Essa separação é feita através de **tabelas**. Por exemplo, no sistema de um banco, é interessante separar o saldo e o limite de uma conta, do nome e CPF de um cliente. Então, poderíamos criar uma tabela para os dados relacionados às contas e outra para os dados relacionados aos clientes.

Cliente		
nome	idade	cpf
José	27	31875638735
Maria	32	30045667856

Conta		
numero	saldo	limite
1	1000	500
2	2000	700

Tabela 1.1: Tabelas para armazenar os dados relacionados aos clientes e às contas

Uma tabela é formada por **registros** (linhas) e os registros são formados por **campos** (colunas). Por exemplo, considere uma tabela para armazenar as informações dos clientes de um banco. Cada registro dessa tabela armazena em seus campos os dados de um determinado cliente.



Criando tabelas no MySQL Server

As tabelas no MySQL Server são criadas através do comando **CREATE TABLE**. Na criação de uma tabela, é necessário definir quais são os nomes e os tipos das colunas.

```
mysql> CREATE TABLE `livraria`.`Livro` (
    -> `titulo` VARCHAR(255),
    -> `preco` DOUBLE
    -> )
    -> ENGINE=MyISAM;
Query OK, 0 rows affected (0.14 sec)
```

Terminal 1.4: Criando uma tabela.

As tabelas de uma base de dados podem ser listadas através do comando **SHOW TABLES**. Antes de utilizar esse comando, devemos selecionar uma base de dados através do comando **USE**.

```
mysql> USE livraria;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
```

```
+--------------------+
| Tables_in_livraria |
+--------------------+
| Livro              |
+--------------------+
1 row in set (0.00 sec)
```

Terminal 1.5: Listando as tabelas de uma base de dados.

Se uma tabela não for mais desejada, ela pode ser removida através do comando **DROP TABLE**.

```
mysql> DROP TABLE Livro;
Query OK, 0 rows affected (0.00 sec)
```

Terminal 1.6: Destruindo uma tabela.



CRUD

As operações básicas para manipular os dados persistidos são: inserir, ler, alterar e remover.

Essas operações são realizadas através de uma linguagem de consulta denominada **SQL** (*Structured Query Language*). Essa linguagem oferece quatro comandos básicos: **INSERT**, **SELECT**, **UPDATE** e **DELETE**. Esses comandos são utilizados para inserir, ler, alterar e remover registros, respectivamente.

```
mysql> INSERT INTO Livro (titulo, preco) VALUES ('Java', 98.75);
Query OK, 1 row affected (0.00 sec)
```

Terminal 1.7: Inserindo um registro.

```
mysql> SELECT * FROM Livro;
+--------+-------+
| titulo | preco |
+--------+-------+
| Java   | 98.75 |
+--------+-------+
1 row in set (0.00 sec)
```

Terminal 1.8: Selecionando registros.

```
mysql> UPDATE Livro SET preco = 115.9 WHERE titulo = 'Java';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Terminal 1.9: Alterando registros.

```
mysql> SELECT * FROM Livro;
+--------+-------+
| titulo | preco |
+--------+-------+
| Java   | 115.9 |
+--------+-------+
1 row in set (0.00 sec)
```

Terminal 1.10: Selecionando registros.

```
mysql> DELETE FROM Livro WHERE titulo = 'Java';
Query OK, 1 row affected (0.00 sec)
```

Terminal 1.11: Removendo registros.

```
mysql> SELECT * FROM Livro;
Empty set (0.00 sec)
```

Terminal 1.12: Selecionando registros.



Chaves Primária e Estrangeira

Suponha que os livros da nossa livraria sejam classificados por editoras. As editoras possuem nome e telefone. Para armazenar esses dados, uma nova tabela deveria ser criada.

Nesse momento, teríamos duas tabelas (Livro e Editora). Constantemente, a aplicação da livraria deverá descobrir qual é a editora de um determinado livro ou quais são os livros de uma determinada editora. Para isso, os registros da tabela Editora devem estar relacionados aos da tabela Livro.

Na tabela Livro, poderíamos adicionar uma coluna para armazenar o nome da editora dos livros. Dessa forma, se alguém quiser recuperar as informações da editora de um determinado livro, deve consultar a tabela Livro para obter o nome da editora correspondente. Depois, com esse nome, deve consultar a tabela Editora para obter as informações da editora.

Porém, há um problema nessa abordagem. A tabela Editora aceita duas editoras com o mesmo nome. Dessa forma, eventualmente, não conseguiríamos descobrir os dados corretos da editora de um determinado livro. Para resolver esse problema, deveríamos criar uma restrição na tabela Editora que proíba a inserção de editoras com o mesmo nome.

Para resolver esse problema no MySQL Server, poderíamos adicionar a propriedade **UNIQUE** no campo nome da tabela Editora. Porém, ainda teríamos mais um problema. Na tabela Livro, poderíamos adicionar registros vinculados a editoras inexistentes, pois não há nenhuma relação explícita entre as tabelas. Para solucionar esses problemas, devemos utilizar o conceito de **chave primária** e **chave estrangeira**.

Toda tabela pode ter uma chave primária, que é um conjunto de um ou mais campos que devem ser únicos para cada registro. Normalmente, um campo numérico é escolhido para ser a chave primária de uma tabela, pois as consultas podem ser realizadas com melhor desempenho.

Então, poderíamos adicionar um campo numérico na tabela Editora e torná-lo chave primária. Vamos chamar esse campo de **id**. Na tabela Livro, podemos adicionar um campo numérico chamado **editora_id** que deve ser utilizado para guardar o valor da chave primária da editora correspondente ao livro. Além disso, o campo editora_id deve estar explicitamente vinculado com o campo id da tabela Editora. Para estabelecer esse vínculo, o campo editora_id da tabela Livro deve ser uma chave estrangeira associada à chave primária da tabela Editora.

Uma chave estrangeira é um conjunto de uma ou mais colunas de uma tabela que possuem valores iguais aos da chave primária de outra tabela.

Com a definição da chave estrangeira, um livro não pode ser inserido com o valor do campo editora_id inválido. Caso tentássemos fazer isso, obteríamos uma mensagem de erro.



Exercícios de Fixação

1 Abra um terminal, crie e acesse uma pasta com o seu nome.

```
cosen@k19:~$ mkdir rafael
cosen@k19:~$ cd rafael/
cosen@k19:~/rafael$
```

Terminal 1.13: Criando e acessando uma pasta com o seu nome.

2 Estando dentro da sua pasta, acesse o **MySQL Server** utilizando o usuário **root** e a senha **root**.

```
k19@k19-11:~/rafael$ mysql -u root -p
Enter password:
```

Terminal 1.14: Logando no MySQL Server.

3 Caso exista uma base de dados chamada **livraria**, remova-a. Utilize o comando **DROP DATABASE** para remover a base livraria se ela existir.

```
mysql> DROP DATABASE IF EXISTS livraria;
Query OK, 1 row affected (0.12 sec)
```

4 Crie uma nova base de dados chamada **livraria**. Utilize o comando **CREATE DATABASE**. Você vai utilizar esta base nos exercícios seguintes.

```
mysql> CREATE DATABASE livraria;
Query OK, 1 row affected (0.00 sec)
```

Terminal 1.16: Criando a base `livraria`.

5 Abra um editor de texto e digite o código abaixo para criar uma tabela com o nome **Editora**. Depois salve o arquivo com o nome `create-table-editora.sql` dentro da pasta com o seu nome.

```
1 USE livraria;
2 CREATE TABLE Editora (
3   id BIGINT NOT NULL AUTO_INCREMENT,
4   nome VARCHAR (255) NOT NULL,
5   email VARCHAR (255) NOT NULL,
6   PRIMARY KEY (id)
7 )
8 ENGINE = InnoDB;
```

Código SQL 1.1: www.k19.com.br/exercicios/k12/01/005/create-table-editora.sql

6 Dentro do terminal, use o comando `source` para executar o arquivo que você acabou de criar.

```
mysql> source create-table-editora.sql
Database changed
Query OK, 0 rows affected (0.08 sec)
```

Terminal 1.17: Executando a tabela `Editora`.

7 Abra um novo editor de texto e digite o código abaixo para criar uma tabela com o nome **Livro**. Em seguida, salve o arquivo com o nome `create-table-livro.sql` dentro da pasta com o seu nome.

```
1  USE livraria;
2  CREATE TABLE Livro (
3    id BIGINT NOT NULL AUTO_INCREMENT,
4    titulo VARCHAR(255) NOT NULL,
5    preco DOUBLE NOT NULL,
6    editora_id BIGINT NOT NULL,
7    PRIMARY KEY(id),
8    FOREIGN KEY fk_editora(editora_id) REFERENCES Editora(id)
9  )
10 ENGINE = InnoDB;
```

Código SQL 1.2: www.k19.com.br/exercicios/k12/01/007/create-table-livro.sql

8 Dentro do terminal, use o comando `source` para executar o código do arquivo `create-table-livro.sql`.

```
mysql> source create-table-livro.sql
Database changed
Query OK, 0 rows affected (0.08 sec)
```

Terminal 1.18: Executando a tabela `Livro`.

9 Abra um novo editor de texto e digite o código abaixo para adicionar alguns registros na tabela **Editora**. Depois salve o arquivo com o nome `adicionando-registros-editora.sql` dentro da pasta com o seu nome.

```
1  INSERT INTO Editora (nome, email) VALUES ('Oreilly', 'oreilly@email.com');
2
3  INSERT INTO Editora (nome, email) VALUES ('Wrox', 'wrox@email.com');
4
5  INSERT INTO Editora (nome, email) VALUES ('Apress', 'apress@email.com');
```

Código SQL 1.3: www.k19.com.br/exercicios/k12/01/009/adicionando-registros-editora.sql

10 Dentro do terminal, execute o arquivo que você acabou de criar para adicionar alguns registro na tabela **Editora**.

```
mysql> source adicionando-registros-editora.sql
Query OK, 1 row affected (0.03 sec)

Query OK, 1 row affected (0.04 sec)

Query OK, 1 row affected (0.04 sec)
```

Terminal 1.19: Inserindo editoras.

11 Abra um novo editor de texto e digite o código abaixo para adicionar alguns registros na tabela **Livro**. Depois salve o arquivo com o nome `adicionando-registros-livro.sql` dentro da pasta com o seu nome.

```
1  INSERT INTO Livro (titulo, preco, editora_id) VALUES ('Aprendendo C#', 89.90, 1);
2
3  INSERT INTO Livro (titulo, preco, editora_id) VALUES ('JSF 2', 122.90, 3);
4
5  INSERT INTO Livro (titulo, preco, editora_id) VALUES ('JSF 2 Avançado', 149.90, 3);
```

Código SQL 1.4: www.k19.com.br/exercicios/k12/01/011/adicionando-registros-livro.sql

12 Dentro do terminal, execute o arquivo que você acabou de criar para adicionar alguns registros na **Livro**.

```
mysql> source adicionando-registros-livro.sql
Query OK, 1 row affected (0.02 sec)

Query OK, 1 row affected (0.04 sec)

Query OK, 1 row affected (0.04 sec)
```

Terminal 1.20: Inserindo livros.

13 Consulte os registros da tabela Editora e da tabela Livro. Utilize o comando **SELECT**.

```
mysql> SELECT * FROM Editora;
+----+---------+-------------------+
| id | nome    | email             |
+----+---------+-------------------+
|  1 | Oreilly | oreilly@email.com |
|  2 | Wrox    | wrox@email.com    |
|  3 | Apress  | apress@email.com  |
+----+---------+-------------------+
3 rows in set (0.00 sec)
```

Terminal 1.21: Selecionando as editoras.

```
mysql> SELECT * FROM Livro;
+----+-----------------------+-------+------------+
| id | titulo                | preco | editora_id |
+----+-----------------------+-------+------------+
|  1 | Aprendendo C#         |  89.9 |          1 |
|  2 | JSF 2                 | 122.9 |          3 |
|  3 | JSF 2 Avançado        | 149.9 |          3 |
+----+-----------------------+-------+------------+
3 rows in set (0.00 sec)
```

Terminal 1.22: Selecionando os livros.

14 Altere alguns dos registros da tabela Livro. Utilize o comando **UPDATE**.

```
mysql> UPDATE Livro SET preco=92.9 WHERE id=1;
Query OK, 1 row affected (0.07 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Terminal 1.23: Alterando livros.

15 Altere alguns dos registros da tabela Editora. Utilize o comando **UPDATE**.

```
mysql> UPDATE Editora SET nome='OReilly' WHERE id=1;
Query OK, 1 row affected (0.09 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Terminal 1.24: Alterando editoras.

16 Remova alguns registros da tabela Livro. Utilize o comando **DELETE**.

```
mysql> DELETE FROM Livro WHERE id=2;
Query OK, 1 row affected (0.07 sec)
```

Terminal 1.25: Removendo livros.

17 Remova alguns registros da tabela Editora. Preste atenção para não remover uma editora que tenha algum livro relacionado já adicionado no banco. Utilize o comando **DELETE**.

```
mysql> DELETE FROM Editora WHERE id=2;
Query OK, 1 row affected (0.05 sec)
```

Terminal 1.26: Removendo editoras.

18 Faça uma consulta para buscar todos os livros de uma determinada editora.

```
mysql> SELECT * FROM Livro as L, Editora as E WHERE L.editora_id=E.id and E.id = 1;
+----+---------------+-------+------------+----+---------+-------------------+
| id | titulo        | preco | editora_id | id | nome    | email             |
+----+---------------+-------+------------+----+---------+-------------------+
|  1 | Aprendendo C# |  92.9 |          1 |  1 | OReilly | oreilly@email.com |
+----+---------------+-------+------------+----+---------+-------------------+
1 row in set (0.00 sec)
```

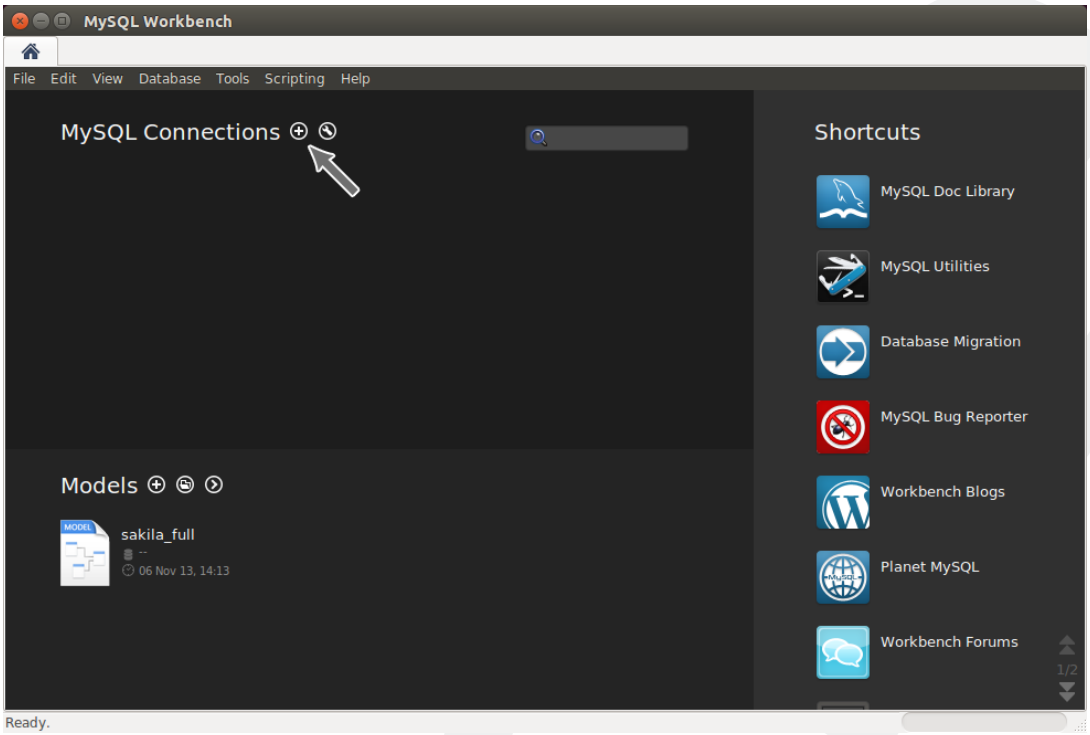
Terminal 1.27: Selecionando os livros de uma editora.

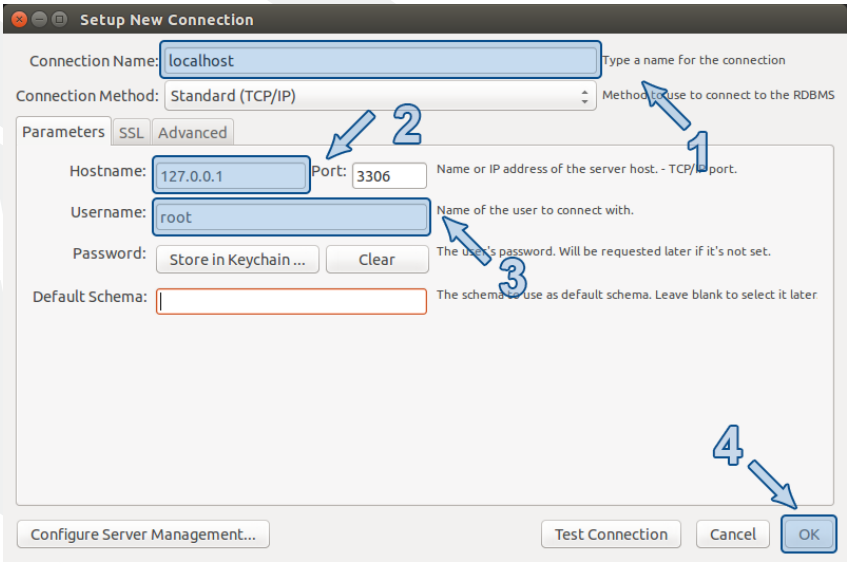


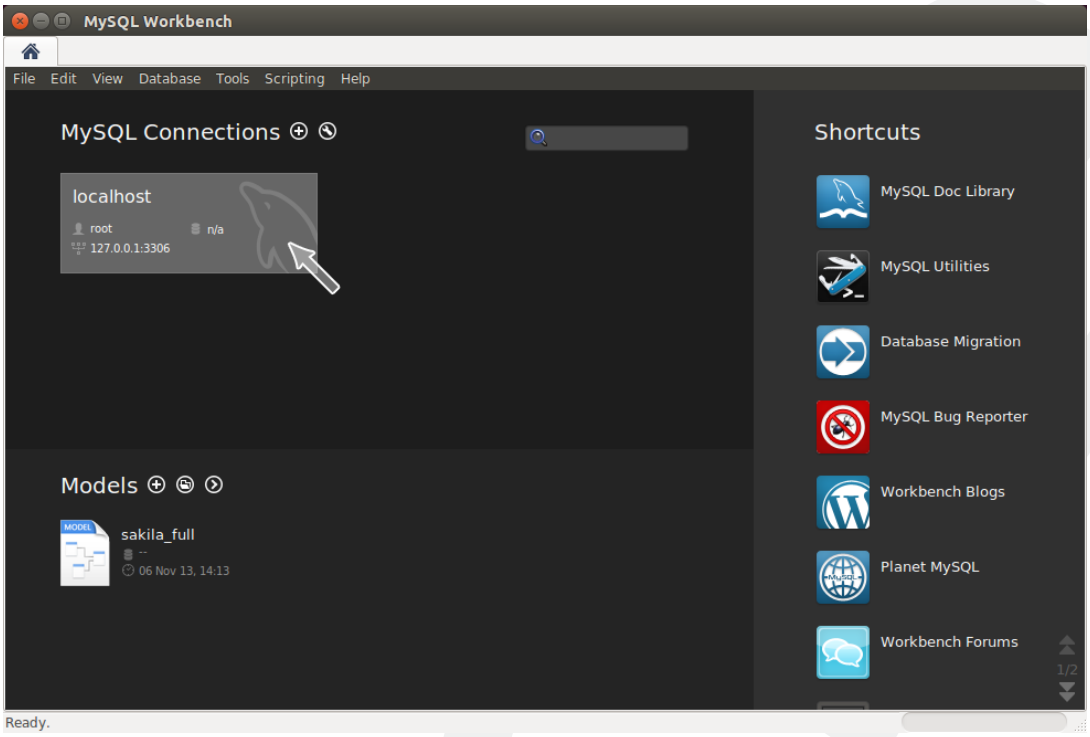
Exercícios Complementares

Utilize o MySQL Workbench para refazer os exercícios anteriores.

1 Abra o **MySQL Workbench** utilizando **localhost** como Server Hostname, **root** como Username e **root** como Password.

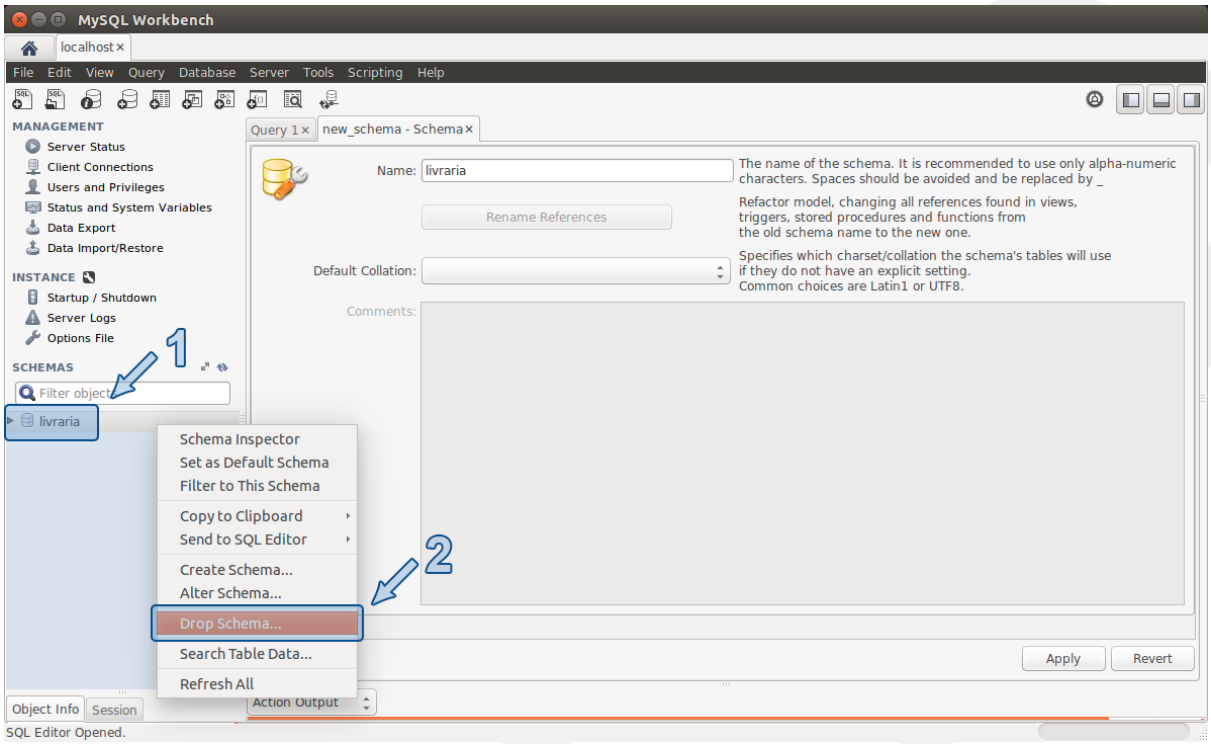

MySQL Workbench
File Edit View Database Tools Scripting Help
MySQL Connections
Shortcuts
MySQL Doc Library
MySQL Utilities
Database Migration
MySQL Bug Reporter
Workbench Blogs
Planet MySQL
Workbench Forums
Models
sakila_full
06 Nov 13, 14:13
Ready.

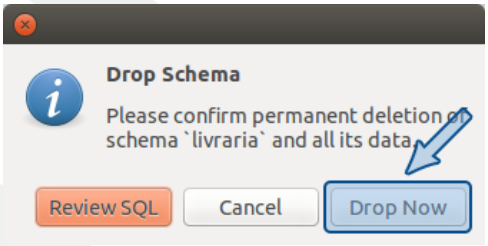

Setup New Connection
Connection Name: localhost
Type a name for the connection
Connection Method: Standard (TCP/IP)
Parameters SSL Advanced
Hostname: 127.0.0.1
Port: 3306
Name or IP address of the server host. - TCP/IP port.
Username: root
Name of the user to connect with.
Password: Store in Keychain ... Clear
Default Schema:
1
2
3
4
Configure Server Management...
Test Connection
Cancel
OK



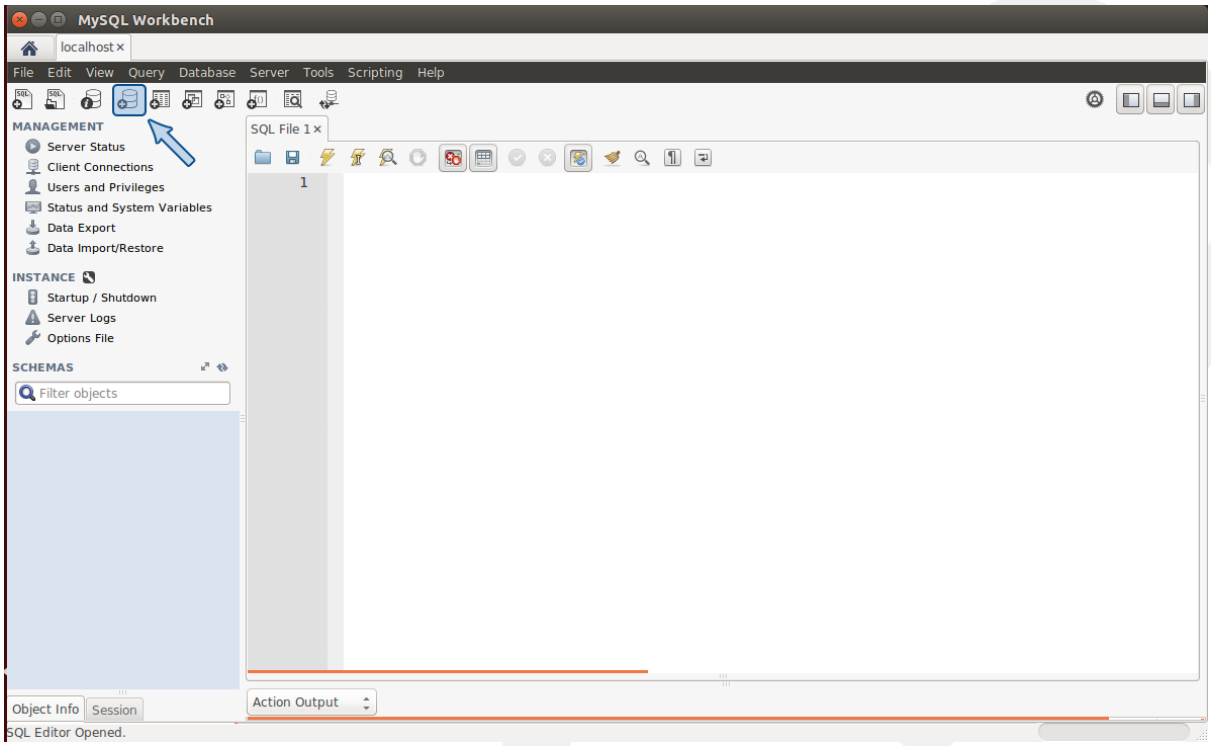


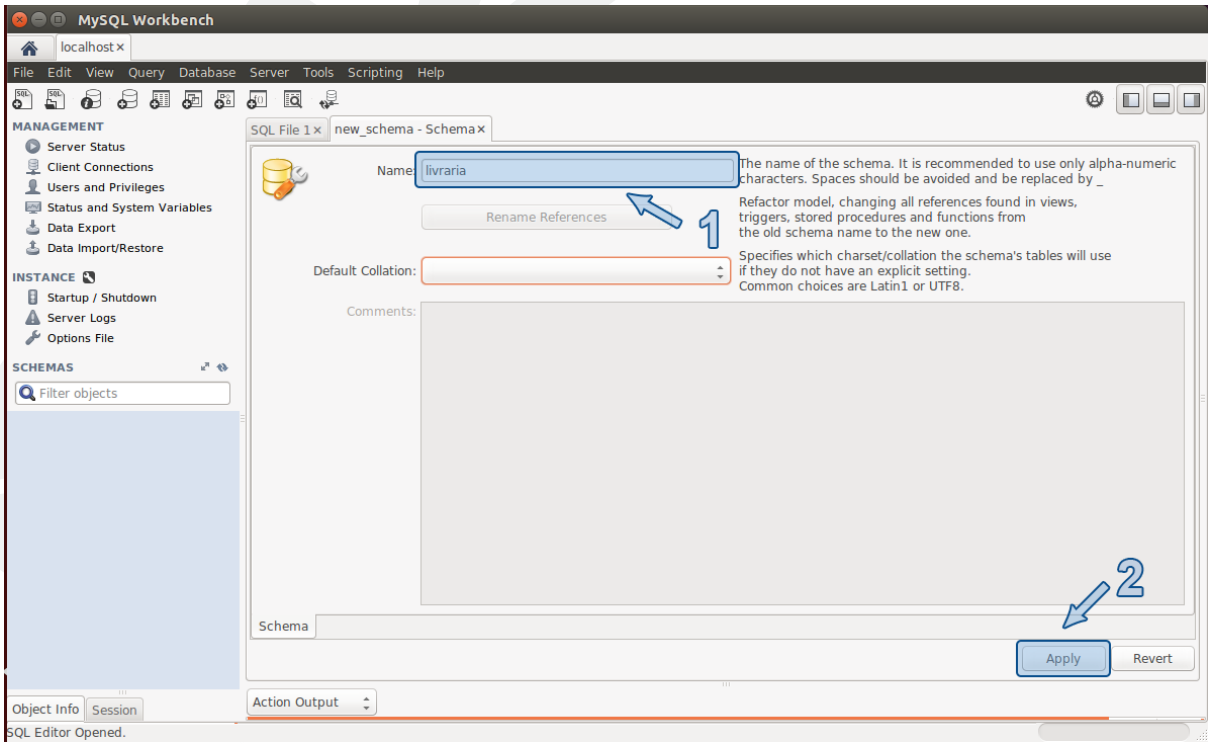
2 Caso exista uma base de dados chamada **livraria**, remova-a conforme a figura abaixo.

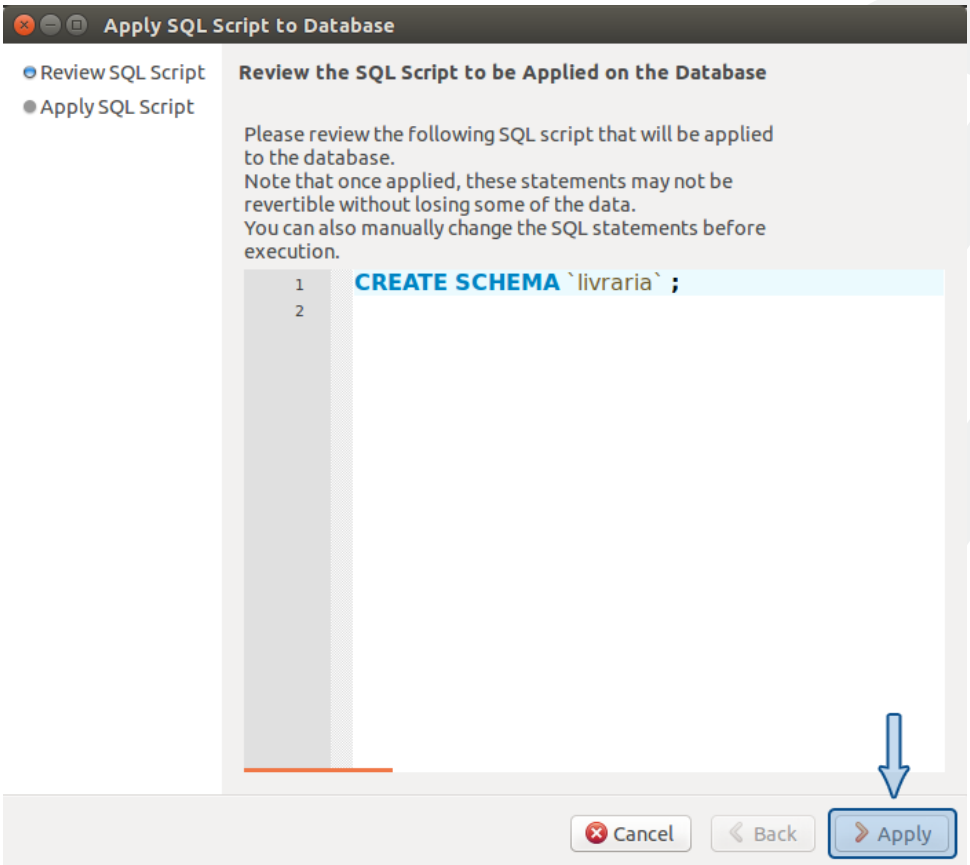


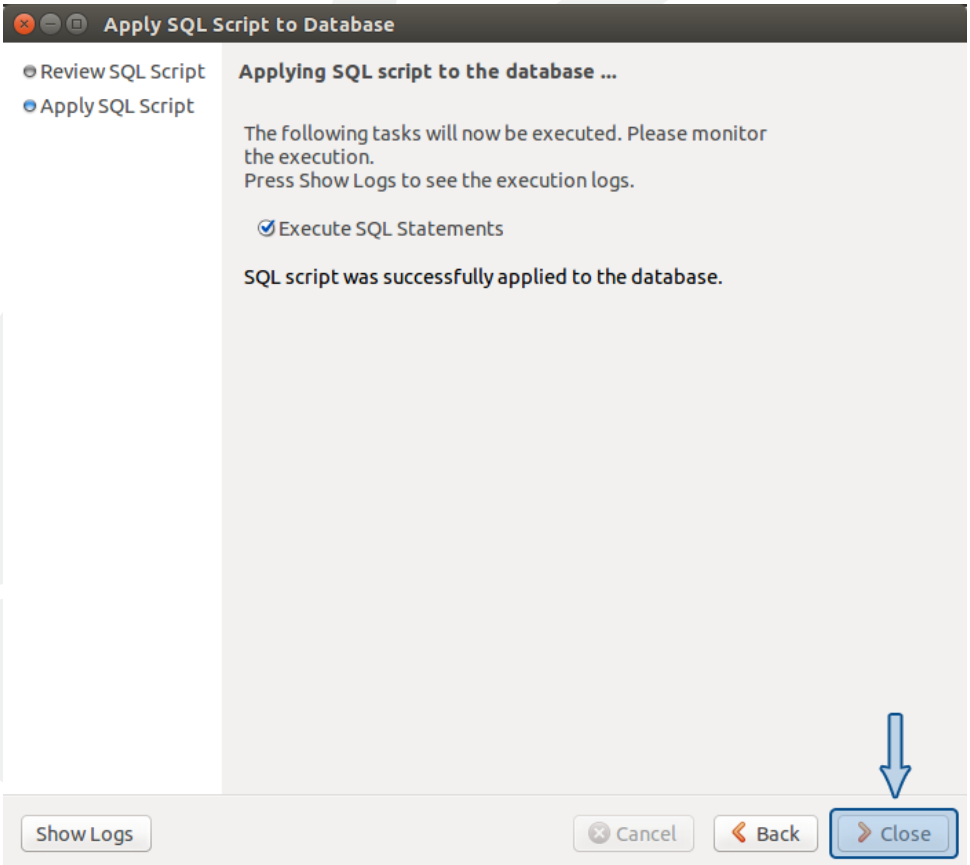


3 Crie uma nova base de dados chamada **livraria**, conforme mostrado na figura abaixo. Você vai utilizar esta base nos exercícios seguintes.

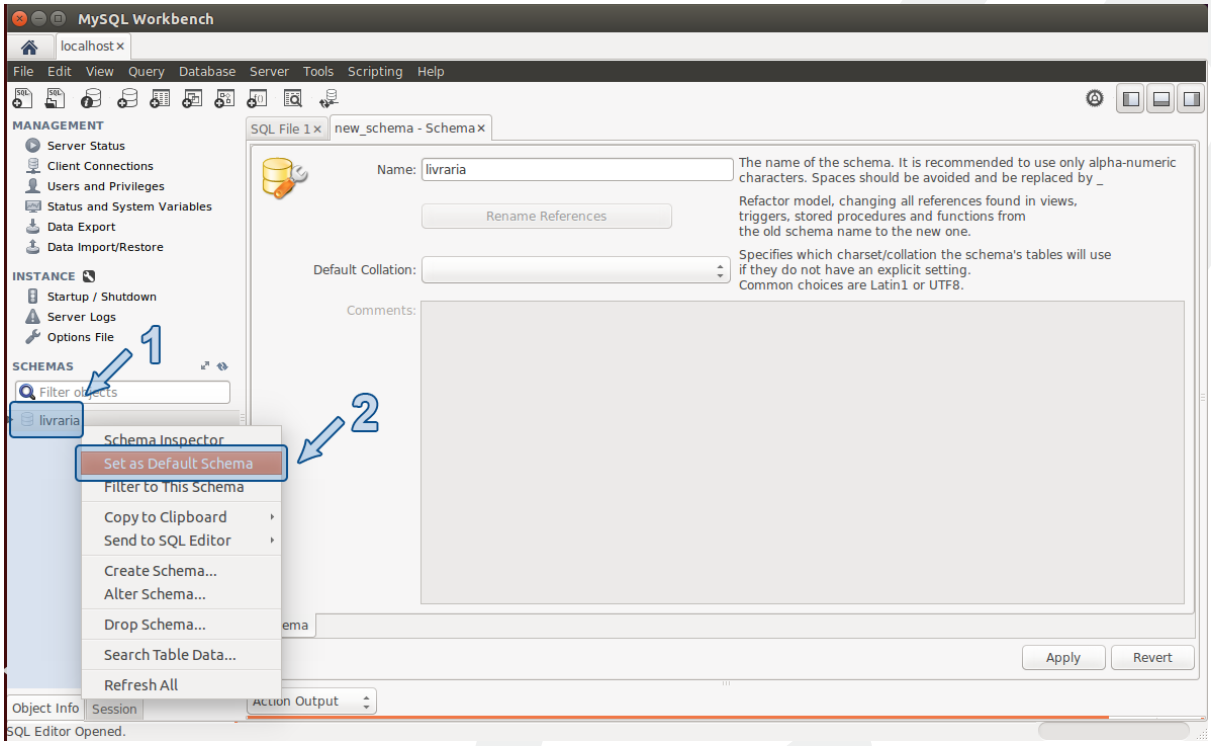

MySQL Workbench
localhost
File Edit View Query Database Server Tools Scripting Help
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
SCHEMAS
Filter objects
SQL File 1
Object Info
Session
Action Output
SQL Editor Opened.


MySQL Workbench
localhost
File Edit View Query Database Server Tools Scripting Help
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
SCHEMAS
Filter objects
SQL File 1
new_schema - Schema
Name: livraria
The name of the schema. It is recommended to use only alpha-numeric characters. Spaces should be avoided and be replaced by _
Rename References
Refactor model, changing all references found in views, triggers, stored procedures and functions from the old schema name to the new one.
1
Default Collation:
Specifies which charset/collation the schema's tables will use if they do not have an explicit setting. Common choices are Latin1 or UTF8.
Comments:
2
Schema
Apply
Revert
Object Info
Session
Action Output
SQL Editor Opened.

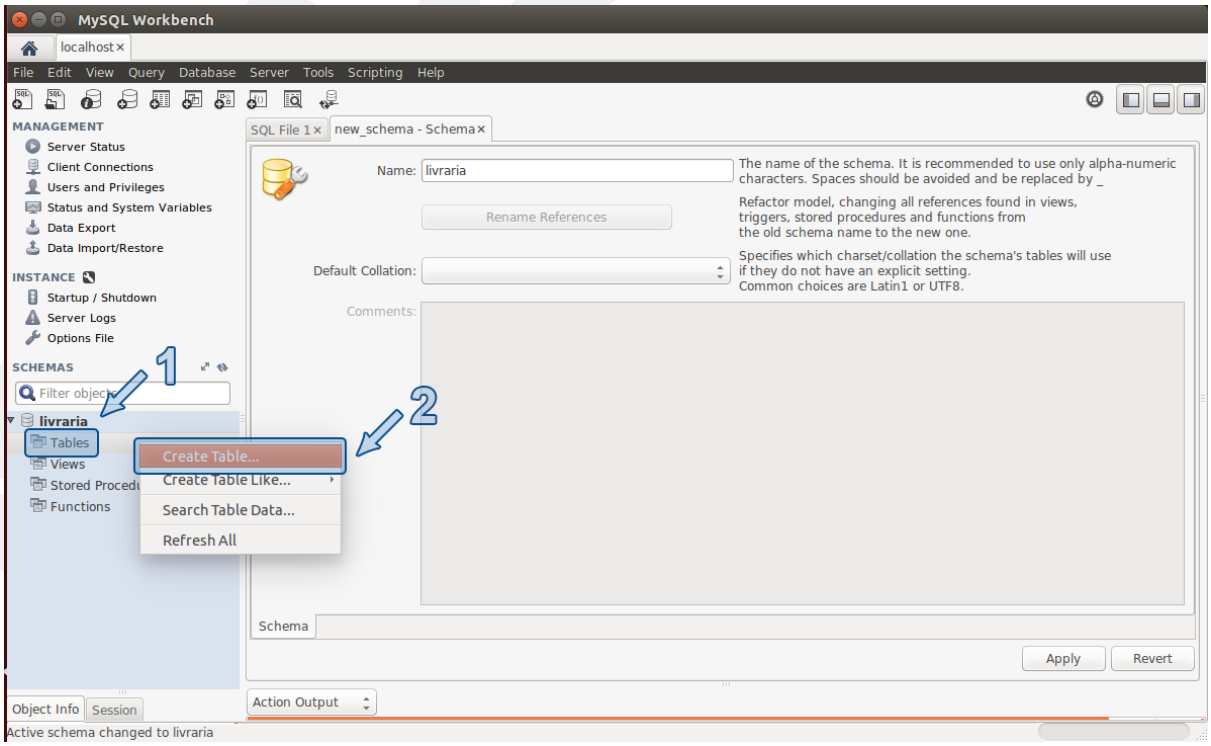

Apply SQL Script to Database
Review SQL Script
Apply SQL Script
Review the SQL Script to be Applied on the Database
Please review the following SQL script that will be applied to the database.
Note that once applied, these statements may not be revertible without losing some of the data.
You can also manually change the SQL statements before execution.
1 CREATE SCHEMA `livraria` ;
2
Cancel
Back
Apply

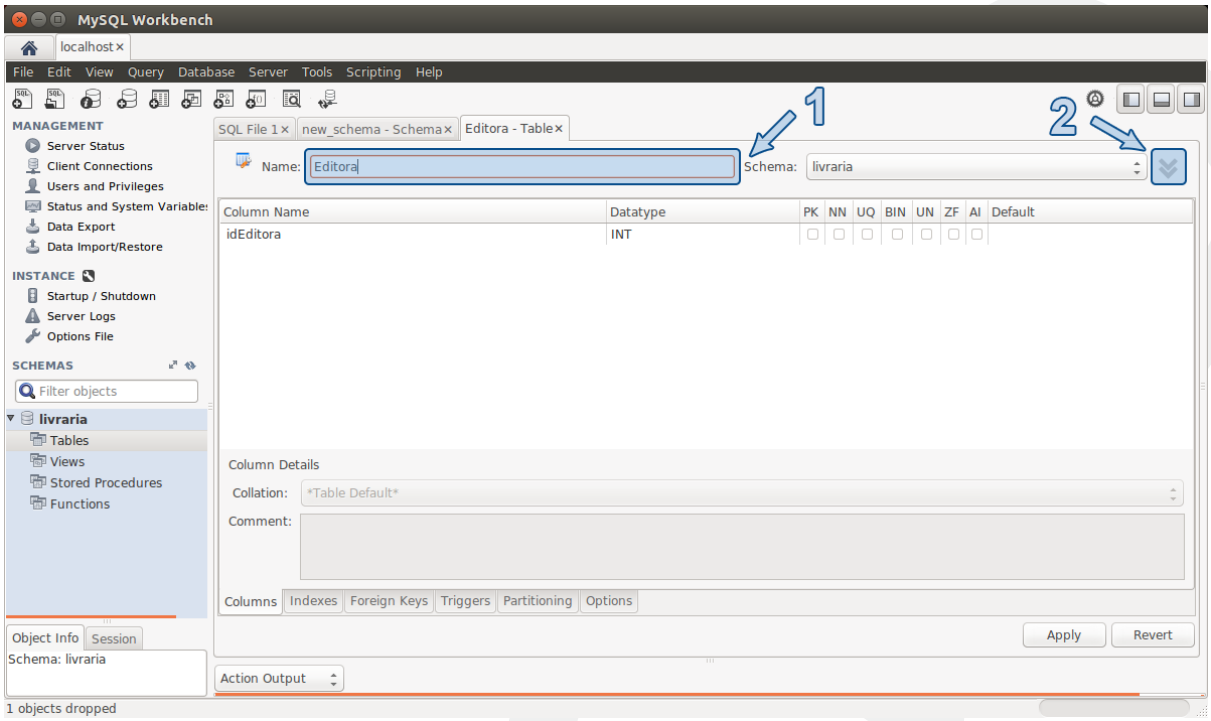

Apply SQL Script to Database
Review SQL Script
Apply SQL Script
Applying SQL script to the database ...
The following tasks will now be executed. Please monitor the execution.
Press Show Logs to see the execution logs.
Execute SQL Statements
SQL script was successfully applied to the database.
Show Logs
Cancel
Back
Close

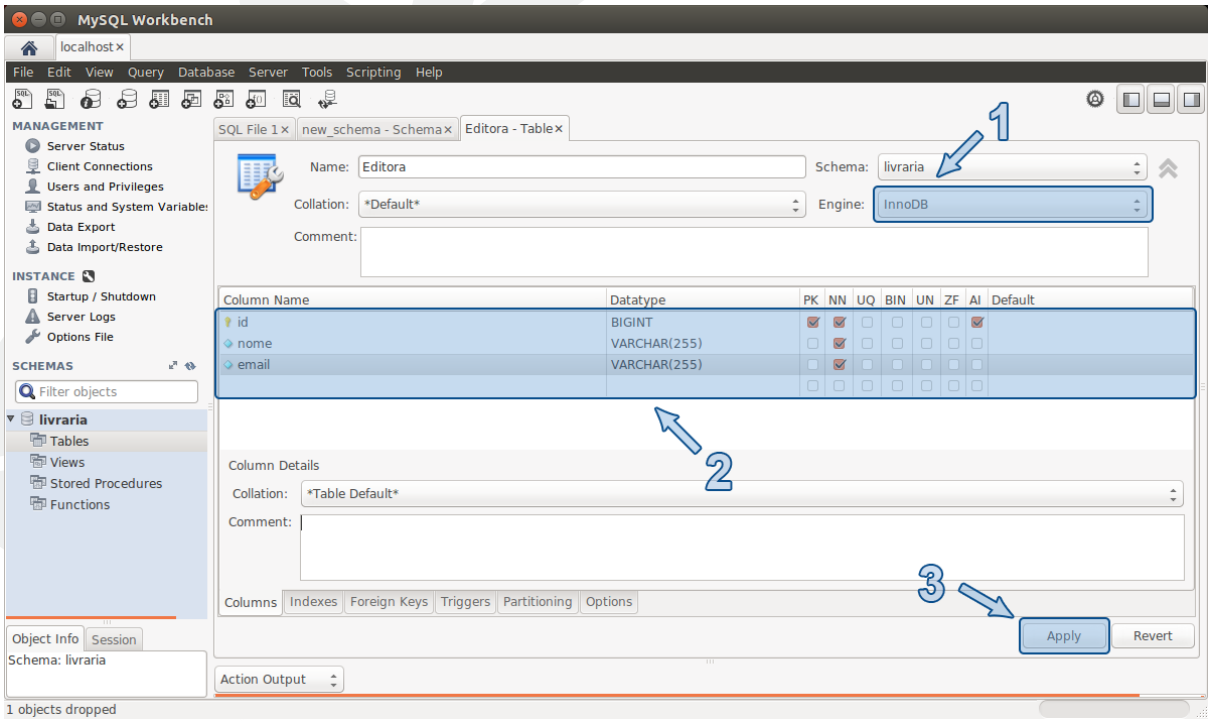
4 Selecione a base de dados livraria como padrão.

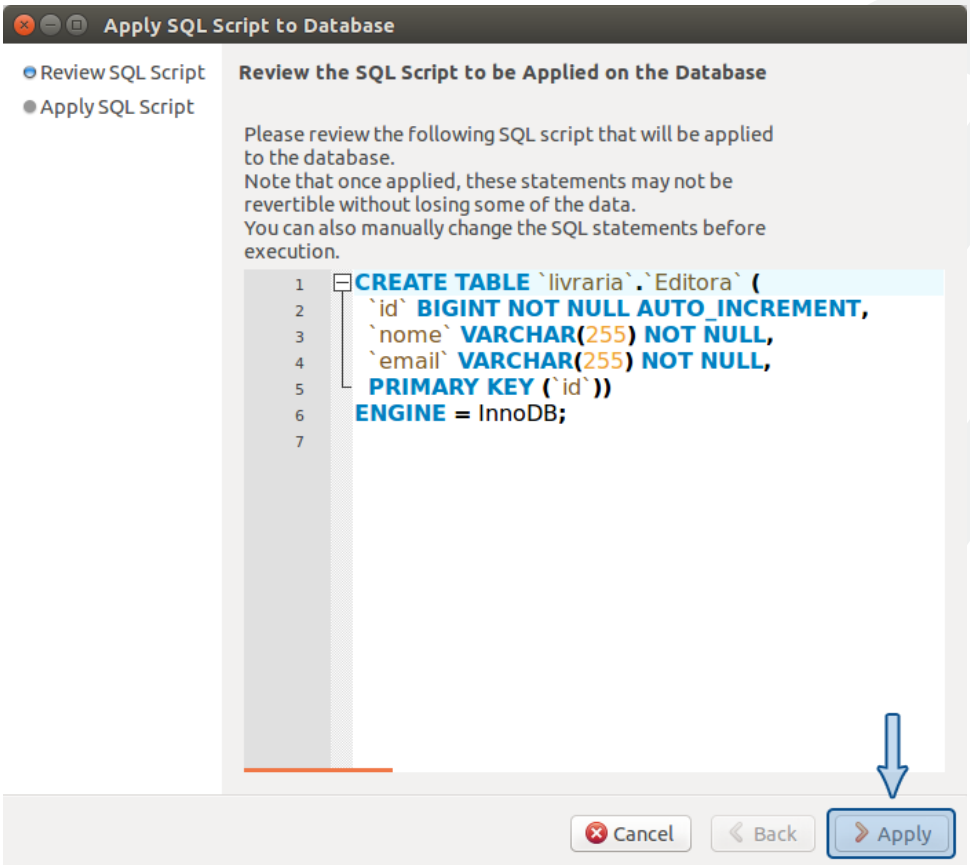


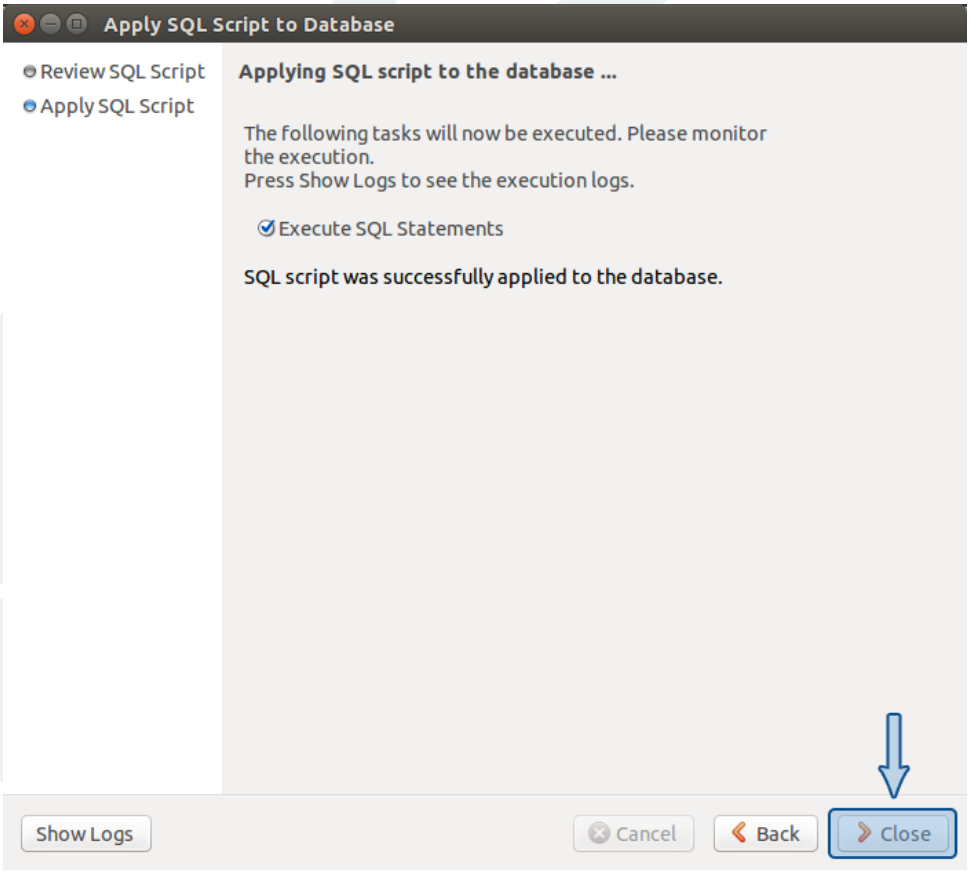
5 Crie uma tabela chamada **Editora** conforme as figuras abaixo.



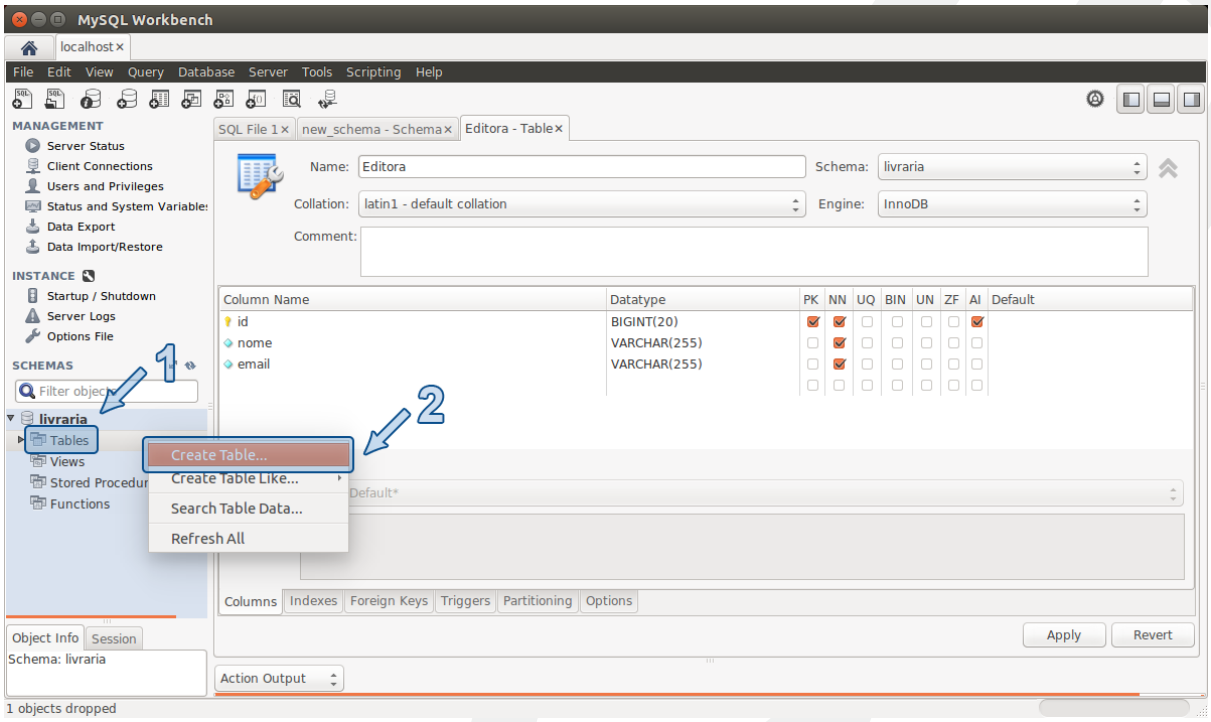

MySQL Workbench
localhost
File Edit View Query Database Server Tools Scripting Help
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variable
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
SCHEMAS
Filter objects
livraria
Tables
Views
Stored Procedures
Functions
Object Info
Session
Schema: livraria
SQL File 1
new_schema - Schema
Editora - Table
Name: Editora
Schema: livraria
1
2
Column Name
Datatype
PK NN UQ BIN UN ZF AI Default
idEditora
INT
Column Details
Collation: *Table Default*
Comment:
Columns
Indexes
Foreign Keys
Triggers
Partitioning
Options
Apply
Revert
Action Output
1 objects dropped

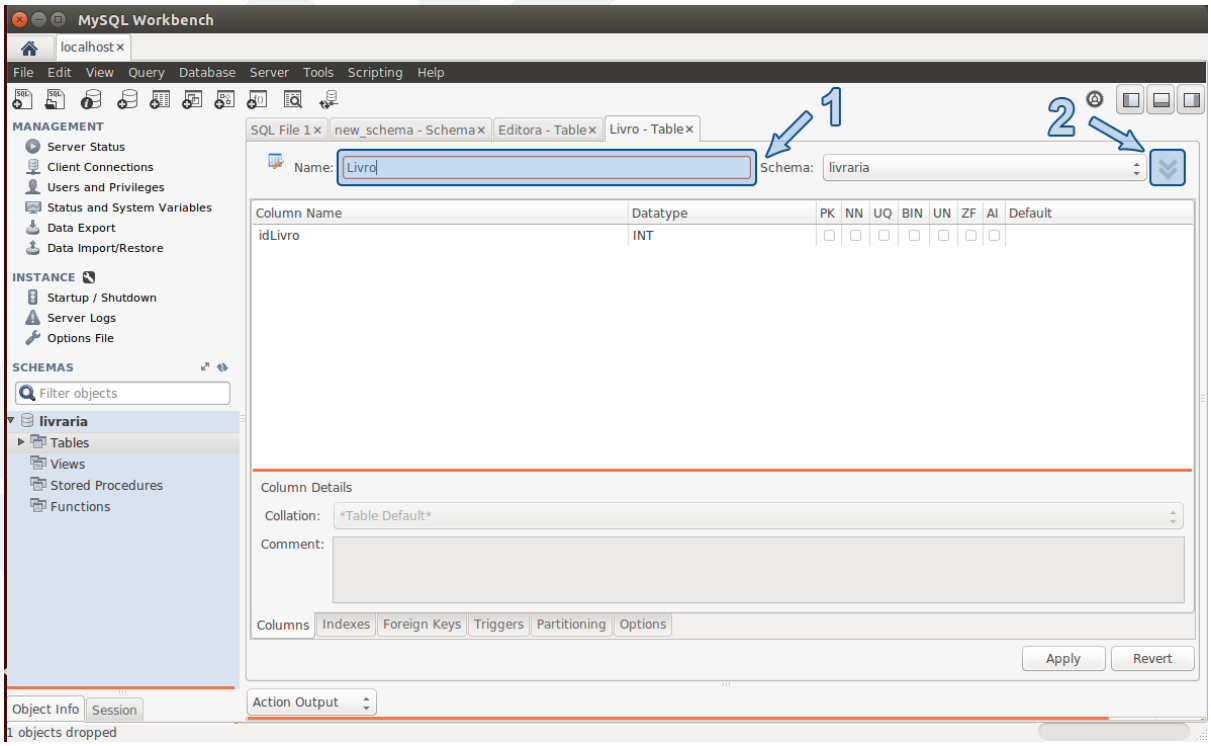

MySQL Workbench
localhost
File Edit View Query Database Server Tools Scripting Help
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variable
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
SCHEMAS
Filter objects
livraria
Tables
Views
Stored Procedures
Functions
Object Info
Session
Schema: livraria
SQL File 1
new_schema - Schema
Editora - Table
Name: Editora
Schema: livraria
Collation: *Default*
Engine: InnoDB
Comment:
1
Column Name
Datatype
PK NN UQ BIN UN ZF AI Default
id
BIGINT
nome
VARCHAR(255)
email
VARCHAR(255)
2
Column Details
Collation: *Table Default*
Comment:
3
Columns
Indexes
Foreign Keys
Triggers
Partitioning
Options
Apply
Revert
Action Output
1 objects dropped

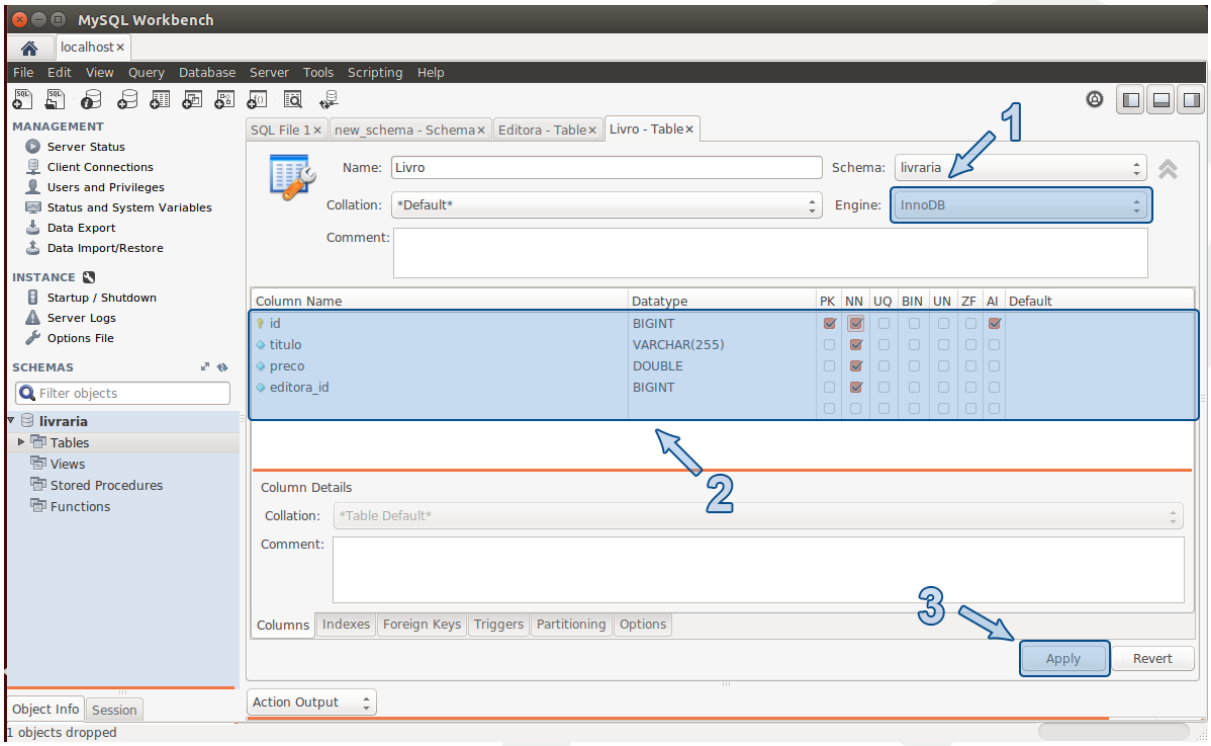

Apply SQL Script to Database
Review SQL Script
Apply SQL Script
Review the SQL Script to be Applied on the Database
Please review the following SQL script that will be applied to the database.
Note that once applied, these statements may not be revertible without losing some of the data.
You can also manually change the SQL statements before execution.
CREATE TABLE `livraria`.`Editora` (
`id` BIGINT NOT NULL AUTO_INCREMENT,
`nome` VARCHAR(255) NOT NULL,
`email` VARCHAR(255) NOT NULL,
PRIMARY KEY (`id`))
ENGINE = InnoDB;
Cancel
Back
Apply

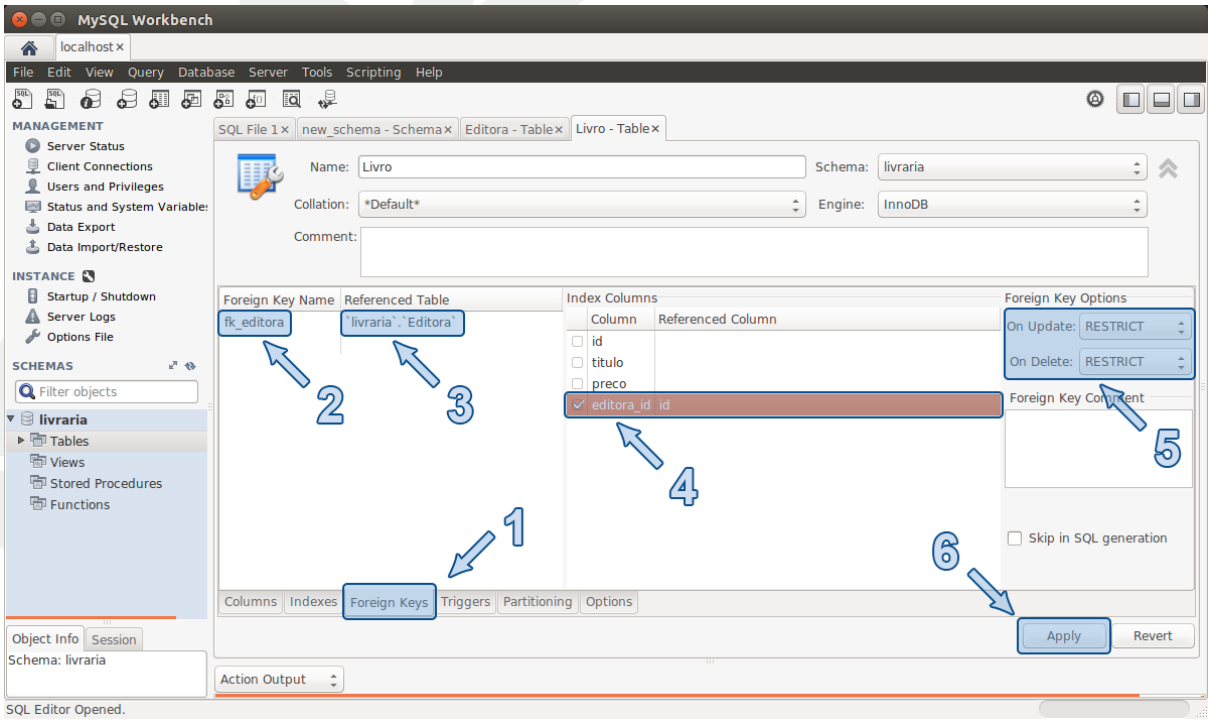

Apply SQL Script to Database
Review SQL Script
Apply SQL Script
Applying SQL script to the database ...
The following tasks will now be executed. Please monitor the execution.
Press Show Logs to see the execution logs.
Execute SQL Statements
SQL script was successfully applied to the database.
Show Logs
Cancel
Back
Close

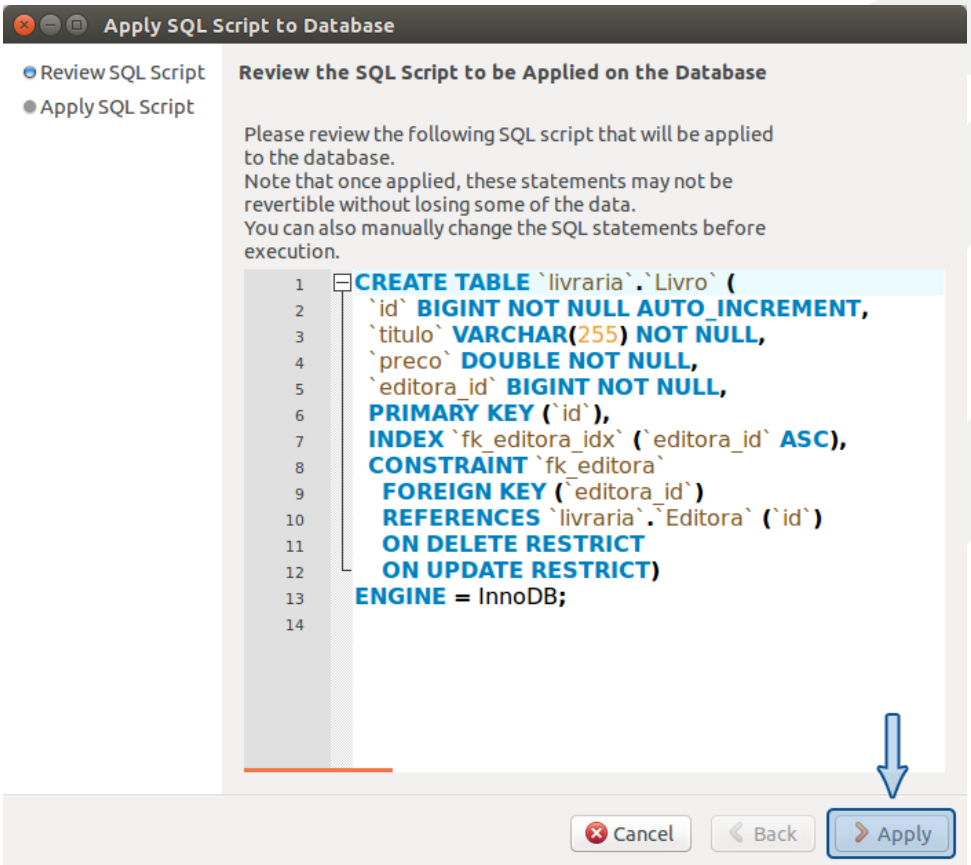
6 Crie uma tabela chamada **Livro** conforme as figuras abaixo.

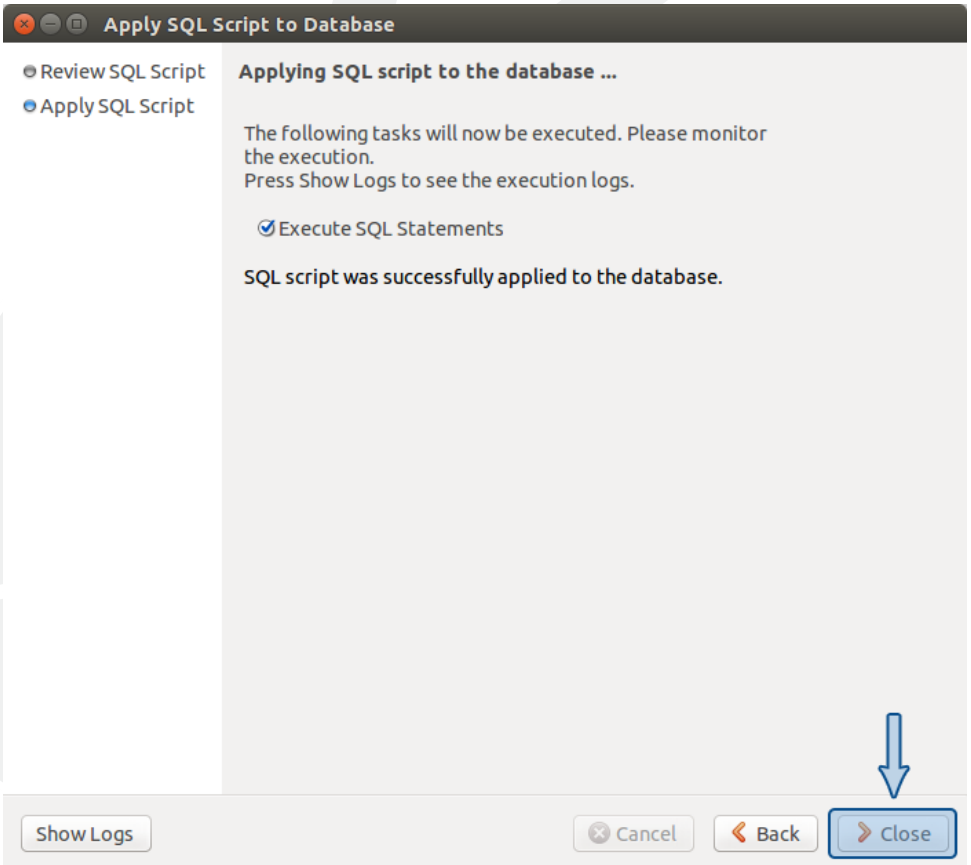




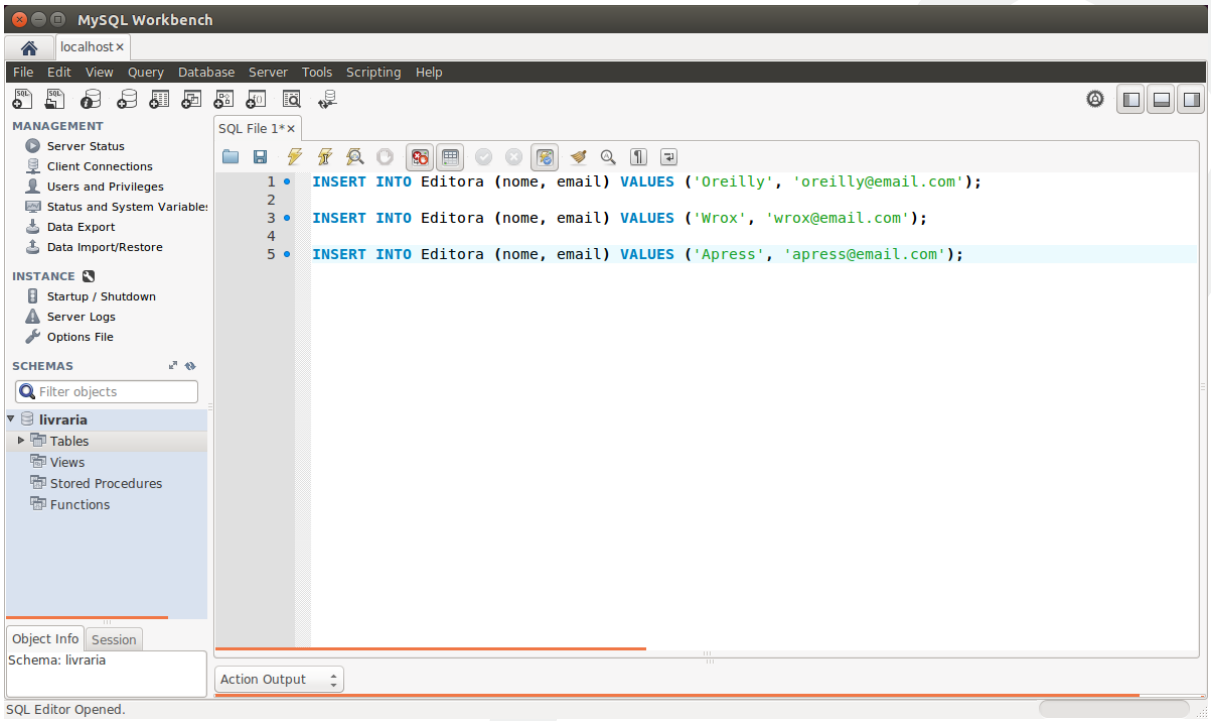

MySQL Workbench
localhost
File Edit View Query Database Server Tools Scripting Help
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
SCHEMAS
Filter objects
livraria
Tables
Views
Stored Procedures
Functions
SQL File 1
new_schema - Schema
Editora - Table
Livro - Table
Name: Livro
Schema: livraria
Collation: *Default*
Engine: InnoDB
Comment:
Column Name
Datatype
PK
NN
UQ
BIN
UN
ZF
AI
Default
id BIGINT
titulo VARCHAR(255)
preco DOUBLE
editora_id BIGINT
Column Details
Collation: *Table Default*
Comment:
Columns
Indexes
Foreign Keys
Triggers
Partitioning
Options
Apply
Revert
Action Output
Object Info
Session
1 objects dropped
1
2
3


MySQL Workbench
localhost
File Edit View Query Database Server Tools Scripting Help
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variable
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
SCHEMAS
Filter objects
livraria
Tables
Views
Stored Procedures
Functions
SQL File 1
new_schema - Schema
Editora - Table
Livro - Table
Name: Livro
Schema: livraria
Collation: *Default*
Engine: InnoDB
Comment:
Foreign Key Name
Referenced Table
fk_editora
`livraria`.`Editora`
Index Columns
Column
Referenced Column
id
titulo
preco
editora_id id
Foreign Key Options
On Update: RESTRICT
On Delete: RESTRICT
Foreign Key Comment
Skip in SQL generation
Columns
Indexes
Foreign Keys
Triggers
Partitioning
Options
Apply
Revert
Object Info
Session
Schema: livraria
Action Output
SQL Editor Opened.
1
2
3
4
5
6

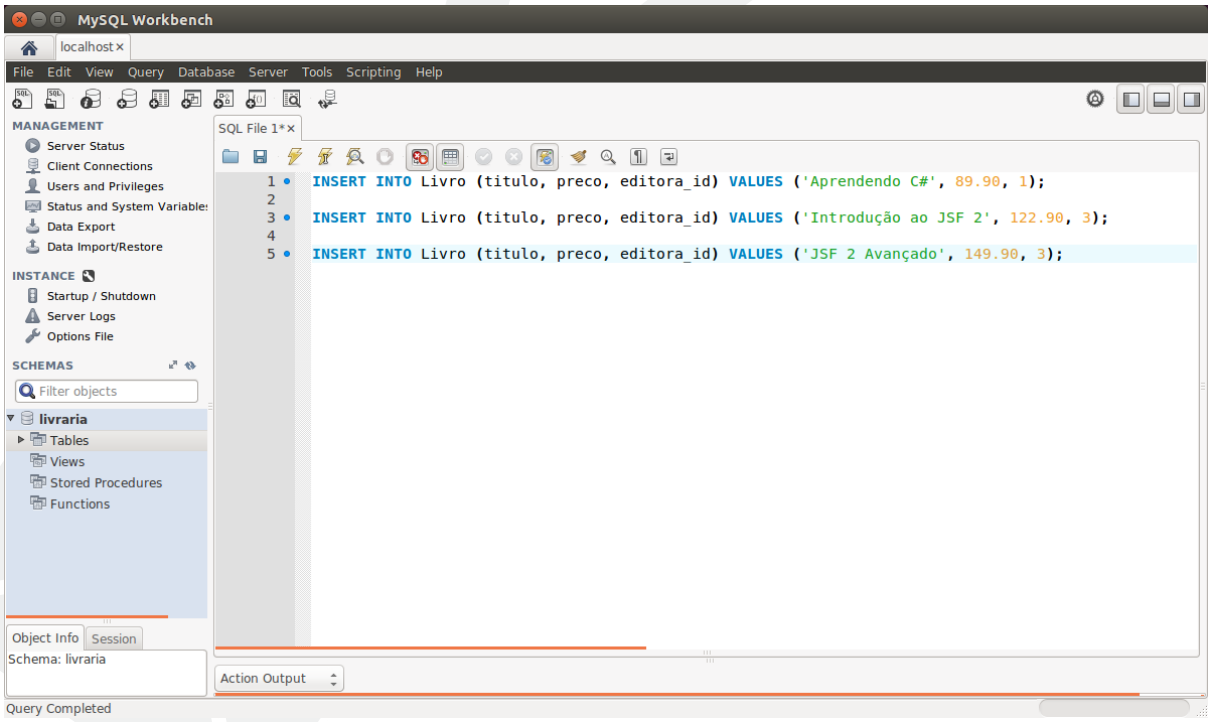

Apply SQL Script to Database
Review SQL Script
Apply SQL Script
Review the SQL Script to be Applied on the Database
Please review the following SQL script that will be applied to the database.
Note that once applied, these statements may not be revertible without losing some of the data.
You can also manually change the SQL statements before execution.
CREATE TABLE `livraria`.`Livro` (
`id` BIGINT NOT NULL AUTO_INCREMENT,
`titulo` VARCHAR(255) NOT NULL,
`preco` DOUBLE NOT NULL,
`editora_id` BIGINT NOT NULL,
PRIMARY KEY (`id`),
INDEX `fk_editora_idx` (`editora_id` ASC),
CONSTRAINT `fk_editora`
FOREIGN KEY (`editora_id`)
REFERENCES `livraria`.`Editora` (`id`)
ON DELETE RESTRICT
ON UPDATE RESTRICT)
ENGINE = InnoDB;
Cancel
Back
Apply


Apply SQL Script to Database
Review SQL Script
Apply SQL Script
Applying SQL script to the database ...
The following tasks will now be executed. Please monitor the execution.
Press Show Logs to see the execution logs.
Execute SQL Statements
SQL script was successfully applied to the database.
Show Logs
Cancel
Back
Close

7 Adicione alguns registros na tabela Editora. Veja exemplos na figura abaixo.

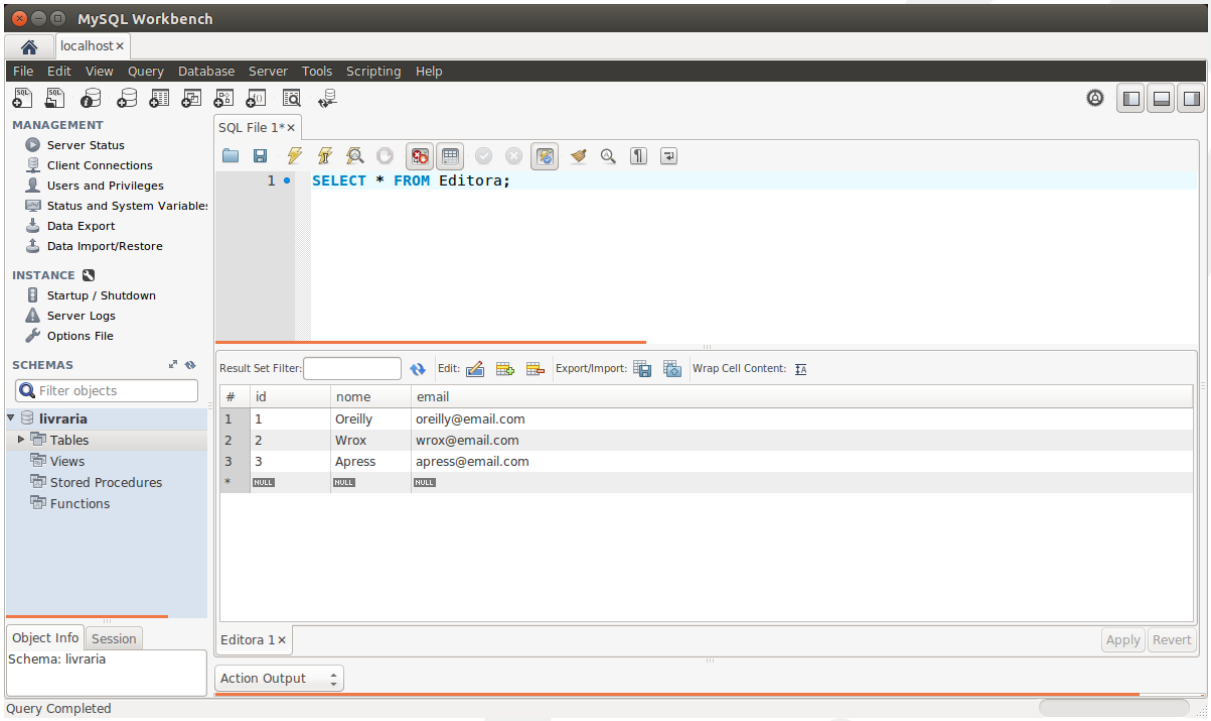


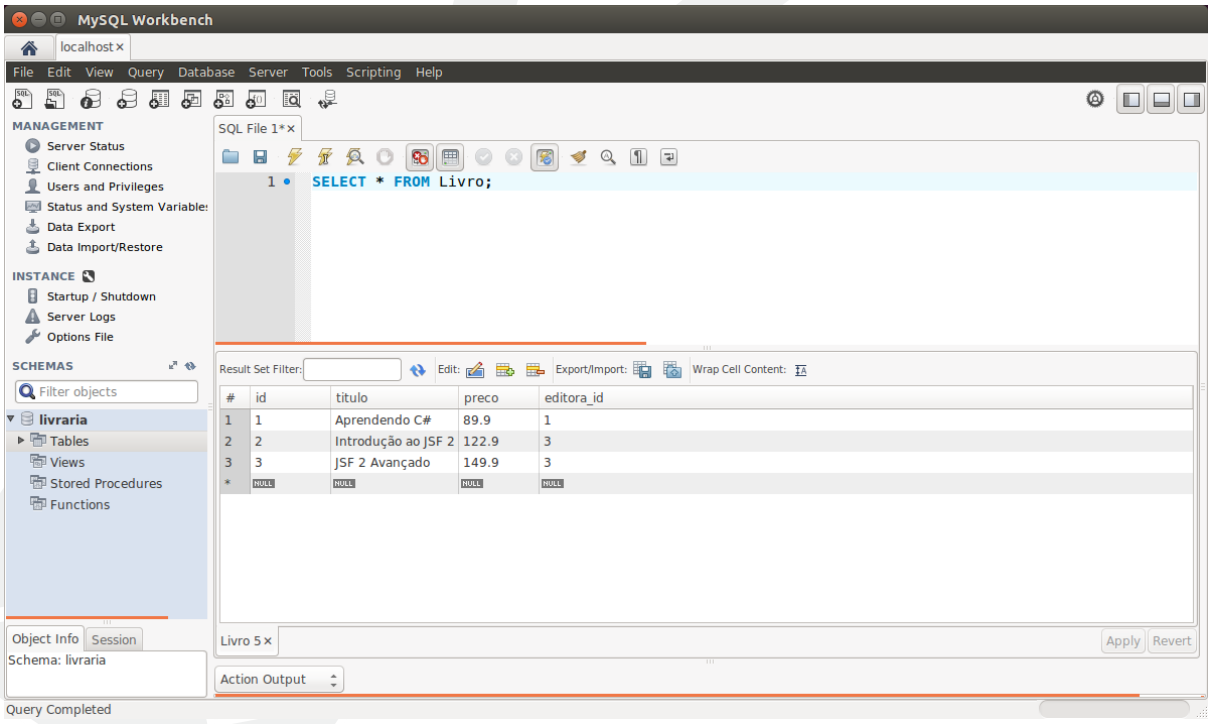
8 Adicione alguns registros na tabela Livro. Veja exemplos na figura abaixo.



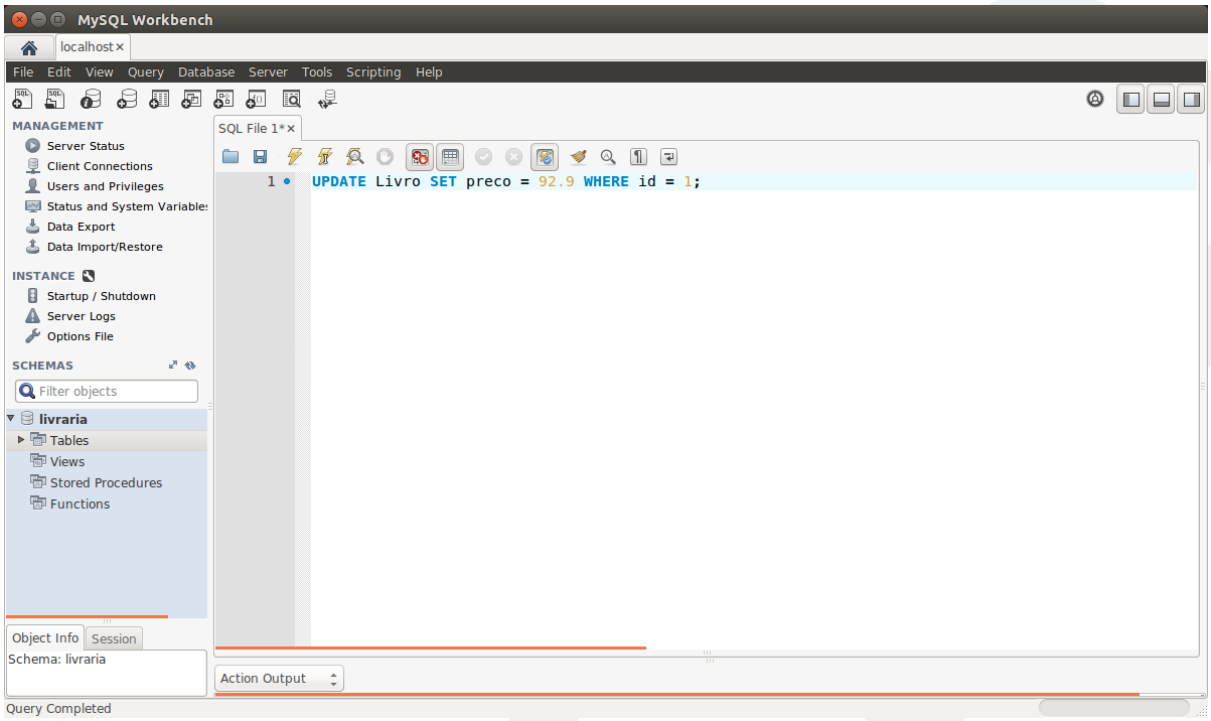
9 Consulte os registros da tabela Editora e, em seguida, consulte a tabela Livro. Veja exemplos logo

abaixo.

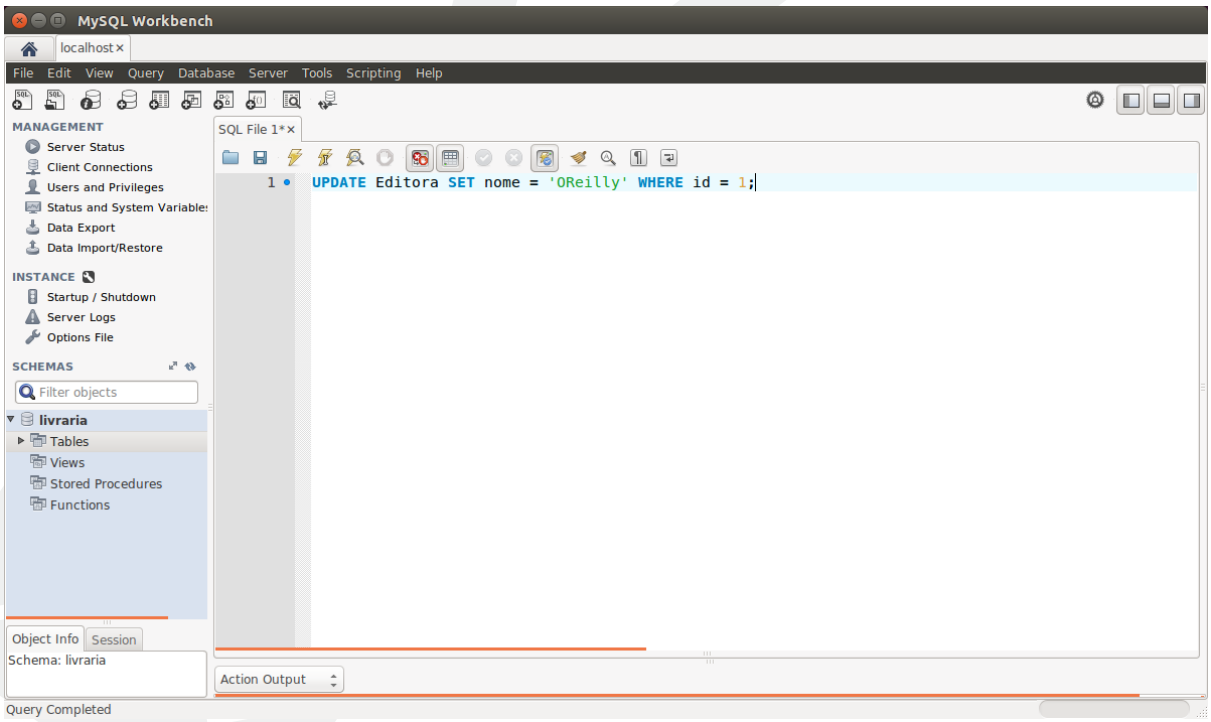




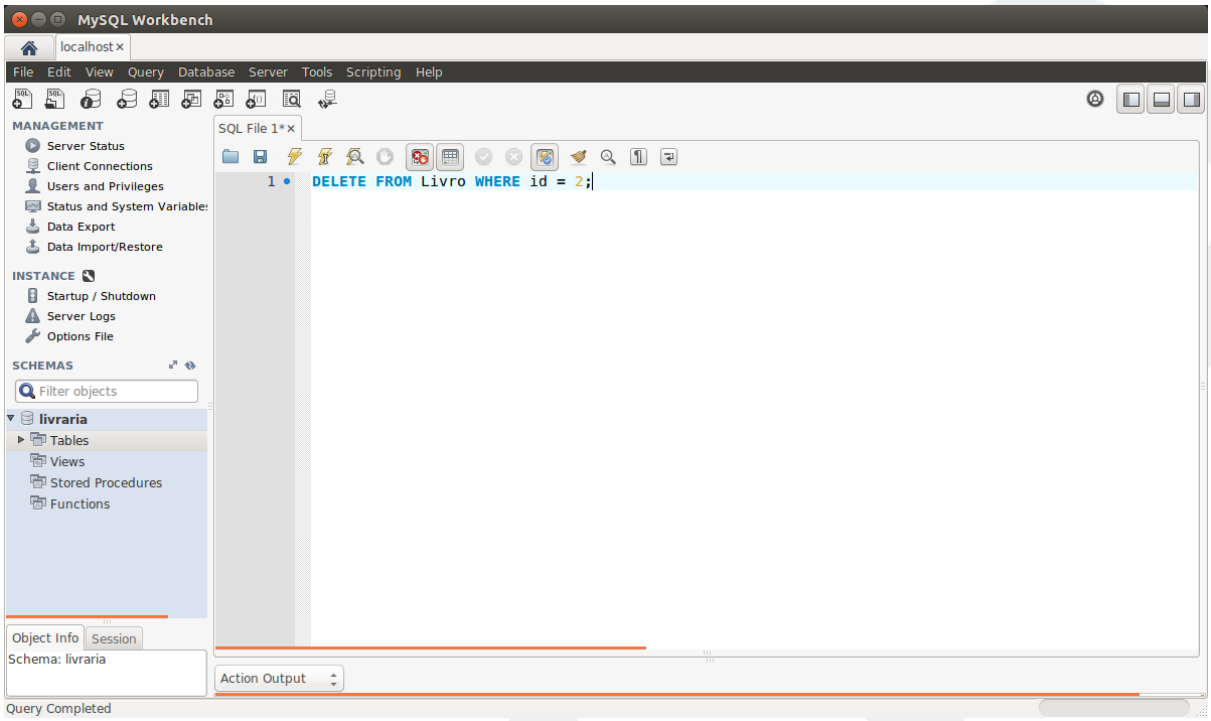
10 Altere alguns dos registros da tabela Livro. Veja o exemplo abaixo.



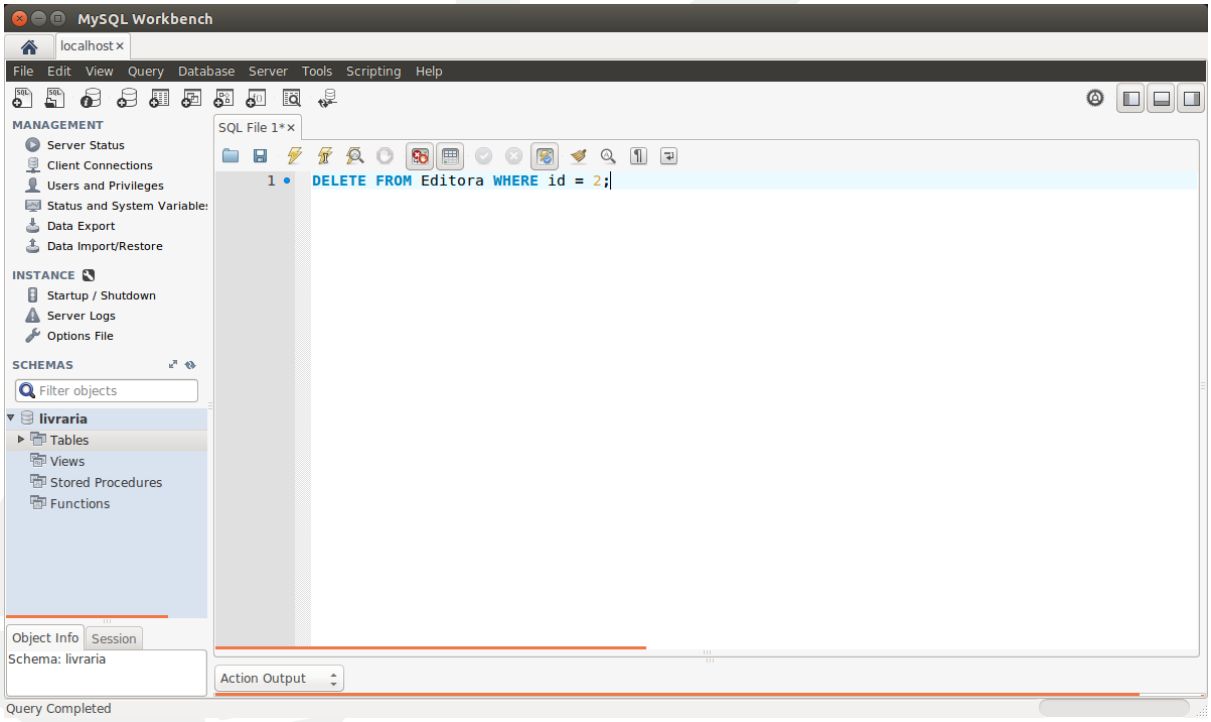
11 Altere alguns dos registros da tabela Editora. Veja o exemplo abaixo.



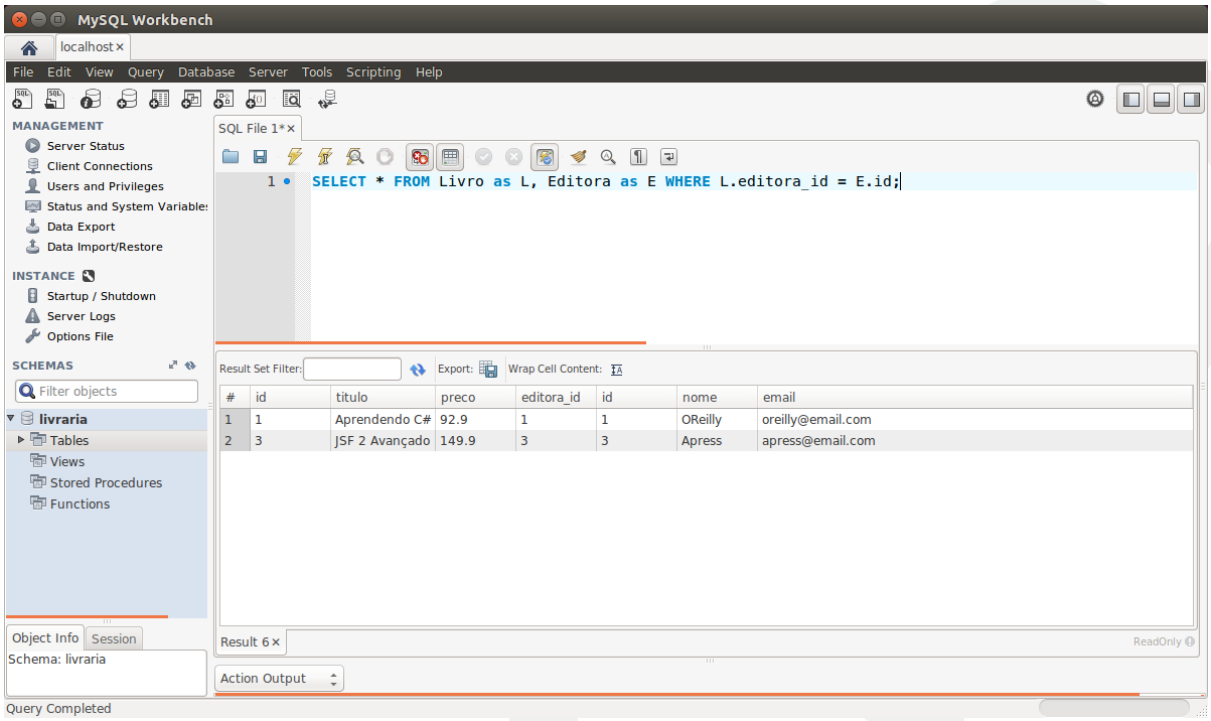
12 Remova alguns registros da tabela Livro. Veja o exemplo abaixo.



13 Remova alguns registros da tabela Editora. Preste atenção para não remover uma editora que tenha algum livro relacionado já adicionado no banco. Veja o exemplo abaixo:



14 Faça uma consulta para buscar todos os livros associados as suas respectivas editoras. Veja um exemplo na figura abaixo.


MySQL Workbench
localhost
File Edit View Query Database Server Tools Scripting Help
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variable
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
SCHEMAS
Filter objects
livraria
Tables
Views
Stored Procedures
Functions
Object Info
Session
Schema: livraria
SQL File 1*
1 SELECT * FROM Livro as L, Editora as E WHERE L.editora_id = E.id;
Result Set Filter:
Export:
Wrap Cell Content:
id titulo preco editora_id id nome email
1 1 Aprendendo C# 92.9 1 1 OReilly oreilly@email.com
2 3 JSF 2 Avançado 149.9 3 3 Apress apress@email.com
Result 6
ReadOnly
Action Output
Query Completed

JDBC

CAPÍTULO 2

No capítulo anterior, aprendemos que utilizar bancos de dados é uma ótima alternativa para armazenar os dados de uma aplicação. Entretanto, você deve ter percebido que as interfaces disponíveis para interagir com o MySQL Server não podem ser utilizadas por qualquer pessoa. Para utilizá-las, é necessário conhecer a linguagem SQL e os conceitos do modelo relacional. Em geral, as interfaces dos outros SGDBs exigem os mesmos conhecimentos.

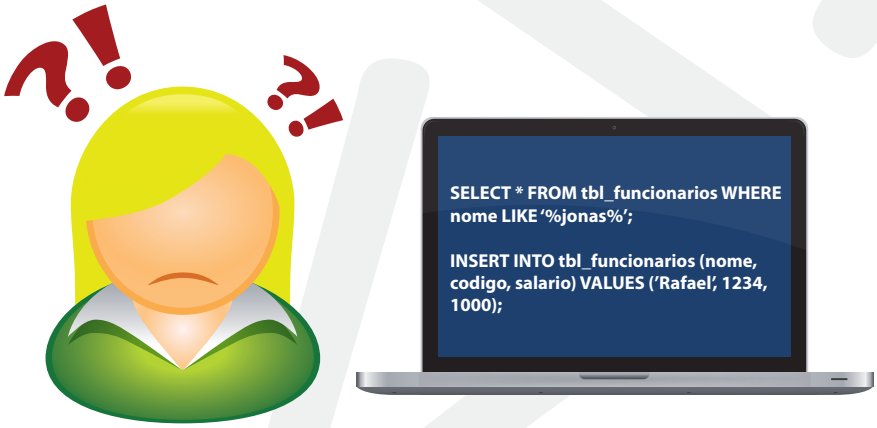


Figura 2.1: Usuários comuns não possuem conhecimento sobre SQL ou sobre o modelo relacional

Para resolver esse problema, podemos desenvolver aplicações com interfaces que não exijam conhecimentos técnicos de SQL ou do modelo relacional para serem utilizadas. Dessa forma, usuários comuns poderiam manipular as informações do banco de dados através dessas aplicações. Nessa abordagem, os usuários interagem com as aplicações e as aplicações interagem com os SGDBs.



Figura 2.2: Usuários comuns devem utilizar interfaces simples

Driver

As aplicações interagem com os SGDBs através de troca de mensagens. Os SGDBs definem o formato das mensagens. Para não sobrecarregar o canal de comunicação entre as aplicações e os SGDBs, as mensagens trocadas devem ocupar o menor espaço possível. Geralmente, protocolos binários são mais apropriados para reduzir o tamanho das mensagens e consequentemente diminuir a carga do canal de comunicação. Por isso, os SGDBs utilizam protocolos binários.

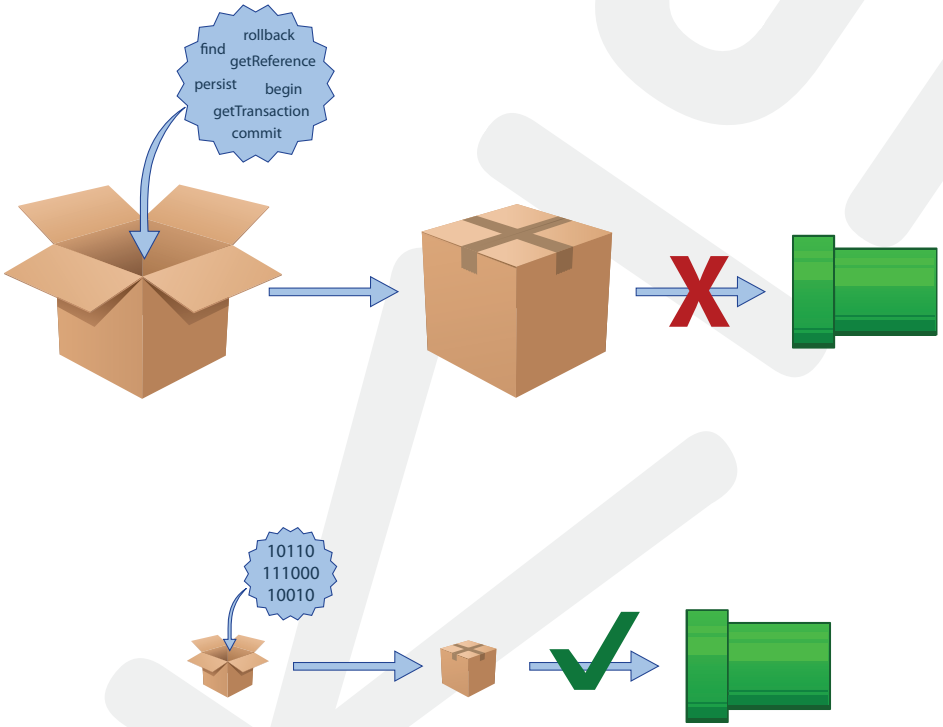


Figura 2.3: Diminuindo o tamanho das mensagens para não sobrecarregar o meio de comunicação

Mensagens binárias são facilmente interpretadas por computadores. Por outro lado, são complexas para um ser humano compreender. Dessa forma, o trabalho dos desenvolvedores seria muito complexo, aumentando o custo para o desenvolvimento e manutenção das aplicações.

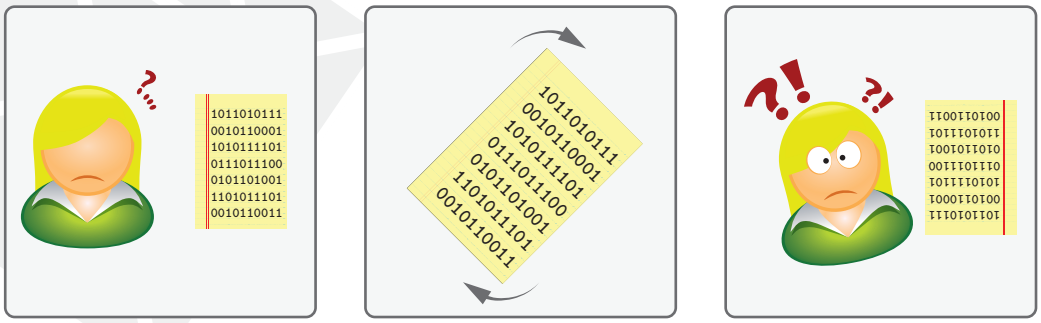


Figura 2.4: Mensagens binárias são altamente complexas para os seres humanos

Para resolver esse problema e facilitar o desenvolvimento das aplicações, as empresas proprietárias dos SGDBs, normalmente, desenvolvem e distribuem **drivers de conexão**. Um driver de conexão atua como um intermediário entre as aplicações e os SGDBs.

Os drivers de conexão são "tradutores" de comandos escritos em uma determinada linguagem de programação para comandos definidos de acordo com o protocolo de um SGDB. Utilizando um driver de conexão, os desenvolvedores das aplicações não manipulam diretamente as mensagens binárias trocadas entre as aplicações e os SGDBs.



Mais Sobre
Em alguns casos, o protocolo binário de um determinado SGDB é fechado. Consequentemente, a única maneira de se comunicar com ele é através de um driver de conexão oferecido pelo fabricante desse SGDB.



JDBC

Suponha que os drivers de conexão fossem desenvolvidos sem nenhum padrão. Cada driver teria sua própria interface, ou seja, seu próprio conjunto de instruções. Consequentemente, os desenvolvedores teriam de conhecer a interface de cada um dos drivers dos respectivos SGDBs que fossem utilizar.

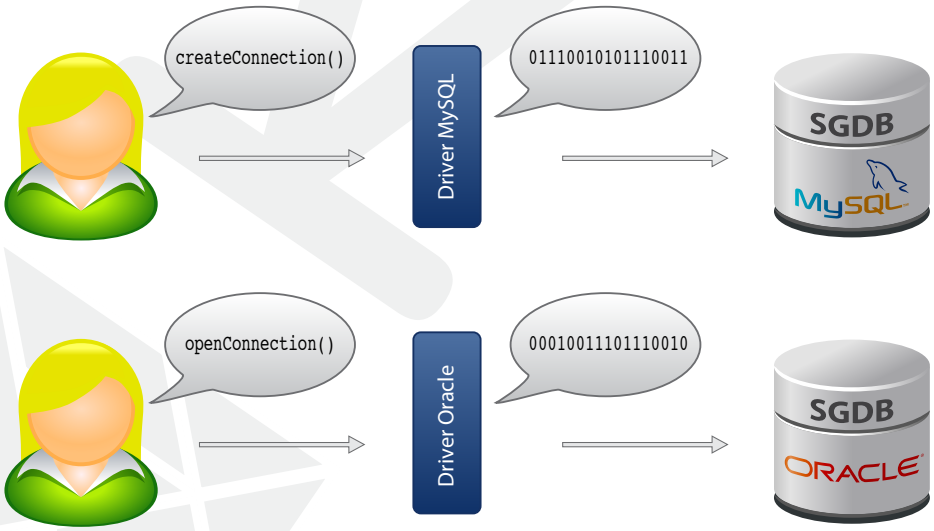


Figura 2.5: Drivers de conexão sem padronização

Para facilitar o trabalho do desenvolvedor da aplicação, a plataforma Java possui uma especificação que padroniza os drivers de conexão. A sigla dessa especificação é **JDBC** (*Java Database Connectivity*). Em geral, as empresas proprietárias dos SGBDs desenvolvem e distribuem drivers de conexão que seguem a especificação JDBC.

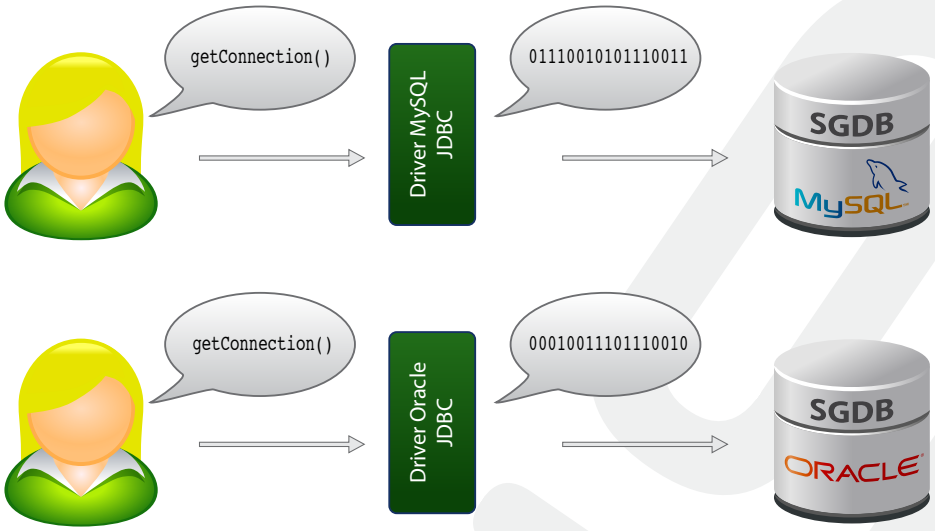


Figura 2.6: Drivers de conexão padronizados pela especificação JDBC

Instalando o Driver JDBC do MySQL Server

Podemos obter um driver de conexão JDBC para o MySQL Server na seguinte url:

http://www.mysql.com/downloads/connector/j/.

A instalação desse driver consiste em descompactar o arquivo obtido no site acima e depois incluir o arquivo **jar** com o driver no **class path** da aplicação.

Criando uma conexão

Com o driver de conexão JDBC adicionado à aplicação, já é possível criar uma conexão. Abaixo, estão as informações geralmente utilizadas para a criação de uma conexão JDBC.

- Nome do driver JDBC.
- Endereço (IP e porta) do SGDB.
- Nome da base de dados.
- Um usuário do SGBD.
- Senha do usuário.

O nome do driver JDBC, o endereço do SGDB e nome da base de dados são definidos na **string de conexão** ou **url de conexão**. Veja o exemplo abaixo:

```
1 String stringDeConexao = "jdbc:mysql://localhost/livraria";
```

Código Java 2.1: String de conexão

A classe responsável pela criação de uma conexão JDBC é a `DriverManager` do pacote `java.sql`. A string de conexão, o usuário e a senha devem ser passados ao método estático `getConnection()` da classe `DriverManager` para que ela possa criar uma conexão JDBC.

```
1 String urlDeConexao = "jdbc:mysql://localhost/livraria";
2 String usuario = "root";
3 String senha = "root";
4
5 Connection conexao = DriverManager.getConnection(urlDeConexao, usuario, senha);
```

Código Java 2.2: Criando uma conexão JDBC



Apagando uma base de dados

Após estabelecer uma conexão JDBC, podemos executar operações. A primeira operação que realizaremos é apagar uma base de dados. O primeiro passo para executar essa operação é definir o código SQL correspondente.

```
1 String sql = "DROP DATABASE IF EXISTS livraria";
```

O código SQL correspondente à operação que desejamos executar deve ser passado como parâmetro para o método `prepareStatement()` de uma conexão JDBC. Esse método criará um objeto que representa a operação que será executada. A operação poderá ser executada posteriormente através do método `execute()`.

```
1 // criando um prepared statement
2 PreparedStatement comando = conexao.prepareStatement(sql);
3
4 // executando o prepared statement
5 comando.execute();
6
7 // fechando o prepared statement
8 comando.close();
```



Importante

A mesma conexão pode ser reaproveitada para executar várias operações. Quando não houver mais operações a serem executadas, devemos finalizar a conexão JDBC através do método `close()`. Finalizar as conexões JDBC que não são mais necessárias é importante pois libera recursos no SGBD.

```
1 conexao.close();
```



Criando uma base de dados

O procedimento para criar uma base de dados é muito parecido com o procedimento para apagar uma base de dados.

```
1 // código SQL da operação
```

```
2  String sql = "CREATE DATABASE livraria";
3
4  // criando um prepared statement
5  PreparedStatement comando = conexao.prepareStatement(sql);
6
7  // executando o prepared statement
8  comando.execute();
9
10 // fechando o prepared statement
11 comando.close();
```



Criando uma tabela

Analogamente, ao que vimos nas seções anteriores, podemos criar uma tabela através de um conexão JDBC.

```
1  String sql =
2    "CREATE TABLE Editora (" +
3      "id BIGINT NOT NULL AUTO_INCREMENT," +
4      "nome VARCHAR (255) NOT NULL," +
5      "email VARCHAR (255) NOT NULL," +
6      "PRIMARY KEY (id)" +
7    ")" +
8    "ENGINE = InnoDB";
9
10 PreparedStatement comando = conexao.prepareStatement(sql);
11
12 comando.execute();
13
14 comando.close();
```



Inserindo dados

Também podemos adicionar dados nas tabelas utilizando conexões JDBC.

```
1  String sql =
2    "INSERT INTO Editora (nome, email) VALUES ('K19','contato@k19.com.br')";
3
4  PreparedStatement comando = conexao.prepareStatement(sql);
5
6  comando.execute();
7
8  comando.close();
```

Normalmente, optamos por deixar os SGBDs gerar os valores das chaves primárias dos registros inseridos em uma tabela. Esses valores podem ser obtidos através dos recursos definidos pela especificação JDBC.

Veja o código a seguir. Na linha 5, a constante `RETURN_GENERATED_KEYS` da interface `Statement` foi utilizada para indicar que queremos recuperar os valores gerados pelo SGBD. Na linha 9, recuperamos esses valores através do método `getGeneratedKeys()`. Esse método devolve um `ResulSet` com todos os valores gerados. Na linha 11, utilizamos o método `next()` para avançar para o primeiro valor gerado. Na linha 13, o método `getLong()` devolve esse valor como `Long`.

```
1  String sql =
```

```
2   "INSERT INTO Editora (nome, email) VALUES ('K19','contato@k19.com.br')";
3
4  PreparedStatement comando =
5    conexao.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
6
7  comando.execute();
8
9  ResultSet generatedKeys = comando.getGeneratedKeys();
10
11 generatedKeys.next();
12
13 Long id = generatedKeys.getLong(1);
14
15 comando.close();
```



Exercícios de Fixação

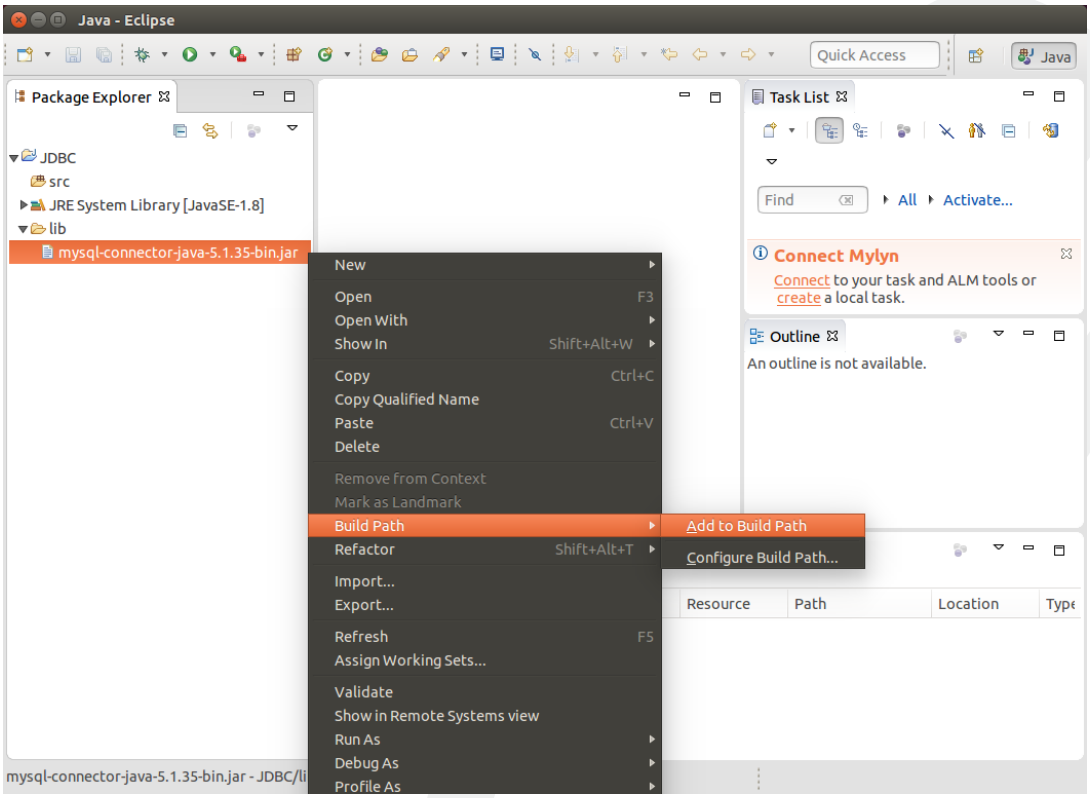
1 Crie um projeto chamado **JDBC**. Esse projeto deve ser do tipo **Java Project**.

2 Acrescente uma pasta chamada **lib** no projeto **JDBC**. Copie o arquivo **mysql-connector-java-VERSAO-bin.jar** da pasta **K19-Arquivos/mysql-connector-java-VERSAO** para a pasta **lib** do projeto **JDBC**. Observação: o termo "VERSAO" deve ser substituído pelo número da versão utilizada.



Importante
Você também pode obter o arquivo **mysql-connector-java-VERSAO-bin.jar** através do site da K19: www.k19.com.br/arquivos.

3 Adicione o arquivo **mysql-connector-java-VERSAO-bin.jar** ao **build path** do projeto **JDBC**. Veja a imagem abaixo.



4 Crie uma classe chamada **CriaBaseDeDadosLivraria** em um pacote chamado **br.com.k19.jdbc** no projeto **JDBC** com o seguinte conteúdo.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.PreparedStatement;
6
7  public class CriaBaseDeDadosLivraria {
8    public static void main(String[] args) throws Exception {
9      String stringDeConexao = "jdbc:mysql://localhost:3306";
10     String usuario = "root";
11     String senha = "root";
12
13     System.out.println("Abrindo conexão...");
14     Connection conexao =
15       DriverManager.getConnection(stringDeConexao, usuario, senha);
16
17     System.out.println("Apagando a base de dados livraria...");
18     String sql = "DROP DATABASE IF EXISTS livraria";
19     PreparedStatement comando = conexao.prepareStatement(sql);
20     comando.execute();
21     comando.close();
22
23     System.out.println("Criando a base de dados livraria...");
24     sql = "CREATE DATABASE livraria";
25     comando = conexao.prepareStatement(sql);
26     comando.execute();
27     comando.close();
28
29     System.out.println("Fechando conexão...");
30     conexao.close();
31   }
32 }
```

Código Java 2.10: www.k19.com.br/exercicios/k12/02/004/CriaBaseDeDadosLivraria.java

Execute a classe **CriaBaseDeDadosLivraria** e verifique se a base dados livraria foi criada com sucesso no SGBD.



Importante
Nos exercícios, não vamos nos preocupar em tratar as exceptions. Por isso, utilizaremos o comando `throws` no método `main`. Em um projeto real, provalvemente, será necessário definir uma estratégia difernte para lidar com as exceptions.

5 Crie uma classe chamada **CriaTabelaEditora** em um pacote chamado **br.com.k19.jdbc** no projeto **JDBC** com o seguinte conteúdo.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.PreparedStatement;
6
7  public class CriaTabelaEditora {
8    public static void main(String[] args) throws Exception {
9      String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
10     String usuario = "root";
11     String senha = "root";
12
13     System.out.println("Abrindo conexão...");
14     Connection conexao =
15       DriverManager.getConnection(stringDeConexao, usuario, senha);
16
17     System.out.println("Criando a tabela Editora...");
18     String sql =
19         "CREATE TABLE Editora (" +
20           "id BIGINT NOT NULL AUTO_INCREMENT," +
21           "nome VARCHAR (255) NOT NULL," +
22           "email VARCHAR (255) NOT NULL," +
23           "PRIMARY KEY (id)" +
24         ")" +
25         "ENGINE = InnoDB";
26     PreparedStatement comando = conexao.prepareStatement(sql);
27     comando.execute();
28     comando.close();
29
30     System.out.println("Fechando conexão...");
31     conexao.close();
32   }
33 }
```

Código Java 2.11: www.k19.com.br/exercicios/k12/02/005/CriaTabelaEditora.java

Execute a classe **CriaTabelaEditora** e verifique se a tabela Editora foi criada corretamente.

6 Crie uma classe chamada **Editora** em um pacote chamado **br.com.k19.jdbc** no projeto **JDBC** com o seguinte conteúdo.

```
1  package br.com.k19.jdbc;
2
```

```
3  public class Editora {
4    private Long id;
5    private String nome;
6    private String email;
7
8    // GETTERS E SETTERS
9  }
```

Código Java 2.12: www.k19.com.br/exercicios/k12/02/006/Editora.java

7 Crie uma classe chamada **InsereEditora** em um pacote chamado **br.com.k19.jdbc** no projeto **JDBC** com o seguinte conteúdo.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.PreparedStatement;
6  import java.sql.ResultSet;
7  import java.sql.Statement;
8  import java.util.Scanner;
9
10 public class InsereEditora {
11   public static void main(String[] args) throws Exception {
12     String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
13     String usuario = "root";
14     String senha = "root";
15
16     System.out.println("Abrindo conexão...");
17     Connection conexao =
18       DriverManager.getConnection(stringDeConexao, usuario, senha);
19
20     Scanner entrada = new Scanner(System.in);
21     Editora e = new Editora();
22
23     System.out.println("Digite o nome da editora: ");
24     e.setNome(entrada.nextLine());
25
26     System.out.println("Digite o email da editora: ");
27     e.setEmail(entrada.nextLine());
28
29     entrada.close();
30
31     String sql = "INSERT INTO Editora (nome, email) " +
32       "VALUES ('" + e.getNome() + "', '" + e.getEmail() + "')";
33
34     PreparedStatement comando = conexao.prepareStatement(sql,
35         Statement.RETURN_GENERATED_KEYS);
36
37     System.out.println("Executando comando...");
38     comando.execute();
39
40     System.out.println("Recuperando o ID gerado pelo SGBD...");
41     ResultSet generatedKeys = comando.getGeneratedKeys();
42     generatedKeys.next();
43     e.setId(generatedKeys.getLong(1));
44     System.out.println("ID: " + e.getId());
45
46     System.out.println("Fechando conexão...");
47     conexao.close();
48   }
49 }
```

Código Java 2.13: www.k19.com.br/exercicios/k12/02/007/InsereEditora.java

Execute a classe **InsereEditora** e verifique se o registro foi inserido com sucesso na tabela Editora.



Exercícios Complementares

1. Crie uma classe chamada **CriaTabelaLivro** para adicionar a tabela **Livro** na base de dados **livraria**.

2. Crie uma classe chamada **Livro** para modelar os livros da livraria.

3. Crie uma classe chamada **InsereLivro** para cadastrar livros na base de dados.



SQL Injection

A implementação da inserção de registros feita anteriormente possui uma falha grave. Os dados obtidos do usuário através do teclado não são tratados antes de serem enviados para o SGDB.

Esses dados podem conter caracteres especiais. Se esses caracteres não são tratados, o comportamento esperado da operação é afetado. Eventualmente, registros não são inseridos como deveriam ou brechas de segurança podem se abrir.

Por exemplo, considere a classe `InsereEditora` do exercício de fixação. Se o usuário digitar "O'Reilly" e "oreilly@email.com", o código SQL gerado pela aplicação seria:

```
1 INSERT INTO Editora (nome, email) VALUES ('O'Reilly', 'oreilly@email.com')
```

Observe que o caractere aspas simples aparece cinco vezes no código SQL acima. O SGDB não saberia dizer onde de fato termina o nome da editora. Ao tentar executar esse código, um erro de sintaxe é lançado pelo MySQL Server. Para resolver esse problema manualmente, devemos adicionar o caractere "\" antes do caractere aspas simples que faz parte do nome da editora. Na sintaxe do MySQL Server, o caractere "\" deve ser acrescentado imediatamente antes de todo caractere especial que deve ser tratado como um caractere comum.

```
1 INSERT INTO Editora (nome, email) VALUES ('O\'Reilly', 'oreilly@email.com')
```

Os valores recebidos dos usuários devem ser analisados e os caracteres especiais contidos nesses valores devem ser tratados. Esse processo é extremamente trabalhoso, pois o conjunto de caracteres especiais e a forma de tratá-los é diferente em cada SGDB.

A responsabilidade do tratamento dos caracteres especiais contidos nos valores de entrada dos usuários pode ser repassada para os drivers JDBC. Dessa forma, o código das aplicações se torna independente das particularidades desse processo para cada SGDB.



Mais Sobre
O processo de tratamento dos caracteres especiais das entradas dos usuários é denominado **sanitize**.

```
// criando um objeto da classe Editora
Editora e = new Editora();

// lendo as entradas do usuário
System.out.println("Digite o nome da editora: ");
e.setNome(entrada.nextLine());

System.out.println("Digite o email da editora: ");
e.setEmail(entrada.nextLine());

// código sql com marcadores para as entradas do usuário
String sql = "INSERT INTO Editora (nome, email) VALUES (?, ?)";

// criando um comando a partir do código SQL
PreparedStatement comando = conexao.prepareStatement(sql);

// adicionando as entradas do usuários no comando
// o processo de sanitização ocorre aqui
comando.setString(1, e.getNome());
comando.setString(2, e.getEmail());
```

Código Java 2.19: "Sanitizando" as entradas dos usuários

Observe que o código SQL foi definido com parâmetros através do caractere "?". Antes de executar o comando, é necessário determinar os valores dos parâmetros. Essa tarefa pode ser realizada através do método `setString()`, que recebe o índice (posição) do parâmetro no código SQL e o valor correspondente. Esse método faz o tratamento dos caracteres especiais contidos nos valores de entrada do usuário de acordo com as regras do SGDB utilizado.



Exercícios de Fixação

8 Tente gerar um erro de SQL Injection no cadastro de editoras. Dica: utilize entradas com aspas simples.

9 Altere o código da classe **InsereEditora** para eliminar o problema do SQL Injection.

```
package br.com.k19.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Scanner;

public class InsereEditora {
  public static void main(String[] args) throws Exception {
    String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
    String usuario = "root";
    String senha = "root";

    System.out.println("Abrindo conexão...");
    Connection conexao =
      DriverManager.getConnection(stringDeConexao, usuario, senha);

    Scanner entrada = new Scanner(System.in);
    Editora e = new Editora();

```

```
23      System.out.println("Digite o nome da editora: ");
24      e.setNome(entrada.nextLine());
25
26      System.out.println("Digite o email da editora: ");
27      e.setEmail(entrada.nextLine());
28
29      entrada.close();
30
31      String sql = "INSERT INTO Editora (nome, email) VALUES (?, ?)";
32
33      PreparedStatement comando = conexao.prepareStatement(sql,
34          Statement.RETURN_GENERATED_KEYS);
35      comando.setString(1, e.getNome());
36      comando.setString(2, e.getEmail());
37
38      System.out.println("Executando comando...");
39      comando.execute();
40
41      System.out.println("Recuperando o ID gerado pelo SGBD...");
42      ResultSet generatedKeys = comando.getGeneratedKeys();
43      generatedKeys.next();
44      e.setId(generatedKeys.getLong(1));
45      System.out.println("ID: " + e.getId());
46
47      System.out.println("Fechando conexão...");
48      conexao.close();
49    }
50  }
```

Código Java 2.20: www.k19.com.br/exercicios/k12/02/009/InsereEditora.java



Exercícios Complementares

4 Tente gerar um erro de SQL Injection no cadastro de livros. Dica: utilize entradas com aspas simples.

5 Altere o código da classe **InsereLivro** para eliminar o problema do SQL Injection.

6 Agora tente causar novamente o problema de SQL Injection ao inserir novos livros.



Listando registros

O processo para executar um comando de consulta é bem parecido com o processo de inserir registros. O primeiro passo é definir a consulta em SQL.

```
1 String sql = "SELECT * FROM Editora";
2
3 PreparedStatement comando = conexao.prepareStatement(sql);
4
5 System.out.println("Executando comando...");
6 ResultSet resultado = comando.executeQuery();
```

Código Java 2.22: Realizando uma consulta.

A diferença é que para executar um comando de consulta é necessário utilizar o método `executeQuery()` ao invés do `execute()`. Esse método devolve um objeto da interface `java.sql.ResultSet`, que é responsável por armazenar os resultados da consulta.

Os dados contidos no `ResultSet` podem ser acessados através de métodos, como o `getString`, `getInt`, `getDouble`, etc, de acordo com o tipo do campo. Esses métodos recebem como parâmetro uma string referente ao nome da coluna correspondente.

```
int id = resultado.getInt("id"),
String nome = resultado.getString("nome"),
String email = resultado.getString("email");
```

Código Java 2.23: Recuperando os dados de um ResultSet

O código acima mostra como os campos dos registros recuperados são acessados. O método `next()` é utilizado para percorrer esses registros.

```
resultado.next();

int id1 = resultado.getInt("id"),
String nome1 = resultado.getString("nome"),
String email1 = resultado.getString("email");

resultado.next();

int id2 = resultado.getInt("id"),
String nome2 = resultado.getString("nome"),
String email2 = resultado.getString("email");
```

O próprio método `next()` devolve um valor booleano para indicar se o `ResultSet` conseguiu avançar para o próximo registro. Quando esse método devolver `false` significa que não há mais registros.

```
while(resultado.next()) {
    int id = resultado.getInt("id"),
    String nome = resultado.getString("nome"),
    String email = resultado.getString("email");
}
```



Exercícios de Fixação

10 Insira algumas editoras utilizando a classe `InsereEditora` que você criou anteriormente.

11 Adicione uma nova classe no projeto **JDBC** chamada **ListaEditoras**.

```
package br.com.k19.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;
import java.util.List;

```

```
10 public class ListaEditoras {
11   public static void main(String[] args) throws Exception {
12     String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
13     String usuario = "root";
14     String senha = "root";
15
16     System.out.println("Abrindo conexão...");
17     Connection conexao =
18       DriverManager.getConnection(stringDeConexao, usuario, senha);
19
20     String sql = "SELECT * FROM Editora;";
21
22     PreparedStatement comando = conexao.prepareStatement(sql);
23
24     System.out.println("Executando comando...");
25     ResultSet resultado = comando.executeQuery();
26
27     List<Editora> lista = new ArrayList<Editora>();
28     while (resultado.next()) {
29       Editora e = new Editora();
30       e.setId(resultado.getLong("id"));
31       e.setNome(resultado.getString("nome"));
32       e.setEmail(resultado.getString("email"));
33       lista.add(e);
34     }
35
36     System.out.println("Resultados encontrados: \n");
37     for (Editora editora : lista) {
38       System.out.println("Id: " + editora.getId());
39       System.out.println("Nome: " + editora.getNome());
40       System.out.println("Email: " + editora.getEmail());
41       System.out.println("----------------------------");
42     }
43
44     System.out.println("\nFechando conexão...");
45     conexao.close();
46   }
47 }
```

Código Java 2.26: www.k19.com.br/exercicios/k12/02/011/ListaEditoras.java



Exercícios Complementares

7 Crie uma classe para listar os livros cadastrados na base de dados.



Connection Factory

Você deve ter percebido que em diversos pontos diferentes da nossa aplicação precisamos de uma conexão JDBC. Se a url de conexão for definida em cada um desses pontos, teremos um problema de manutenção. Imagine que o driver do banco seja atualizado ou que o IP do SGBD seja alterado. Teríamos que alterar o código da nossa aplicação em muitos lugares. Mais precisamente, cada ocorrência da url de conexão precisaria ser modificada. A probabilidade de algum ponto não ser corrigido é grande.

Para diminuir o trabalho de manutenção, podemos implementar uma classe responsável pela criação e distribuição de conexões. A url de conexão deve ser definida nessa classe e somente nela. Consequentemente, alterações nas informações contidas na url de conexão afetariam apenas uma

classe da aplicação.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.SQLException;
6
7  public class ConnectionFactory {
8    public static Connection createConnection() throws SQLException {
9      String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
10     String usuario = "root";
11     String senha = "root";
12
13     return DriverManager.getConnection(stringDeConexao, usuario, senha);
14   }
15 }
```

Código Java 2.28: ConnectionFactory.java

Agora, podemos obter uma nova conexão apenas chamando o método `createConnection()`. O resto do sistema não precisa mais conhecer os detalhes sobre a criação das conexões com o banco de dados, diminuindo o acoplamento da aplicação.



Exercícios de Fixação

12 Adicione uma classe chamada **ConnectionFactory** no projeto **JDBC** com o código abaixo.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.SQLException;
6
7  public class ConnectionFactory {
8    public static Connection createConnection() throws SQLException {
9      String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
10     String usuario = "root";
11     String senha = "root";
12
13     return DriverManager.getConnection(stringDeConexao, usuario, senha);
14   }
15 }
```

Código Java 2.29: www.k19.com.br/exercicios/k12/02/012/ConnectionFactory.java

13 Altere as classes **CriaBaseDeDadosLivraria**, **CriaTabelaEditora**, **InsereEditora** e **ListaEditoras** para que elas utilizem a **ConnectionFactory**. Depois, execute-as novamente nessa ordem.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5
6  public class CriaBaseDeDadosLivraria {
7    public static void main(String[] args) throws Exception {
8      System.out.println("Abrindo conexão...");
9      Connection conexao = ConnectionFactory.createConnection();
```

```
10
11      System.out.println("Apagando a base de dados livraria...");
12      String sql = "DROP DATABASE IF EXISTS livraria";
13      PreparedStatement comando = conexao.prepareStatement(sql);
14      comando.execute();
15      comando.close();
16
17      System.out.println("Criando a base de dados livraria...");
18      sql = "CREATE DATABASE livraria";
19      comando = conexao.prepareStatement(sql);
20      comando.execute();
21      comando.close();
22
23      System.out.println("Fechando conexão...");
24      conexao.close();
25    }
26  }
```

Código Java 2.30: www.k19.com.br/exercicios/k12/02/013/CriaBaseDeDadosLivraria.java

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5
6  public class CriaTabelaEditora {
7    public static void main(String[] args) throws Exception {
8      System.out.println("Abrindo conexão...");
9      Connection conexao = ConnectionFactory.createConnection();
10
11     System.out.println("Criando a tabela Editora...");
12     String sql =
13         "CREATE TABLE Editora (" +
14           "id BIGINT NOT NULL AUTO_INCREMENT," +
15           "nome VARCHAR (255) NOT NULL," +
16           "email VARCHAR (255) NOT NULL," +
17           "PRIMARY KEY (id)" +
18         ")" +
19         "ENGINE = InnoDB";
20     PreparedStatement comando = conexao.prepareStatement(sql);
21     comando.execute();
22     comando.close();
23
24     System.out.println("Fechando conexão...");
25     conexao.close();
26   }
27 }
```

Código Java 2.31: www.k19.com.br/exercicios/k12/02/013/CriaTabelaEditora.java

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5  import java.sql.ResultSet;
6  import java.sql.Statement;
7  import java.util.Scanner;
8
9  public class InsereEditora {
10   public static void main(String[] args) throws Exception {
11     System.out.println("Abrindo conexão...");
12     Connection conexao = ConnectionFactory.createConnection();
13
14     Scanner entrada = new Scanner(System.in);
15     Editora e = new Editora();
16
17     System.out.println("Digite o nome da editora: ");
18     e.setNome(entrada.nextLine());
```

```
19
20      System.out.println("Digite o email da editora: ");
21      e.setEmail(entrada.nextLine());
22
23      entrada.close();
24
25      String sql = "INSERT INTO Editora (nome, email) VALUES (?, ?)";
26
27      PreparedStatement comando = conexao.prepareStatement(sql,
28          Statement.RETURN_GENERATED_KEYS);
29      comando.setString(1, e.getNome());
30      comando.setString(2, e.getEmail());
31
32      System.out.println("Executando comando...");
33      comando.execute();
34
35      System.out.println("Recuperando o ID gerado pelo SGBD...");
36      ResultSet generatedKeys = comando.getGeneratedKeys();
37      generatedKeys.next();
38      e.setId(generatedKeys.getLong(1));
39      System.out.println("ID: " + e.getId());
40
41      System.out.println("Fechando conexão...");
42      conexao.close();
43    }
44  }
```

Código Java 2.32: www.k19.com.br/exercicios/k12/02/013/InsereEditora.java

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5  import java.sql.ResultSet;
6  import java.util.ArrayList;
7  import java.util.List;
8
9  public class ListaEditoras {
10   public static void main(String[] args) throws Exception {
11     System.out.println("Abrindo conexão...");
12     Connection conexao = ConnectionFactory.createConnection();
13
14     String sql = "SELECT * FROM Editora;";
15
16     PreparedStatement comando = conexao.prepareStatement(sql);
17
18     System.out.println("Executando comando...");
19     ResultSet resultado = comando.executeQuery();
20
21     List<Editora> lista = new ArrayList<Editora>();
22     while (resultado.next()) {
23       Editora e = new Editora();
24       e.setId(resultado.getLong("id"));
25       e.setNome(resultado.getString("nome"));
26       e.setEmail(resultado.getString("email"));
27       lista.add(e);
28     }
29
30     System.out.println("Resultados encontrados: \n");
31     for (Editora editora : lista) {
32       System.out.println("Id: " + editora.getId());
33       System.out.println("Nome: " + editora.getNome());
34       System.out.println("Email: " + editora.getEmail());
35       System.out.println("----------------------------");
36     }
37
38     System.out.println("\nFechando conexão...");
39     conexao.close();
40   }
```

```
41 }
```

Código Java 2.33: www.k19.com.br/exercicios/k12/02/013/ListaEditoras.java

14 Crie uma classe chamada **AlteraEditora** em um pacote chamado **br.com.k19.jdbc** no projeto **JDBC** com o seguinte conteúdo.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5  import java.util.Scanner;
6
7  public class AlteraEditora {
8    public static void main(String[] args) throws Exception {
9      System.out.println("Abrindo conexão...");
10     Connection conexao = ConnectionFactory.createConnection();
11
12     Scanner entrada = new Scanner(System.in);
13     Editora e = new Editora();
14
15     System.out.println("Digite o id da editora que deseja alterar: ");
16     e.setId(Long.parseLong(entrada.nextLine()));
17
18     System.out.println("Digite o novo nome da editora: ");
19     e.setNome(entrada.nextLine());
20
21     System.out.println("Digite o novo email da editora: ");
22     e.setEmail(entrada.nextLine());
23
24     entrada.close();
25
26     String sql = "UPDATE Editora SET nome = ?, email = ?  WHERE id = ?";
27
28     PreparedStatement comando = conexao.prepareStatement(sql);
29     comando.setString(1, e.getNome());
30     comando.setString(2, e.getEmail());
31     comando.setLong(3, e.getId());
32
33     System.out.println("Executando comando...");
34     comando.execute();
35
36     System.out.println("Fechando conexão...");
37     conexao.close();
38   }
39 }
```

Código Java 2.34: www.k19.com.br/exercicios/k12/02/014/AlteraEditora.java

Execute a classe **AlteraEditora** e verifique se o registro foi alterado com sucesso na tabela Editora.

15 Crie uma classe chamada **RemoveEditora** em um pacote chamado **br.com.k19.jdbc** no projeto **JDBC** com o seguinte conteúdo.

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5  import java.util.Scanner;
6
7  public class RemoveEditora {
8    public static void main(String[] args) throws Exception {
```

```
 9       System.out.println("Abrindo conexão...");
10       Connection conexao = ConnectionFactory.createConnection();
11
12       Scanner entrada = new Scanner(System.in);
13
14       System.out.println("Digite o id da editora que deseja remover: ");
15       Long id = Long.parseLong(entrada.nextLine());
16
17       entrada.close();
18
19       String sql = "DELETE FROM Editora WHERE id = ?";
20
21       PreparedStatement comando = conexao.prepareStatement(sql);
22       comando.setLong(1, id);
23
24       System.out.println("Executando comando...");
25       comando.execute();
26
27       System.out.println("Fechando conexão...");
28       conexao.close();
29     }
30 }
```

Código Java 2.35: www.k19.com.br/exercicios/k12/02/015/RemoveEditora.java

Execute a classe **RemoveEditora** e verifique se o registro foi removido com sucesso na tabela Editora.



Exercícios Complementares

8 Altere as classes **CriaTabelaLivro**, **InsereLivro** e **ListaLivros** para que elas utilizem a fábrica de conexão. Depois, execute-as novamente nessa ordem.

9 Implemente a alteração dos dados dos livros pelo `id`.

10 Implemente a remoção de livros pelo `id`.

JPA 2.1 e Hibernate

CAPÍTULO 3

Múltiplas sintaxes da linguagem SQL

No capítulo anterior, utilizamos a especificação JDBC para fazer uma aplicação Java interagir com os SGDBs. Nessa interação, as consultas são definidas com a linguagem SQL. A sintaxe dessa linguagem é diferente em cada SGDB. Dessa forma, a complexidade do trabalho dos desenvolvedores aumenta. Para resolver esse problema, as consultas deveriam ser definidas através de um mecanismo independente da linguagem SQL.

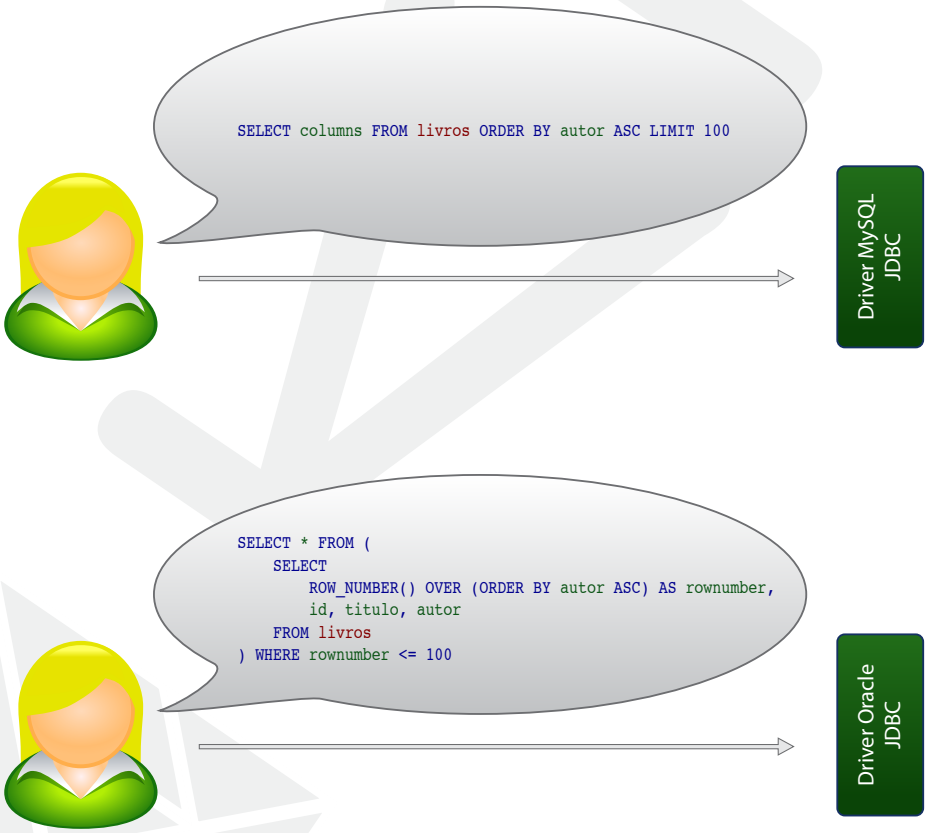


Figura 3.1: Diferentes sintaxes da linguagem SQL

Orientação a Objetos VS Modelo Relacional

Outro problema na comunicação entre uma aplicação Java e um SGDB é o conflito de paradigmas. Os SGDBs são organizados seguindo o modelo relacional. Por outro lado, as aplicações Java

utilizam o modelo orientado a objetos.

A transição de dados entre o modelo relacional e o modelo orientado a objetos não é simples. Para realizar essa transição, é necessário definir um mapeamento entre os conceitos desses dois paradigmas. Por exemplo, classes podem ser mapeadas para tabelas, objetos para registros, atributos para campos e referência entre objetos para chaves estrangeiras.

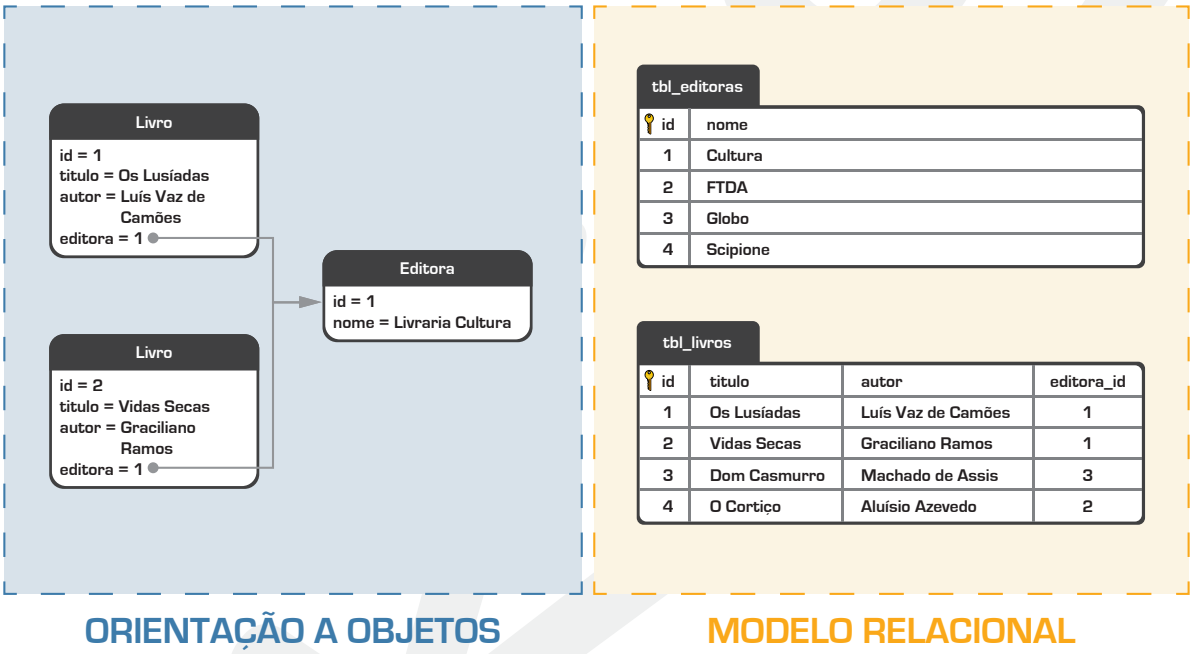


Figura 3.2: Modelo Orientado a Objetos vs Modelo Relacional



Ferramentas ORM

Para facilitar a comunicação entre aplicações Java que seguem o modelo orientado a objetos e os SGDBs que seguem o modelo relacional, podemos utilizar ferramentas que automatizam a transição de dados entre as aplicações e os SGDBs. Essas ferramentas são conhecidas como ferramentas **ORM** (*Object Relational Mapper*).

As ferramentas ORM oferecem mecanismos de consultas independentes da linguagem SQL. Dessa forma, o acoplamento entre as aplicações e os SGDBs diminui drasticamente. A principal ferramenta ORM para Java utilizada no mercado é o Hibernate. Mas, existem outras que possuem o mesmo objetivo.

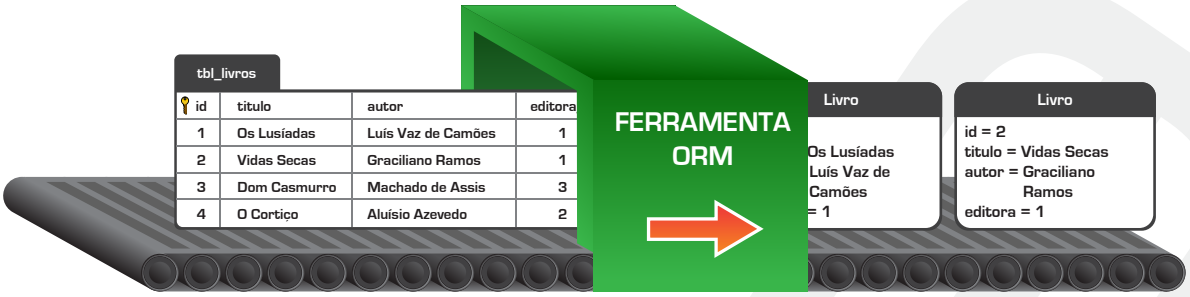


Figura 3.3: Transformação dos dados do Modelo Relacional para o Modelo Orientado a Objetos

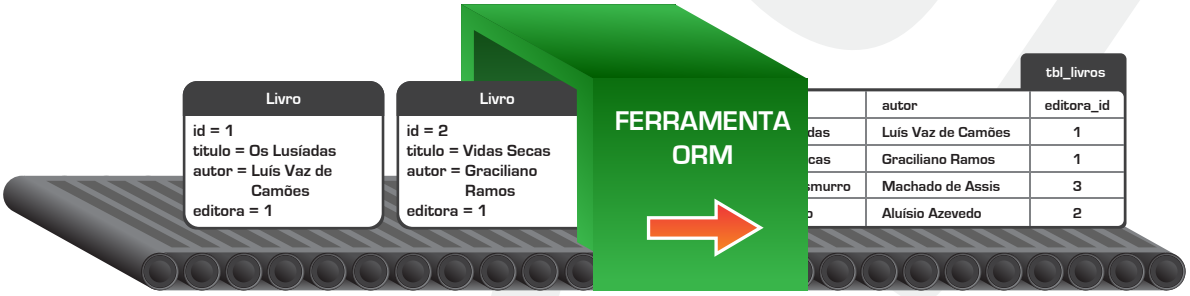


Figura 3.4: Transformação dos dados do Modelo Orientado a Objetos para o Modelo Relacional



O que é JPA e Hibernate?

Após o sucesso do Hibernate, a especificação **JPA** (*Java Persistence API*) foi criada com o objetivo de padronizar as ferramentas ORM para aplicações Java e consequentemente diminuir a complexidade do desenvolvimento. Atualmente, essa especificação está na sua segunda versão.

Ela especifica um conjunto de classes e métodos que as ferramentas ORM devem implementar. Veja que a JPA é apenas uma especificação. Ela não implementa nenhum código. Para isso, utilizamos alguma das diversas implementações da JPA. Neste curso, utilizaremos o Hibernate como implementação de JPA. As outras implementações de JPA mais conhecidas são **EclipseLink** e **OpenJPA**. Optamos por utilizar o Hibernate por ele ser o mais antigo e mais utilizado atualmente.

Caso você queira utilizar outra ferramenta ORM, poderá aplicar os conceitos aqui aprendidos justamente porque eles seguem a mesma especificação. Assim, podemos programar voltado à especificação e substituir uma implementação pela outra, sem precisar reescrever o código da nossa aplicação. Claro que teríamos que alterar alguns arquivos de configuração, mas o código da aplicação permaneceria o mesmo.



Bibliotecas

Antes de começar a utilizar o Hibernate, é necessário baixar do site oficial o **bundle** que inclui os jar's do hibernate e todas as suas dependências. Neste curso, utilizaremos a versão 4.3.10. A url do site oficial do Hibernate é `http://www.hibernate.org/`.



Configuração

Para configurar o Hibernate em uma aplicação, devemos criar um arquivo chamado **persistence.xml**. O conteúdo desse arquivo possuirá informações sobre o banco de dados, como a url de conexão, usuário e senha, além de dados sobre a implementação de JPA que será utilizada.

O arquivo `persistence.xml` deve estar em uma pasta chamada **META-INF**, que deve estar no classpath da aplicação. Veja abaixo um exemplo de configuração para o `persistence.xml`.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
  http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
  version="2.1">
  <persistence-unit name="livraria-pu" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties>
      <property
        name="javax.persistence.schema-generation.database.action"
        value="drop-and-create"/>
      <property
        name="javax.persistence.jdbc.driver"
        value="com.mysql.jdbc.Driver" />
      <property
        name="javax.persistence.jdbc.user"
        value="root" />
      <property
        name="javax.persistence.jdbc.password"
        value="root" />
      <property
        name="javax.persistence.jdbc.url"
        value="jdbc:mysql://localhost:3306/livraria" />
    </properties>
  </persistence-unit>
</persistence>
```

Código XML 3.1: persistence.xml

No arquivo `persistence.xml`, podemos definir diversas unidades de persistência. Para cada unidade de persistência, devemos definir um nome único e um conjunto de propriedades. Normalmente, nesse conjunto aparecem as propriedades necessárias para criação das conexões JDBC.

- javax.persistence.jdbc.driver
- javax.persistence.jdbc.user
- javax.persistence.jdbc.password
- javax.persistence.jdbc.url

A propriedade `javax.persistence.schema-generation.database.action` determina qual estratégia será utilizada na criação das tabelas da base de dados. Há quatro valores aceitos por essa propriedade:

none: As tabelas não serão criadas nem apagadas pelo provedor JPA

create: Na inicialização da unidade de persistência, se as tabelas não existirem na base de dados elas serão criadas pelo provedor JPA. Caso contrário, se as tabelas existirem, nada será criado ou apagado.

drop-and-create: Na inicialização da unidade de persistência, se as tabelas não existirem na base de dados elas serão criadas pelo provedor JPA. Caso contrário, se as tabelas existirem, elas serão apagadas e criadas novamente.

drop: Na inicialização da unidade de persistência, se as tabelas existirem na base de dados elas serão apagadas pelo provedor JPA.



Mapeamento

Um dos principais objetivos das ferramentas ORM é estabelecer o mapeamento entre os conceitos do modelo orientado a objetos e os conceitos do modelo relacional. Esse mapeamento pode ser definido através de XML ou de maneira mais prática com anotações Java.

Considere as entidades `Livro` e `Editora` definidas abaixo.

```
1  @Entity
2  public class Livro {
3
4    @Id
5    @GeneratedValue
6    private Long id;
7
8    private String titulo;
9
10   private Double preco;
11
12   // GETTERS E SETTERS
13 }
```

Código Java 3.1: Livro.java

```
1  @Entity
2  public class Editora {
3
4    @Id
5    @GeneratedValue
6    private Long id;
7
8    private String nome;
9
10   private String email;
11
12   // GETTERS E SETTERS
13 }
```

Código Java 3.2: Editora.java

As classes anotadas com `@Entity` são mapeadas para tabelas no banco de dados. Por padrão, as entidades `Editora` e `Livro` serão mapeadas para tabelas com o mesmo nome.

Cada atributo da entidade `Editora` será mapeado para uma coluna com o mesmo nome na tabela `Editora`. Os tipos dessas colunas são escolhidos automaticamente. Analogamente, cada atri-

buto da entidade `Livro` será mapeado para uma coluna com o mesmo nome na tabela `Livro`. Os tipos dessas colunas também são escolhidos automaticamente.

Devido à utilização da anotação `@Id`, a coluna correspondente ao atributo `id` da entidade `Livro` será definida como chave primária da tabela `Livro`. Analogamente, a coluna correspondente ao atributo `id` da entidade `Editora` será definida como chave primária da tabela `Editora`. Além disso, devido à utilização da anotação `GeneratedValue`, essas chaves primárias serão geradas automaticamente pelo banco de dados.

Anotações

A seguir, veremos as principais anotações Java de mapeamento do JPA. Essas anotações estão no pacote **javax.persistence**.

@Entity É a principal anotação do JPA. Ela deve aparecer antes do nome de uma classe e deve ser definida em todas as classes que terão objetos persistidos no banco de dados.

As classes anotadas com `@Entity` são mapeadas para tabelas. Por convenção, as tabelas possuem os mesmos nomes das classes. Mas, podemos alterar esse comportamento utilizando a anotação `@Table`.

Os atributos declarados em uma classe anotada com `@Entity` são mapeados para colunas na tabela correspondente à classe. Outra vez, por convenção, as colunas possuem os mesmos nomes dos atributos. Novamente, podemos alterar esse padrão utilizando a anotação `@Column`.

```
1 @Entity
2 public class Editora {
3   ...
4 }
```

Código Java 3.3: Editora.java

@Id Utilizada para indicar qual atributo de uma classe anotada com `@Entity` será mapeado para a chave primária da tabela correspondente à classe. Geralmente o atributo anotado com `@Id` é do tipo `Long`.

```
1 @Entity
2 public class Editora {
3
4   @Id
5   private Long id;
6   ...
7 }
```

Código Java 3.4: Editora.java

@GeneratedValue Geralmente vem acompanhado da anotação `@Id`. Serve para indicar que o atributo é gerado pelo banco, no momento em que um novo registro é inserido.

```
1 @Entity
2 public class Editora {
3
4   @Id
5   @GeneratedValue
6   private Long id;
7   ...
8 }
```

Código Java 3.5: Editora.java

@Table Utilizada para alterar o nome padrão da tabela. Ela recebe o parâmetro `name` para indicar qual nome deve ser utilizado na tabela. Veja o exemplo:

```
@Table(name="Publisher")
@Entity
public class Editora {
    // ...
}
```

Código Java 3.6: Editora.java

@Column Utilizado para alterar o nome da coluna que será usado na tabela. Caso você esteja utilizando um banco de dados legado, no qual os nomes das colunas já foram definidos, você pode mudar através dessa anotação. Além disso, podemos estabelecer certas restrições, como determinar se o campo pode ou não ser nulo e a quantidade máxima de caracteres de um texto.

```
@Entity
public class Editora {
    @Column(name="publisher_name", nullable=false, length=30)
    private String nome;

}
```

Código Java 3.7: Editora.java

Podemos utilizar o atributo `columnDefinition` se houver a necessidade de definir o tipo e as restrições da coluna diretamente em SQL.

```
@Entity
public class Editora {
    @Column(name="publisher_name", columnDefinition="VARCHAR(255)")
    private String nome;

}
```

Código Java 3.8: Editora.java

@Transient Serve para indicar que um atributo não deve ser persistido, ou seja, os atributos anotados com `@Transient` não são mapeados para colunas.

```
@Entity
public class Aluno {
  @Id
  @GeneratedValue
  private Long id;

  private Calendar dataDeNascimento;

  @Transient
  private Integer idade;

}
```

Código Java 3.9: Aluno.java

Observe que o atributo `idade` pode ser calculado a partir do atributo `dataDeNascimento`. Portanto, não é necessário armazenar o valor da idade no banco de dados. Nesses casos, podemos utilizar a anotação **@Transient** e indicar quais atributos não devem ser mapeados nas tabelas.



Gerando Tabelas

Uma das vantagens de se utilizar um provedor JPA é que ele é capaz de gerar as tabelas do banco para a nossa aplicação. Ele faz isso de acordo com as anotações colocadas nas classes e as informações presentes no `persistence.xml`.

As tabelas são geradas através de um método da classe `Persistence`, o `createEntityManagerFactory(String entityUnit)`. O parâmetro `entityUnit` permite escolher, pelo nome, uma unidade de persistência definida no `persistence.xml`.

A política de criação das tabelas pode ser alterada no arquivo `persistence.xml`. Podemos, por exemplo, fazer com que o provedor JPA sempre sobrescreva as tabelas existentes. Para isso, a seguinte propriedade:

```
1 <property
2   name="javax.persistence.schema-generation.database.action"
3   value="drop-and-create"/>
```



Exercícios de Fixação

1 Crie um projeto chamado **JPA**. Esse projeto deve ser do tipo **Java Project**.

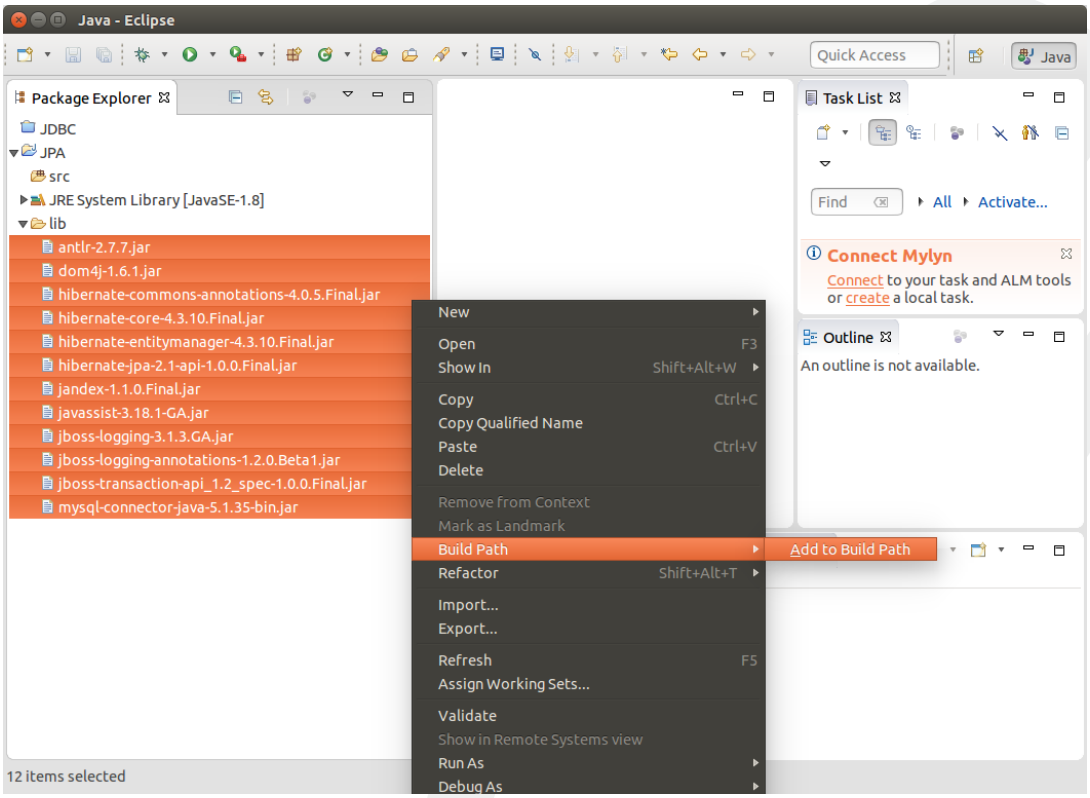
2 Crie uma pasta chamada **lib** no projeto **JPA**. Copie os JARs do Hibernate e do driver JDBC do MySQL para a pasta **lib**. Esses JARs estão nos diretórios:

- K19-Arquivos/hibernate-release-4.3.10.Final/lib/required
- K19-Arquivos/hibernate-release-4.3.10.Final/lib/jpa
- K19-Arquivos/mysql-connector-java-VERSAO



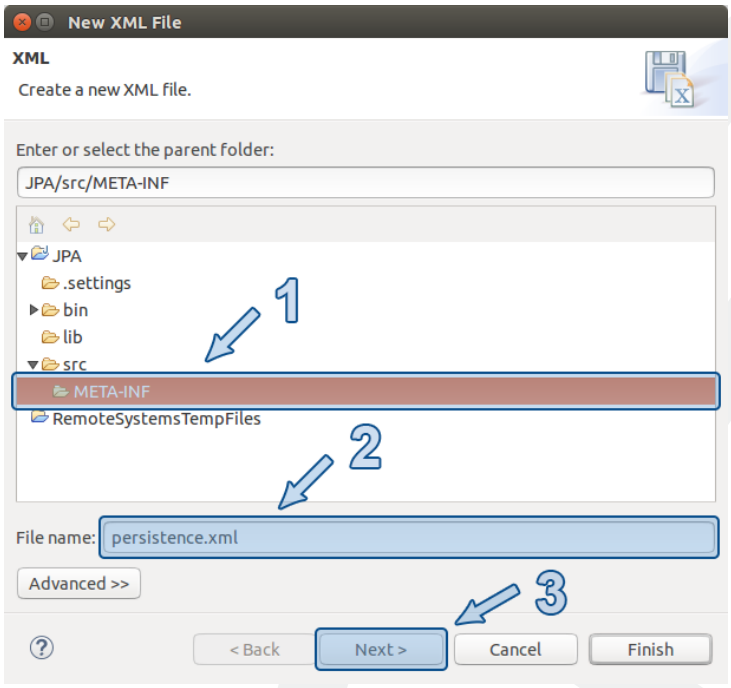
Importante
Você também pode obter os JARs do Hibernate e do driver JDBC do MySQL através do site da K19: www.k19.com.br/arquivos.

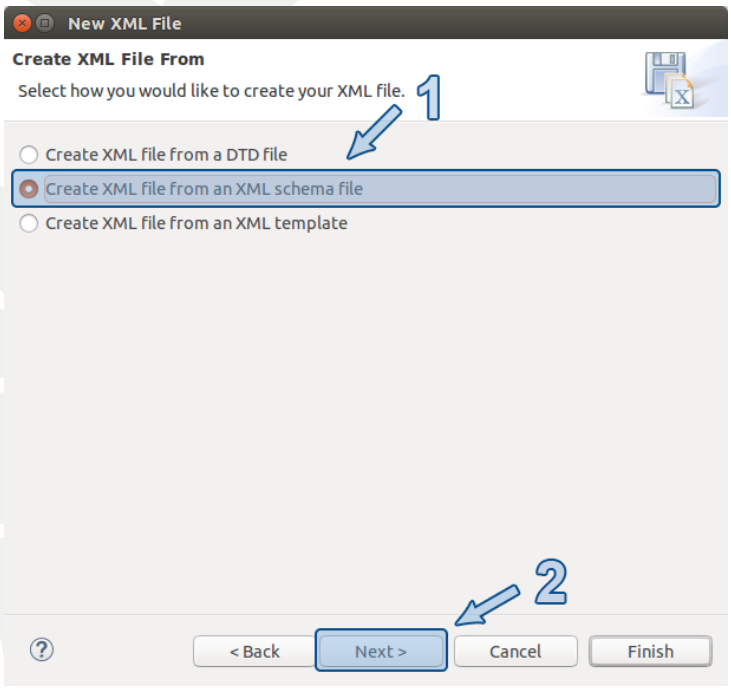
3 Adicione os JARs da pasta **lib** do projeto **JPA** no **build path**.

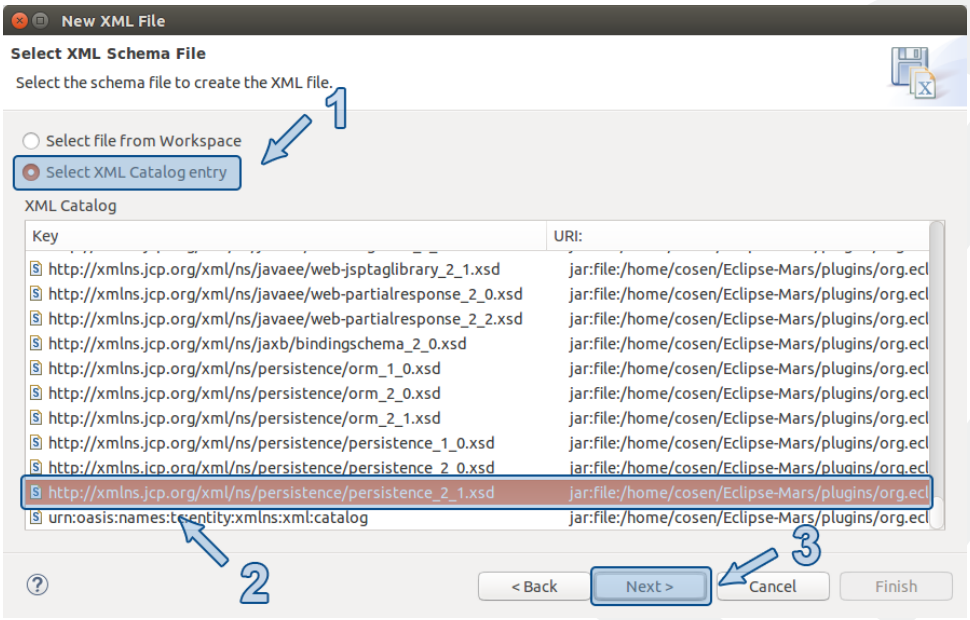


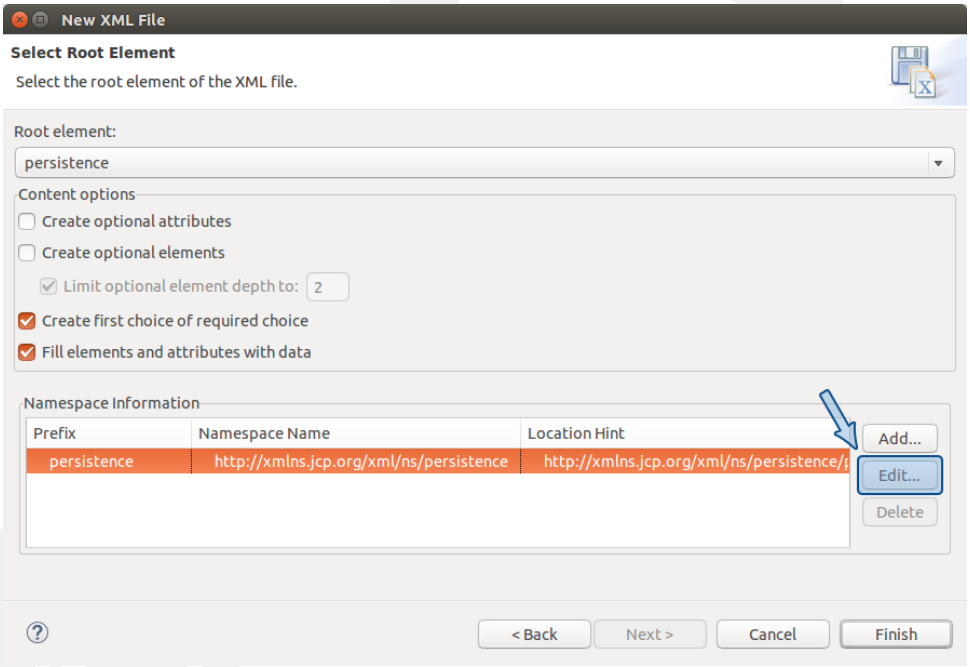
4 Configure uma unidade de persistência no projeto **JPA**. Para isso, crie uma pasta chamada **META-INF** na pasta **src** do projeto **JPA**. Adicione nessa pasta o arquivo **persistence.xml** com o seguinte conteúdo.

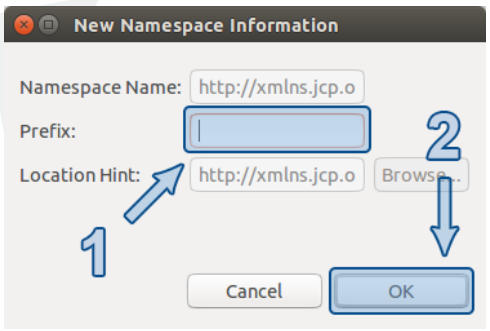
Para criar o arquivo `persistence.xml` no Eclipse, digite "CTRL + 3" e pesquise por "Create a new XML file". Selecione a opção correspondente e siga as imagens abaixo.

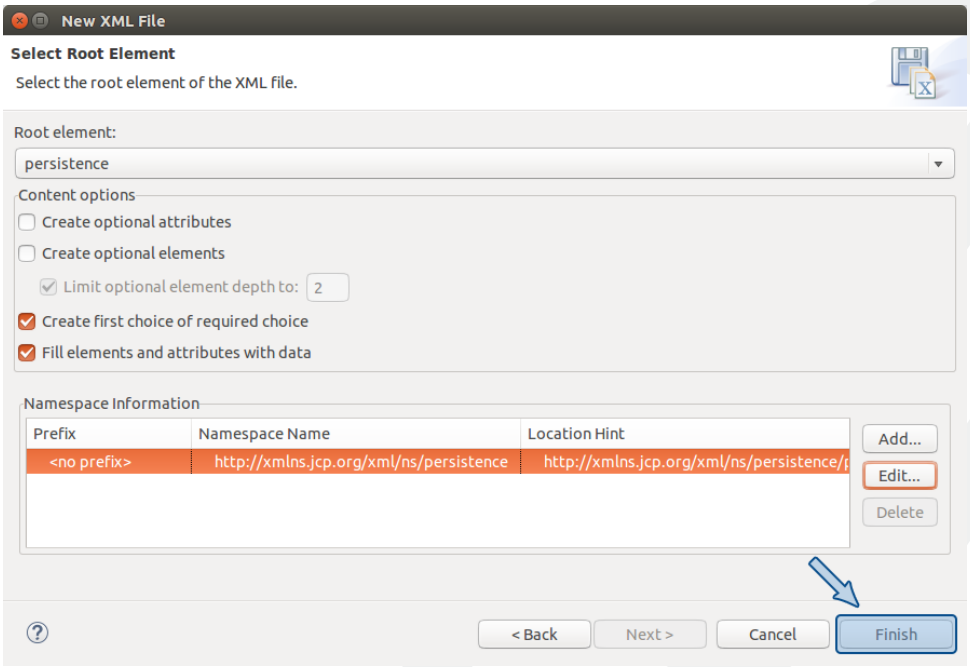

New XML File
XML
Create a new XML file.
Enter or select the parent folder:
JPA/src/META-INF
JPA
.settings
bin
lib
src
META-INF
RemoteSystemsTempFiles
1
2
File name:
persistence.xml
Advanced >>
3
< Back
Next >
Cancel
Finish


New XML File
Create XML File From
Select how you would like to create your XML file.
1
Create XML file from a DTD file
Create XML file from an XML schema file
Create XML file from an XML template
2
< Back
Next >
Cancel
Finish


New XML File
Select XML Schema File
Select the schema file to create the XML file.
Select file from Workspace
Select XML Catalog entry
XML Catalog
Key
URI:
http://xmlns.jcp.org/xml/ns/javaee/web-jsptaglibrary_2_1.xsd
http://xmlns.jcp.org/xml/ns/javaee/web-partialresponse_2_0.xsd
http://xmlns.jcp.org/xml/ns/javaee/web-partialresponse_2_2.xsd
http://xmlns.jcp.org/xml/ns/jaxb/bindingschema_2_0.xsd
http://xmlns.jcp.org/xml/ns/persistence/orm_1_0.xsd
http://xmlns.jcp.org/xml/ns/persistence/orm_2_0.xsd
http://xmlns.jcp.org/xml/ns/persistence/orm_2_1.xsd
http://xmlns.jcp.org/xml/ns/persistence/persistence_1_0.xsd
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_0.xsd
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd
urn:oasis:names:tc:entity:xmlns:xml:catalog
jar:file:/home/cosen/Eclipse-Mars/plugins/org.ecl
1
2
3
< Back
Next >
Cancel
Finish


New XML File
Select Root Element
Select the root element of the XML file.
Root element:
persistence
Content options
Create optional attributes
Create optional elements
Limit optional element depth to: 2
Create first choice of required choice
Fill elements and attributes with data
Namespace Information
Prefix
Namespace Name
Location Hint
persistence
http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/p
Add...
Edit...
Delete
< Back
Next >
Cancel
Finish


New Namespace Information
Namespace Name:
http://xmlns.jcp.o
Prefix:
Location Hint:
http://xmlns.jcp.o
Browse...
1
2
Cancel
OK




```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <persistence
3    version="2.1"
4    xmlns="http://xmlns.jcp.org/xml/ns/persistence"
5    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
6    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
7      http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd ">
8
9    <persistence-unit name="livraria-pu" transaction-type="RESOURCE_LOCAL">
10     <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
11     <exclude-unlisted-classes>false</exclude-unlisted-classes>
12     <properties>
13       <property
14         name="javax.persistence.schema-generation.database.action"
15         value="drop-and-create"/>
16
17       <property
18         name="javax.persistence.jdbc.driver"
19         value="com.mysql.jdbc.Driver" />
20
21       <property
22         name="javax.persistence.jdbc.user"
23         value="root" />
24
25       <property
26         name="javax.persistence.jdbc.password"
27         value="root" />
28
29       <property
30         name="javax.persistence.jdbc.url"
31         value="jdbc:mysql://localhost:3306/livraria" />
32     </properties>
33   </persistence-unit>
34 </persistence>
```

Código XML 3.3: www.k19.com.br/exercicios/k12/03/004/persistence.xml

5 Crie uma classe chamada **Editora** em um pacote chamado **br.com.k19.jpa** e acrescente as anotações necessárias para fazer o mapeamento.

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.Entity;
4  import javax.persistence.GeneratedValue;
5  import javax.persistence.Id;
6
7  @Entity
8  public class Editora {
9
10   @Id
11   @GeneratedValue
12   private Long id;
13
14   private String nome;
15
16   private String email;
17
18   // GETTERS AND SETTERS
19 }
```

Código Java 3.10: www.k19.com.br/exercicios/k12/03/005/Editora.java

6 Apague a tabela **Livro** e depois a tabela **Editora**.

7 Inicialize a unidade de persistência através da classe **Persistence**. Para isso, crie uma classe chamada **Teste** com o seguinte código.

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.EntityManagerFactory;
4  import javax.persistence.Persistence;
5
6  public class Teste {
7    public static void main(String[] args) {
8      EntityManagerFactory factory =
9        Persistence.createEntityManagerFactory("livraria-pu");
10
11     factory.close();
12   }
13 }
```

Código Java 3.11: www.k19.com.br/exercicios/k12/03/007/Teste.java

Execute a classe **Teste** e verifique se a tabela **Editora** foi criada corretamente.



Manipulando entidades

Para manipular as entidades da nossa aplicação, devemos utilizar um `EntityManager` que é obtido através de uma `EntityManagerFactory`.

```
1  EntityManagerFactory factory =
2    Persistence.createEntityManagerFactory("K19");
3
4  EntityManager manager = factory.createEntityManager();
```

Código Java 3.12: Obtendo um EntityManager

Persistindo

Para armazenar as informações de um objeto no banco de dados, o primeiro passo é utilizar o método `persist()` do `EntityManager`.

```
1 Editora novaEditora = new Editora();
2 novaEditora.setNome("K19 - Livros")
3 novaEditora.setEmail("contato@k19.com.br");
4
5 manager.persist(novaEditora);
```

Código Java 3.13: Marcando um objeto para ser persistido

É importante destacar que o método `persist()` apenas marca os objetos que devem ser armazenados no banco de dados. Os objetos serão armazenados após a chamada do método `commit()`, como veremos adiante.

Buscando

Para obter um objeto que contenha informações do banco de dados, podemos utilizar o método `find()` ou o método `getReference()` do `EntityManager`.

```
1 Editora editora1 = manager.find(Editora.class, 1L);
2 Editora editora2 = manager.getReference(Editora.class, 2L);
```

Código Java 3.14: Obtendo objetos com informações do banco de dados

Há uma diferença entre os dois métodos básicos de busca `find()` e `getReference()`. O método `find()` recupera os dados desejados imediatamente. Já o método `getReference()` posterga essa tarefa até a primeira chamada de um método `get` do objeto.

Removendo

Para remover o registro correspondente a um objeto, devemos utilizar o método `remove()` do `EntityManager`.

```
1 Editora editora1 = manager.find(Editora.class, 1L);
2 manager.remove(editora1);
```

Código Java 3.15: Marcando um objeto para ser removido

Atualizando

Para alterar os dados do registro correspondente a um objeto, basta utilizar os próprios métodos setters desse objeto.

```
1 Editora editora = manager.find(Editora.class, 1L);
2 editora.setNome("K19 - Livros e Publicações");
```

Código Java 3.16: Alterando os dados de um registro

Listando

Para obter uma listagem com todos os objetos referentes aos registros de uma tabela, podemos utilizar a linguagem de consulta do JPA, a **JPQL**, que é muito parecida com a linguagem SQL. A principal vantagem do JPQL em relação ao SQL é que a sintaxe do JPQL não depende do SGDB utilizado.

```
Query query = manager.createQuery("SELECT e FROM Editora e");
List<Editora> editoras = query.getResultList();
```

Código Java 3.17: Obtendo uma lista de objetos com informações do banco de dados

Transações

As modificações realizadas nos objetos administrados por um `EntityManager` são mantidas em memória. Em certos momentos, é necessário sincronizar os dados da memória com os dados do banco de dados. Essa sincronização deve ser realizada através de uma transação JPA criada pelo `EntityManager` que administra os objetos que desejamos sincronizar.

Para abrir uma transação, utilizamos o método `begin()`.

```
manager.getTransaction().begin();
```

Código Java 3.18: Abrindo uma transação

Com uma transação aberta, podemos sincronizar os dados da memória com os dados do banco através do método `commit()`.

```
Editora editora = manager.find(Editora.class, 1L);
editora.setNome("K19 - Livros e Publicações");

manager.getTransaction().begin();
manager.getTransaction().commit();
```

Código Java 3.19: Sincronizando com o método `commit()`



Exercícios de Fixação

8 Altere o arquivo **persistence.xml**.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence
  version="2.1"
  xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
    http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd ">

  <persistence-unit name="livraria-pu" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties>
      <property
        name="javax.persistence.schema-generation.database.action"
        value="none"/>

      <property
        name="javax.persistence.jdbc.driver"
        value="com.mysql.jdbc.Driver" />

      <property
        name="javax.persistence.jdbc.user"
        value="root" />
```

```
24
25      <property
26        name="javax.persistence.jdbc.password"
27        value="root" />
28
29      <property
30        name="javax.persistence.jdbc.url"
31        value="jdbc:mysql://localhost:3306/livraria" />
32    </properties>
33  </persistence-unit>
34 </persistence>
```

Código XML 3.4: www.k19.com.br/exercicios/k12/03/008/persistence.xml

9 Crie uma classe chamada **InsereEditoraComJPA** no pacote **br.com.k19.jpa**.

```
1  package br.com.k19.jpa;
2
3  import java.util.Scanner;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7
8  public class InsereEditoraComJPA {
9
10   public static void main(String[] args) {
11     EntityManagerFactory factory =
12       Persistence.createEntityManagerFactory("livraria-pu");
13
14     EntityManager manager = factory.createEntityManager();
15
16     Editora novaEditora = new Editora();
17
18     Scanner entrada = new Scanner(System.in);
19
20     System.out.println("Digite o nome da editora: ");
21     novaEditora.setNome(entrada.nextLine());
22
23     System.out.println("Digite o email da editora: ");
24     novaEditora.setEmail(entrada.nextLine());
25
26     entrada.close();
27
28     manager.persist(novaEditora);
29
30     manager.getTransaction().begin();
31     manager.getTransaction().commit();
32
33     System.out.println("Editora cadastrada com id:" + novaEditora.getId());
34
35     manager.close();
36     factory.close();
37   }
38 }
```

Código Java 3.20: www.k19.com.br/exercicios/k12/03/009/InsereEditoraComJPA.java

Execute a classe **InsereEditoraComJPA** e verifique se o registro foi inserido com sucesso na tabela Editora.

10 Crie uma classe chamada **ListaEditorasComJPA** no pacote **br.com.k19.jpa**.

```
1  package br.com.k19.jpa;
2
```

```
3  import java.util.List;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7  import javax.persistence.Query;
8
9  public class ListaEditorasComJPA {
10
11   public static void main(String[] args) {
12     EntityManagerFactory factory =
13       Persistence.createEntityManagerFactory("livraria-pu");
14
15     EntityManager manager = factory.createEntityManager();
16
17     Query query = manager.createQuery("SELECT e FROM Editora e");
18     List<Editora> editoras = query.getResultList();
19
20     for(Editora e : editoras) {
21       System.out.println("EDITORA: " + e.getNome() + " - " + e.getEmail());
22     }
23
24     manager.close();
25     factory.close();
26   }
27 }
```

Código Java 3.21: www.k19.com.br/exercicios/k12/03/010/ListaEditorasComJPA.java

Execute a classe **ListaEditorasComJPA** para listar as editoras.

11 Crie uma classe chamada **AlteraEditoraComJPA** no pacote **br.com.k19.jpa**.

```
1  package br.com.k19.jpa;
2
3  import java.util.Scanner;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7
8  public class AlteraEditoraComJPA {
9
10   public static void main(String[] args) {
11     EntityManagerFactory factory =
12       Persistence.createEntityManagerFactory("livraria-pu");
13
14     EntityManager manager = factory.createEntityManager();
15
16     Scanner entrada = new Scanner(System.in);
17
18     System.out.println("Digite o id da editora que deseja alterar: ");
19     Long id = Long.ParseLong(entrada.nextLine());
20
21     Editora e = manager.find(Editora.class, id);
22
23     System.out.println("Digite o novo nome da editora: ");
24     e.setNome(entrada.nextLine());
25
26     System.out.println("Digite o novo email da editora: ");
27     e.setEmail(entrada.nextLine());
28
29     entrada.close();
30
31     manager.getTransaction().begin();
32     manager.getTransaction().commit();
33
34     manager.close();
35     factory.close();
```

```
36    }
37 }
```

Código Java 3.22: www.k19.com.br/exercicios/k12/03/011/AlteraEditoraComJPA.java

Execute a classe **AlteraEditoraComJPA** e verifique se o registro foi alterado com sucesso na tabela Editora.

12 Crie uma classe chamada **RemoveEditoraComJPA** no pacote **br.com.k19.jpa**.

```
1  package br.com.k19.jpa;
2
3  import java.util.Scanner;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7
8  public class RemoveEditoraComJPA {
9
10   public static void main(String[] args) {
11     EntityManagerFactory factory =
12       Persistence.createEntityManagerFactory("livraria-pu");
13
14     EntityManager manager = factory.createEntityManager();
15
16     Scanner entrada = new Scanner(System.in);
17
18     System.out.println("Digite o id da editora que deseja remover: ");
19     Long id = Long.ParseLong(entrada.nextLine());
20
21     entrada.close();
22
23     Editora e = manager.find(Editora.class, id);
24
25     manager.remove(e);
26
27     manager.getTransaction().begin();
28     manager.getTransaction().commit();
29
30     manager.close();
31     factory.close();
32   }
33 }
```

Código Java 3.23: www.k19.com.br/exercicios/k12/03/012/RemoveEditoraComJPA.java

Execute a classe **RemoveEditoraComJPA** e verifique se o registro foi removido com sucesso da tabela Editora.



Objetos Embutidos

Considere que é necessário cadastrar os endereços das editoras no sistema da livraria. Esses endereços devem ser formados por país, estado, cidade, logradouro, número, complemento e CEP. Essas informações podem ser definidas na classe `Editora`.

```
1  @Entity
2  public class Editora {
3
4    @Id
5    @GeneratedValue
```

```
6   private Long id;
7
8   private String nome;
9
10  private String email;
11
12  private String pais;
13
14  private String estado;
15
16  private String cidade;
17
18  private String logradouro;
19
20  private String numero;
21
22  private String complemento;
23
24  private String cep;
25
26  // GETTERS E SETTERS
27
28 }
```

Código Java 3.24: Editora.java

Cada atributo acima será mapeado para uma coluna da tabela correspondente a classe `Editora`.

Agora, considere que o sistema da livraria deve permitir também o cadastro dos endereços dos clientes. Seguindo a mesma abordagem, as informações dos endereços dos clientes podem ser definidas na classe `Cliente`.

```
1  @Entity
2  public class Cliente {
3
4    @GeneratedValue
5    private Long id;
6
7    private String nome;
8
9    private String cpf;
10
11   private String pais;
12
13   private String estado;
14
15   private String cidade;
16
17   private String logradouro;
18
19   private String numero;
20
21   private String complemento;
22
23   private String cep;
24
25   // GETTERS E SETTERS
26
27 }
```

Código Java 3.25: Cliente.java

Cada atributo acima será mapeado para uma coluna da tabela correspondente a classe `Cliente`.

Contudo, podemos melhorar a modelagem das classes `Editora` e `Cliente` criando uma terceira

classe para definir os edereços.

```
1  public class Endereco {
2    private String pais;
3
4    private String estado;
5
6    private String cidade;
7
8    private String logradouro;
9
10   private String numero;
11
12   private String complemento;
13
14   private String cep;
15
16   // GETTERS E SETTERS
17 }
```

Código Java 3.26: Endereco.java

```
1  @Entity
2  public class Editora {
3
4    @Id
5    @GeneratedValue
6    private Long id;
7
8    private String nome;
9
10   private String email;
11
12   private Endereco endereco;
13
14   // GETTERS E SETTERS
15 }
```

Código Java 3.27: Editora.java

```
1  @Entity
2  public class Cliente {
3
4    @Id
5    @GeneratedValue
6    private Long id;
7
8    private String nome;
9
10   private String cpf;
11
12   private Endereco endereco;
13
14   // GETTERS E SETTERS
15 }
```

Código Java 3.28: Cliente.java

A criação da classe `Endereco` simplifica a modelagem das classes `Editora` e `Cliente`. Permite um melhor reaproveitamento de código. Para mapear o relacionamento entre essas classes, temos duas possibilidades. A primeira é adicionar a anotação `@Embedded` nos atributos do tipo `Endereco` nas classes `Editora` e `Cliente`.

```
1  @Entity
2  public class Editora {
```

```
3
4   @Id
5   @GeneratedValue
6   private Long id;
7
8   private String nome;
9
10  private String email;
11
12  @Embedded
13  private Endereco endereco;
14
15  // GETTERS E SETTERS
16 }
```

Código Java 3.29: Editora.java

```
1  @Entity
2  public class Cliente {
3
4    @Id
5    @GeneratedValue
6    private Long id;
7
8    private String nome;
9
10   private String cpf;
11
12   @Embedded
13   private Endereco endereco;
14
15   // GETTERS E SETTERS
16 }
```

Código Java 3.30: Cliente.java

A segunda possibilidade é adicionar a anotação `@Embeddable` na classe `Endereco`. Nesse casso, nenhuma anotação precisa ser adicionada nas classes `Editora` e `Cliente`.

```
1  @Embeddable
2  public class Endereco {
3    private String pais;
4
5    private String estado;
6
7    private String cidade;
8
9    private String logradouro;
10
11   private String numero;
12
13   private String complemento;
14
15   private String cep;
16
17   // GETTERS E SETTERS
18 }
```

Código Java 3.31: Endereco.java



Exercícios de Fixação

13 Crie uma classe chamada **Endereco** no pacote **br.com.k19.jpa**.

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.Embeddable;
4
5  @Embeddable
6  public class Endereco {
7    private String pais;
8
9    private String estado;
10
11   private String cidade;
12
13   private String logradouro;
14
15   private String numero;
16
17   private String complemento;
18
19   private String cep;
20
21   // GETTERS E SETTERS
22 }
```

Código Java 3.32: www.k19.com.br/exercicios/k12/03/013/Endereco.java

14 Crie uma classe chamada **Cliente** no pacote **br.com.k19.jpa**.

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.Embedded;
4  import javax.persistence.Entity;
5  import javax.persistence.GeneratedValue;
6  import javax.persistence.Id;
7
8  @Entity
9  public class Cliente {
10
11   @Id
12   @GeneratedValue
13   private Long id;
14
15   private String nome;
16
17   private String cpf;
18
19   @Embedded
20   private Endereco endereco;
21
22   // GETTERS E SETTERS
23 }
```

Código Java 3.33: www.k19.com.br/exercicios/k12/03/014/Cliente.java

15 Altere o arquivo **persistence.xml**.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <persistence
3    version="2.1"
4    xmlns="http://xmlns.jcp.org/xml/ns/persistence"
5    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
6    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
7      http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd ">
```

```
8
9   <persistence-unit name="livraria-pu" transaction-type="RESOURCE_LOCAL">
10    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
11    <exclude-unlisted-classes>false</exclude-unlisted-classes>
12    <properties>
13      <property
14        name="javax.persistence.schema-generation.database.action"
15        value="drop-and-create"/>
16
17      <property
18        name="javax.persistence.jdbc.driver"
19        value="com.mysql.jdbc.Driver" />
20
21      <property
22        name="javax.persistence.jdbc.user"
23        value="root" />
24
25      <property
26        name="javax.persistence.jdbc.password"
27        value="root" />
28
29      <property
30        name="javax.persistence.jdbc.url"
31        value="jdbc:mysql://localhost:3306/livraria" />
32    </properties>
33  </persistence-unit>
34 </persistence>
```

Código XML 3.5: www.k19.com.br/exercicios/k12/03/015/persistence.xml

16 Crie uma classe chamada **InsereClienteComJPA** no pacote **br.com.k19.jpa**.

```
1  package br.com.k19.jpa;
2
3  import java.util.Scanner;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7
8  public class InsereClienteComJPA {
9
10   public static void main(String[] args) {
11     EntityManagerFactory factory =
12       Persistence.createEntityManagerFactory("livraria-pu");
13
14     EntityManager manager = factory.createEntityManager();
15
16     Cliente c = new Cliente();
17
18     Scanner entrada = new Scanner(System.in);
19
20     System.out.println("Digite o nome do cliente: ");
21     c.setNome(entrada.nextLine());
22
23     System.out.println("Digite o cpf do cliente: ");
24     c.setCpf(entrada.nextLine());
25
26     Endereco e = new Endereco();
27
28     c.setEndereco(e);
29
30     System.out.println("Digite o endereço do cliente: ");
31
32     System.out.println("País: ");
33     e.setPais(entrada.nextLine());
34
35     System.out.println("Estado: ");
```

```
36      e.setEstado(entrada.nextLine());
37
38      System.out.println("Cidade: ");
39      e.setCidade(entrada.nextLine());
40
41      System.out.println("Logradouro: ");
42      e.setLogradouro(entrada.nextLine());
43
44      System.out.println("Número: ");
45      e.setNumero(entrada.nextLine());
46
47      System.out.println("Complemento: ");
48      e.setComplemento(entrada.nextLine());
49
50      System.out.println("Cep: ");
51      e.setCep(entrada.nextLine());
52
53      entrada.close();
54
55      manager.persist(c);
56
57      manager.getTransaction().begin();
58      manager.getTransaction().commit();
59
60      System.out.println("Cliente cadastrado com id:" + c.getId());
61
62      manager.close();
63      factory.close();
64    }
65  }
```

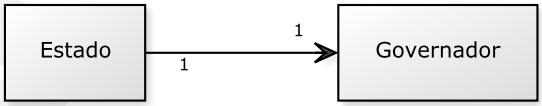
Código Java 3.34: www.k19.com.br/exercicios/k12/03/016/InsereClienteComJPA.java

Execute a classe **InsereClienteComJPA** e verifique se o registro foi inserido com sucesso na tabela Cliente.

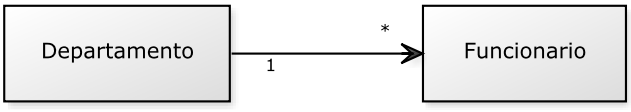
Relacionamentos

Os relacionamentos entre as entidades de um domínio devem ser expressos na modelagem através de vínculos entre classes. De acordo com a JPA, podemos definir quatro tipos de relacionamentos de acordo com a cardinalidade.

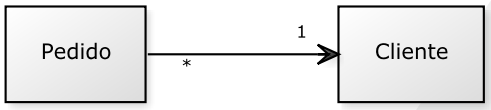
One to One (Um para Um): Por exemplo, um estado é governado por apenas um governador e um governador governa apenas um estado.



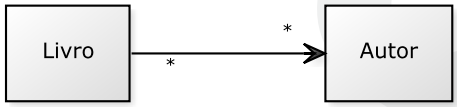
One to Many (Um para Muitos): Por exemplo, um departamento possui muitos funcionários e um funcionário trabalha em apenas em um departamento.



Many to One (Muitos para Um): Por exemplo, um pedido pertence a apenas um cliente e um cliente faz muitos pedidos.



Many to Many (Muitos para Muitos): Por exemplo, um livro possui muitos autores e um autor possui muitos livros.





One to One

Suponha que em nosso domínio existam duas entidades: Estado e Governador. Devemos criar uma classe para cada entidade e aplicar nelas as anotações básicas de mapeamento.

```
1 @Entity
2 class Estado {
3   @Id
4   @GeneratedValue
5   private Long id;
6 }
```

Código Java 3.35: Estado.java

```
1 @Entity
2 class Governador {
3   @Id
4   @GeneratedValue
5   private Long id;
6 }
```

Código Java 3.36: Governador.java

Como existe um relacionamento entre estados e governadores, devemos expressar esse vínculo através de um atributo que pode ser inserido na classe `Estado`.

```
1 @Entity
2 class Estado {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   private Governador governador;
8 }
```

Código Java 3.37: Estado.java

Além disso, devemos informar ao provedor JPA que o relacionamento que existe entre um estado e um governador é do tipo One to One. Fazemos isso aplicando a anotação **@OneToOne** no atributo que expressa o relacionamento.

```
1 @Entity
2 class Estado {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   @OneToOne
8   private Governador governador;
9 }
```

Código Java 3.38: Estado.java

No banco de dados, a tabela referente à classe `Estado` possuirá uma coluna de relacionamento chamada de **join column**. Em geral, essa coluna será definida como uma chave estrangeira associada à tabela referente à classe `Governador`.

Por padrão, o nome da coluna de relacionamento é formado pelo nome do atributo que estabelece o relacionamento, seguido pelo caractere "_" e pelo nome do atributo que define a chave primária da entidade alvo. No exemplo de estados e governadores, a join column teria o nome **governador_id**.



Figura 3.5: Tabelas correspondentes às classes `Estado` e `Governador`

Podemos alterar o nome padrão das join columns aplicando a anotação **@JoinColumn**, conforme apresentado no exemplo abaixo.

```
1 @Entity
2 class Estado {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   @OneToOne
8   @JoinColumn(name="gov_id")
9   private Governador governador;
10 }
```

Código Java 3.39: Estado.java

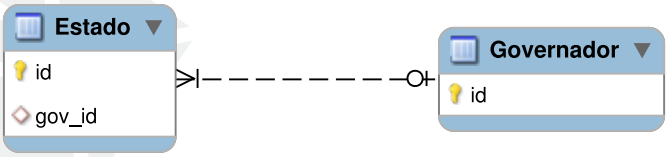


Figura 3.6: Personalizando o nome da coluna de relacionamento



Mais Sobre

Por padrão, em um relacionamento One to One, um objeto da primeira entidade não

precisa estar necessariamente relacionado a um objeto da segunda entidade. Para exigir que cada objeto da primeira entidade esteja relacionado a um objeto da segunda entidade, devemos usar o atributo **optional** da anotação `OneToOne`.

```
1 @Entity
2 class Estado {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   @OneToOne(optional=false)
8   private Governador governador;
9 }
```

Código Java 3.40: Estado.java



Exercícios de Fixação

17 Crie duas entidades no pacote `br.com.k19.jpa`: `Estado` e `Governador`.

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.Entity;
4  import javax.persistence.GeneratedValue;
5  import javax.persistence.Id;
6
7  @Entity
8  public class Governador {
9    @Id
10   @GeneratedValue
11   private Long id;
12
13   private String nome;
14
15   // GETTERS E SETTERS
16 }
```

Código Java 3.41: www.k19.com.br/exercicios/k12/03/017/Governador.java

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.Entity;
4  import javax.persistence.GeneratedValue;
5  import javax.persistence.Id;
6  import javax.persistence.OneToOne;
7
8  @Entity
9  public class Estado {
10   @Id
11   @GeneratedValue
12   private Long id;
13
14   private String nome;
15
16   @OneToOne
17   private Governador governador;
18
19   // GETTERS E SETTERS
```

```
20 }
```

Código Java 3.42: www.k19.com.br/exercicios/k12/03/017/Estado.java

18 Adicione um governador e um estado no banco de dados. Crie uma classe chamada `AdicionaEstadoGovernador` no pacote `br.com.k19.jpa`.

```
1  package br.com.k19.jpa;
2
3  import java.util.Scanner;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7
8  public class AdicionaEstadoGovernador {
9    public static void main(String[] args) {
10     EntityManagerFactory factory =
11       Persistence.createEntityManagerFactory("livraria-pu");
12     EntityManager manager = factory.createEntityManager();
13
14     manager.getTransaction().begin();
15
16     Scanner entrada = new Scanner(System.in);
17
18     Governador g = new Governador();
19     System.out.println("Digite o nome do governador:");
20     g.setNome(entrada.nextLine());
21
22     Estado e = new Estado();
23     System.out.println("Digite o nome do estado:");
24     e.setNome(entrada.nextLine());
25
26     entrada.close();
27
28     e.setGovernador(g);
29
30     manager.persist(g);
31     manager.persist(e);
32
33     System.out.println("Governador cadastrado com id " + g.getId());
34     System.out.println("Estado cadastrado com id " + e.getId());
35
36     manager.getTransaction().commit();
37
38     manager.close();
39     factory.close();
40   }
41 }
```

Código Java 3.43: www.k19.com.br/exercicios/k12/03/018/AdicionaEstadoGovernador.java

Execute a classe `AdicionaEstadoGovernador`. Verifique as tabelas `Estado` e `Governador`.



One to Many

Suponha que em nosso domínio existam as entidades Departamento e Funcionário. Criaríamos duas classes com as anotações básicas de mapeamento.

```
1 @Entity
2 class Departamento {
3   @Id
```

```
4   @GeneratedValue
5   private Long id;
6 }
```

Código Java 3.44: Departamento.java

```
1 @Entity
2 class Funcionario {
3   @Id
4   @GeneratedValue
5   private Long id;
6 }
```

Código Java 3.45: Funcionario.java

Como existe um relacionamento entre departamentos e funcionários, devemos expressar esse vínculo através de um atributo que pode ser inserido na classe `Departamento`. Supondo que um departamento possa ter muitos funcionários, devemos utilizar uma coleção para expressar esse relacionamento.

```
1 @Entity
2 class Departamento {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   private Collection<Funcionario> funcionarios;
8 }
```

Código Java 3.46: Departamento.java

Para informar a cardinalidade do relacionamento entre departamentos e funcionários, devemos utilizar a anotação **@OneToMany** na coleção.

```
1 @Entity
2 class Departamento {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   @OneToMany
8   private Collection<Funcionario> funcionarios;
9 }
```

Código Java 3.47: Departamento.java

No banco de dados, além das duas tabelas correspondentes às classes `Departamento` e `Funcionario`, deve existir uma terceira tabela para relacionar os registros dos departamentos com os registros dos funcionários. Essa terceira tabela é chamada de tabela de relacionamento ou **join table**.

Por padrão, o nome da join table é a concatenação com "_" dos nomes das duas entidades. No exemplo de departamentos e funcionários, o nome do join table seria **Departamento_Funcionario**. Essa tabela possuirá duas colunas vinculadas às entidades que formam o relacionamento. No exemplo, a join table `Departamento_Funcionario` possuirá uma coluna chamada **Departamento_id** e outra chamada **funcionarios_id**.

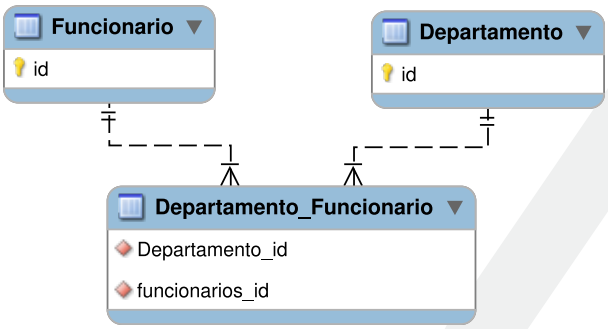


Figura 3.7: Tabelas correspondentes às classes `Departamento` *e* `Funcionario`

Para personalizar os nomes das colunas da join table e da própria join table, podemos aplicar a anotação **@JoinTable** no atributo que define o relacionamento.

```
1  @Entity
2  class Departamento {
3    @Id
4    @GeneratedValue
5    private Long id;
6
7    @OneToMany
8    @JoinTable(name="DEP_FUNC",
9      joinColumns=@JoinColumn(name="DEP_ID"),
10     inverseJoinColumns=@JoinColumn(name="FUNC_ID"))
11   private Collection<Funcionario> funcionarios;
12 }
```

Código Java 3.48: Departamento.java

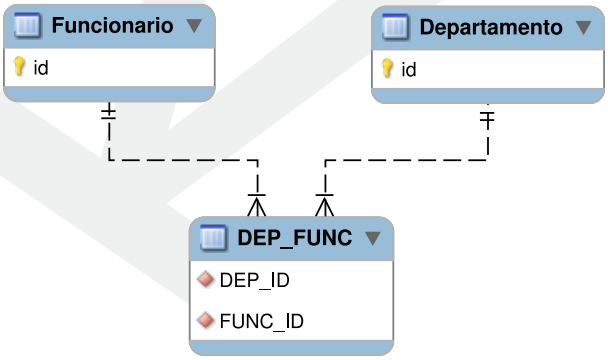


Figura 3.8: Personalizando a tabela de relacionamento



Exercícios de Fixação

19 Crie duas entidades no pacote `br.com.k19.jpa`: `Funcionario` e `Departamento`.

```
1 package br.com.k19.jpa;
2
3 import javax.persistence.Entity;
4 import javax.persistence.GeneratedValue;
5 import javax.persistence.Id;
```

```
6
7  @Entity
8  public class Funcionario {
9    @Id
10   @GeneratedValue
11   private Long id;
12
13   private String nome;
14
15   // GETTERS E SETTERS
16 }
```

Código Java 3.49: www.k19.com.br/exercicios/k12/03/019/Funcionario.java

```
1  package br.com.k19.jpa;
2
3  import java.util.ArrayList;
4  import java.util.Collection;
5
6  import javax.persistence.Entity;
7  import javax.persistence.GeneratedValue;
8  import javax.persistence.Id;
9  import javax.persistence.OneToMany;
10
11 @Entity
12 public class Departamento {
13   @Id
14   @GeneratedValue
15   private Long id;
16
17   private String nome;
18
19   @OneToMany
20   private Collection<Funcionario> funcionarios = new ArrayList<Funcionario>();
21
22   // GETTERS E SETTERS
23 }
```

Código Java 3.50: www.k19.com.br/exercicios/k12/03/019/Departamento.java

20 Adicione um departamento e um funcionário no banco de dados. Crie uma classe chamada `AdicionaDepartamentoFuncionario` no pacote `br.com.k19.jpa`.

```
1  package br.com.k19.jpa;
2
3  import java.util.Scanner;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7
8  public class AdicionaDepartamentoFuncionario {
9    public static void main(String[] args) {
10     EntityManagerFactory factory =
11         Persistence.createEntityManagerFactory("livraria-pu");
12     EntityManager manager = factory.createEntityManager();
13
14     manager.getTransaction().begin();
15
16     Scanner entrada = new Scanner(System.in);
17
18     Funcionario f = new Funcionario();
19     System.out.println("Digite o nome do funcionário:");
20     f.setNome(entrada.nextLine());
21
22     Departamento d = new Departamento();
23     System.out.println("Digite o nome do departamento:");
```

```
24      d.setNome(entrada.nextLine());
25
26      entrada.close();
27
28      d.getFuncionarios().add(f);
29
30      manager.persist(f);
31      manager.persist(d);
32
33      System.out.println("Funcionário cadastrado com id " + f.getId());
34      System.out.println("Departamento cadastrado com id " + d.getId());
35
36      manager.getTransaction().commit();
37
38      manager.close();
39      factory.close();
40    }
41  }
```

Código Java 3.51: www.k19.com.br/exercicios/k12/03/020/AdicionaDepartamentoFuncionario.java

Execute a classe `AdicionaDepartamentoFuncionario`. Verifique as tabelas `Departamento`, `Funcionario` e `Departamento_Funcionario`.



Many to One

Suponha que em nosso domínio existam as entidades Pedido e Cliente. As duas classes que modelariam essas entidades seriam definidas com as anotações principais de mapeamento.

```
1  @Entity
2  class Pedido {
3    @Id
4    @GeneratedValue
5    private Long id;
6  }
```

Código Java 3.52: Pedido.java

```
1  @Entity
2  class Cliente {
3    @Id
4    @GeneratedValue
5    private Long id;
6  }
```

Código Java 3.53: Cliente.java

Como existe um relacionamento entre pedidos e clientes, devemos expressar esse vínculo através de um atributo que pode ser inserido na classe `Pedido`. Supondo que um pedido pertença a um único cliente, devemos utilizar um atributo simples para expressar esse relacionamento.

```
1  @Entity
2  class Pedido {
3    @Id
4    @GeneratedValue
5    private Long id;
6
7    private Cliente cliente;
8  }
```

Código Java 3.54: Pedido.java

Para informar a cardinalidade do relacionamento entre pedidos e clientes, devemos utilizar a anotação **@ManyToOne** no atributo.

```
1 @Entity
2 class Pedido {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   @ManyToOne
8   private Cliente cliente;
9 }
```

Código Java 3.55: Pedido.java

No banco de dados, a tabela referente à classe `Pedido` possuirá uma **join column** vinculada à tabela da classe `Cliente`. Por padrão, o nome da join column é formado pelo nome da entidade alvo do relacionamento, seguido pelo caractere "_" e pelo nome do atributo que define a chave primária da entidade alvo.



Figura 3.9: Tabelas correspondentes às classes `Pedido` *e* `Cliente`

No exemplo de pedidos e clientes, o nome da join column seria **cliente_id**. Podemos alterar o nome padrão das join columns aplicando a anotação **@JoinColumn**.

```
1  @Entity
2  class Pedido {
3    @Id
4    @GeneratedValue
5    private Long id;
6
7    @ManyToOne
8    @JoinColumn(name="cli_id")
9    private Cliente cliente;
10 }
```

Código Java 3.56: Pedido.java

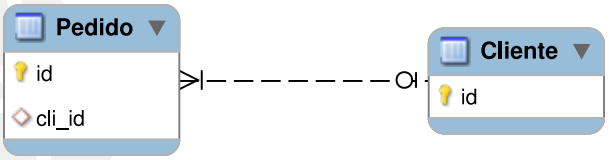


Figura 3.10: Personalizando a tabela `Pedido`



Mais Sobre

Por padrão, em um relacionamento Many to One, um objeto da primeira entidade não precisa estar necessariamente relacionado a um objeto da segunda entidade. Para exi-

gir que cada objeto da primeira entidade esteja relacionado a um objeto da segunda entidade, devemos usar o atributo **optional** da anotação `ManyToOne`.

```
1 @Entity
2 class Pedido {
3   @Id
4   @GeneratedValue
5   private Long id;
6
7   @ManyToOne(optional=false)
8   private Cliente cliente;
9 }
```

Código Java 3.57: Pedido.java



Exercícios de Fixação

21 Crie duas classes no pacote `br.com.k19.jpa`: `Pedido` e `Consumidor`.

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.Entity;
4  import javax.persistence.GeneratedValue;
5  import javax.persistence.Id;
6
7  @Entity
8  public class Consumidor {
9    @Id
10   @GeneratedValue
11   private Long id;
12
13   private String nome;
14
15   // GETTERS E SETTERS
16 }
```

Código Java 3.58: www.k19.com.br/exercicios/k12/03/021/Consumidor.java

```
1  package br.com.k19.jpa;
2
3  import java.util.Calendar;
4
5  import javax.persistence.Entity;
6  import javax.persistence.GeneratedValue;
7  import javax.persistence.Id;
8  import javax.persistence.ManyToOne;
9  import javax.persistence.Temporal;
10 import javax.persistence.TemporalType;
11
12 @Entity
13 public class Pedido {
14   @Id
15   @GeneratedValue
16   private Long id;
17
18   @Temporal(TemporalType.DATE)
19   private Calendar data;
20
21   @ManyToOne
```

```
22    private Consumidor consumidor;
23
24    // GETTERS E SETTERS
25  }
```

Código Java 3.59: www.k19.com.br/exercicios/k12/03/021/Pedido.java

22 Adicione um cliente e um departamento no banco de dados. Crie uma classe chamada `AdicionaPedidoConsumidor` no pacote `br.com.k19.jpa`.

```
1  package br.com.k19.jpa;
2
3  import java.util.Calendar;
4  import java.util.Scanner;
5
6  import javax.persistence.EntityManager;
7  import javax.persistence.EntityManagerFactory;
8  import javax.persistence.Persistence;
9
10 public class AdicionaPedidoConsumidor {
11   public static void main(String[] args) {
12     EntityManagerFactory factory = Persistence
13         .createEntityManagerFactory("livraria-pu");
14     EntityManager manager = factory.createEntityManager();
15
16     manager.getTransaction().begin();
17
18     Scanner entrada = new Scanner(System.in);
19
20     Consumidor c = new Consumidor();
21     System.out.println("Digite o nome do consumidor");
22     c.setNome(entrada.nextLine());
23
24     Pedido p = new Pedido();
25     p.setData(Calendar.getInstance());
26
27     p.setConsumidor(c);
28
29     entrada.close();
30
31     manager.persist(c);
32     manager.persist(p);
33
34     System.out.println("Consumidor cadastrado com id " + c.getId());
35     System.out.println("Pedido cadastrado com id " + p.getId());
36
37     manager.getTransaction().commit();
38
39     manager.close();
40     factory.close();
41   }
42 }
```

Código Java 3.60: www.k19.com.br/exercicios/k12/03/022/AdicionaPedidoConsumidor.java

Execute a classe `AdicionaPedidoConsumidor`. Verifique as tabelas `Consumidor` e `Pedido`.



Many to Many

Suponha que em nosso domínio existam as entidades Livro e Autor. As classes com as anotações básicas de mapeamento seriam mais ou menos assim:

```
@Entity
class Livro {
  @Id
  @GeneratedValue
  private Long id;
}
```

Código Java 3.61: Livro.java

```
@Entity
class Autor {
  @Id
  @GeneratedValue
  private Long id;
}
```

Código Java 3.62: Autor.java

Como existe um relacionamento entre livros e autores, devemos expressar esse vínculo através de um atributo que pode ser inserido na classe `Livro`. Supondo que um livro possa ser escrito por muitos autores, devemos utilizar uma coleção para expressar esse relacionamento.

```
@Entity
class Livro {
  @Id
  @GeneratedValue
  private Long id;

  private Collection<Autor> autores;
}
```

Código Java 3.63: Livro.java

Para informar a cardinalidade do relacionamento entre livros e autores, devemos utilizar a anotação **@ManyToMany** na coleção.

```
@Entity
class Livro {
  @Id
  @GeneratedValue
  private Long id;

  @ManyToMany
  private Collection<Autor> autores;
}
```

Código Java 3.64: Livro.java

No banco de dados, além das duas tabelas correspondentes às classes `Livro` e `Autor`, uma join table é criada para relacionar os registros dos livros com os registros dos autores. Por padrão, o nome da join table é a concatenação com "_" dos nomes das duas entidades. No exemplo de livros e autores, o nome do join table seria **Livro_Autor**. Essa tabela possuirá duas colunas vinculadas às entidades que formam o relacionamento. No exemplo, a join table `Livro_Autor` possuirá uma coluna chamada **Livro_id** e outra chamada **autores_id**.

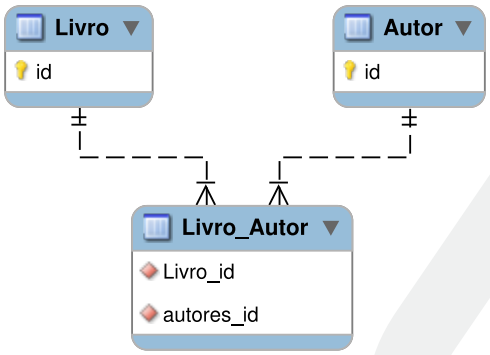


Figura 3.11: Tabelas correspondentes às classes `Livro` *e* `Autor`

Para personalizar o nome join table e os nomes de suas colunas, podemos aplicar a anotação **@JoinTable** no atributo que define o relacionamento.

```
1  @Entity
2  class Livro {
3    @Id
4    @GeneratedValue
5    private Long id;
6
7    @ManyToMany
8    @JoinTable(name="Liv_Aut",
9      joinColumns=@JoinColumn(name="Liv_ID"),
10     inverseJoinColumns=@JoinColumn(name="Aut_ID"))
11   private Collection<Autor> autores;
12 }
```

Código Java 3.65: Livro.java

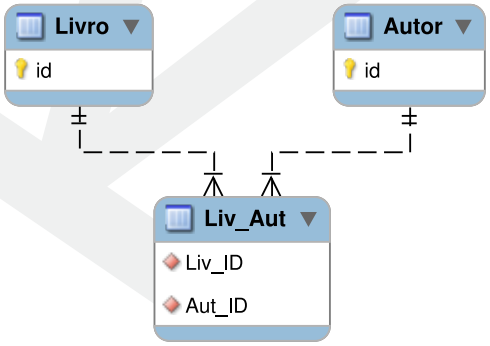


Figura 3.12: Personalizando a tabela de relacionamento



Exercícios de Fixação

23 Crie duas entidades no pacote `br.com.k19.jpa`: `Livro` e `Autor`.

```
1 package br.com.k19.jpa;
2
3 import javax.persistence.Entity;
4 import javax.persistence.GeneratedValue;
5 import javax.persistence.Id;
```

```
6
7  @Entity
8  public class Autor {
9    @Id
10   @GeneratedValue
11   private Long id;
12
13   private String nome;
14
15   // GETTERS E SETTERS
16 }
```

Código Java 3.66: www.k19.com.br/exercicios/k12/03/023/Autor.java

```
1  package br.com.k19.jpa;
2
3  import java.util.ArrayList;
4  import java.util.Collection;
5
6  import javax.persistence.Entity;
7  import javax.persistence.GeneratedValue;
8  import javax.persistence.Id;
9  import javax.persistence.ManyToMany;
10
11 @Entity
12 public class Livro {
13   @Id
14   @GeneratedValue
15   private Long id;
16
17   private String nome;
18
19   @ManyToMany
20   private Collection<Autor> autores = new ArrayList<Autor>();
21
22   // GETTERS E SETTERS
23 }
```

Código Java 3.67: www.k19.com.br/exercicios/k12/03/023/Livro.java

24 Adicione um livro e um autor no banco de dados. Crie uma classe chamada `AdicionaLivroAutor` no pacote `br.com.k19.jpa`.

```
1  package br.com.k19.jpa;
2
3  import java.util.Scanner;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7
8  public class AdicionaLivroAutor {
9    public static void main(String[] args) {
10     EntityManagerFactory factory =
11       Persistence.createEntityManagerFactory("livraria-pu");
12     EntityManager manager = factory.createEntityManager();
13
14     manager.getTransaction().begin();
15
16     Scanner entrada = new Scanner(System.in);
17
18     Autor a = new Autor();
19     System.out.println("Digite o nome do autor:");
20     a.setNome(entrada.nextLine());
21
22     Livro l = new Livro();
23     System.out.println("Digite o nome do livro:");
```

```
24      l.setNome(entrada.nextLine());
25
26      l.getAutores().add(a);
27
28      entrada.close();
29
30      manager.persist(a);
31      manager.persist(l);
32
33      System.out.println("Autor cadastrado com id " + a.getId());
34      System.out.println("Livro cadastrado com id " + l.getId());
35
36      manager.getTransaction().commit();
37
38      manager.close();
39      factory.close();
40    }
41  }
```

Código Java 3.68: www.k19.com.br/exercicios/k12/03/024/AdicionaLivroAutor.java

Execute a classe `AdicionaLivroAutor`. Verifique as tabelas `Livro`, `Autor` e `Livro_Autor`.



Relacionamentos Bidirecionais

Quando expressamos um relacionamento colocando um atributo em uma das entidades, podemos acessar a outra entidade a partir da primeira. Por exemplo, considere o relacionamento entre governadores e estados.

```
1  @Entity
2  class Estado {
3    @Id
4    @GeneratedValue
5    private Long id;
6
7    @OneToOne
8    private Governador governador;
9
10   // GETTERS E SETTERS
11 }
```

Código Java 3.69: Estado.java

Como o relacionamento está definido na classe `Estado`, podemos acessar o governador a partir de um estado.

```
1  Estado e = manager.find(Estado.class, 1L);
2  Governador g = e.getGovernador();
```

Código Java 3.70: Acessando o governador a partir de um estado

Também podemos expressar o relacionamento na classe `Governador`. Dessa forma, poderíamos acessar um estado a partir de um governador.

```
1  @Entity
2  class Governador {
3    @Id
4    @GeneratedValue
5    private Long id;
6
```

```
7   @OneToOne
8   private Estado estado;
9
10  // GETTERS E SETTERS
11 }
```

Código Java 3.71: Governador.java

```
1 Governador g = manager.find(Governador.class, 1L);
2 Estado e = g.getEstado();
```

Código Java 3.72: Acessando um estado a partir de um governador

A figura abaixo ilustra as tabelas `Estado` e `Governador` no banco de dados, assim como as join columns correspondentes aos relacionamentos.

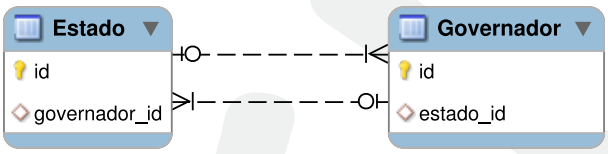


Figura 3.13: Tabelas `Estado` e `Governador` no banco de dados

Note que foram criadas duas colunas de relacionamentos. A primeira na tabela `Estado` com o nome `governador_id` e a segunda na tabela `Governador` com o nome `estado_id`. Nesse caso, o provedor JPA está considerando dois relacionamentos unidirecionais distintos entre essas entidades.

No entanto, de acordo com o modelo relacional, a relação entre estados e governadores deveria ser expressa com apenas uma coluna de relacionamento. Ou seja, o relacionamento entre governadores e estados deveria ser bidirecional. Assim, devemos indicar em uma das classes que esse relacionamento bidirecional é a junção de dois relacionamentos unidirecionais. Para isso, devemos adicionar o atributo **mappedBy** na anotação `@OneToOne` em uma das classes. O valor do `mappedBy` deve ser o nome do atributo que expressa o mesmo relacionamento na outra entidade.

```
1  @Entity
2  class Governador {
3    @Id
4    @GeneratedValue
5    private Long id;
6
7    @OneToOne(mappedBy="governador")
8    private Estado estado;
9
10   // GETTERS E SETTERS
11 }
```

Código Java 3.73: Governador.java

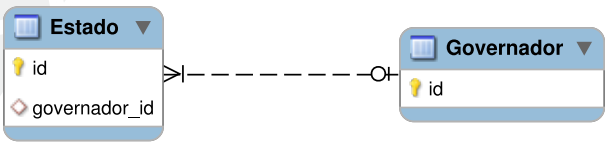


Figura 3.14: Tabelas `Estado` e `Governador` no banco de dados



Exercícios de Fixação

25 Considere um sistema de cobrança de ligações telefônicas. Nesse sistema, temos uma entidade chamada Ligação e uma entidade chamada Fatura. Cada ligação está associada a uma única fatura, enquanto que uma fatura está associada a múltiplas ligações. Implemente classes para modelar essas duas entidades no pacote `br.com.k19.jpa`.

```
1  package br.com.k19.jpa;
2
3  import javax.persistence.Entity;
4  import javax.persistence.GeneratedValue;
5  import javax.persistence.Id;
6  import javax.persistence.ManyToOne;
7
8  @Entity
9  public class Ligacao {
10   @Id
11   @GeneratedValue
12   private Long id;
13
14   @ManyToOne
15   private Fatura fatura;
16
17   private Integer duracao;
18
19   // GETTERS E SETTERS
20 }
```

Código Java 3.74: www.k19.com.br/exercicios/k12/03/025/Ligacao.java

```
1  package br.com.k19.jpa;
2
3  import java.util.ArrayList;
4  import java.util.Calendar;
5  import java.util.Collection;
6
7  import javax.persistence.Entity;
8  import javax.persistence.GeneratedValue;
9  import javax.persistence.Id;
10 import javax.persistence.OneToMany;
11 import javax.persistence.Temporal;
12 import javax.persistence.TemporalType;
13
14 @Entity
15 public class Fatura {
16   @Id @GeneratedValue
17   private Long id;
18
19   @OneToMany
20   private Collection<Ligacao> ligacoes = new ArrayList<Ligacao>();
21
22   @Temporal(TemporalType.DATE)
23   private Calendar vencimento;
24
25   // GETTERS E SETTERS
26 }
```

Código Java 3.75: www.k19.com.br/exercicios/k12/03/025/Fatura.java

26 Faça um teste para adicionar algumas ligações e uma fatura. Adicione no pacote `br.com.k19.jpa`

uma classe chamada AdicionaFaturaLigacao.

```
package br.com.k19.jpa;

import java.util.GregorianCalendar;
import java.util.Scanner;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class AdicionaFaturaLigacao {
  public static void main(String[] args) {
    EntityManagerFactory factory =
      Persistence.createEntityManagerFactory("livraria-pu");
    EntityManager manager = factory.createEntityManager();

    manager.getTransaction().begin();

    Scanner entrada = new Scanner(System.in);

    Ligacao ligacao1 = new Ligacao();
    System.out.println("Digite o tempo de duração da ligação:");
    ligacao1.setDuracao(Integer.parseInt(entrada.nextLine()));

    Ligacao ligacao2 = new Ligacao();
    System.out.println("Digite o tempo de duração da ligação:");
    ligacao2.setDuracao(Integer.parseInt(entrada.nextLine()));

    Fatura fatura = new Fatura();
    fatura.setVencimento(new GregorianCalendar(2012, 11, 20));

    fatura.getLigacoes().add(ligacao1);
    fatura.getLigacoes().add(ligacao2);

    ligacao1.setFatura(fatura);
    ligacao2.setFatura(fatura);

    entrada.close();

    manager.persist(fatura);
    manager.persist(ligacao1);
    manager.persist(ligacao2);

    System.out.println("Fatura cadastrada com id " + fatura.getId());
    System.out.println("Ligação cadastrada com id " + ligacao1.getId());
    System.out.println("Ligação cadastrada com id " + ligacao2.getId());

    manager.getTransaction().commit();

    manager.close();
    factory.close();
  }
}
```

Código Java 3.76: www.k19.com.br/exercicios/k12/03/026/AdicionaFaturaLigacao.java

Execute a classe AdicionaFaturaLigacao. Verifique as tabelas. Observe que a tabela Ligacao possui uma coluna de relacionamento chamada fatura_id e a tabela Fatura_Ligacao vincula os registros das tabelas Ligacao e Fatura.

27 Altere o código da classe Fatura de forma a criar um relacionamento bidirecional entre as faturas e as ligações.

```
package br.com.k19.jpa;
```

```
2
3  import java.util.ArrayList;
4  import java.util.Calendar;
5  import java.util.Collection;
6
7  import javax.persistence.Entity;
8  import javax.persistence.GeneratedValue;
9  import javax.persistence.Id;
10 import javax.persistence.OneToMany;
11 import javax.persistence.Temporal;
12 import javax.persistence.TemporalType;
13
14 @Entity
15 public class Fatura {
16   @Id
17   @GeneratedValue
18   private Long id;
19
20   @OneToMany(mappedBy="fatura")
21   private Collection<Ligacao> ligacoes = new ArrayList<Ligacao>();
22
23   @Temporal(TemporalType.DATE)
24   private Calendar vencimento;
25
26   // GETTERS E SETTERS
27 }
```

Código Java 3.77: www.k19.com.br/exercicios/k12/03/027/Fatura.java

28 Execute a classe `AdicionaFaturaLigacao` para adicionar uma fatura e algumas ligações. Verifique as tabelas criadas. Note que foram criadas apenas duas tabelas: `Fatura` e `Ligacao`.



Consultas

Os provedores JPA permitem que as aplicações criem consultas utilizando a linguagem **JPQL**. Essas consultas são traduzidas para a linguagem SQL do banco de dados que estiver sendo utilizado. Dessa forma, podemos definir uma consulta uma única vez em JPQL é utilizá-la em vários bancos de dados diferentes.

Recuperando a quantidade de registros

```
1 String jpql = "SELECT COUNT(x) FROM Editora x";
2
3 TypedQuery<Long> consulta = manager.createQuery(jpql, Long.class);
4
5 Long resultado = consulta.getSingleResult();
```

Código Java 3.78: Quantidade de editoras

```
1 String jpql = "SELECT COUNT(x) FROM Livro x WHERE x.preco >= 100";
2
3 TypedQuery<Long> consulta = manager.createQuery(jpql, Long.class);
4
5 Long resultado = consulta.getSingleResult();
```

Código Java 3.79: Quantidade de livros com preço maior ou igual a 100

Somando valores

```
String jpql = "SELECT SUM(x.preco) FROM Livro x";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.80: Soma dos preços dos livros

```
String jpql = "SELECT SUM(x.preco) FROM Livro x WHERE x.preco >= 100";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.81: Soma dos preços dos livros com preço maior ou igual a 100

Recuperando valores mínimos ou máximos

```
String jpql = "SELECT MIN(x.preco) FROM Livro x";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.82: O preço do livro mais barato

```
String jpql = "SELECT MIN(x.preco) FROM Livro x WHERE x.preco >= 100";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.83: O menor preço de um livro que seja maior ou igual a 100

```
String jpql = "SELECT MAX(x.preco) FROM Livro x";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.84: O preço do livro mais caro

```
String jpql = "SELECT MAX(x.preco) FROM Livro x WHERE x.preco <= 100";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.85: O maior preço de um livro que seja menor ou igual a 100

Recuperando médias

```
String jpql = "SELECT AVG(x.preco) FROM Livro x";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.86: O preço médio dos livros

```
String jpql = "SELECT AVG(x.preco) FROM Livro x WHERE x.preco >= 100";

TypedQuery<Double> consulta = manager.createQuery(jpql, Double.class);

Double resultado = consulta.getSingleResult();
```

Código Java 3.87: O preço médio dos livros com preço maior ou igual a 100

Recuperando listas

```
String jpql = "SELECT x FROM Livro x";

TypedQuery<Livro> consulta = manager.createQuery(jpql, Livro.class);

List<Livro> resultado = consulta.getResultList();
```

Código Java 3.88: Lista de livros

```
String jpql = "SELECT x FROM Livro x WHERE x.preco >= 100";

TypedQuery<Livro> consulta = manager.createQuery(jpql, Livro.class);

List<Livro> resultado = consulta.getResultList();
```

Código Java 3.89: Lista do livros com preço maior ou igual a 100

```
String jpql = "SELECT x.nome FROM Livro x";

TypedQuery<String> consulta = manager.createQuery(jpql, String.class);

List<String> resultado = consulta.getResultList();
```

Código Java 3.90: Lista dos nomes dos livros

```
String jpql = "SELECT x.nome FROM Livro x WHERE x.preco >= 100";

TypedQuery<String> consulta = manager.createQuery(jpql, String.class);

List<String> resultado = consulta.getResultList();
```

Código Java 3.91: Lista dos nomes dos livros com preço maior ou igual a 100

Ordenando resultados

```
String jpql = "SELECT x FROM Livro x ORDER BY x.nome";

TypedQuery<Livro> consulta = manager.createQuery(jpql, Livro.class);

List<Livro> resultado = consulta.getResultList();
```

Código Java 3.92: Lista dos livros ordenados pelo nome

```
String jpql = "SELECT x.nome FROM Livro x ORDER BY x.nome";

TypedQuery<String> consulta = manager.createQuery(jpql, String.class);

List<String> resultado = consulta.getResultList();
```

Código Java 3.93: Lista ordenada dos nomes dos livros

Limitando resultados

```
String jpql = "SELECT x FROM Livro x ORDER BY x.nome";

TypedQuery<Livro> consulta = manager.createQuery(jpql, Livro.class);

consulta.maxResults(10);

List<Livro> resultado = consulta.getResultList();
```

Código Java 3.94: Lista dos 10 primeiros livros ordenados pelo nome

```
String jpql = "SELECT x FROM Livro x ORDER BY x.nome";

TypedQuery<Livro> consulta = manager.createQuery(jpql, Livro.class);

consulta.setFirstResult(2);

List<Livro> resultado = consulta.getResultList();
```

Código Java 3.95: Lista dos livros ordenados pelo nome a partir do terceiro

Utilizando parâmetros

```
String jpql = "SELECT x FROM Livro x WHERE x.preco >= :preco";

TypedQuery<Livro> consulta = manager.createQuery(jpql, Livro.class);

consulta.setParameter("preco", 100.5);

List<Livro> resultado = consulta.getResultList();
```

Código Java 3.96: Lista dos livros com preço maior ou igual a 100.5



Exercícios de Fixação

29 Crie uma entidade no pacote `br.com.k19.jpa` chamada `Revista`.

```
package br.com.k19.jpa;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;

@Entity
public class Revista {
  @Id
  @GeneratedValue
  private Long id;

  private String nome;

  private Double preco;

  // GETTERS E SETTERS
}
```

Código Java 3.97: www.k19.com.br/exercicios/k12/03/028/Revista.java

30 Adicione diversas revistas. Crie uma classe chamada `AdicionaRevista` no pacote `br.com.k19.jpa`.

```
package br.com.k19.jpa;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class AdicionaRevista {
  public static void main(String[] args) {
    EntityManagerFactory factory =
      Persistence.createEntityManagerFactory("livraria-pu");
    EntityManager manager = factory.createEntityManager();

    manager.getTransaction().begin();

    for(int i = 0; i < 15; i++) {
      Revista revista = new Revista();
      revista.setNome("Revista " + (i + 1));
      revista.setPreco(10.0 * (i + 1));
      manager.persist(revista);
    }

    manager.getTransaction().commit();

    manager.close();
    factory.close();
  }
}
```

Código Java 3.98: www.k19.com.br/exercicios/k12/03/029/AdicionaRevista.java

Execute a classe `AdicionaRevista` e verifique os dados no banco de dados.

31 Altere o arquivo `persistence.xml`.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence
  version="2.1"
  xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
    http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd ">

  <persistence-unit name="livraria-pu" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties>
      <property
        name="javax.persistence.schema-generation.database.action"
        value="none"/>

      <property
        name="javax.persistence.jdbc.driver"
        value="com.mysql.jdbc.Driver" />

      <property
        name="javax.persistence.jdbc.user"
        value="root" />

      <property
        name="javax.persistence.jdbc.password"
        value="root" />

      <property
        name="javax.persistence.jdbc.url"
```

```
31          value="jdbc:mysql://localhost:3306/livraria" />
32      </properties>
33    </persistence-unit>
34  </persistence>
```

Código XML 3.6: www.k19.com.br/exercicios/k12/03/030/persistence.xml

32 Realize diversas consultas. Crie uma classe chamada `Consultas` no pacote `br.com.k19.jpa`.

```
1  package br.com.k19.jpa;
2
3  import java.util.List;
4
5  import javax.persistence.EntityManager;
6  import javax.persistence.EntityManagerFactory;
7  import javax.persistence.Persistence;
8  import javax.persistence.TypedQuery;
9
10 public class Consultas {
11   public static void main(String[] args) {
12     EntityManagerFactory factory =
13       Persistence.createEntityManagerFactory("livraria-pu");
14     EntityManager manager = factory.createEntityManager();
15
16     /* Recuperando a quantidade de revistas */
17     String jpql = "SELECT COUNT(x) FROM Revista x";
18
19     TypedQuery<Long> consulta1 = manager.createQuery(jpql, Long.class);
20
21     Long contador = consulta1.getSingleResult();
22
23     System.out.println("Há " + contador + " revistas.\n");
24
25     /* Recuperando a quantidade de revistas com preço acima de 100 */
26     jpql = "SELECT COUNT(x) FROM Revista x WHERE x.preco > 100";
27
28     TypedQuery<Long> consulta2 = manager.createQuery(jpql, Long.class);
29
30     contador = consulta2.getSingleResult();
31
32     System.out.println("Há " + contador + " revistas com preço acima de 100.\n");
33
34     /* Somando os preços das revistas */
35     jpql = "SELECT SUM(x.preco) FROM Revista x";
36
37     TypedQuery<Double> consulta3 = manager.createQuery(jpql, Double.class);
38
39     Double soma = consulta3.getSingleResult();
40
41     System.out.println("A soma dos preços das revistas é " + soma + "\n");
42
43     /* Recuperando o preço da revista mais cara */
44     jpql = "SELECT MAX(x.preco) FROM Revista x";
45
46     TypedQuery<Double> consulta4 = manager.createQuery(jpql, Double.class);
47
48     Double precoMaximo = consulta4.getSingleResult();
49
50     System.out.println("O preço da revista mais cara é " + precoMaximo + "\n");
51
52     /* Recuperando a média dos preços das revistas */
53     jpql = "SELECT AVG(x.preco) FROM Revista x";
54
55     TypedQuery<Double> consulta5 = manager.createQuery(jpql, Double.class);
56
57     Double media = consulta5.getSingleResult();
58
```

```
59      System.out.println("A média dos preços das revistas é " + media + "\n");
60
61      /* Recuperando todas as revistas */
62      jpql = "SELECT x FROM Revista x";
63
64      TypedQuery<Revista> consulta6 = manager.createQuery(jpql, Revista.class);
65
66      List<Revista> revistas = consulta6.getResultList();
67
68      System.out.println("Todas as revistas: ");
69
70      for(Revista revista : revistas) {
71        System.out.println("Id: " + revista.getId());
72        System.out.println("Nome: " + revista.getNome());
73        System.out.println("Preço: " + revista.getPreco());
74        System.out.println();
75      }
76
77      /* Recuperando as revistas com preço acima de 100 */
78      jpql = "SELECT x FROM Revista x WHERE x.preco > 100";
79
80      TypedQuery<Revista> consulta7 = manager.createQuery(jpql, Revista.class);
81
82      revistas = consulta7.getResultList();
83
84      System.out.println("Revistas com preço acima de 100: ");
85
86      for(Revista revista : revistas) {
87        System.out.println("Id: " + revista.getId());
88        System.out.println("Nome: " + revista.getNome());
89        System.out.println("Preço: " + revista.getPreco());
90        System.out.println();
91      }
92
93      manager.close();
94      factory.close();
95    }
96  }
```

Código Java 3.99: www.k19.com.br/exercicios/k12/03/031/Consultas.java

Execute a classe `Consultas` e verifique as informações exibidas na saída padrão.



Sistema de Controle de Finanças Pessoais

Para praticar os conhecimentos obtidos nesse capítulo, vamos implementar um pequeno sistema de controle de finanças pessoais.



Exercícios de Fixação

33 Crie duas entidades em pacote chamado `br.com.k19.jpa.grana`: `Despesa` e `Receita`.

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.SimpleDateFormat;
4  import java.util.Calendar;
5
6  import javax.persistence.Entity;
7  import javax.persistence.GeneratedValue;
```

```
8  import javax.persistence.Id;
9
10 @Entity
11 public class Despesa {
12
13   @Id
14   @GeneratedValue
15   private Long id;
16
17   private String nome;
18
19   private Double valor;
20
21   private Calendar data;
22
23   private String tipo;
24
25   @Override
26   public String toString() {
27     String s = this.nome + " - ";
28     s += this.valor + " - ";
29     s += this.tipo + " - ";
30
31     SimpleDateFormat sdf =
32         new SimpleDateFormat("dd/MM/yyyy");
33
34     s += sdf.format(this.data.getTime());
35
36     return s;
37   }
38
39   // GETTERS E SETTERS
40 }
```

Código Java 3.100: www.k19.com.br/exercicios/k12/03/032/Despesa.java

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.SimpleDateFormat;
4  import java.util.Calendar;
5
6  import javax.persistence.Entity;
7  import javax.persistence.GeneratedValue;
8  import javax.persistence.Id;
9
10 @Entity
11 public class Receita {
12
13   @Id
14   @GeneratedValue
15   private Long id;
16
17   private String nome;
18
19   private Double valor;
20
21   private Calendar data;
22
23   private String tipo;
24
25   @Override
26   public String toString() {
27     String s = this.nome + " - ";
28     s += this.valor + " - ";
29     s += this.tipo + " - ";
30
31     SimpleDateFormat sdf =
32         new SimpleDateFormat("dd/MM/yyyy");
33
```

```
34      s += sdf.format(this.data.getTime());
35
36      return s;
37    }
38
39    // GETTERS E SETTERS
40  }
```

Código Java 3.101: www.k19.com.br/exercicios/k12/03/032/Receita.java

34 Crie dois repositórios no pacote `br.com.k19.jpa.grana`: `DespesaRepositorio` e `ReceitaRepositorio`.

```
1  package br.com.k19.jpa.grana;
2
3  import java.util.Calendar;
4  import java.util.List;
5
6  import javax.persistence.EntityManager;
7  import javax.persistence.TypedQuery;
8
9  public class DespesaRepositorio {
10
11   private EntityManager manager;
12
13   public DespesaRepositorio(EntityManager manager) {
14     this.manager = manager;
15   }
16
17   public void adiciona(Despesa despesa) {
18     this.manager.persist(despesa);
19   }
20
21   public void remove(Despesa despesa) {
22     this.manager.remove(despesa);
23   }
24
25   public Double somaDespesasAte(Calendar data) {
26     String jpql =
27       "SELECT SUM(x.valor) FROM Despesa x WHERE x.data <= :data";
28
29     TypedQuery<Double> query =
30       this.manager.createQuery(jpql, Double.class);
31
32     query.setParameter("data", data);
33
34     return query.getSingleResult();
35   }
36
37   public Double somaDespesas(
38     Calendar dataInicial,
39     Calendar dataFinal) {
40
41     String jpql =
42       "SELECT SUM(x.valor) FROM Despesa x WHERE "
43       + "x.data >= :dataInicial AND x.data <= :dataFinal";
44
45     TypedQuery<Double> query =
46       this.manager.createQuery(jpql, Double.class);
47
48     query.setParameter("dataInicial", dataInicial);
49     query.setParameter("dataFinal", dataFinal);
50
51     return query.getSingleResult();
52   }
53
54   public List<Despesa> buscaPorPeriodo(
55     Calendar dataInicial,
```

```
56      Calendar dataFinal) {
57
58      String jpql =
59        "SELECT x FROM Despesa x WHERE x.data >= :dataInicial "
60        + "AND x.data <= :dataFinal ORDER BY x.data";
61
62      TypedQuery<Despesa> query =
63        this.manager.createQuery(jpql, Despesa.class);
64
65      query.setParameter("dataInicial", dataInicial);
66      query.setParameter("dataFinal", dataFinal);
67
68      return query.getResultList();
69    }
70
71    public List<Despesa> buscaRecentes() {
72      String jpql =
73        "SELECT x FROM Despesa x ORDER BY x.id DESC";
74
75      TypedQuery<Despesa> query =
76        this.manager.createQuery(jpql, Despesa.class);
77
78      query.setMaxResults(10);
79
80      return query.getResultList();
81    }
82  }
```

Código Java 3.102: www.k19.com.br/exercicios/k12/03/033/DespesaRepositorio.java

```
1   package br.com.k19.jpa.grana;
2
3   import java.util.Calendar;
4   import java.util.List;
5
6   import javax.persistence.EntityManager;
7   import javax.persistence.TypedQuery;
8
9   public class ReceitaRepositorio {
10
11    private EntityManager manager;
12
13    public ReceitaRepositorio(EntityManager manager) {
14      this.manager = manager;
15    }
16
17    public void adiciona(Receita receita) {
18      this.manager.persist(receita);
19    }
20
21    public void remove(Receita receita) {
22      this.manager.remove(receita);
23    }
24
25    public Double somaReceitasAte(Calendar data) {
26      String jpql =
27        "SELECT SUM(x.valor) FROM Receita x WHERE x.data <= :data";
28
29      TypedQuery<Double> query =
30        this.manager.createQuery(jpql, Double.class);
31
32      query.setParameter("data", data);
33
34      return query.getSingleResult();
35    }
36
37    public Double somaReceitas(Calendar dataInicial, Calendar dataFinal) {
38      String jpql =
39        "SELECT SUM(x.valor) FROM Receita x WHERE "
```

```
40        + "x.data >= :dataInicial AND x.data <= :dataFinal";
41
42      TypedQuery<Double> query =
43        this.manager.createQuery(jpql, Double.class);
44
45      query.setParameter("dataInicial", dataInicial);
46      query.setParameter("dataFinal", dataFinal);
47
48      return query.getSingleResult();
49    }
50
51    public List<Receita> buscaPorPeriodo(
52      Calendar dataInicial,
53      Calendar dataFinal) {
54
55      String jpql =
56        "SELECT x FROM Receita x WHERE x.data >= :dataInicial "
57        + "AND x.data <= :dataFinal ORDER BY x.data";
58
59      TypedQuery<Receita> query =
60        this.manager.createQuery(jpql, Receita.class);
61
62      query.setParameter("dataInicial", dataInicial);
63      query.setParameter("dataFinal", dataFinal);
64
65      return query.getResultList();
66    }
67
68    public List<Receita> buscaRecentes() {
69      String jpql =
70        "SELECT x FROM Receita x ORDER BY x.id DESC";
71
72      TypedQuery<Receita> query =
73        this.manager.createQuery(jpql, Receita.class);
74
75      query.setMaxResults(10);
76
77      return query.getResultList();
78    }
79  }
```

Código Java 3.103: www.k19.com.br/exercicios/k12/03/033/ReceitaRepositorio.java

35 Crie uma interface chamada `Tela` no pacote `br.com.k19.jpa.grana`.

```
1  package br.com.k19.jpa.grana;
2
3  public interface Tela {
4
5    Tela mostra();
6
7    String getNome();
8  }
```

Código Java 3.104: www.k19.com.br/exercicios/k12/03/034/Tela.java

36 Crie uma classe chamada `TelaMenu` no pacote `br.com.k19.jpa.grana`.

```
1  package br.com.k19.jpa.grana;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class TelaMenu implements Tela {
```

```
7
8   private String nome;
9
10  private List<Tela> filhas = new ArrayList<Tela>();
11
12  public TelaMenu(String nome) {
13    this.nome = nome;
14  }
15
16  public Tela mostra() {
17    System.out.println(">>> " + this.nome + " <<<");
18    System.out.println();
19
20    for (int i = 0; i < this.filhas.size(); i++) {
21      System.out.println((i + 1) + ". " + this.filhas.get(i).getNome());
22    }
23
24    System.out.println();
25    System.out.print("Escolha a opção: ");
26    int indiceDaOpcao = Grana.getEntrada().nextInt() - 1;
27
28    System.out.println();
29
30    return this.filhas.get(indiceDaOpcao);
31  }
32
33  public void adicionaFilha(Tela tela) {
34    this.filhas.add(tela);
35  }
36
37  public String getNome() {
38    return this.nome;
39  }
40 }
```

Código Java 3.105: www.k19.com.br/exercicios/k12/03/035/TelaMenu.java

37 Crie uma classe chamada `TelaGeraDados` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.util.Calendar;
4  import java.util.GregorianCalendar;
5
6  import javax.persistence.EntityManager;
7
8  public class TelaGeraDados implements Tela {
9
10   private Tela anterior;
11
12   public TelaGeraDados(Tela anterior) {
13     this.anterior = anterior;
14   }
15
16   @Override
17   public Tela mostra() {
18     System.out.println(">>> " + this.getNome() + " <<<");
19     System.out.println();
20
21     System.out.println("Gerando dados...");
22
23     EntityManager manager = Grana.getEntityManager();
24
25     ReceitaRepositorio receitaRepositorio =
26       new ReceitaRepositorio(manager);
27
```

```
28      DespesaRepositorio despesaRepositorio =
29        new DespesaRepositorio(manager);
30
31      manager.getTransaction().begin();
32
33      /* Receitas */
34      for (int i = 0; i < 36; i++) {
35        Receita receita = new Receita();
36        receita.setNome("Salário K19");
37        receita.setTipo("Salário");
38        receita.setValor(8000.0);
39
40        Calendar data = new GregorianCalendar(2014, 0, 10);
41        data.add(Calendar.MONTH, i);
42        receita.setData(data);
43
44        receitaRepositorio.adiciona(receita);
45      }
46
47      for (int i = 0; i < 36; i++) {
48        Receita receita = new Receita();
49        receita.setNome("VA");
50        receita.setTipo("Vale Alimentação");
51        receita.setValor(400.00);
52
53        Calendar data = new GregorianCalendar(2014, 0, 15);
54        data.add(Calendar.MONTH, i);
55        receita.setData(data);
56
57        receitaRepositorio.adiciona(receita);
58      }
59
60      for (int i = 0; i < 36; i++) {
61        Receita receita = new Receita();
62        receita.setNome("VR");
63        receita.setTipo("Vale Refeição");
64        receita.setValor(500.0);
65
66        Calendar data = new GregorianCalendar(2014, 0, 20);
67        data.add(Calendar.MONTH, i);
68        receita.setData(data);
69
70        receitaRepositorio.adiciona(receita);
71      }
72
73      for (int i = 0; i < 36; i++) {
74        Receita receita = new Receita();
75        receita.setNome("Aluguel Casa BH");
76        receita.setTipo("Aluguel");
77        receita.setValor(1000.0);
78
79        Calendar data = new GregorianCalendar(2014, 0, 1);
80        data.add(Calendar.MONTH, i);
81        receita.setData(data);
82
83        receitaRepositorio.adiciona(receita);
84      }
85
86      /* Despesas */
87      for (int i = 0; i < 36; i++) {
88        Despesa despesa = new Despesa();
89        despesa.setNome("Aluguel Apto SP");
90        despesa.setTipo("Aluguel");
91        despesa.setValor(1800.0);
92
93        Calendar data = new GregorianCalendar(2014, 0, 1);
94        data.add(Calendar.MONTH, i);
95        despesa.setData(data);
96
97        despesaRepositorio.adiciona(despesa);
```

```
98      }
99
100     for (int i = 0; i < 36; i++) {
101       Despesa despesa = new Despesa();
102       despesa.setNome("Compras Supermercado");
103       despesa.setTipo("Alimentação");
104       despesa.setValor(1000.0);
105
106       Calendar data = new GregorianCalendar(2014, 0, 10);
107       data.add(Calendar.MONTH, i);
108       despesa.setData(data);
109
110       despesaRepositorio.adiciona(despesa);
111     }
112
113     for (int i = 0; i < 36; i++) {
114       Despesa despesa = new Despesa();
115       despesa.setNome("Combustível");
116       despesa.setTipo("Transporte");
117       despesa.setValor(400.00);
118
119       Calendar data = new GregorianCalendar(2014, 0, 15);
120       data.add(Calendar.MONTH, i);
121       despesa.setData(data);
122
123       despesaRepositorio.adiciona(despesa);
124     }
125
126     for (int i = 0; i < 36; i++) {
127       Despesa despesa = new Despesa();
128       despesa.setNome("Cinema");
129       despesa.setTipo("Lazer");
130       despesa.setValor(200.0);
131
132       Calendar data = new GregorianCalendar(2014, 0, 20);
133       data.add(Calendar.MONTH, i);
134       despesa.setData(data);
135
136       despesaRepositorio.adiciona(despesa);
137     }
138
139     manager.getTransaction().commit();
140     manager.close();
141
142     System.out.println("Dados gerados");
143     System.out.println();
144
145     return this.anterior;
146   }
147
148   @Override
149   public String getNome() {
150     return "Gera Dados";
151   }
152 }
```

Código Java 3.106: www.k19.com.br/exercicios/k12/03/036/TelaGeraDados.java

38 Crie uma classe chamada `TelaLimpaDados` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1 package br.com.k19.jpa.grana;
2
3 import javax.persistence.EntityManager;
4 import javax.persistence.Query;
5
6 public class TelaLimpaDados implements Tela {
```

```
7
8   private Tela anterior;
9
10  public TelaLimpaDados(Tela anterior) {
11    this.anterior = anterior;
12  }
13
14  @Override
15  public Tela mostra() {
16    System.out.println(">>> " + this.getNome() + " <<<");
17    System.out.println();
18
19    System.out.println("Limpando dados...");
20
21    EntityManager manager = Grana.getEntityManager();
22    manager.getTransaction().begin();
23
24    Query query = manager.createQuery("DELETE FROM Receita");
25    query.executeUpdate();
26
27    query = manager.createQuery("DELETE FROM Despesa");
28    query.executeUpdate();
29
30    manager.getTransaction().commit();
31    manager.close();
32
33    System.out.println("Dados limpos");
34
35    return this.anterior;
36  }
37
38  @Override
39  public String getNome() {
40    return "Limpa Dados";
41  }
42 }
```

Código Java 3.107: www.k19.com.br/exercicios/k12/03/037/TelaLimpaDados.java

39 Crie uma classe chamada `TelaSair` no pacote `br.com.k19.jpa.grana`.

```
1  package br.com.k19.jpa.grana;
2
3  public class TelaSair implements Tela {
4
5    @Override
6    public String getNome() {
7      return "Sair";
8    }
9
10   @Override
11   public Tela mostra() {
12     return null;
13   }
14 }
```

Código Java 3.108: www.k19.com.br/exercicios/k12/03/038/TelaSair.java

40 Crie uma classe chamada `TelaDespesaAdicionar` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.ParseException;
```

```
4  import java.text.SimpleDateFormat;
5  import java.util.Calendar;
6  import java.util.Date;
7  import java.util.InputMismatchException;
8  import java.util.LinkedHashMap;
9  import java.util.Map;
10
11 import javax.persistence.EntityManager;
12
13 public class TelaDespesaAdicionar implements Tela {
14
15   private Tela anterior;
16
17   public TelaDespesaAdicionar(Tela anterior) {
18     this.anterior = anterior;
19   }
20
21   @Override
22   public Tela mostra() {
23     System.out.println(">>> " + this.getNome() + " <<<");
24     System.out.println();
25
26     System.out.print("Digite o nome: ");
27     String nome = Grana.getEntrada().next();
28
29     double valor = -1;
30     while (valor < 0) {
31       System.out.print("Digite o valor: ");
32
33       try {
34         valor = Grana.getEntrada().nextDouble();
35       } catch (InputMismatchException e) {
36         System.out.println("Valor incorreto");
37         continue;
38       }
39
40       if (valor < 0) {
41         System.out.println("Valor incorreto");
42       }
43     }
44
45     SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
46
47     Calendar data = null;
48     while (data == null) {
49       System.out.print("Digite a data (ex: 05/08/2016): ");
50
51       try {
52         Date d = sdf.parse(Grana.getEntrada().next());
53         data = Calendar.getInstance();
54         data.setTime(d);
55       } catch (ParseException e) {
56         System.out.println("Data incorreta");
57       }
58     }
59
60     System.out.println();
61     Map<Integer, String> tipos =
62         new LinkedHashMap<Integer, String>();
63     tipos.put(1, "Aluguel");
64     tipos.put(2, "Alimentação");
65     tipos.put(3, "Transporte");
66     tipos.put(4, "Lazer");
67     tipos.put(5, "Outros");
68
69     for (Integer key : tipos.keySet()) {
70       System.out.println(key + ". " + tipos.get(key));
71     }
72
73     int tipo = -1;
```

```
74      while (tipo < 1 || tipo > 5) {
75        System.out.print("Escolha o tipo (número de 1 a 5): ");
76
77        try {
78          tipo = Grana.getEntrada().nextInt();
79        } catch (InputMismatchException e) {
80          System.out.println("Tipo incorreto");
81          continue;
82        }
83
84        if (tipo < 1 || tipo > 5) {
85          System.out.println("Tipo incorreto");
86        }
87      }
88
89      EntityManager manager = Grana.getEntityManager();
90      manager.getTransaction().begin();
91
92      Despesa despesa = new Despesa();
93      despesa.setNome(nome);
94      despesa.setValor(valor);
95      despesa.setTipo(tipos.get(tipo));
96      despesa.setData(data);
97
98      DespesaRepositorio despesaRepositorio =
99          new DespesaRepositorio(manager);
100     despesaRepositorio.adiciona(despesa);
101
102     manager.getTransaction().commit();
103     manager.close();
104
105     System.out.println("Despesa adicionada");
106
107     return this.anterior;
108   }
109
110   @Override
111   public String getNome() {
112     return "Adicionar Despesa";
113   }
114 }
```

Código Java 3.109: www.k19.com.br/exercicios/k12/03/039/TelaDespesaAdicionar.java

41 Crie uma classe chamada `TelaDespesaConsultaPeriodo` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.ParseException;
4  import java.text.SimpleDateFormat;
5  import java.util.Calendar;
6  import java.util.Date;
7  import java.util.InputMismatchException;
8  import java.util.List;
9
10 import javax.persistence.EntityManager;
11
12 public class TelaDespesaConsultaPeriodo implements Tela {
13   private Tela anterior;
14
15   public TelaDespesaConsultaPeriodo(Tela anterior) {
16     this.anterior = anterior;
17   }
18
19   @Override
20   public Tela mostra() {
```

```
21      System.out.println(">>> " + this.getNome() + " <<<");
22      System.out.println();
23
24      SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
25
26      Calendar dataInicial = null;
27
28      while (dataInicial == null) {
29        System.out.print("Digite a data inicial (ex: 05/08/2016): ");
30
31        try {
32          Date d = sdf.parse(Grana.getEntrada().next());
33          dataInicial = Calendar.getInstance();
34          dataInicial.setTime(d);
35        } catch (ParseException e) {
36          System.out.println("Data incorreta");
37        }
38      }
39
40      Calendar dataFinal = null;
41
42      while (dataFinal == null) {
43        System.out.print("Digite a data final (ex: 05/12/2016): ");
44
45        try {
46          Date d = sdf.parse(Grana.getEntrada().next());
47          dataFinal = Calendar.getInstance();
48          dataFinal.setTime(d);
49        } catch (ParseException e) {
50          System.out.println("Data incorreta");
51        }
52      }
53
54
55      EntityManager manager = Grana.getEntityManager();
56
57      DespesaRepositorio despesaRepositorio =
58          new DespesaRepositorio(manager);
59
60      List<Despesa> despesas =
61        despesaRepositorio.buscaPorPeriodo(dataInicial, dataFinal);
62
63      int opcao = -1;
64
65      while(opcao < 0 || opcao > despesas.size()) {
66        for (int i = 0; i < despesas.size(); i++) {
67          System.out.println((i + 1) + ". " + despesas.get(i));
68        }
69
70        System.out.println();
71        System.out.println("Digite o número da despesa que deseja remover.");
72        System.out.println("Digite 0 para voltar.");
73
74        try {
75          opcao = Grana.getEntrada().nextInt();
76        } catch (InputMismatchException e) {
77          System.out.println("Opção incorreta");
78          continue;
79        }
80
81        if (opcao < 0 || opcao > 1000) {
82          System.out.println("Opção incorreto");
83          continue;
84        }
85
86        if(opcao != 0) {
87          manager.getTransaction().begin();
88
89          despesaRepositorio.remove(despesas.get(opcao - 1));
90
```

```
91          manager.getTransaction().commit();
92
93          despesas.remove(opcao - 1);
94
95          opcao = -1;
96
97          System.out.println();
98          System.out.println("Despesa Removida");
99          System.out.println();
100       }
101     }
102
103     System.out.println();
104
105     return this.anterior;
106   }
107
108   @Override
109   public String getNome() {
110     return "Despesas por Período";
111   }
112 }
```

Código Java 3.110: www.k19.com.br/exercicios/k12/03/040/TelaDespesaConsultaPeriodo.java

42 Crie uma classe chamada `TelaDespesaConsultaRecentes` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.util.InputMismatchException;
4  import java.util.List;
5
6  import javax.persistence.EntityManager;
7
8  public class TelaDespesaConsultaRecentes implements Tela {
9
10   private Tela anterior;
11
12   public TelaDespesaConsultaRecentes(Tela anterior) {
13     this.anterior = anterior;
14   }
15
16   @Override
17   public Tela mostra() {
18     System.out.println(">>> " + this.getNome() + " <<<");
19     System.out.println();
20
21     EntityManager manager = Grana.getEntityManager();
22
23     DespesaRepositorio despesaRepositorio =
24         new DespesaRepositorio(manager);
25
26     List<Despesa> despesas =
27       despesaRepositorio.buscaRecentes();
28
29     int opcao = -1;
30
31     while(opcao < 0 || opcao > despesas.size()) {
32       for (int i = 0; i < despesas.size(); i++) {
33         System.out.println((i + 1) + ". " + despesas.get(i));
34       }
35
36       System.out.println();
37       System.out.println("Digite o número da receita que deseja remover.");
38       System.out.println("Digite 0 para voltar.");
39
```

```
40        try {
41          opcao = Grana.getEntrada().nextInt();
42        } catch (InputMismatchException e) {
43          System.out.println("Opção incorreta");
44          continue;
45        }
46
47        if (opcao < 0 || opcao > 1000) {
48          System.out.println("Opção incorreto");
49          continue;
50        }
51
52        if(opcao != 0) {
53          manager.getTransaction().begin();
54
55          despesaRepositorio.remove(despesas.get(opcao - 1));
56
57          manager.getTransaction().commit();
58
59          despesas.remove(opcao - 1);
60
61          opcao = -1;
62
63          System.out.println();
64          System.out.println("Despesa Removida");
65          System.out.println();
66        }
67      }
68
69      System.out.println();
70
71      return this.anterior;
72    }
73
74    @Override
75    public String getNome() {
76      return "Últimas Despesas Adicionadas";
77    }
78  }
```

Código Java 3.111: www.k19.com.br/exercicios/k12/03/041/TelaDespesaConsultaRecentes.java

43 Crie uma classe chamada `TelaReceitaAdicionar` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.ParseException;
4  import java.text.SimpleDateFormat;
5  import java.util.Calendar;
6  import java.util.Date;
7  import java.util.InputMismatchException;
8  import java.util.LinkedHashMap;
9  import java.util.Map;
10
11 import javax.persistence.EntityManager;
12
13 public class TelaReceitaAdicionar implements Tela {
14
15   private Tela anterior;
16
17   public TelaReceitaAdicionar(Tela anterior) {
18     this.anterior = anterior;
19   }
20
21   @Override
22   public Tela mostra() {
```

```
23      System.out.println(">>> " + this.getNome() + " <<<");
24      System.out.println();
25
26      System.out.print("Digite o nome: ");
27      String nome = Grana.getEntrada().next();
28
29      double valor = -1;
30      while (valor < 0) {
31        System.out.print("Digite o valor: ");
32
33        try {
34          valor = Grana.getEntrada().nextDouble();
35        } catch (InputMismatchException e) {
36          System.out.println("Valor incorreto");
37          continue;
38        }
39
40        if (valor < 0) {
41          System.out.println("Valor incorreto");
42        }
43      }
44
45      SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
46
47      Calendar data = null;
48      while (data == null) {
49        System.out.print("Digite a data (ex: 05/08/2016): ");
50
51        try {
52          Date d = sdf.parse(Grana.getEntrada().next());
53          data = Calendar.getInstance();
54          data.setTime(d);
55        } catch (ParseException e) {
56          System.out.println("Data incorreta");
57        }
58      }
59
60      System.out.println();
61      Map<Integer, String> tipos = new LinkedHashMap<Integer, String>();
62      tipos.put(1, "Salário");
63      tipos.put(2, "Vale Alimentação");
64      tipos.put(3, "Vale Refeição");
65      tipos.put(4, "Aluguel");
66      tipos.put(5, "Outros");
67
68      for (Integer key : tipos.keySet()) {
69        System.out.println(key + ". " + tipos.get(key));
70      }
71
72      int tipo = -1;
73      while (tipo < 1 || tipo > 5) {
74        System.out.print("Escolha o tipo (número de 1 a 5): ");
75
76        try {
77          tipo = Grana.getEntrada().nextInt();
78        } catch (InputMismatchException e) {
79          System.out.println("Tipo incorreto");
80          continue;
81        }
82
83        if (tipo < 1 || tipo > 5) {
84          System.out.println("Tipo incorreto");
85        }
86      }
87
88      EntityManager manager = Grana.getEntityManager();
89      manager.getTransaction().begin();
90
91      Receita receita = new Receita();
92      receita.setNome(nome);
```

```
93      receita.setValor(valor);
94      receita.setTipo(tipos.get(tipo));
95      receita.setData(data);
96
97      ReceitaRepositorio receitaRepositorio = new ReceitaRepositorio(manager);
98      receitaRepositorio.adiciona(receita);
99
100     manager.getTransaction().commit();
101     manager.close();
102
103     System.out.println("Receita adicionada");
104     System.out.println();
105
106     return this.anterior;
107   }
108
109   @Override
110   public String getNome() {
111     return "Adicionar Receita";
112   }
113 }
```

Código Java 3.112: www.k19.com.br/exercicios/k12/03/042/TelaReceitaAdicionar.java

44 Crie uma classe chamada `TelaReceitaConsultaPeriodo` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.ParseException;
4  import java.text.SimpleDateFormat;
5  import java.util.Calendar;
6  import java.util.Date;
7  import java.util.InputMismatchException;
8  import java.util.List;
9
10 import javax.persistence.EntityManager;
11
12 public class TelaReceitaConsultaPeriodo implements Tela {
13
14   private Tela anterior;
15
16   public TelaReceitaConsultaPeriodo(Tela anterior) {
17     this.anterior = anterior;
18   }
19
20   @Override
21   public Tela mostra() {
22     System.out.println(">>> " + this.getNome() + " <<<");
23     System.out.println();
24
25     SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
26
27     Calendar dataInicial = null;
28
29     while (dataInicial == null) {
30       System.out.print("Digite a data inicial (ex: 05/08/2016): ");
31
32       try {
33         Date d = sdf.parse(Grana.getEntrada().next());
34         dataInicial = Calendar.getInstance();
35         dataInicial.setTime(d);
36       } catch (ParseException e) {
37         System.out.println("Data incorreta");
38       }
39     }
40
```

```
41      Calendar dataFinal = null;
42
43      while (dataFinal == null) {
44        System.out.print("Digite a data final (ex: 05/12/2016): ");
45
46        try {
47          Date d = sdf.parse(Grana.getEntrada().next());
48          dataFinal = Calendar.getInstance();
49          dataFinal.setTime(d);
50        } catch (ParseException e) {
51          System.out.println("Data incorreta");
52        }
53      }
54
55
56      EntityManager manager = Grana.getEntityManager();
57
58      ReceitaRepositorio receitaRepositorio =
59          new ReceitaRepositorio(manager);
60
61      List<Receita> receitas =
62        receitaRepositorio.buscaPorPeriodo(dataInicial, dataFinal);
63
64      int opcao = -1;
65
66      while(opcao < 0 || opcao > receitas.size()) {
67        for (int i = 0; i < receitas.size(); i++) {
68          System.out.println((i + 1) + ". " + receitas.get(i));
69        }
70
71        System.out.println();
72        System.out.println("Digite o número da despesa que deseja remover.");
73        System.out.println("Digite 0 para voltar.");
74
75        try {
76          opcao = Grana.getEntrada().nextInt();
77        } catch (InputMismatchException e) {
78          System.out.println("Opção incorreta");
79          continue;
80        }
81
82        if (opcao < 0 || opcao > 1000) {
83          System.out.println("Opção incorreto");
84          continue;
85        }
86
87        if(opcao != 0) {
88          manager.getTransaction().begin();
89
90          receitaRepositorio.remove(receitas.get(opcao - 1));
91
92          manager.getTransaction().commit();
93
94          receitas.remove(opcao - 1);
95
96          opcao = -1;
97
98          System.out.println();
99          System.out.println("Receita Removida");
100         System.out.println();
101       }
102     }
103
104     System.out.println();
105
106     return this.anterior;
107   }
108
109   @Override
110   public String getNome() {
```

```
111     return "Receitas por Período";
112   }
113 }
```

Código Java 3.113: www.k19.com.br/exercicios/k12/03/043/TelaReceitaConsultaPeriodo.java

45 Crie uma classe chamada `TelaReceitaConsultaRecentes` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.util.InputMismatchException;
4  import java.util.List;
5
6  import javax.persistence.EntityManager;
7
8  public class TelaReceitaConsultaRecentes implements Tela {
9
10   private Tela anterior;
11
12   public TelaReceitaConsultaRecentes(Tela anterior) {
13     this.anterior = anterior;
14   }
15
16   @Override
17   public Tela mostra() {
18     System.out.println(">>> " + this.getNome() + " <<<");
19     System.out.println();
20
21     EntityManager manager = Grana.getEntityManager();
22
23     ReceitaRepositorio receitaRepositorio =
24         new ReceitaRepositorio(manager);
25
26     List<Receita> receitas =
27       receitaRepositorio.buscaRecentes();
28
29     int opcao = -1;
30
31     while(opcao < 0 || opcao > receitas.size()) {
32       for (int i = 0; i < receitas.size(); i++) {
33         System.out.println((i + 1) + ". " + receitas.get(i));
34       }
35
36       System.out.println();
37       System.out.println("Digite o número da receita que deseja remover.");
38       System.out.println("Digite 0 para voltar.");
39
40       try {
41         opcao = Grana.getEntrada().nextInt();
42       } catch (InputMismatchException e) {
43         System.out.println("Opção incorreta");
44         continue;
45       }
46
47       if (opcao < 0 || opcao > 1000) {
48         System.out.println("Opção incorreto");
49         continue;
50       }
51
52       if(opcao != 0) {
53         manager.getTransaction().begin();
54
55         receitaRepositorio.remove(receitas.get(opcao - 1));
56
57         manager.getTransaction().commit();
58
```

```
59          receitas.remove(opcao - 1);
60
61          opcao = -1;
62
63          System.out.println();
64          System.out.println("Receita Removida");
65          System.out.println();
66        }
67      }
68
69      System.out.println();
70
71      return this.anterior;
72    }
73
74    @Override
75    public String getNome() {
76      return "Últimas Receitas Adicionadas";
77    }
78  }
```

Código Java 3.114: www.k19.com.br/exercicios/k12/03/044/TelaReceitaConsultaRecentes.java

46 Crie uma classe chamada `TelaRelatorioConsolidado` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.ParseException;
4  import java.text.SimpleDateFormat;
5  import java.util.Calendar;
6  import java.util.Date;
7
8  import javax.persistence.EntityManager;
9
10 public class TelaRelatorioConsolidado implements Tela {
11
12   private Tela anterior;
13
14   public TelaRelatorioConsolidado(Tela anterior) {
15     this.anterior = anterior;
16   }
17
18   @Override
19   public Tela mostra() {
20     System.out.println(">>> " + this.getNome() + " <<<");
21     System.out.println();
22
23     Calendar data = null;
24     SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
25
26     while (data == null) {
27       System.out.print("Digite uma data (ex: 05/08/2017): ");
28
29       try {
30         Date d = sdf.parse(Grana.getEntrada().next());
31         data = Calendar.getInstance();
32         data.setTime(d);
33       } catch (ParseException e) {
34         System.out.println("Data incorreta");
35       }
36     }
37
38     EntityManager manager = Grana.getEntityManager();
39
40     ReceitaRepositorio receitaRepositorio =
41         new ReceitaRepositorio(manager);
```

```
42
43      DespesaRepositorio despesaRepositorio =
44          new DespesaRepositorio(manager);
45
46      Double receitas = receitaRepositorio.somaReceitasAte(data);
47      Double despesas = despesaRepositorio.somaDespesasAte(data);
48
49      manager.close();
50
51      System.out.println("Saldo: " + (receitas - despesas));
52
53      System.out.println();
54
55      return this.anterior;
56    }
57
58    @Override
59    public String getNome() {
60      return "Relatório Consolidado";
61    }
62  }
```

Código Java 3.115: www.k19.com.br/exercicios/k12/03/045/TelaRelatorioConsolidado.java

47 Crie uma classe chamada `TelaRelatorioMensal` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.util.Calendar;
4  import java.util.GregorianCalendar;
5  import java.util.InputMismatchException;
6
7  import javax.persistence.EntityManager;
8
9  public class TelaRelatorioMensal implements Tela {
10
11   private Tela anterior;
12
13   public TelaRelatorioMensal(Tela anterior) {
14     this.anterior = anterior;
15   }
16
17   @Override
18   public String getNome() {
19     return "Relatório Mensal";
20   }
21
22   @Override
23   public Tela mostra() {
24     System.out.println(">>> " + this.getNome() + " <<<");
25     System.out.println();
26
27     int mes = -1;
28
29     while (mes < 1 || mes > 12) {
30       System.out.print("Digite o mês (número de 1 a 12): ");
31
32       try {
33         mes = Grana.getEntrada().nextInt();
34       } catch (InputMismatchException e) {
35         System.out.println("Mês incorreto");
36         continue;
37       }
38
39       if (mes < 1 || mes > 12) {
40         System.out.println("Mês incorreto");
```

```
41        }
42      }
43
44      int ano = -1;
45
46      while (ano < 1900 || ano > 3000) {
47        System.out.print("Digite o ano (número de 1900 a 3000): ");
48
49        try {
50          ano = Grana.getEntrada().nextInt();
51        } catch (InputMismatchException e) {
52          System.out.println("Ano incorreto");
53          continue;
54        }
55
56        if (ano < 1900 || ano > 3000) {
57          System.out.println("Ano incorreto");
58        }
59      }
60
61      Calendar dataInicial = new GregorianCalendar(ano, mes - 1, 1);
62      Calendar dataFinal = new GregorianCalendar(ano, mes - 1, 1);
63      dataFinal.add(Calendar.MONTH, 1);
64      dataFinal.add(Calendar.DAY_OF_MONTH, -1);
65
66      EntityManager manager = Grana.getEntityManager();
67
68      ReceitaRepositorio receitaRepositorio =
69          new ReceitaRepositorio(manager);
70
71      DespesaRepositorio despesaRepositorio =
72          new DespesaRepositorio(manager);
73
74      Double receitas =
75        receitaRepositorio.somaReceitas(dataInicial, dataFinal);
76      Double despesas =
77        despesaRepositorio.somaDespesas(dataInicial, dataFinal);
78
79      manager.close();
80
81      System.out.println("Receitas: " + receitas);
82      System.out.println("Despesas: " + despesas);
83      System.out.println("Saldo do Período: " + (receitas - despesas));
84
85      System.out.println();
86
87      return this.anterior;
88    }
89  }
```

Código Java 3.116: www.k19.com.br/exercicios/k12/03/046/TelaRelatorioMensal.java

48 Crie uma classe chamada `TelaRelatorioPersonalizado` no pacote `br.com.k19.jpa.grana`. Por enquanto, não se preocupe com os erros de compilação.

```
1  package br.com.k19.jpa.grana;
2
3  import java.text.ParseException;
4  import java.text.SimpleDateFormat;
5  import java.util.Calendar;
6  import java.util.Date;
7
8  import javax.persistence.EntityManager;
9
10 public class TelaRelatorioPersonalizado implements Tela {
11
12   private Tela anterior;
```

```
13
14    public TelaRelatorioPersonalizado(Tela anterior) {
15      this.anterior = anterior;
16    }
17
18    @Override
19    public String getNome() {
20      return "Relatório Personalizado";
21    }
22
23    @Override
24    public Tela mostra() {
25      System.out.println(">>> " + this.getNome() + " <<<");
26      System.out.println();
27
28      SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
29
30      Calendar dataInicial = null;
31
32      while (dataInicial == null) {
33        System.out.print("Digite a data inicial (ex: 05/08/2016): ");
34
35        try {
36          Date d = sdf.parse(Grana.getEntrada().next());
37          dataInicial = Calendar.getInstance();
38          dataInicial.setTime(d);
39        } catch (ParseException e) {
40          System.out.println("Data incorreta");
41        }
42      }
43
44      Calendar dataFinal = null;
45
46      while (dataFinal == null) {
47        System.out.print("Digite a data final (ex: 05/12/2016): ");
48
49        try {
50          Date d = sdf.parse(Grana.getEntrada().next());
51          dataFinal = Calendar.getInstance();
52          dataFinal.setTime(d);
53        } catch (ParseException e) {
54          System.out.println("Data incorreta");
55        }
56      }
57
58      EntityManager manager = Grana.getEntityManager();
59
60      ReceitaRepositorio receitaRepositorio =
61          new ReceitaRepositorio(manager);
62
63      DespesaRepositorio despesaRepositorio =
64          new DespesaRepositorio(manager);
65
66      Double receitas =
67        receitaRepositorio.somaReceitas(dataInicial, dataFinal);
68      Double despesas =
69        despesaRepositorio.somaDespesas(dataInicial, dataFinal);
70
71      manager.close();
72
73      System.out.println("Receitas: " + receitas);
74      System.out.println("Despesas: " + despesas);
75      System.out.println("Saldo do Período: " + (receitas - despesas));
76
77      System.out.println();
78
79      return this.anterior;
80    }
81  }
```

Código Java 3.117: www.k19.com.br/exercicios/k12/03/047/TelaRelatorioPersonalizado.java

49 Altere o arquivo `persistence.xml`.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence
  version="2.1"
  xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
    http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd ">

  <persistence-unit name="livraria-pu" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties>
      <property
        name="javax.persistence.schema-generation.database.action"
        value="none"/>

      <property
        name="javax.persistence.jdbc.driver"
        value="com.mysql.jdbc.Driver" />

      <property
        name="javax.persistence.jdbc.user"
        value="root" />

      <property
        name="javax.persistence.jdbc.password"
        value="root" />

      <property
        name="javax.persistence.jdbc.url"
        value="jdbc:mysql://localhost:3306/livraria" />
    </properties>
  </persistence-unit>

  <persistence-unit name="grana-pu" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>br.com.k19.jpa.grana.Despesa</class>
    <class>br.com.k19.jpa.grana.Receita</class>
    <exclude-unlisted-classes>true</exclude-unlisted-classes>
    <properties>
      <property
        name="javax.persistence.schema-generation.database.action"
        value="drop-and-create"/>

      <property
        name="javax.persistence.jdbc.driver"
        value="com.mysql.jdbc.Driver" />

      <property
        name="javax.persistence.jdbc.user"
        value="root" />

      <property
        name="javax.persistence.jdbc.password"
        value="root" />

      <property
        name="javax.persistence.jdbc.url"
        value="jdbc:mysql://localhost:3306/grana" />
    </properties>
  </persistence-unit>
```

```
</persistence>
```

Código XML 3.7: www.k19.com.br/exercicios/k12/03/048/persistence.xml

50 Crie uma classe chamada `Grana` no pacote `br.com.k19.jpa.grana`.

```
package br.com.k19.jpa.grana;

import java.util.Scanner;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class Grana {

  private static Scanner entrada = new Scanner(System.in);

  private static EntityManagerFactory factory;

  public static void main(String[] args) {
    /* Menu Principal */
    TelaMenu principal =
      new TelaMenu("Menu Principal");

    TelaMenu relatorios =
      new TelaMenu("Relatórios");
    TelaMenu receitas =
      new TelaMenu("Receitas");
    TelaMenu despesas =
      new TelaMenu("Despesas");
    TelaGeraDados geraDados =
      new TelaGeraDados(principal);
    TelaLimpaDados limpaDados =
      new TelaLimpaDados(principal);
    TelaSair sair =
      new TelaSair();

    principal.adicionaFilha(relatorios);
    principal.adicionaFilha(receitas);
    principal.adicionaFilha(despesas);
    principal.adicionaFilha(geraDados);
    principal.adicionaFilha(limpaDados);
    principal.adicionaFilha(sair);

    /* Relatórios */
    TelaRelatorioConsolidado consolidado =
      new TelaRelatorioConsolidado(relatorios);
    TelaRelatorioMensal mensal =
      new TelaRelatorioMensal(relatorios);
    TelaRelatorioPersonalizado personalizado =
      new TelaRelatorioPersonalizado(relatorios);

    relatorios.adicionaFilha(consolidado);
    relatorios.adicionaFilha(mensal);
    relatorios.adicionaFilha(personalizado);
    relatorios.adicionaFilha(principal);

    /* Receitas */
    TelaReceitaAdicionar receitaAdicionar =
      new TelaReceitaAdicionar(receitas);
    TelaMenu receitaConsultar =
      new TelaMenu("Consultar");

    receitas.adicionaFilha(receitaAdicionar);
    receitas.adicionaFilha(receitaConsultar);
    receitas.adicionaFilha(principal);
```

```
62
63      /* Receitas - Consultar */
64      TelaReceitaConsultaPeriodo receitaConsultaPeriodo =
65        new TelaReceitaConsultaPeriodo(receitaConsultar);
66      TelaReceitaConsultaRecentes receitaConsultaRecentes =
67        new TelaReceitaConsultaRecentes(receitaConsultar);
68
69      receitaConsultar.adicionaFilha(receitaConsultaPeriodo);
70      receitaConsultar.adicionaFilha(receitaConsultaRecentes);
71      receitaConsultar.adicionaFilha(receitas);
72
73      /* Despesas */
74      TelaDespesaAdicionar despesaAdicionar =
75        new TelaDespesaAdicionar(despesas);
76      TelaMenu despesaConsultar =
77        new TelaMenu("Consultar");
78
79      despesas.adicionaFilha(despesaAdicionar);
80      despesas.adicionaFilha(despesaConsultar);
81      despesas.adicionaFilha(principal);
82
83      /* Despesas - Consultar */
84      TelaDespesaConsultaPeriodo despesaConsultaPeriodo =
85        new TelaDespesaConsultaPeriodo(despesaConsultar);
86      TelaDespesaConsultaRecentes despesaConsultaRecentes =
87        new TelaDespesaConsultaRecentes(despesaConsultar);
88
89      despesaConsultar.adicionaFilha(despesaConsultaPeriodo);
90      despesaConsultar.adicionaFilha(despesaConsultaRecentes);
91      despesaConsultar.adicionaFilha(despesas);
92
93      Grana.factory = Persistence.createEntityManagerFactory("grana-pu");
94
95      Tela atual = principal;
96      while (atual != null) {
97        atual = atual.mostra();
98      }
99
100     Grana.factory.close();
101     Grana.entrada.close();
102   }
103
104   public static Scanner getEntrada() {
105     return entrada;
106   }
107
108   public static EntityManager getEntityManager() {
109     return Grana.factory.createEntityManager();
110   }
111 }
```

Código Java 3.118: www.k19.com.br/exercicios/k12/03/049/Grana.java

Execute a classe `Grana` e teste todas as funcionalidades.



Desafios

1 Acrescente no sistema de controle de finanças pessoais um relatório que informe a soma dos valores das receitas de um determinado tipo e outro que informe a soma dos valores das despesas de um determinado tipo.

WEB CONTAINER

CAPÍTULO 4



Necessidades de uma aplicação web

HTTP

Os usuários de uma aplicação web utilizam navegadores (*browsers*) para interagir com essa aplicação. A comunicação entre navegadores e uma aplicação web é realizada através de requisições e respostas definidas pelo protocolo HTTP. Dessa forma, os desenvolvedores de aplicação web devem estar preparados para trabalhar com o protocolo HTTP.

Acesso simultâneo

Além disso, na grande maioria dos casos, as aplicações web devem ser acessadas por diversos usuários ao mesmo tempo. Consequentemente, os desenvolvedores web devem criar ou utilizar algum mecanismo eficiente que permita esse tipo de acesso.

Conteúdo dinâmico

As páginas de uma aplicação web devem ser geradas dinamicamente. Por exemplo, quando um usuário de uma aplicação de email acessa a sua caixa de entrada, ele deseja ver todos os emails enviados até aquele momento. A página contendo a lista de emails deve ser gerada novamente toda vez que essa página for requisitada. Consequentemente, os desenvolvedores web devem criar ou utilizar algum mecanismo eficiente que permita que o conteúdo das páginas das aplicações web seja gerado dinamicamente.

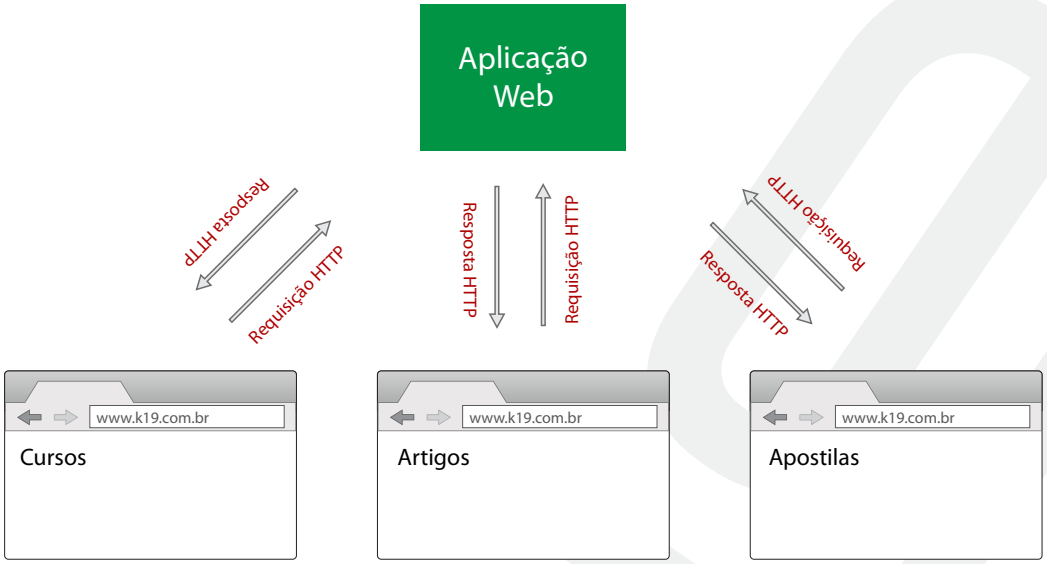


Figura 4.1: Necessidades de uma aplicação web

Solução

Resolver os três problemas apresentados tomaria boa parte do tempo de desenvolvimento, além de exigir conhecimentos técnicos extremamente específicos por parte dos desenvolvedores. Para facilitar o desenvolvimento de aplicações web, a plataforma Java oferece uma solução genérica que pode ser utilizada para desenvolver aplicações web. Conheceremos essa solução a seguir.

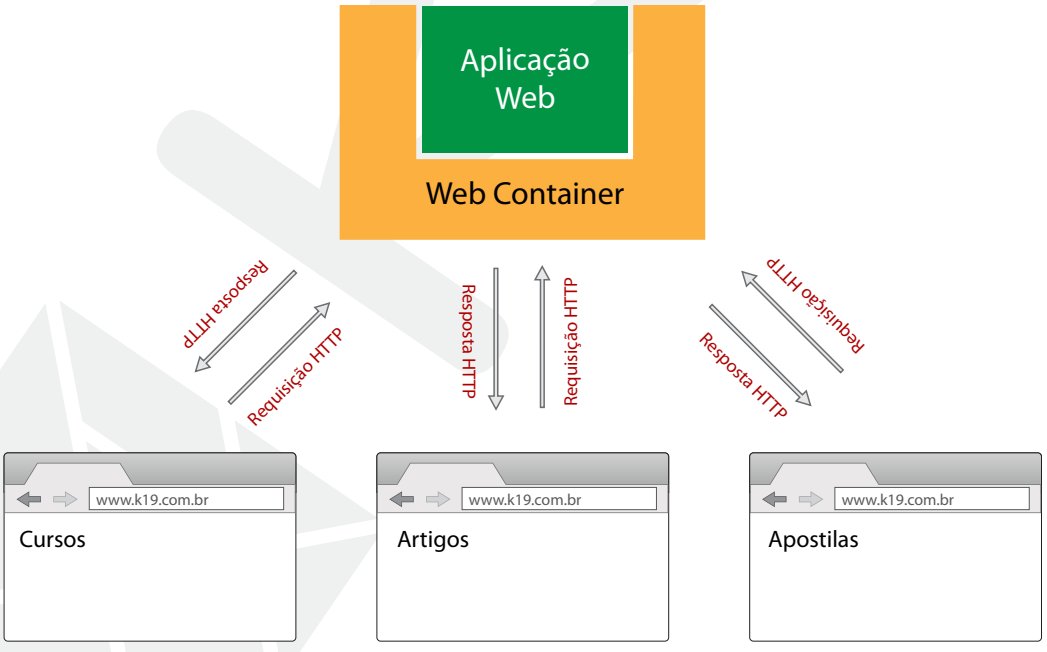


Figura 4.2: Web Container



Web Container

Uma aplicação web Java deve ser implantada em um **Web Container** para obter os recursos fundamentais que ela necessita. Um Web Container é responsável:

- Pelo envio e recebimento de mensagens HTTP.
- Por permitir que as aplicações sejam acessadas simultaneamente por vários usuários de uma maneira eficiente.
- Por permitir que as páginas de uma aplicação web sejam geradas dinamicamente.

Os dois Web Containers mais importantes do mercado são **Tomcat** e **Jetty**. Também podemos utilizar um servidor de aplicação Java EE como o **Wildfly**, **JBoss**, **GlassFish** ou **WebSphere**, pois eles possuem um Web Container internamente.



Servlet e Java EE

Como é comum na plataforma Java, foi definida uma especificação para padronizar a interface dos recursos oferecidos pelos Web Containers. Essa especificação é chamada **Servlet** e atualmente está na versão 3.1. Para consultá-la, acesse `https://jcp.org/en/jsr/detail?id=340`.

A especificação Servlet faz parte do **Java EE**. O Java EE é uma especificação que agrupa diversas outras especificações. Para consultá-la, acesse `https://jcp.org/en/jsr/detail?id=342`.

Apesar das especificações, os Web Containers possuem algumas diferenças nas configurações que devem ser realizadas pelos desenvolvedores. Dessa forma, não há 100% de portabilidade. Contudo, a maior parte das configurações e do modelo de programação é padronizado. Sendo assim, se você conhece bem um dos Web Containers, também conhece bastante dos outros.

Neste treinamento, optamos pela utilização do servidor de aplicação GlassFish 4.1 e o Wildfly 8.2. Esses servidores seguem a especificação Java EE 7 e portanto contém um Web Container. Mostraremos, a seguir, a sua instalação e configuração.



Exercícios de Fixação

1 Copie o arquivo **wildfly-8.2.0.Final.zip** da pasta **K19-Arquivos** para a sua Área de Trabalho. Depois, descompacte esse arquivo.



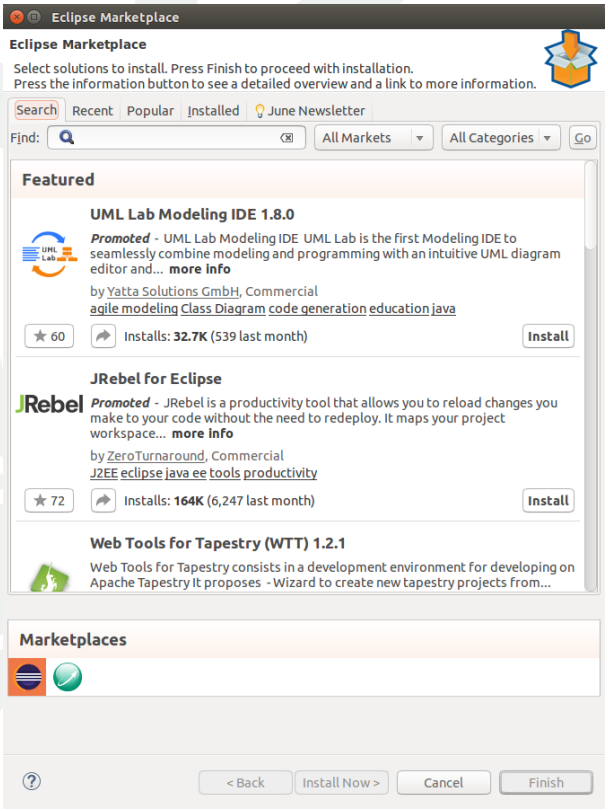
Importante
Você também pode obter o arquivo **wildfly-8.2.0.Final.zip** através do site da K19: `www.k19.com.br/arquivos`.

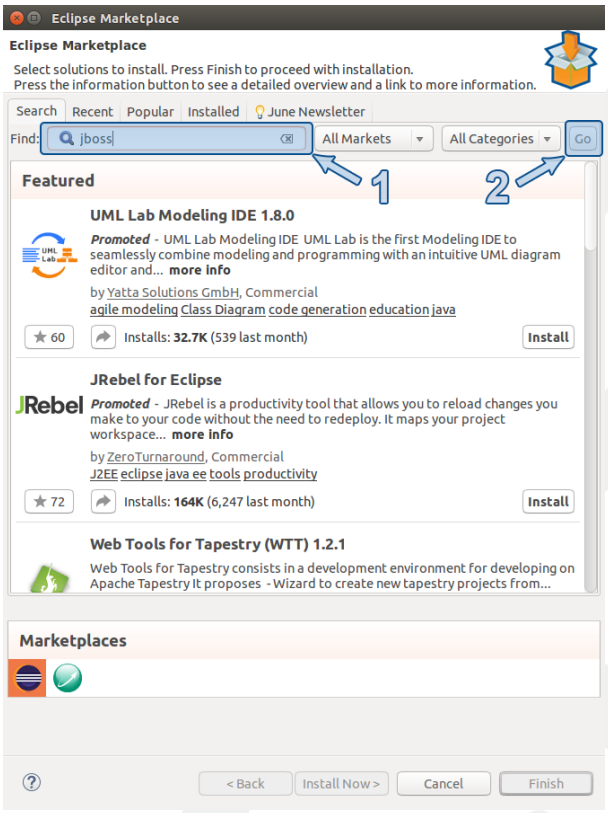
2 Copie o arquivo **glassfish-4.1.zip** da pasta **K19-Arquivos** para a sua Área de Trabalho. Depois, descompacte esse arquivo.

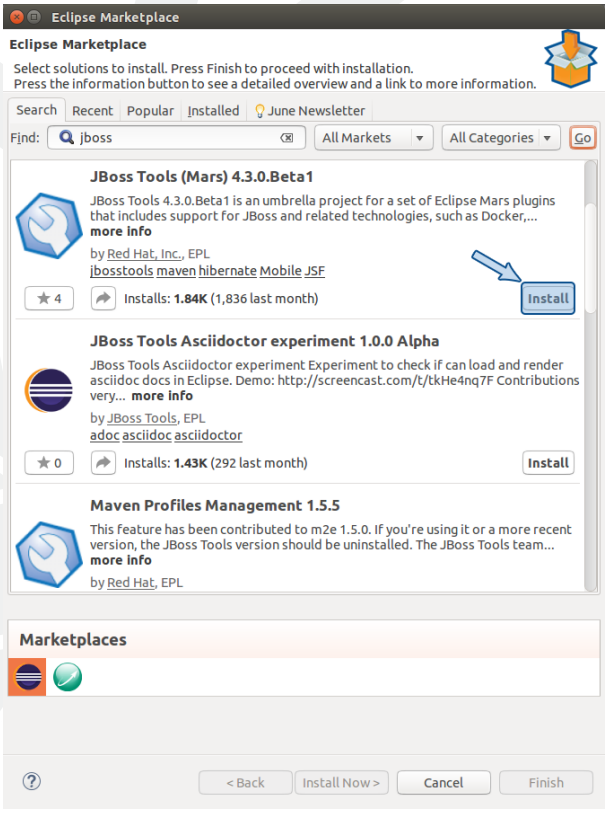


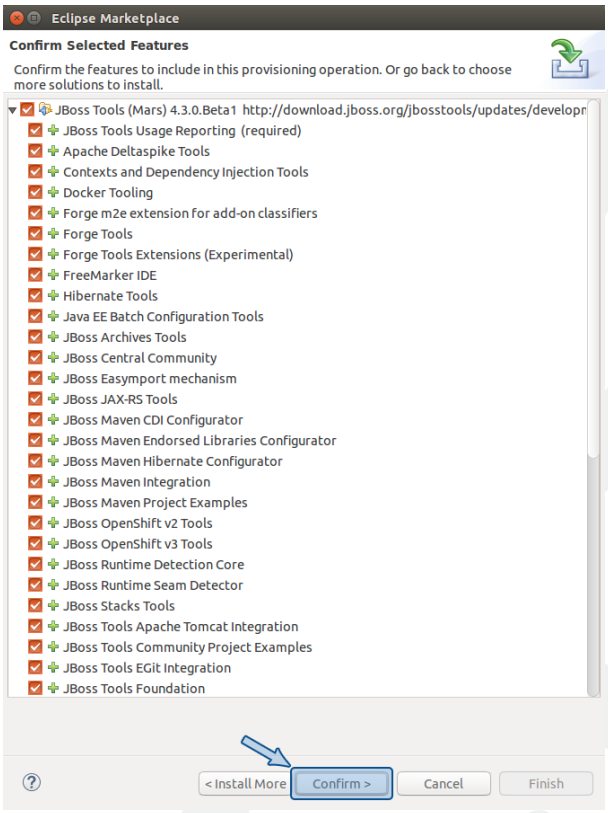
Importante
Você também pode obter o arquivo **glassfish-4.1.zip** através do site da K19: www.k19.com.br/arquivos.

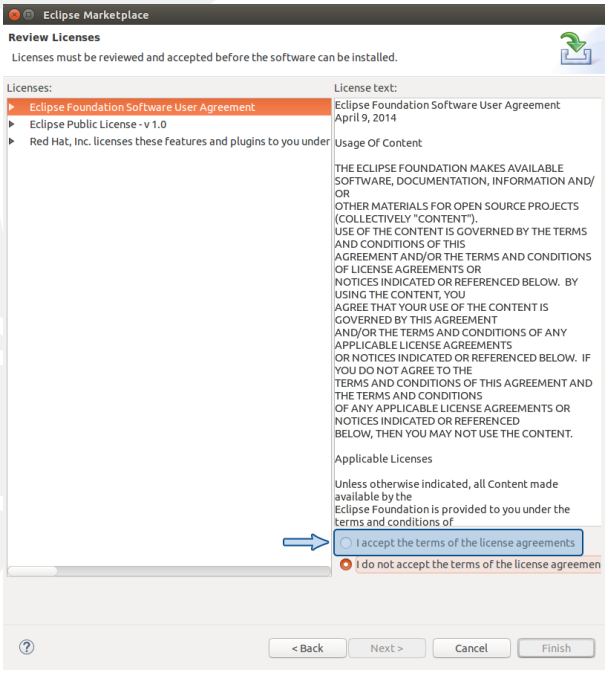
3 Utilizando o Eclipse Marketplace, adicione no Eclipse Mars o suporte ao Wildfly. Digite “CTRL + 3” para abrir o Quick Access. Em seguida, pesquise por “Eclipse Marketplace”.

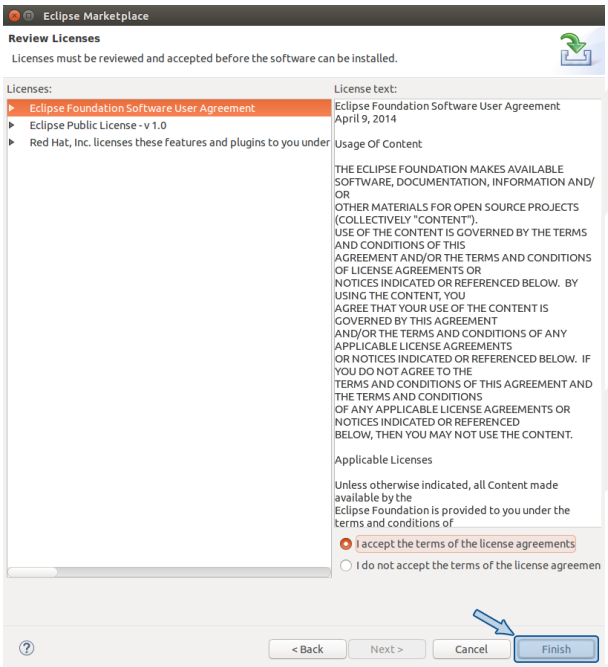


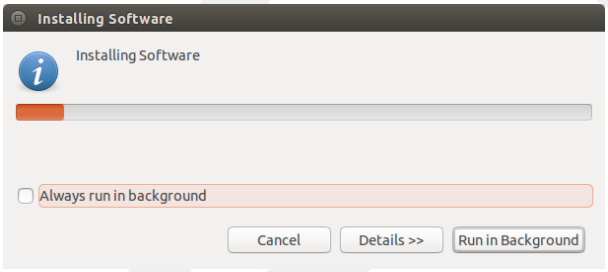

Eclipse Marketplace
Eclipse Marketplace
Select solutions to install. Press Finish to proceed with installation.
Press the information button to see a detailed overview and a link to more information.
Search
Recent
Popular
Installed
June Newsletter
Find:
jboss
All Markets
All Categories
Go
1
2
Featured
UML Lab Modeling IDE 1.8.0
Promoted - UML Lab Modeling IDE UML Lab is the first Modeling IDE to seamlessly combine modeling and programming with an intuitive UML diagram editor and... more info
by Yatta Solutions GmbH, Commercial
agile modeling Class Diagram code generation education java
60
Installs: 32.7K (539 last month)
Install
JRebel for Eclipse
Promoted - JRebel is a productivity tool that allows you to reload changes you make to your code without the need to redeploy. It maps your project workspace... more info
by ZeroTurnaround, Commercial
J2EE eclipse java ee tools productivity
72
Installs: 164K (6,247 last month)
Install
Web Tools for Tapestry (WTT) 1.2.1
Web Tools for Tapestry consists in a development environment for developing on Apache Tapestry It proposes - Wizard to create new tapestry projects from...
Marketplaces
< Back
Install Now >
Cancel
Finish

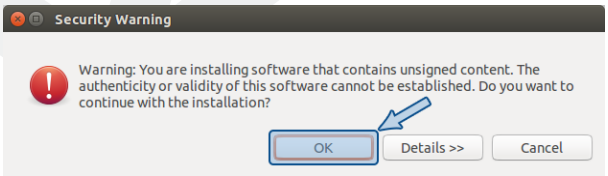

Eclipse Marketplace
Eclipse Marketplace
Select solutions to install. Press Finish to proceed with installation.
Press the information button to see a detailed overview and a link to more information.
Search
Recent
Popular
Installed
June Newsletter
Find:
jboss
All Markets
All Categories
Go
JBoss Tools (Mars) 4.3.0.Beta1
JBoss Tools 4.3.0.Beta1 is an umbrella project for a set of Eclipse Mars plugins that includes support for JBoss and related technologies, such as Docker,... more info
by Red Hat, Inc., EPL
jbosstools maven hibernate Mobile JSF
4
Installs: 1.84K (1,836 last month)
Install
JBoss Tools Asciidoctor experiment 1.0.0 Alpha
JBoss Tools Asciidoctor experiment Experiment to check if can load and render asciidoc docs in Eclipse. Demo: http://screencast.com/t/tkHe4nq7F Contributions very... more info
by JBoss Tools, EPL
adoc asciidoc asciidoctor
0
Installs: 1.43K (292 last month)
Install
Maven Profiles Management 1.5.5
This feature has been contributed to m2e 1.5.0. If you're using it or a more recent version, the JBoss Tools version should be uninstalled. The JBoss Tools team... more info
by Red Hat, EPL
Marketplaces
< Back
Install Now >
Cancel
Finish

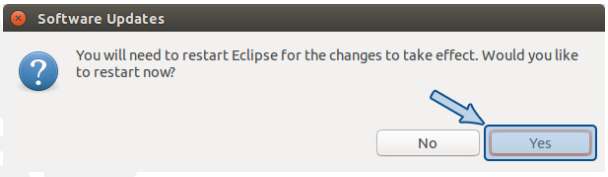

Eclipse Marketplace
Confirm Selected Features
Confirm the features to include in this provisioning operation. Or go back to choose more solutions to install.
JBoss Tools (Mars) 4.3.0.Beta1 http://download.jboss.org/jbosstools/updates/developr
JBoss Tools Usage Reporting (required)
Apache Deltaspike Tools
Contexts and Dependency Injection Tools
Docker Tooling
Forge m2e extension for add-on classifiers
Forge Tools
Forge Tools Extensions (Experimental)
FreeMarker IDE
Hibernate Tools
Java EE Batch Configuration Tools
JBoss Archives Tools
JBoss Central Community
JBoss Easymport mechanism
JBoss JAX-RS Tools
JBoss Maven CDI Configurator
JBoss Maven Endorsed Libraries Configurator
JBoss Maven Hibernate Configurator
JBoss Maven Integration
JBoss Maven Project Examples
JBoss OpenShift v2 Tools
JBoss OpenShift v3 Tools
JBoss Runtime Detection Core
JBoss Runtime Seam Detector
JBoss Stacks Tools
JBoss Tools Apache Tomcat Integration
JBoss Tools Community Project Examples
JBoss Tools EGit Integration
JBoss Tools Foundation
< Install More
Confirm >
Cancel
Finish

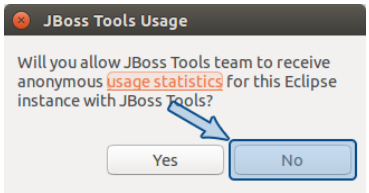

Eclipse Marketplace
Review Licenses
Licenses must be reviewed and accepted before the software can be installed.
Licenses:
Eclipse Foundation Software User Agreement
Eclipse Public License - v 1.0
Red Hat, Inc. licenses these features and plugins to you under
License text:
Eclipse Foundation Software User Agreement
April 9, 2014
Usage Of Content
THE ECLIPSE FOUNDATION MAKES AVAILABLE SOFTWARE, DOCUMENTATION, INFORMATION AND/OR
OTHER MATERIALS FOR OPEN SOURCE PROJECTS (COLLECTIVELY "CONTENT").
USE OF THE CONTENT IS GOVERNED BY THE TERMS AND CONDITIONS OF THIS
AGREEMENT AND/OR THE TERMS AND CONDITIONS OF LICENSE AGREEMENTS OR
NOTICES INDICATED OR REFERENCED BELOW. BY USING THE CONTENT, YOU
AGREE THAT YOUR USE OF THE CONTENT IS GOVERNED BY THIS AGREEMENT
AND/OR THE TERMS AND CONDITIONS OF ANY APPLICABLE LICENSE AGREEMENTS
OR NOTICES INDICATED OR REFERENCED BELOW. IF YOU DO NOT AGREE TO THE
TERMS AND CONDITIONS OF THIS AGREEMENT AND THE TERMS AND CONDITIONS
OF ANY APPLICABLE LICENSE AGREEMENTS OR NOTICES INDICATED OR REFERENCED
BELOW, THEN YOU MAY NOT USE THE CONTENT.
Applicable Licenses
Unless otherwise indicated, all Content made available by the
Eclipse Foundation is provided to you under the terms and conditions of
I accept the terms of the license agreements
I do not accept the terms of the license agreemen
< Back
Next >
Cancel
Finish


Eclipse Marketplace
Review Licenses
Licenses must be reviewed and accepted before the software can be installed.
Licenses:
Eclipse Foundation Software User Agreement
Eclipse Public License - v 1.0
Red Hat, Inc. licenses these features and plugins to you under
License text:
Eclipse Foundation Software User Agreement
April 9, 2014
Usage Of Content
THE ECLIPSE FOUNDATION MAKES AVAILABLE SOFTWARE, DOCUMENTATION, INFORMATION AND/OR
OTHER MATERIALS FOR OPEN SOURCE PROJECTS (COLLECTIVELY "CONTENT").
USE OF THE CONTENT IS GOVERNED BY THE TERMS AND CONDITIONS OF THIS
AGREEMENT AND/OR THE TERMS AND CONDITIONS OF LICENSE AGREEMENTS OR
NOTICES INDICATED OR REFERENCED BELOW. BY USING THE CONTENT, YOU
AGREE THAT YOUR USE OF THE CONTENT IS GOVERNED BY THIS AGREEMENT
AND/OR THE TERMS AND CONDITIONS OF ANY APPLICABLE LICENSE AGREEMENTS
OR NOTICES INDICATED OR REFERENCED BELOW. IF YOU DO NOT AGREE TO THE
TERMS AND CONDITIONS OF THIS AGREEMENT AND THE TERMS AND CONDITIONS
OF ANY APPLICABLE LICENSE AGREEMENTS OR NOTICES INDICATED OR REFERENCED
BELOW, THEN YOU MAY NOT USE THE CONTENT.
Applicable Licenses
Unless otherwise indicated, all Content made available by the
Eclipse Foundation is provided to you under the terms and conditions of
I accept the terms of the license agreements
I do not accept the terms of the license agreemen
< Back
Next >
Cancel
Finish

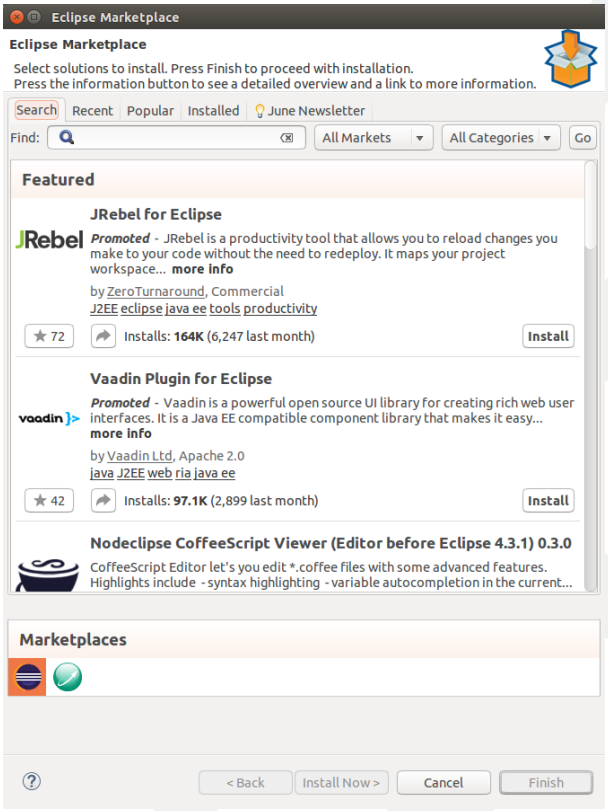

Installing Software
Installing Software
Always run in background
Cancel
Details >>
Run in Background

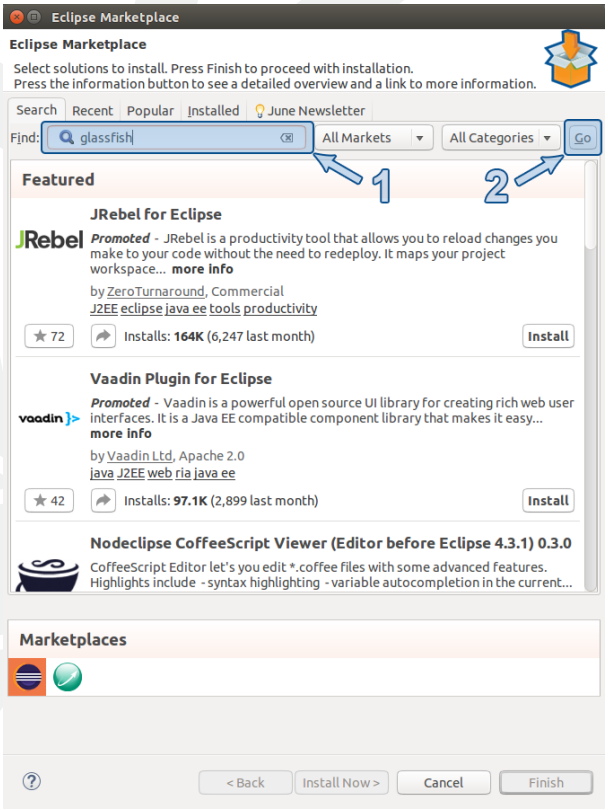

Security Warning
Warning: You are installing software that contains unsigned content. The authenticity or validity of this software cannot be established. Do you want to continue with the installation?
OK
Details >>
Cancel

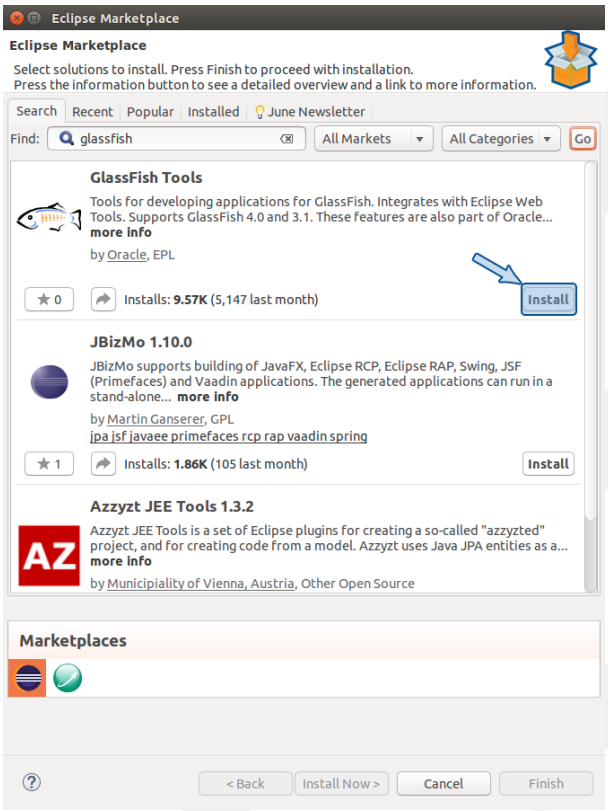

Software Updates
You will need to restart Eclipse for the changes to take effect. Would you like to restart now?
No
Yes

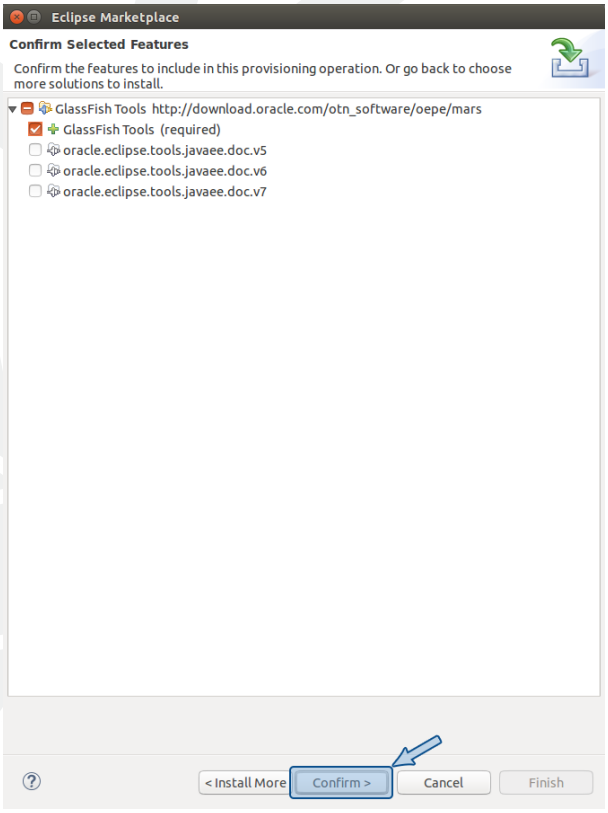

JBoss Tools Usage
Will you allow JBoss Tools team to receive anonymous usage statistics for this Eclipse instance with JBoss Tools?
Yes
No

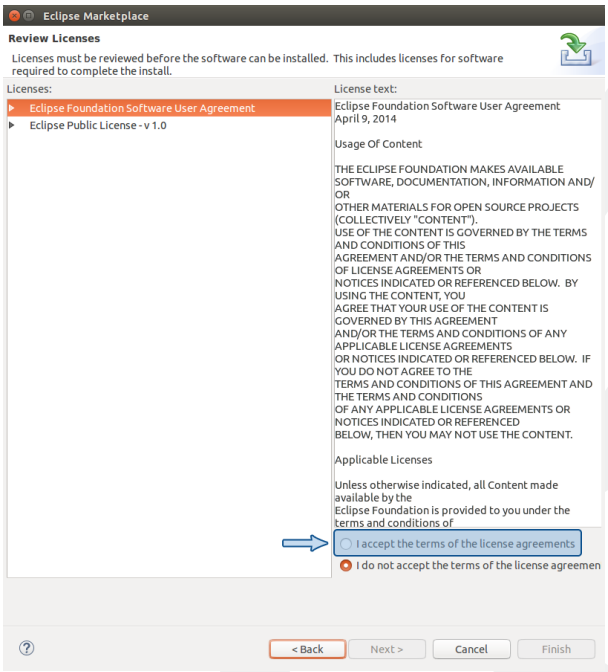
4 Utilizando o Eclipse Marketplace, adicione no Eclipse Mars o suporte ao GlassFish. Digite “CTRL + 3” para abrir o Quick Access. Em seguida, pesquise por “Eclipse Marketplace”.

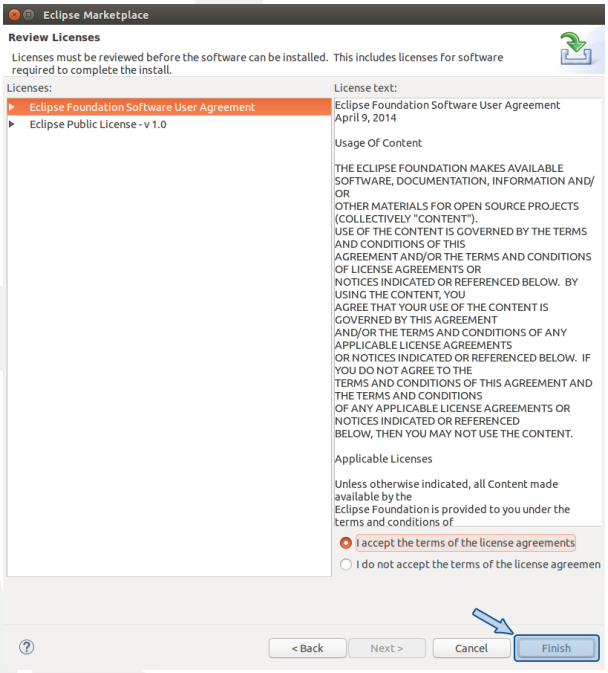


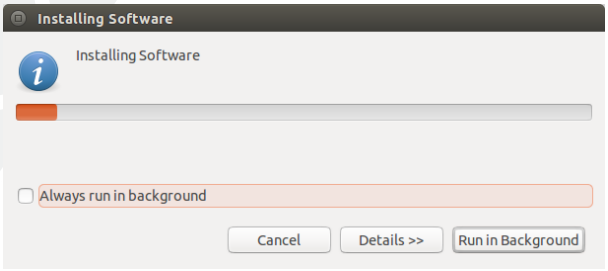


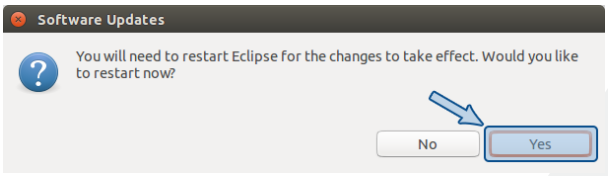

Eclipse Marketplace
Eclipse Marketplace
Select solutions to install. Press Finish to proceed with installation.
Press the information button to see a detailed overview and a link to more information.
Search
Recent
Popular
Installed
June Newsletter
Find:
glassfish
All Markets
All Categories
Go
GlassFish Tools
Tools for developing applications for GlassFish. Integrates with Eclipse Web Tools. Supports GlassFish 4.0 and 3.1. These features are also part of Oracle...
more info
by Oracle, EPL
0
Installs: 9.57K (5,147 last month)
Install
JBizMo 1.10.0
JBizMo supports building of JavaFX, Eclipse RCP, Eclipse RAP, Swing, JSF (Primefaces) and Vaadin applications. The generated applications can run in a stand-alone... more info
by Martin Ganserer, GPL
jpa jsf javaee primefaces rcp rap vaadin spring
1
Installs: 1.86K (105 last month)
Install
Azzyzt JEE Tools 1.3.2
Azzyzt JEE Tools is a set of Eclipse plugins for creating a so-called "azzyzted" project, and for creating code from a model. Azzyzt uses Java JPA entities as a...
more info
by Municipiality of Vienna, Austria, Other Open Source
Marketplaces
< Back
Install Now >
Cancel
Finish


Eclipse Marketplace
Confirm Selected Features
Confirm the features to include in this provisioning operation. Or go back to choose more solutions to install.
GlassFish Tools http://download.oracle.com/otn_software/oepe/mars
GlassFish Tools (required)
oracle.eclipse.tools.javaee.doc.v5
oracle.eclipse.tools.javaee.doc.v6
oracle.eclipse.tools.javaee.doc.v7
< Install More
Confirm >
Cancel
Finish

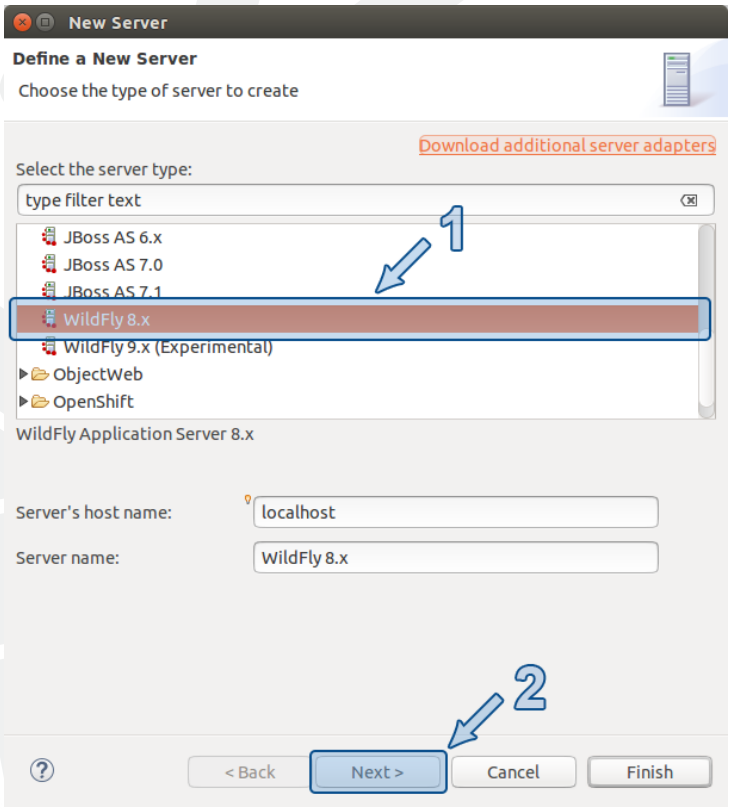

Eclipse Marketplace
Review Licenses
Licenses must be reviewed before the software can be installed. This includes licenses for software required to complete the install.
Licenses:
Eclipse Foundation Software User Agreement
Eclipse Public License - v 1.0
License text:
Eclipse Foundation Software User Agreement
April 9, 2014
Usage Of Content
THE ECLIPSE FOUNDATION MAKES AVAILABLE SOFTWARE, DOCUMENTATION, INFORMATION AND/OR
OTHER MATERIALS FOR OPEN SOURCE PROJECTS (COLLECTIVELY "CONTENT").
USE OF THE CONTENT IS GOVERNED BY THE TERMS AND CONDITIONS OF THIS
AGREEMENT AND/OR THE TERMS AND CONDITIONS OF LICENSE AGREEMENTS OR
NOTICES INDICATED OR REFERENCED BELOW. BY USING THE CONTENT, YOU
AGREE THAT YOUR USE OF THE CONTENT IS GOVERNED BY THIS AGREEMENT
AND/OR THE TERMS AND CONDITIONS OF ANY APPLICABLE LICENSE AGREEMENTS
OR NOTICES INDICATED OR REFERENCED BELOW. IF YOU DO NOT AGREE TO THE
TERMS AND CONDITIONS OF THIS AGREEMENT AND THE TERMS AND CONDITIONS
OF ANY APPLICABLE LICENSE AGREEMENTS OR NOTICES INDICATED OR REFERENCED
BELOW, THEN YOU MAY NOT USE THE CONTENT.
Applicable Licenses
Unless otherwise indicated, all Content made available by the
Eclipse Foundation is provided to you under the terms and conditions of
I accept the terms of the license agreements
I do not accept the terms of the license agreemen
< Back
Next >
Cancel
Finish

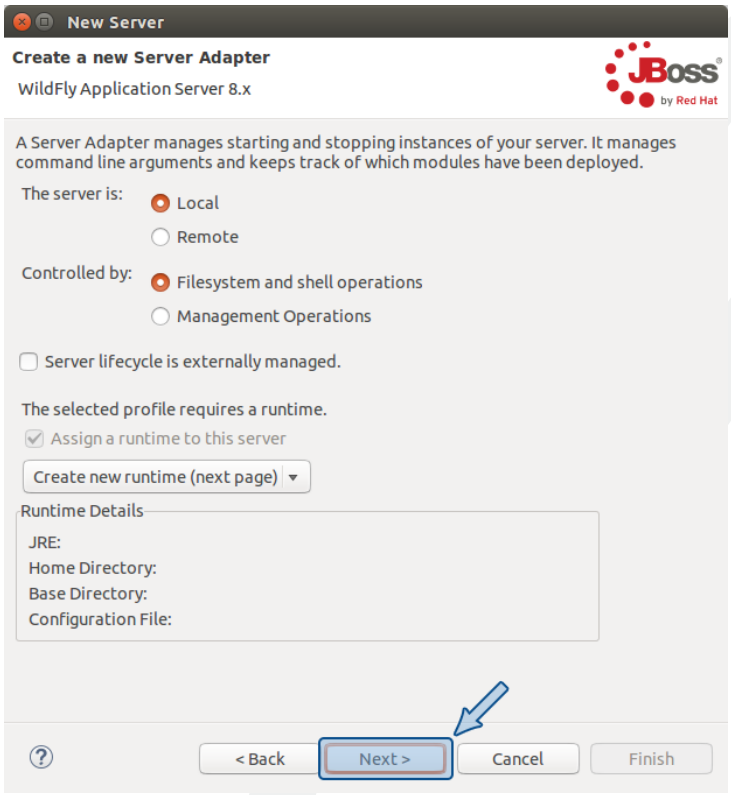

Eclipse Marketplace
Review Licenses
Licenses must be reviewed before the software can be installed. This includes licenses for software required to complete the install.
Licenses:
Eclipse Foundation Software User Agreement
Eclipse Public License - v 1.0
License text:
Eclipse Foundation Software User Agreement
April 9, 2014
Usage Of Content
THE ECLIPSE FOUNDATION MAKES AVAILABLE SOFTWARE, DOCUMENTATION, INFORMATION AND/OR
OTHER MATERIALS FOR OPEN SOURCE PROJECTS (COLLECTIVELY "CONTENT").
USE OF THE CONTENT IS GOVERNED BY THE TERMS AND CONDITIONS OF THIS
AGREEMENT AND/OR THE TERMS AND CONDITIONS OF LICENSE AGREEMENTS OR
NOTICES INDICATED OR REFERENCED BELOW. BY USING THE CONTENT, YOU
AGREE THAT YOUR USE OF THE CONTENT IS GOVERNED BY THIS AGREEMENT
AND/OR THE TERMS AND CONDITIONS OF ANY APPLICABLE LICENSE AGREEMENTS
OR NOTICES INDICATED OR REFERENCED BELOW. IF YOU DO NOT AGREE TO THE
TERMS AND CONDITIONS OF THIS AGREEMENT AND THE TERMS AND CONDITIONS
OF ANY APPLICABLE LICENSE AGREEMENTS OR NOTICES INDICATED OR REFERENCED
BELOW, THEN YOU MAY NOT USE THE CONTENT.
Applicable Licenses
Unless otherwise indicated, all Content made available by the
Eclipse Foundation is provided to you under the terms and conditions of
I accept the terms of the license agreements
I do not accept the terms of the license agreemen
< Back
Next >
Cancel
Finish


Installing Software
Installing Software
Always run in background
Cancel
Details >>
Run in Background



5 Configure o Wildfly no Eclipse Mars. Digite “CTRL + 3” para abrir o Quick Access. Em seguida, pesquise por “Define a new server”.

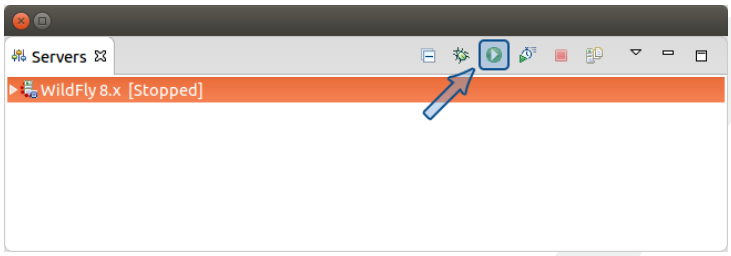


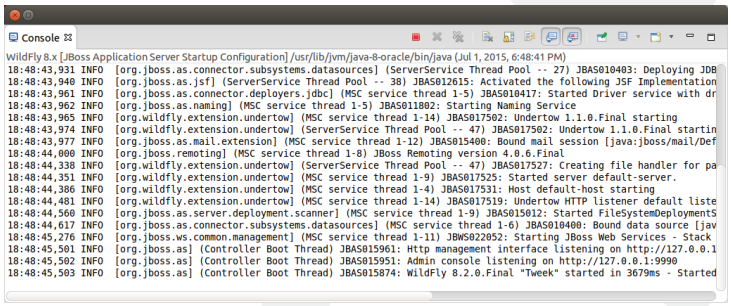

New Server
Create a new Server Adapter
WildFly Application Server 8.x
JBoss by Red Hat
A Server Adapter manages starting and stopping instances of your server. It manages command line arguments and keeps track of which modules have been deployed.
The server is:
Local
Remote
Controlled by:
Filesystem and shell operations
Management Operations
Server lifecycle is externally managed.
The selected profile requires a runtime.
Assign a runtime to this server
Create new runtime (next page)
Runtime Details
JRE:
Home Directory:
Base Directory:
Configuration File:
< Back
Next >
Cancel
Finish


New Server
JBoss Runtime
WildFly Application Server 8.x
JBoss by Red Hat
A JBoss Server runtime references a JBoss installation directory.
It can be used to set up classpaths for projects which depend on this runtime,
as well as by a "server" which will be able to start and stop instances of JBoss.
Name
WildFly 8.x Runtime
Home Directory
Download and install runtime...
Browse...
Runtime JRE
Execution Environment:
JavaSE-1.7
Environments...
Alternate JRE:
java-8-oracle
Installed JREs...
Server base directory:
standalone
Browse...
Configuration file:
standalone.xml
Browse...
< Back
Next >
Cancel
Finish


New Server
JBoss Runtime
WildFly Application Server 8.x
A JBoss Server runtime references a JBoss installation directory.
It can be used to set up classpaths for projects which depend on this runtime,
as well as by a "server" which will be able to start and stop instances of JBoss.
Name
WildFly 8.x Runtime
Home Directory
Download and install runtime...
/home/cosen/Desktop/wildfly-8.2.0.Final
Browse...
Runtime JRE
Execution Environment:
JavaSE-1.7
Environments...
Alternate JRE:
java-8-oracle
Installed JREs...
Server base directory:
standalone
Browse...
Configuration file:
standalone.xml
Browse...
< Back
Next >
Cancel
Finish


New Server
JBoss Runtime
WildFly Application Server 8.x
A JBoss Server runtime references a JBoss installation directory.
It can be used to set up classpaths for projects which depend on this runtime,
as well as by a "server" which will be able to start and stop instances of JBoss.
Name
WildFly 8.x Runtime
Home Directory
Download and install runtime...
/home/cosen/Desktop/wildfly-8.2.0.Final
Browse...
Runtime JRE
Execution Environment:
JavaSE-1.7
Environments...
Alternate JRE:
java-8-oracle
Installed JREs...
Server base directory:
standalone
Browse...
Configuration file:
standalone.xml
Browse...
< Back
Next >
Cancel
Finish

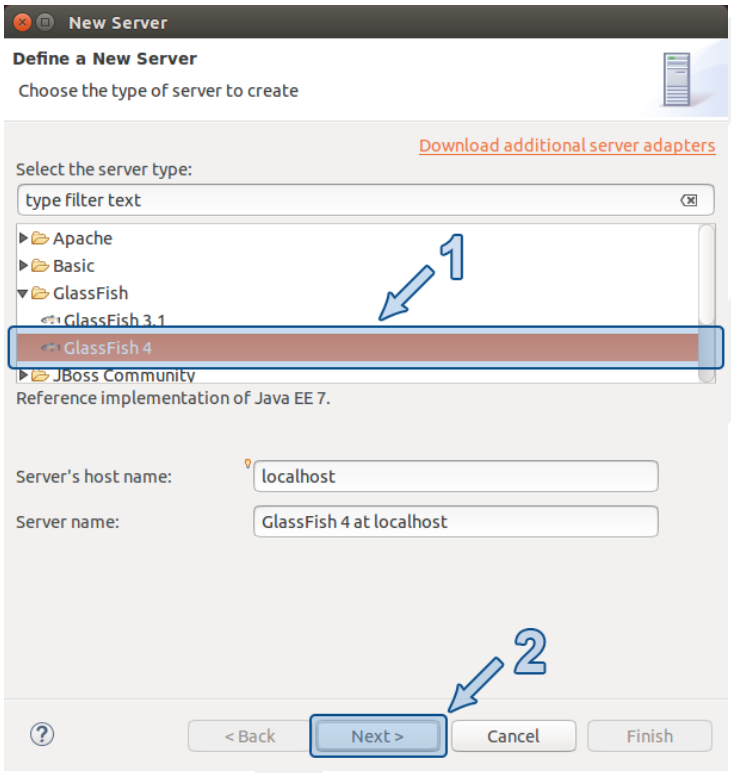


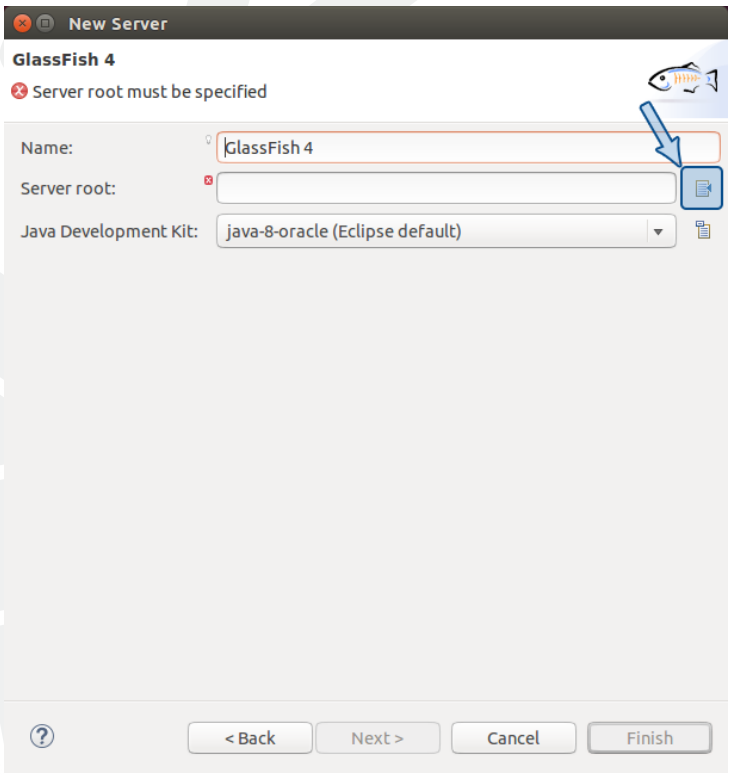


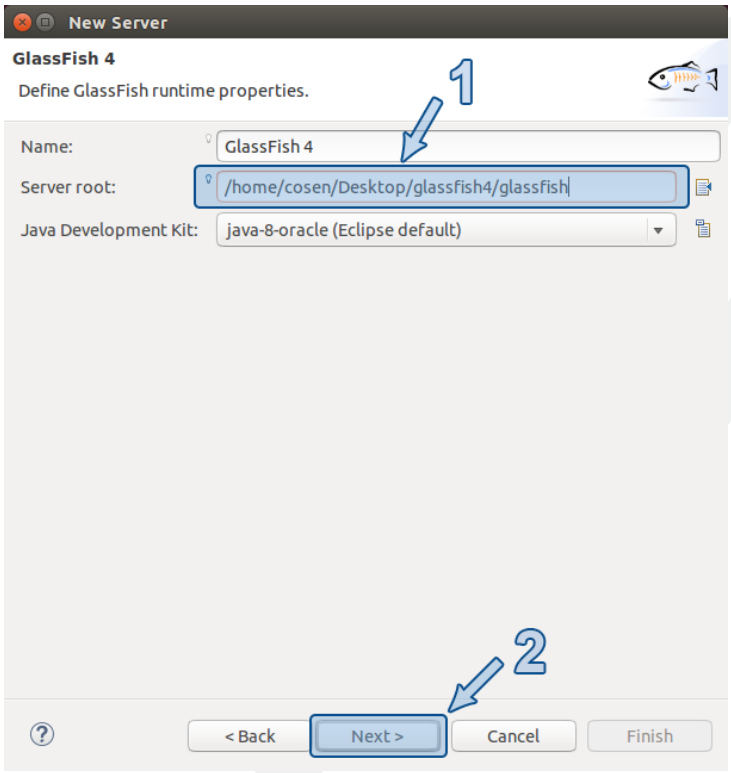


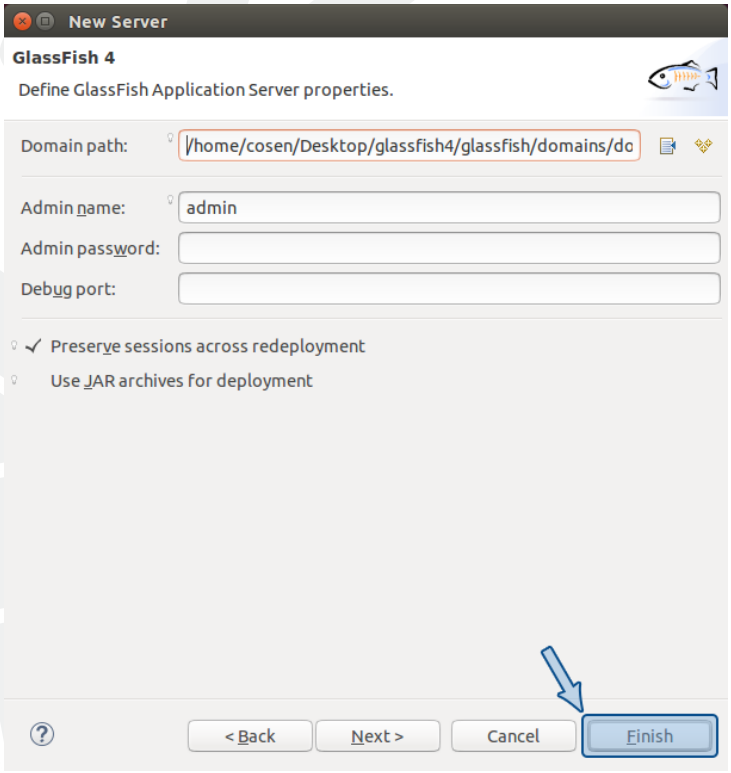
6 Na visão Servers, com um clique duplo sobre o Wildfly abra as configurações desse servidor. No menu “Publishing”, marque a opção “Automatically publish when resources change”. Por fim, digite “CTRL + S” para salvar as alterações.

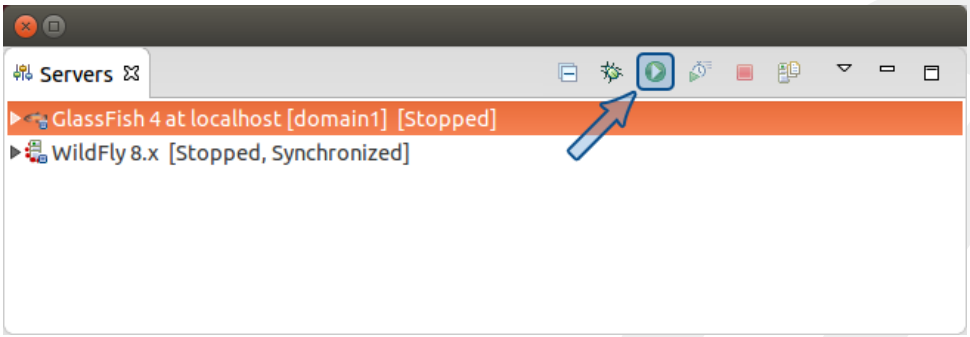
7 Pare o Wildfly e configure o GlassFish no Eclipse Mars. Digite “CTRL + 3” para abrir o Quick Access. Em seguida, pesquise por “Define a new server”.

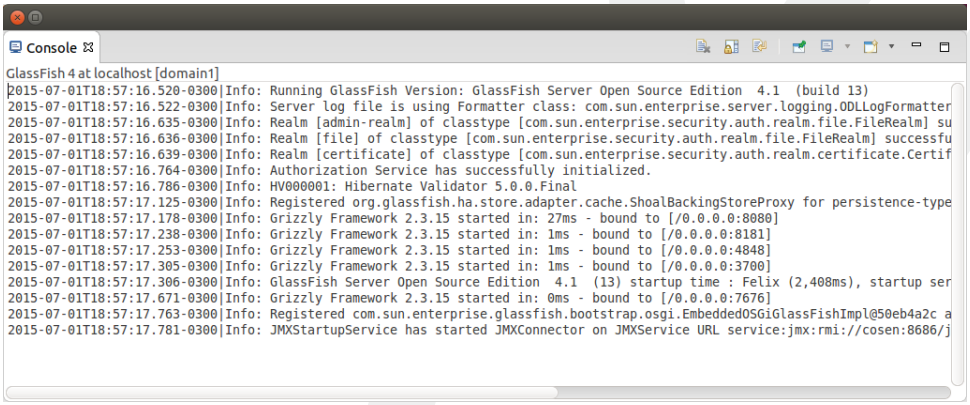

New Server
Define a New Server
Choose the type of server to create
Download additional server adapters
Select the server type:
type filter text
Apache
Basic
GlassFish
GlassFish 3.1
GlassFish 4
JBoss Community
Reference implementation of Java EE 7.
Server's host name:
localhost
Server name:
GlassFish 4 at localhost
1
2
< Back
Next >
Cancel
Finish

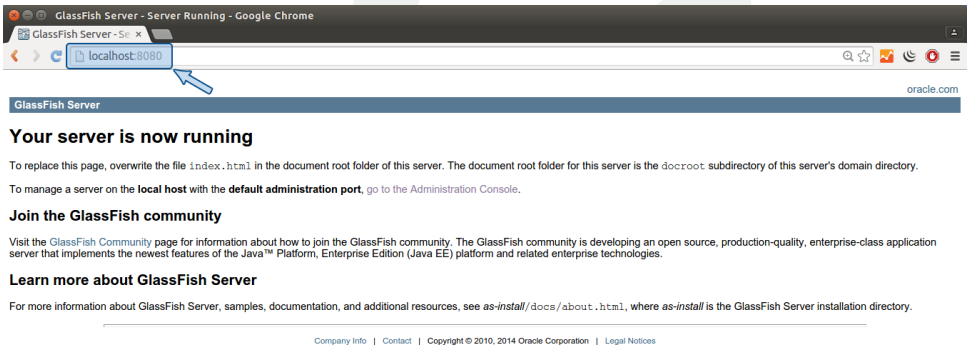

New Server
GlassFish 4
Server root must be specified
Name:
GlassFish 4
Server root:
Java Development Kit:
java-8-oracle (Eclipse default)
< Back
Next >
Cancel
Finish


New Server
GlassFish 4
Define GlassFish runtime properties.
1
Name:
GlassFish 4
Server root:
/home/cosen/Desktop/glassfish4/glassfish
Java Development Kit:
java-8-oracle (Eclipse default)
2
< Back
Next >
Cancel
Finish


New Server
GlassFish 4
Define GlassFish Application Server properties.
Domain path:
/home/cosen/Desktop/glassfish4/glassfish/domains/do
Admin name:
admin
Admin password:
Debug port:
Preserve sessions across redeployment
Use JAR archives for deployment
< Back
Next >
Cancel
Finish



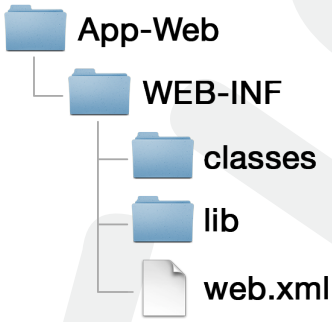




8 Na visão Servers, com um clique duplo sobre o GlassFish abra as configurações desse servidor. No menu "Publishing", marque a opção "Automatically publish when resources change". Por fim, digite "CTRL + S" para salvar as alterações.

Aplicação Web Java

Para que uma aplicação web Java possa ser implantada em um Web Container, a estrutura de pastas precisa seguir algumas regras.



A pasta `K19-App` é a raiz da aplicação. Ela pode ter qualquer nome. A pasta `WEB-INF` deve ser criada dentro da pasta raiz. O conteúdo da pasta `WEB-INF` não pode ser acessado diretamente pelos usuários da aplicação. Por outro lado, os arquivos dentro da pasta raiz da aplicação mas fora da pasta `WEB-INF` podem ser acessados diretamente através de um navegador.

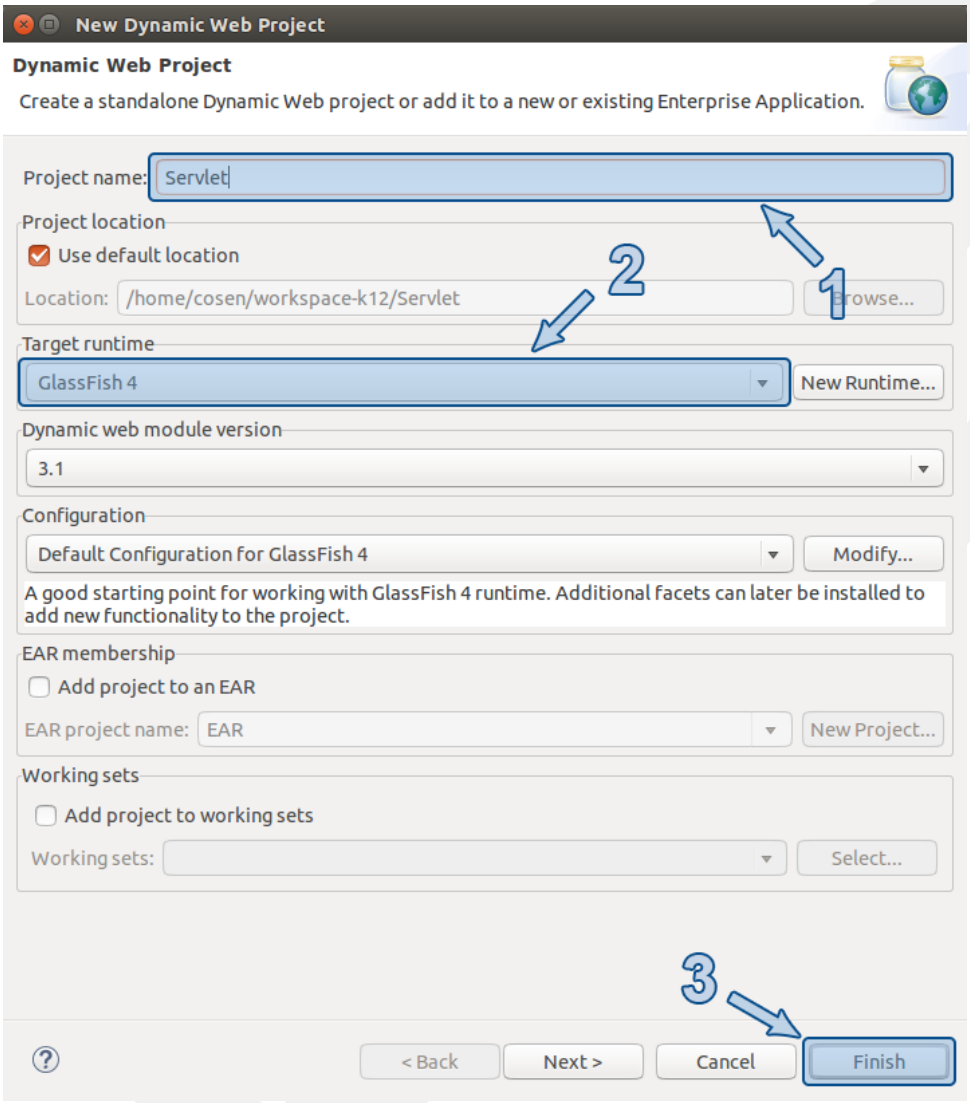
As pastas `classes` e `lib` devem ser criadas dentro da pasta `WEB-INF`. O código compilado da aplicação deve ser salvo na pasta `classes`. Os jar's das bibliotecas extras que serão utilizadas devem ser colocados na pasta `lib`. O arquivo `web.xml` contém configurações do Web Container e deve ser criado na pasta `WEB-INF`.

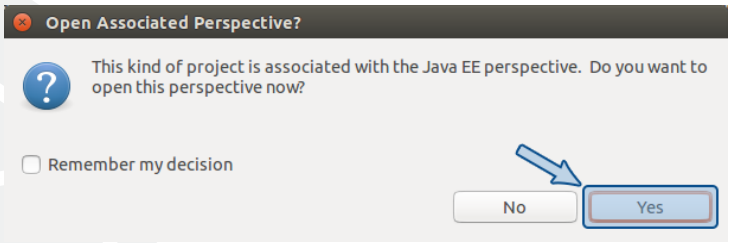
Em geral, as IDEs criam toda a estrutura de pastas exigidas pelos Web Containers. Então, na prática, não temos o trabalho de criar esses diretórios manualmente.



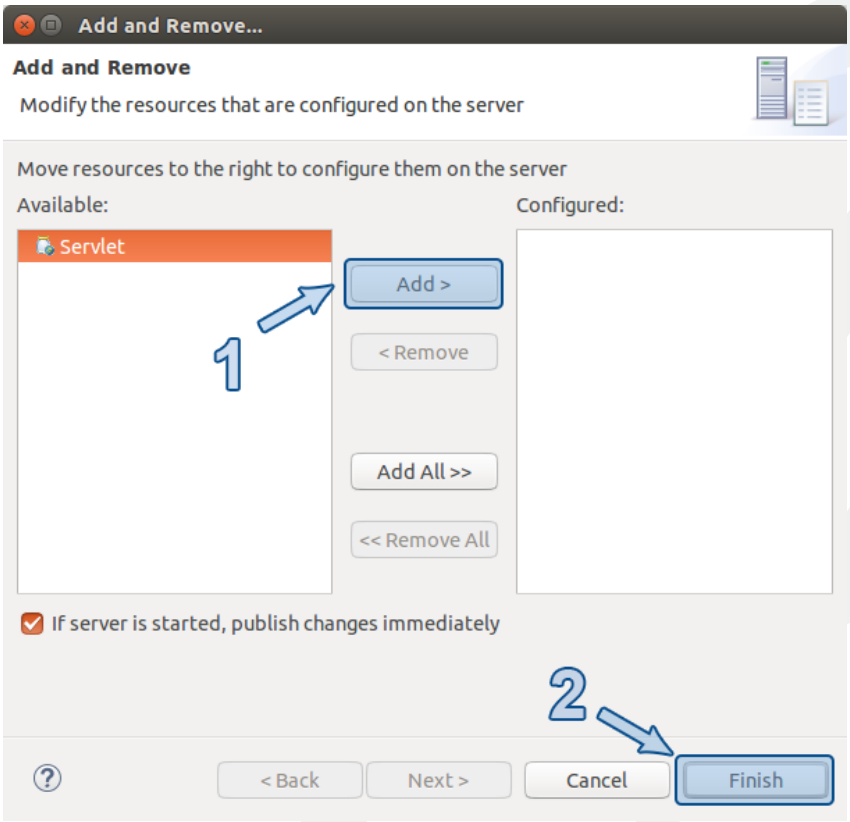
Exercícios de Fixação

9 Crie um projeto web chamado **Servlet**. Digite "CTRL + 3" e pesquise por "Dynamic Web Project". Selecione a opção correspondente e siga as imagens abaixo.





10 Implante o projeto **Servlet** no GlassFish. Na visão Servers, clique com o botão direito do mouse sobre o GlassFish, selecione a opção "Add and Remove..." e siga a imagem abaixo.



Processando requisições

Após implantar a nossa aplicação web Java em um Web Container, as requisições e respostas HTTP já estão sendo processadas pelo Web Container, que também já permite o acesso de múltiplos usuários à nossa aplicação.

Em seguida, devemos definir como o conteúdo das páginas da aplicação é gerado. Para isso, podemos criar uma **Servlet**.

Servlet

Para criar uma Servlet, podemos seguir os seguintes passos:

1. Criar uma classe.
2. Herdar da classe `javax.servlet.http.HttpServlet`.
3. Reescrever o método `service()`.
4. Utilizar a anotação `@WebServlet` para definir a url que será utilizada para acessar a Servlet. Essa anotação existe após a especificação Servlet 3.0. Antes, essa configuração era realizada através do arquivo `web.xml`.

```java
@WebServlet("/ola-mundo")
public class OlaMundoServlet extends HttpServlet {

  @Override
  protected void service(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    // Lógica para processar as regras de negócio e gerar conteúdo
  }
}
```

Código Java 4.1: OlaMundoServlet.java

O método `service()` é executado toda vez que uma requisição HTTP é realizada para a url definida na anotação `@WebServlet`. Esse método recebe dois parâmetros. O primeiro é a referência do objeto da classe `HttpServletRequest` que armazena todos os dados da requisição. O segundo parâmetro é a referência do objeto da classe `HttpServletResponse` que armazenará o conteúdo gerado pela Servlet.

Inserindo conteúdo na resposta

Para inserir conteúdo texto na resposta HTTP que será enviada para o navegador do usuário, devemos utilizar os métodos `getWriter()` e `println()`. Em geral, o conteúdo inserido na resposta HTTP é texto HTML. Veja o código abaixo.

```java
@WebServlet("/ola-mundo")
public class OlaMundoServlet extends HttpServlet {

  @Override
  protected void service(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    PrintWriter writer = resp.getWriter();
    writer.println("<html><body><h1>Olá Mundo</h1></body></html>");
  }
}
```

Código Java 4.2: OlaMundoServlet.java



Exercícios de Fixação

11 Crie um pacote chamado **br.com.k19.servlets** no projeto **Servlet**.

12 Crie uma classe chamada **OlaMundoServlet** no pacote **br.com.k19.servlets**.

```java
package br.com.k19.servlets;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/ola-mundo")
public class OlaMundoServlet extends HttpServlet {
```

```
13
14   private static final long serialVersionUID = 1L;
15
16   @Override
17   protected void service(HttpServletRequest req, HttpServletResponse resp)
18       throws ServletException, IOException {
19     PrintWriter writer = resp.getWriter();
20     writer.println("<html><body><h1>Olá Mundo</h1></body></html>");
21   }
22 }
```

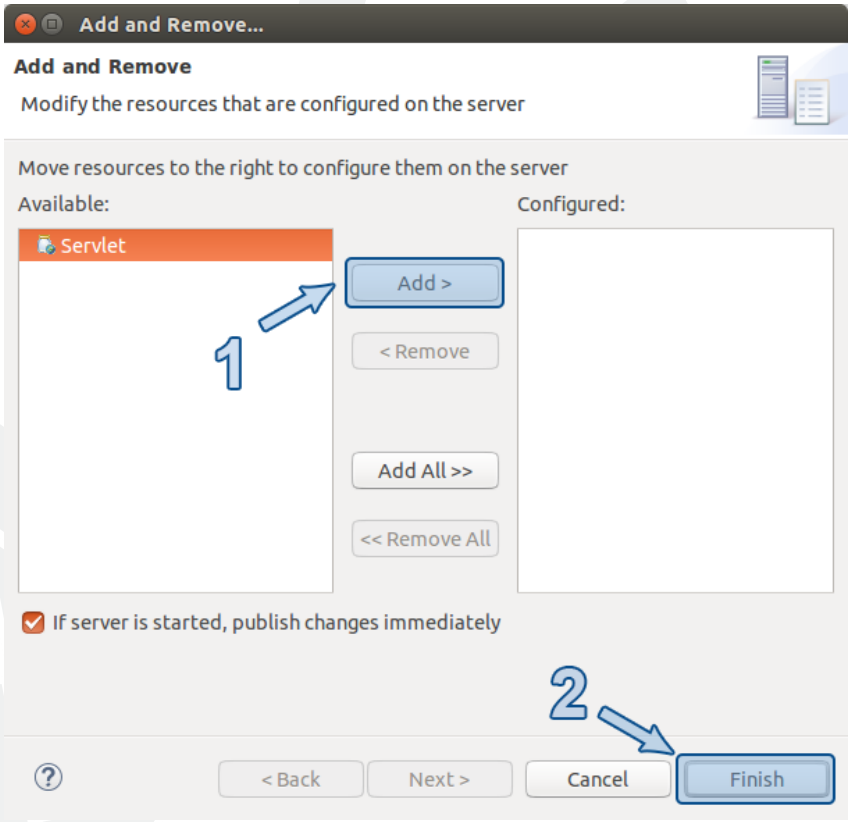
Código Java 4.3: www.k19.com.br/exercicios/k12/04/012/OlaMundoServlet.java

13 Verifique o funcionamento da OlaMundoServlet acessando a url abaixo em um navegador.

```
http://localhost:8080/Servlet/ola-mundo
```

14 Na visão Servers, clique com o botão direito do mouse sobre o GlassFish e selecione a opção "Stop".

15 Implante o projeto **Servlet** no Wildfly. Na visão Servers, clique com o botão direito do mouse sobre o Wildfly, selecione a opção "Add and Remove..." e siga a imagem abaixo.



16 Na visão Servers, clique com o botão direito do mouse sobre o Wildfly e selecione a opção "Start".

17 Verifique o funcionamento da OlaMundoServlet acessando a url abaixo em um navegador.

```
http://localhost:8080/Servlet/ola-mundo
```



Frameworks

Hoje em dia, é improvável que uma empresa decida começar um projeto utilizando diretamente Servlets, pois a produtividade seria pequena e a manutenção muito custosa. Vários frameworks foram criados para facilitar o desenvolvimento e a manutenção de aplicações web. Apesar de serem baseados em Servlets, esses frameworks oferecem diversos recursos adicionais para as aplicações. Eis uma lista dos principais frameworks para aplicações web Java:

- JSF
- Struts 1.x
- Struts 2.x
- Spring MVC

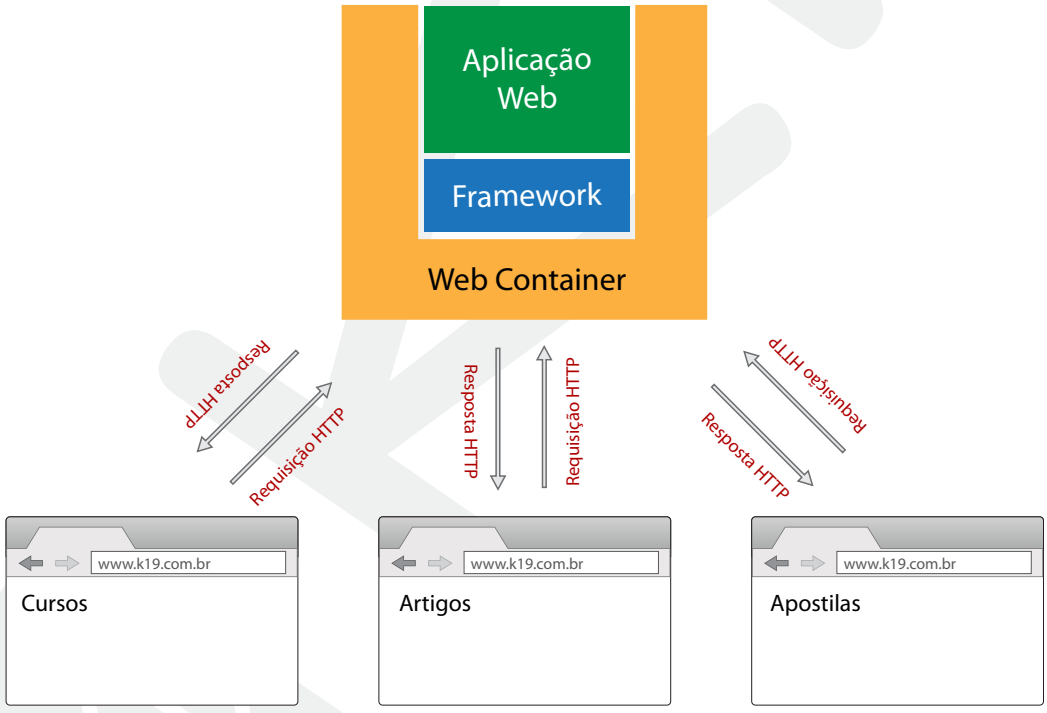


Figura 4.3: Framework para aplicações web

Nos próximos capítulos, mostraremos o funcionamento e explicaremos os conceitos relacionados ao framework JSF.

Visão Geral do JSF 2.2

CAPÍTULO 5

Atualmente, o principal framework para desenvolvimento de aplicações web em Java é o JSF (*Java Server Faces*). A especificação desse framework pode ser obtida em https://jcp.org/en/jsr/detail?id=344. Além disso, recomendamos a consulta ao javadoc da API do JSF que pode ser obtido no mesmo link. O JSF é fortemente baseado nos padrões *MVC* e *Front Controller*.



MVC e Front Controller

O MVC (*model-view-controller*) é um padrão de arquitetura que tem por objetivo isolar a lógica de negócio da lógica de apresentação de uma aplicação.

Esse padrão (ou alguma variação) é amplamente adotado nas principais plataformas de desenvolvimento atuais. Em particular, ele é bastante utilizado na plataforma Java.

O padrão MVC divide uma aplicação em três tipos de componentes: modelo, visão e controlador.

Modelo: encapsula os dados e as funcionalidades da aplicação.

Visão: é responsável pela exibição de informações, cujos dados são obtidos do modelo.

Controlador: recebe as requisições do usuário e aciona o modelo e/ou a visão.

Para mais detalhes sobre o padrão MVC, uma boa referência é o livro *Pattern-Oriented Software Architecture Volume 1: A System of Patterns* (editora Wiley, 1996) dos autores Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, Michael Stal e Michael Stal.

No padrão *Front Controller*, todas as requisições do usuário são recebidas pelo mesmo componente. Dessa forma, tarefas que devem ser realizadas em todas as requisições podem ser implementadas por esse componente. Isso evita a repetição de código e facilita a manutenção do sistema.

Para mais informações sobre esse padrão, consulte, por exemplo, o livro *Core J2EE Patterns: Best Practices and Design Strategies* (editora Prentice Hall, 2003, segunda edição) dos autores Deepak Alur, Dan Malks e John Crupi.



Configurando uma aplicação JSF

Uma aplicação JSF deve respeitar a estrutura geral de uma aplicação web Java que foi descrita no capítulo anterior. Essa estrutura é defina pela especificação *Servlet* que está disponível em https://jcp.org/en/jsr/detail?id=340.

web.xml

A Faces Servlet deve ser configurada no arquivo `WEB-INF/web.xml`, indicando a classe que a implementa e o padrão de url que será associado a essa servlet.

Por exemplo, na configuração abaixo, todas as requisições cujas urls possuam o sufixo *.xhtml* serão processadas pela Faces Servlet.

```
<servlet>
  <servlet-name>Faces Servlet</servlet-name>
  <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>Faces Servlet</servlet-name>
  <url-pattern>*.xhtml</url-pattern>
</servlet-mapping>
```

Código XML 5.1: web.xml

faces-config.xml

Devemos adicionar um arquivo chamado `faces-config.xml` no diretório `WEB-INF`. Nesse arquivo, podemos alterar diversas configurações do JSF. Mas, a princípio, podemos deixá-lo apenas com a tag `faces-config`.

```
<?xml version="1.0" encoding="UTF-8"?>
<faces-config
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
      http://xmlns.jcp.org/xml/ns/javaee/web-facesconfig_2_2.xsd"
    version="2.2">

</faces-config>
```

Código XML 5.2: faces-config.xml

Bibliotecas

Para utilizar os recursos do JSF, é necessário que a aplicação possua uma implementação JSF. Essa implementação pode ser adicionada manualmente no diretório `WEB-INF/lib` da aplicação.

Se um servidor de aplicação Java EE for utilizado, a inclusão de uma implementação JSF manualmente não é necessária, já que esse tipo de servidor possui, por padrão, uma implementação de JSF.



Managed Beans

Os managed beans são objetos fundamentais de uma aplicação JSF. Suas principais tarefas são:

1. Fornecer dados que serão exibidos nas telas.
2. Receber os dados enviados nas requisições.
3. Executar tarefas de acordo com as ações dos usuários.

Criando um Managed Bean

Um managed bean pode ser definido de duas maneiras. A primeira maneira é criar uma classe Java e registrá-la no arquivo `faces-config.xml`. Veja o exemplo abaixo.

```
1 package br.com.k19;
2
3 public class TesteBean {
4   ...
5 }
```

Código Java 5.1: TesteBean.java

```
1 ...
2 <managed-bean>
3   <managed-bean-name>testeBean</managed-bean-name>
4   <managed-bean-class>br.com.k19.TesteBean</managed-bean-class>
5   <managed-bean-scope>request</managed-bean-scope>
6 </managed-bean>
7 ...
```

Código XML 5.3: faces-config.xml

No registro de um managed bean, devemos definir o nome, a classe e o escopo do managed bean. O nome será utilizado para acessar esse managed bean nas páginas da aplicação. O escopo será explicado em detalhes no Capítulo 9.

A segunda forma é criar uma classe Java com a anotação `@ManagedBean` do pacote `javax.faces.bean`. Essa anotação só pode ser utilizada a partir da versão 2 do JSF. Observe o exemplo abaixo.

```
1 @ManagedBean
2 public class TesteBean {
3   ...
4 }
```

Código Java 5.2: TesteBean.java

Utilizando a anotação `@ManagedBean`, por padrão, o JSF assumirá que o nome do managed bean é o nome da classe com a primeira letra minúscula. Para o exemplo acima, o nome padrão do managed bean é `testeBean`. Além disso, o escopo *request* será assumido como padrão.

Propriedades

Considere o seguinte managed bean.

```
1 @ManagedBean
2 public class TesteBean {
3   private int numero;
4 }
```

Código Java 5.3: TesteBean.java

Para acessar o valor do atributo `numero` em uma tela JSF, precisamos definir um método de leitura. Esse método deve seguir a convenção de nomenclatura do Java. Veja o exemplo abaixo:

```
1 @ManagedBean
2 public class TesteBean {
3   private int numero;
4
```

```
5   public int getNumero() {
6     return numero;
7   }
8 }
```

Código Java 5.4: TesteBean.java

Note que o nome do método começa com `get` e é seguido pelo nome do atributo com a primeira letra em caixa alta.

Para alterar o valor do atributo `numero` com valores obtidos através de uma página web, precisamos definir um método de escrita.

```
1  @ManagedBean
2  public class TesteBean {
3    private int numero;
4
5    public int setNumero(int numero) {
6      this.numero = numero;
7    }
8
9    public int getNumero() {
10     return numero;
11   }
12 }
```

Código Java 5.5: TesteBean.java

O nome do método de escrita deve necessariamente começar com a palavra `set` e terminar com o nome do atributo com a primeira letra em caixa alta.

Com os métodos de acesso já implementados, podemos exibir o valor do atributo `numero` utilizando *expression language* (`#{}`). Veja o exemplo a seguir.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html">
6
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10
11 <h:body>
12   Valor: #{testeBean.numero}
13 </h:body>
14 </html>
```

Código XHTML 5.1: Exibindo o valor do atributo `numero`

Para alterar o valor do atributo `numero` do managed bean `testeBean`, podemos vinculá-lo, por exemplo, a uma caixa de texto em um formulário. Observe o código abaixo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html">
6
7  <h:head>
8    <title>K19 Treinamentos</title>
```

```
</h:head>

<h:body>

  Valor: #{testeBean.numero}

  <h:form>
    <h:inputText value="#{testeBean.numero}" />
    <h:commandButton value="Altera"/>
  </h:form>
</h:body>
</html>
```

Código XHTML 5.2: Alterando o valor do atributo `numero`



Importante
É importante destacar que o vínculo com uma propriedade de um managed bean dá-se por meio dos nomes dos métodos getters e setters, e não pelo nome do atributo.

No exemplo acima, se mantivéssemos o nome do atributo do managed bean mas substituíssemos os nomes dos métodos `getNumero()` e `setNumero()` por `getValor()` e `setValor()`, respectivamente, então os trechos de código XHTML em destaque deveriam ser alterados para o seguinte:

```
Valor: #{testeBean.valor}
<h:inputText value="#{testeBean.valor}" />
```

Ações

Para implementar as lógicas que devem ser executadas assim que o usuário clicar em um botão ou link, basta criar métodos nas classes dos managed beans.

Seguindo o exemplo acima, podemos criar um método que incrementa o valor do atributo `numero`, como no código abaixo.

```
@ManagedBean
public class TesteBean {
  private int numero;

  public void incrementaNumero() {
    this.numero = numero + 1;
  }

  // GETTERS E SETTERS
}
```

Código Java 5.6: TesteBean.java

Esses métodos podem ser `void` quando desejamos manter os usuários na mesma tela ou devolver `String` quando desejamos realizar uma navegação entre telas. Veremos mais detalhes em capítulos posteriores.

Para incrementar o valor do atributo `numero` do managed bean `testeBean`, podemos criar uma página que possui um botão para executar essa ação e associá-lo ao método `incrementaNumero()`, conforme o código abaixo.

```
1 <h:form>
2   <h:commandButton value="Incrementa" action="#{testeBean.incrementaNumero}" />
3 </h:form>
```

Código XHTML 5.4: Botão associado a um método de um managed bean



Mais Sobre

Todo managed bean possui um nome único que é utilizado para acessá-lo dentro dos trechos escritos com expression language. Quando utilizamos a anotação `@ManagedBean`, por padrão, o JSF assume que o nome do managed bean é o nome da classe com a primeira letra em caixa baixa. Porém, podemos alterar esse nome acrescentado um argumento na anotação.

```
1 @ManagedBean(name="teste")
2 class TesteBean {
3   ...
4 }
```

Código Java 5.7: Alterando o nome de um managed bean



Processamento de uma requisição

Em uma aplicação JSF, toda requisição realizada através de um navegador é recebida pela *Faces Servlet*. Ao receber uma requisição, a Faces Servlet executa os seguintes passos:

Restore View: Na primeira requisição de um usuário, uma nova árvore de componentes que representa a tela desse usuário é gerada. Nas demais requisições (*postback*) desse mesmo usuário, a árvore de componentes que representa a tela anteriormente enviada a ele é reconstruída.

Apply Request Values: Nesta etapa, a árvore construída na etapa anterior é percorrida e cada um dos seus componentes é "decodificado". No processo de decodificação, cada componente extrai da requisição atual os dados associados a essa componente e se atualiza com essas informações.

Ainda nessa etapa, os eventos de ação (como um clique em um *link* ou em um botão, por exemplo) são identificados. Por padrão, esses eventos são adicionados às filas correspondentes para serem tratados posteriormente (na fase *Invoke Application*).

Process Validations: Nesta fase, os componentes que possuem valores submetidos pelo usuário através de formulários são convertidos e validados (caso haja algum validador registrado para esse componente).

Se ocorrer algum erro de conversão ou validação, mensagens de erro são adicionadas no contexto da requisição atual e o fluxo é redirecionado para a fase *Render Response*. Caso contrário, processo continua na fase *Update Model Values*.

Ainda nesta fase, os eventos de mudança de valor são identificados e tratados ao final da mesma.

Update Model Values: Os valores contidos em cada componente da árvore, já convertidos e validados na fase anterior, são armazenados em propriedades de objetos definidos pela aplicação (managed beans)

Invoke Application: Uma vez que os dados dos componentes já foram convertidos, validados e armazenados nos objetos do modelo, as tarefas correspondentes ao evento que disparou a requisição (normalmente um clique em um botão ou link) serão executadas.

Também nesta fase, a próxima tela a ser apresentada ao usuário é determinada pelo resultado do método que implementa a lógica de negócio executado nesta fase.

Render Response: Nesta etapa, a próxima tela é gerada e enviada ao navegador do usuário. Uma representação desta tela também é armazenada a fim de ser usada na fase *Restore View* da próxima requisição.

O diagrama abaixo ilustra a estrutura geral de uma aplicação JSF. O processamento de uma requisição enviada por um navegador começa na Faces Servlet logo após a sua chegada. A Faces Servlet controla a execução das seis etapas descritas acima.

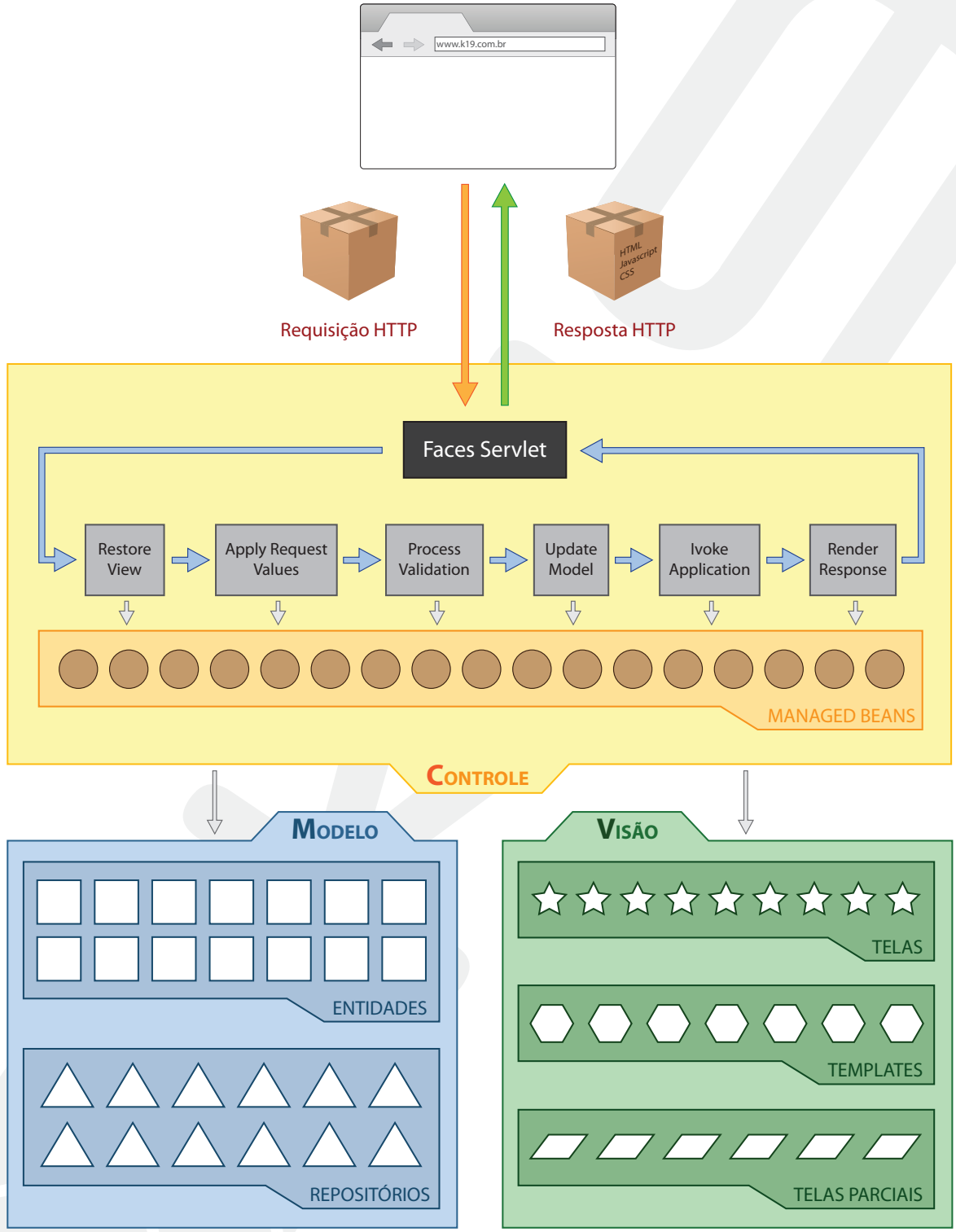


Figura 5.1: Estrutura geral de uma aplicação JSF

Os managed beans estão à disposição da Faces Servlet durante todo o processamento da requisição. Por exemplo, nas etapas *Render Response* e *Restore View*, a Faces Servlet aciona os managed beans para recuperar os dados que devem ser usados na construção ou reconstrução da árvore de componentes. Na etapa *Update Model*, a Faces Servlet armazena nos managed beans os dados já convertidos e validados. Na etapa *Invoke Application*, a Faces Servlet dispara um método em um managed bean responsável pelo processamento da regra de negócio correspondente à requisição atual.

Todas as regras de negócio são implementadas no modelo, que também administra os dados da aplicação. Os managed beans acionam o modelo para executar alguma regra de negócio, recuperar ou alterar os dados administrados pelo modelo.

As telas da aplicação são definidas na camada de visão. A Faces Servlet acessa essa camada toda vez que necessita construir ou reconstruir a árvore de componentes de uma determinada tela. Isso ocorre nas etapas *Restore View* e *Render Response*.



Exemplo Prático

Com as configurações já realizadas, implementaremos uma aplicação que mostra o funcionamento básico do JSF. Essa aplicação deverá receber um texto do usuário e exibi-lo em letras maiúsculas.

Managed Bean

Vamos começar criando um managed bean para armazenar o texto enviado pelo usuário e a lógica para transformá-lo.

```
1  import javax.faces.bean.ManagedBean;
2
3  @ManagedBean
4  public class TextoBean {
5    private String texto;
6
7    public String transformaEmCaixaAlta() {
8      this.texto = this.texto.toUpperCase();
9      return "resposta";
10   }
11
12   public String getTexto() {
13     return this.texto;
14   }
15
16   public void setTexto(String texto) {
17     this.texto = texto;
18   }
19 }
```

Código Java 5.8: TextoBean.java

A classe que implementa o managed bean deve ser anotada com `@ManagedBean`. O atributo `texto` armazenará o texto enviado pelo usuário e esse texto será modificado pelo método `transformaEmCaixaAlta()`. Esse método devolve uma string para indicar qual deve ser a próxima tela a ser enviada para o usuário.

A Faces Servlet utilizará o método `setTexto()` para armazenar o texto enviado pelo usuário no managed bean. Por outro lado, utilizará o método `getTexto()` para recuperar o texto e exibi-lo após a sua modificação.

Telas

Uma vez que o managed bean foi criado, podemos associá-lo a um formulário que receberá o texto do usuário.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:ui="http://java.sun.com/jsf/facelets"
6    xmlns:h="http://java.sun.com/jsf/html"
7    xmlns:f="http://java.sun.com/jsf/core">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12
13 <h:body>
14   <h:form>
15     <h:outputLabel value="Texto: " />
16     <h:inputTextarea value="#{textoBean.texto}"/>
17     <h:commandButton value="Transformar" action="#{textoBean.transformaEmCaixaAlta}"/>
18   </h:form>
19 </h:body>
20 </html>
```

Código XHTML 5.5: formulario.xhtml

Observe nas linhas em destaque a ligação entre essa tela e o managed bean. A caixa de entrada de texto foi associada à propriedade `texto` do managed bean `TextoBean`. O botão, por sua vez, foi associado ao método `transformaEmCaixaAlta()` do managed bean `TextoBean`.

Para exibir o texto transformado, podemos criar uma outra tela.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:ui="http://java.sun.com/jsf/facelets"
6    xmlns:h="http://java.sun.com/jsf/html"
7    xmlns:f="http://java.sun.com/jsf/core">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12
13 <h:body>
14   <h:outputText value="#{textoBean.texto}"/>
15 </h:body>
16 </html>
```

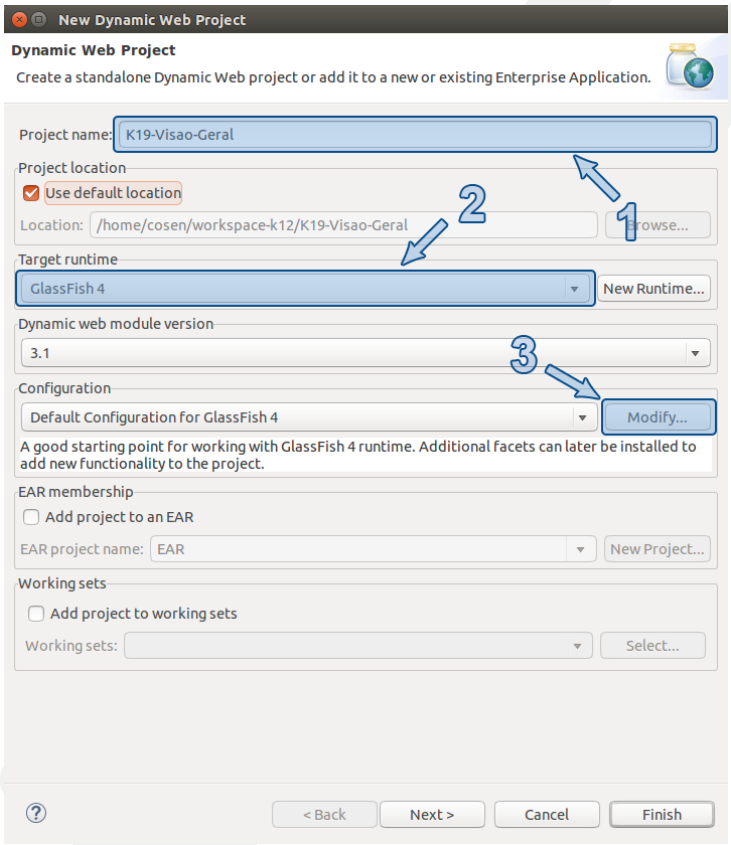
Código XHTML 5.6: resposta.xhtml

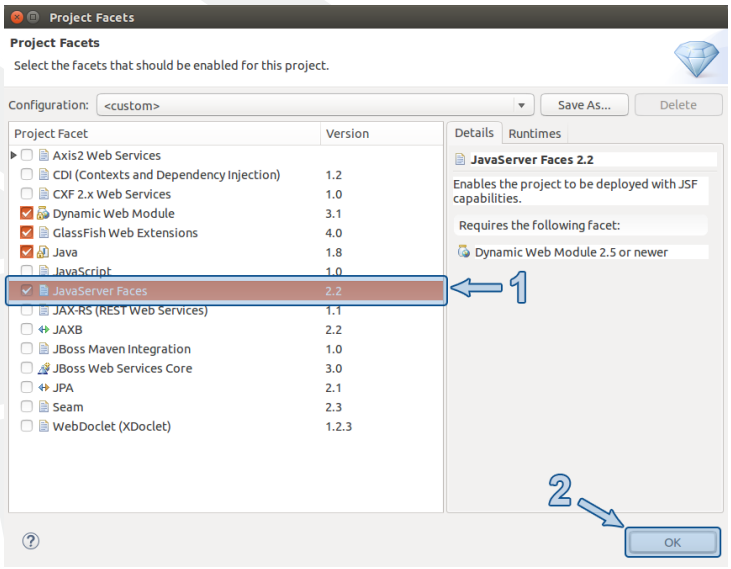
Analogamente, a caixa de saída de texto está associada à propriedade `texto` do managed bean `TextoBean`.

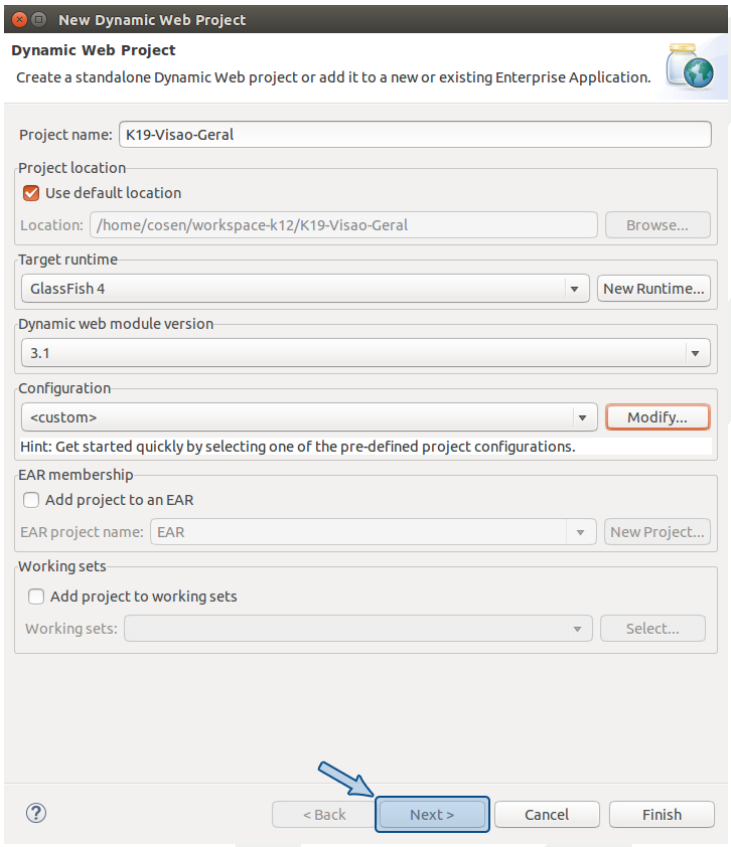


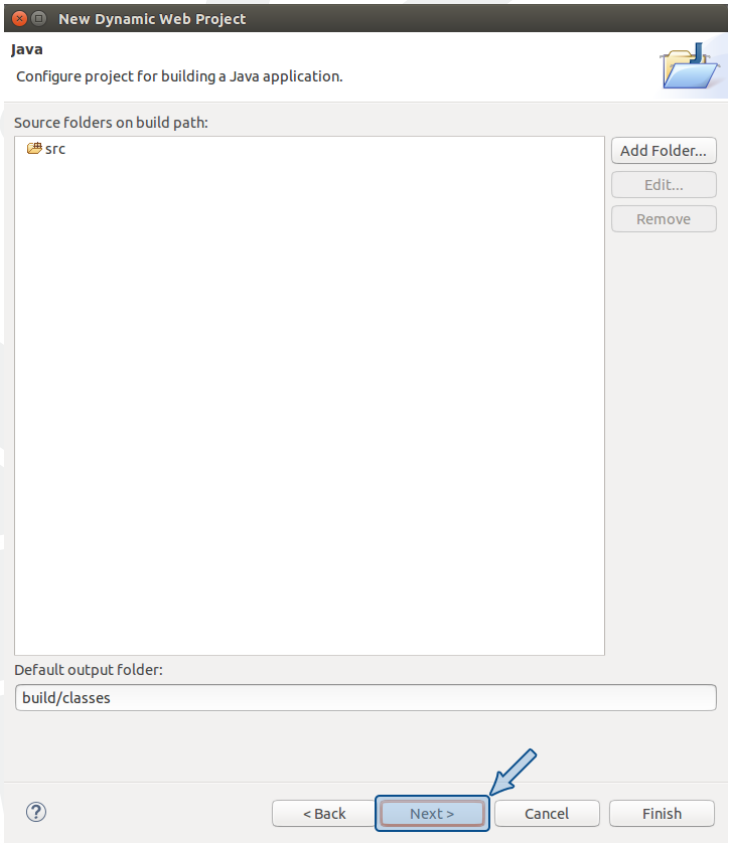
Exercícios de Fixação

1 Crie um projeto chamado **K19-Visao-Geral**. Digite "CTRL + 3" e pesquise por "Dynamic Web Project". Selecione a opção correspondente e siga as imagens abaixo.

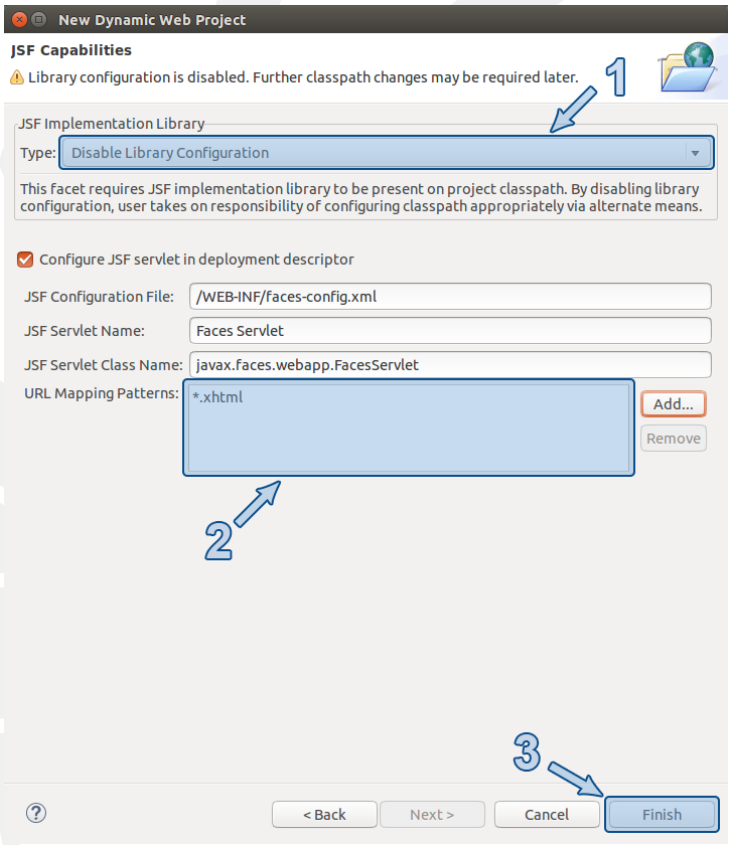





New Dynamic Web Project
Dynamic Web Project
Create a standalone Dynamic Web project or add it to a new or existing Enterprise Application.
Project name: K19-Visao-Geral
Project location
Use default location
Location: /home/cosen/workspace-k12/K19-Visao-Geral
Browse...
Target runtime
GlassFish 4
New Runtime...
Dynamic web module version
3.1
Configuration
<custom>
Modify...
Hint: Get started quickly by selecting one of the pre-defined project configurations.
EAR membership
Add project to an EAR
EAR project name: EAR
New Project...
Working sets
Add project to working sets
Working sets:
Select...
< Back
Next >
Cancel
Finish


New Dynamic Web Project
Java
Configure project for building a Java application.
Source folders on build path:
src
Add Folder...
Edit...
Remove
Default output folder:
build/classes
< Back
Next >
Cancel
Finish


New Dynamic Web Project
Web Module
Configure web module settings.
Context root:
K19-Visao-Geral
Content directory:
WebContent
Generate web.xml deployment descriptor
< Back
Next >
Cancel
Finish


New Dynamic Web Project
JSF Capabilities
Library configuration is disabled. Further classpath changes may be required later.
1
JSF Implementation Library
Type:
Disable Library Configuration
This facet requires JSF implementation library to be present on project classpath. By disabling library configuration, user takes on responsibility of configuring classpath appropriately via alternate means.
Configure JSF servlet in deployment descriptor
JSF Configuration File:
/WEB-INF/faces-config.xml
JSF Servlet Name:
Faces Servlet
JSF Servlet Class Name:
javax.Faces.webapp.FacesServlet
URL Mapping Patterns:
*.xhtml
Add...
Remove
2
3
< Back
Next >
Cancel
Finish

2 Altere o arquivo de configuração **web.xml** do projeto **K19-Visao-Geral**.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  version="3.1">
  <display-name>K19-Visao-Geral</display-name>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.xhtml</url-pattern>
  </servlet-mapping>

  <context-param>
    <param-name>javax.faces.PROJECT_STAGE</param-name>
    <param-value>Development</param-value>
  </context-param>
</web-app>
```

Código XML 5.4: www.k19.com.br/exercicios/k12/05/002/web.xml

3 Em um pacote chamado **br.com.k19.controle** no projeto **K19-Visao-Geral**, crie um managed bean para armazenar um número inteiro *n* enviado pelo usuário, gerar um número aleatório entre zero e *n* e armazená-lo em uma propriedade.

```
package br.com.k19.controle;

import javax.faces.bean.ManagedBean;

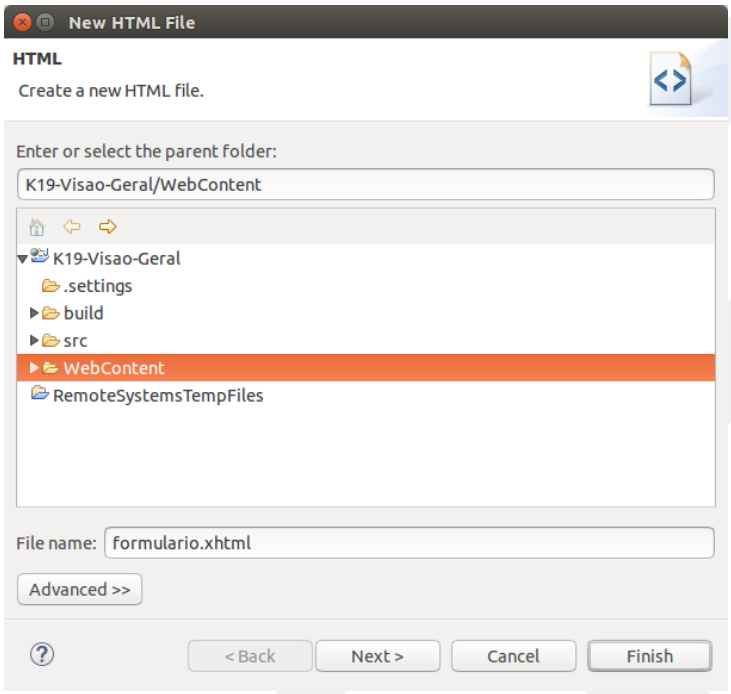
@ManagedBean
public class NumeroAleatorioBean {
  private int maximo;
  private int numeroAleatorio;

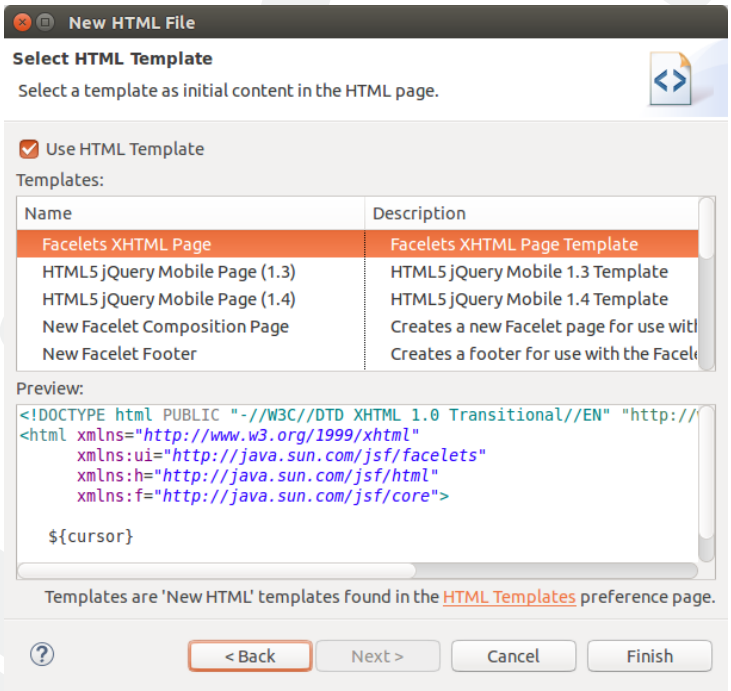
  public String geraNumeroAleatorio() {
    this.numeroAleatorio = (int) (Math.random() * this.maximo);
    return "resposta";
  }

  // GETTERS E SETTERS
}
```

Código Java 5.9: www.k19.com.br/exercicios/k12/05/003/NumeroAleatorioBean.java

4 Crie um arquivo chamado **formulario.xhtml**. Digite “CTRL + 3” e pesquise por “Create a new HTML file”. Selecione a opção correspondente e siga as imagens abaixo.





5 Altere o código do arquivo **formulario.xhtml**

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3 <html xmlns="http://www.w3.org/1999/xhtml"
4   xmlns:ui="http://java.sun.com/jsf/facelets"
5   xmlns:h="http://java.sun.com/jsf/html"
6   xmlns:f="http://java.sun.com/jsf/core">
7 <h:head>
8   <title>K19 Treinamentos</title>
```

```
</h:head>

<h:body>
  <h:form>
    <h:outputLabel value="Número máximo: " />
    <h:inputText value="#{numeroAleatorioBean.maximo}" />
    <h:commandButton value="Gera número aleatório"
      action="#{numeroAleatorioBean.geraNumeroAleatorio}" />
  </h:form>
</h:body>
</html>
```

Código XHTML 5.7: www.k19.com.br/exercicios/k12/05/005/formulario.xhtml

6 Agora, crie um arquivo chamado **resposta.xhtml** com o conteúdo abaixo.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h:outputText value="#{numeroAleatorioBean.numeroAleatorio}" />
</h:body>
</html>
```

Código XHTML 5.8: www.k19.com.br/exercicios/k12/05/006/resposta.xhtml

7 Remova o projeto **Servlet** do GlassFish e do Wildfly. Implante o projeto **K19-Visao-Geral** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

http://localhost:8080/K19-Visao-Geral/formulario.xhtml.

8 Implante o projeto **K19-Visao-Geral** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

http://localhost:8080/K19-Visao-Geral/formulario.xhtml.

Componentes Visuais

CAPÍTULO 6

No JSF 2, as telas são definidas em arquivos XHTML. Os componentes visuais que constituem as telas são adicionados por meio de tags. A especificação do JSF define uma grande quantidade de tags e as classifica em bibliotecas. As principais bibliotecas de tags do JSF são:

- Core (http://java.sun.com/jsf/core)
- HTML (http://java.sun.com/jsf/html)
- Facelets (http://java.sun.com/jsf/facelets)

A seguir, apresentaremos o funcionamento geral das principais tags dessas três bibliotecas. A documentação completa pode ser encontrada em http://www.jcp.org/en/jsr/detail?id=314.

Estrutura Básica de uma Página JSF

A estrutura básica de uma página JSF é muito semelhante à estrutura de uma página HTML. Observe o código abaixo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:ui="http://java.sun.com/jsf/facelets"
6    xmlns:h="http://java.sun.com/jsf/html"
7    xmlns:f="http://java.sun.com/jsf/core">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12
13 <h:body>
14   <h:outputText value="Estrutura básica de uma tela JSF" />
15 </h:body>
16 </html>
```

Código XHTML 6.1: Estrutura Básica de uma Tela JSF

O conteúdo da página é definido no corpo da tag <html>. Esse conteúdo é dividido em duas partes: o cabeçalho, delimitado pela tag <h:head> (e não pela tag <head>), e o corpo, delimitado pela tag <h:body> (e não pela tag <body>).

As bibliotecas de tags que serão utilizadas para construir a página devem ser "importadas" através do pseudo-atributo xmlns aplicado à tag <html>. Observe no exemplo acima que as três principais bibliotecas do JSF foram importadas.



Mais Sobre
Devemos utilizar a declaração `<!DOCTYPE>` para informar aos navegadores qual é a versão do HTML utilizado na página. Com essa informação, os navegadores podem processar o documento corretamente.



Mais Sobre
Quando uma biblioteca de tags é "importada" através do pseudo-atributo `xmlns`, é possível definir um prefixo para essa biblioteca. Esse prefixo é utilizado, no documento, para acessar as tags da biblioteca.



Importante
A utilização das tags `<h:head>` e `<h:body>` é fundamental para o funcionamento completo das páginas HTML geradas pelo JSF. Ao processar essas tags, na etapa `Render Response`, o JSF adiciona recursos como scripts e arquivos de estilo na tela HTML que será enviada para o usuário. Esses recursos são necessários para o funcionamento correto dos componentes.



Formulários

Os formulários no JSF são definidos através da tag `<h:form>` (e não pela tag `<form>`). Um formulário é composto basicamente por caixas de texto, caixas de seleção, rótulos, botões e links. Ao ser processado, o componente `<h:form>` gera um formulário HTML configurado para o método POST do HTTP.

Nas seções a seguir, descreveremos os elementos de um formulário.



Caixas de Texto

O JSF define três tipos de caixas de texto para coletar dados do usuário.

- `<h:inputText>`
  Permite que o usuário digite uma linha de texto.
- `<h:inputTextarea>`
  Permite que o usuário digite várias linhas de texto.
- `<h:inputSecret>`
  Igual ao `<h:inputText>` mas não exibe o que foi digitado.

Uma caixa de texto pode ser associada a uma propriedade de um managed bean através do atributo `value`. Esse vínculo é denominado **binding**. Considere o seguinte managed bean.

```
1 @ManagedBean
2 public class UsuarioBean {
```

```
3   private String nome;
4
5   public String getNome() {
6     return nome;
7   }
8
9   public void setNome(String nome) {
10    this.nome = nome;
11  }
12 }
```

Código Java 6.1: UsuarioBean.java

Devemos utilizar *expression language* (`#{}`) para estabelecer uma associação entre a propriedade `nome` a uma caixa de texto. Observe o código abaixo.

```
1 <h:inputText value="#{usuarioBean.nome}" />
```

Quando o JSF constrói a tela a ser enviada para o usuário, ele recupera o valor da propriedade `nome` através do método `getNome()` e guarda esse valor na caixa de texto correspondente.

Analogamente, no processamento de uma requisição, o JSF extrai o valor presente na caixa de texto e utiliza o método `setNome()` para armazenar esse valor na propriedade `nome` do managed bean `UsuarioBean`.

Rótulos

Rótulos são utilizados para indicar ao usuário o que ele deve preencher em determinada caixa de texto. Para acrescentar um rótulo em uma página, devemos utilizar o componente `<h:outputLabel>`. O texto do rótulo é definido pelo atributo `value` e pode ser associado a uma caixa de texto através do atributo `for`. O valor do atributo `for` deve ser igual ao valor do atributo `id` da caixa de texto que desejamos associar ao rótulo.

```
1 <h:outputLabel value="Nome: " for="nome"/>
2 <h:inputText id="nome"/>
```

Código XHTML 6.3: Exemplo de uso do `<h:outputLabel>`

Exemplos

A seguir, alguns exemplos de utilização das caixas de texto serão apresentados. Mostraremos o código XHTML das telas, o código HTML produzido pelo JSF e as telas renderizadas por um navegador.

1. XHTML:

```
<h:outputLabel value="Nome: " for="nome" />
<h:inputText value="Jonas K. Hirata" id="nome" />
```

HTML gerado:

```
<label for="j_idt6:nome">Nome: </label>
<input id="j_idt6:nome" type="text" name="j_idt6:nome" value="Jonas K. Hirata" />
```

Resultado:

Nome: Jonas K. Hirata

2. XHTML:

```
<h:outputLabel value="Nome: " for="nome" />
<h:inputText value="#{usuarioBean.nome}" id="nome" />
```

HTML gerado:

```
<label for="j_idt6:nome">Nome: </label>
<input id="j_idt6:nome" type="text" name="j_idt6:nome" value="Jonas Hirata"/>
```

Resultado:

Nome: Jonas Hirata

3. XHTML:

```
<h:outputLabel value="Nome: " for="nome" />
<h:inputText value="#{usuarioBean.nome}" readonly="true" id="nome" />
```

HTML gerado:

```
<label for="j_idt6:nome">Nome: </label>
<input id="j_idt6:nome" type="text" name="j_idt6:nome" value="Jonas Hirata"
readonly="readonly" />
```

Resultado:

Nome: Jonas Hirata

4. XHTML:

```
<h:outputLabel value="CEP: " for="cep" />
<h:inputText value="#{usuarioBean.cep}" maxlength="9" id="cep" />
```

HTML gerado:

```
<label for="j_idt6:cep">Cep: </label>
<input id="j_idt6:cep" type="text" name="j_idt6:cep" value="01452-001" maxlength="9" ↩
    />
```

Resultado:

CEP: 01452-001

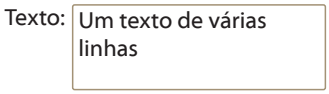
5. XHTML:

```
<h:outputLabel value="Texto: " for="texto" />
<h:inputTextarea value="Um texto de várias linhas" id="texto" />
```

HTML gerado:

```
<label for="j_idt6:texto">Texto: </label>
<textarea id="j_idt6:texto" name="j_idt6:texto">Um texto de várias linhas</textarea>
```

Resultado:



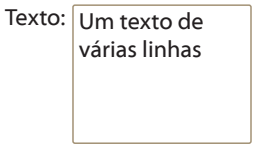
6. XHTML:

```
<h:outputLabel value="Texto: " for="texto" />
<h:inputTextarea value="Um texto de várias linhas" cols="10" rows="3" id="texto"/>
```

HTML gerado:

```
<label for="j_idt6:texto">Texto: </label>
<textarea id="j_idt6:texto" name="j_idt6:texto" cols="10" rows="3">
  Um texto de várias linhas
</textarea>
```

Resultado:



7. XHTML:

```
<h:outputLabel value="Senha: " for="senha" />
<h:inputSecret value="#{usuarioBean.senha}" id="senha" />
```

HTML gerado:

```
<label for="j_idt6:senha">Senha: </label>
<input id="j_idt6:senha" type="password" name="j_idt6:senha" value="" />
```

Resultado:



Campos Ocultos

Podemos adicionar informações que são submetidas automaticamente quando um formulário é enviado ao servidor. Essas informações podem ser inseridas com o componente `<h:inputHidden>`. Essa tag possui dois atributos principais: `value` e `id`. Podemos vincular um campo oculto a uma propriedade de um managed bean, como no exemplo abaixo.

```
1 <h:inputHidden id="curso-id" value="#{cursosBean.curso.id}" />
```

Código XHTML 6.11: Exemplo de um campo oculto vinculado a uma propriedade de um managed bean

Caixas de Seleção

O JSF define sete tipos de caixas de seleção.

- `<h:selectBooleanCheckbox>`
  Permite que o usuário faça seleções do tipo "sim ou não".
- `<h:selectManyCheckbox>`
  Permite que o usuário faça múltiplas seleções do tipo "sim ou não".
- `<h:selectOneRadio>`, `<h:selectOneMenu>`, `<h:selectOneListbox>`
  Permitem a seleção de uma única opção de um conjunto de opções. A principal diferença entre essas tags é modo como elas são apresentadas visualmente.
- `<h:selectManyListbox>`, `<h:selectManyMenu>`
  Possibilita a seleção de diversas opções de um conjunto de opções. A diferença principal entre essas duas tags é modo como elas são apresentadas visualmente.

Podemos vincular uma caixa de seleção a uma propriedade de um managed bean através do atributo `value`. No exemplo abaixo, uma caixa de seleção é associada à propriedade `aceito` do managed bean `ContratoBean`.

```
1 <h:selectBooleanCheckbox value="#{contratoBean.aceito}" />
```

Código XHTML 6.12: Caixa de seleção vinculada à propriedade `aceito` *do managed bean* `ContratoBean`

Caixas de seleção do tipo `<h:selectBooleanCheckbox>` devem ser vinculadas a propriedades booleanas.

Opções Estáticas e Dinâmicas

As opções de uma caixa de seleção (com exceção da `<h:selectBooleanCheckbox>`) podem ser definidas estática ou dinamicamente. Estaticamente, cada opção pode ser adicionada através da tag `<f:selectItem>`. Considere o seguinte exemplo.

```
1 <h:selectOneMenu value="#{cursosBean.siglaDoCursoEscolhido}">
2   <f:selectItem itemValue="K11" itemLabel="Orientação a Objetos em Java">
3   <f:selectItem itemValue="K12" itemLabel="Desenvolvimento Web com JSF2 e JPA2">
4 </h:selectOneMenu>
```

Código XHTML 6.13: Caixa de seleção com opções definidas estaticamente

O atributo `itemValue` define o que será enviado para a aplicação se a opção correspondente for selecionada. O atributo `itemLabel` define a descrição associada à opção correspondente. Essa descrição é exibida para o usuário.

Definir as opções estaticamente pode dificultar a adição ou remoção de opções. Por exemplo, o cadastramento ou a remoção de um curso no sistema implicaria em uma mudança no arquivo XHTML.

Seria interessante eliminar essa dependência fazendo com que o arquivo XHTML recupere dinamicamente todos os cursos cadastrados. Isso pode ser realizado através da tag `<f:selectItems>`.

```
1 <h:selectOneMenu value="#{cursosBean.siglaDoCursoEscolhido}">
2   <f:selectItems
3     value="#{cursosBean.cursos}"
```

```
4       var="curso"
5       itemValue="#{curso.sigla}"
6       itemLabel="#{curso.nome}">
7   </h:selectOneMenu>
```

Código XHTML 6.14: Caixa de seleção com opções definidas dinamicamente

A atributo `value` da tag `<f:selectItems>` deve ser associado a uma coleção de itens. Todos os itens serão percorridos e adicionados como opção na caixa de seleção. O atributo `var` é utilizado para definir a variável que armazenará, a cada rodada, o item atual. Essa variável permite que o `itemValue` e o `itemLabel` de cada opção sejam definidos.

Seleção Única ou Múltipla

Algumas caixas de seleção permitem que apenas uma opção seja selecionada, enquanto outras permitem que múltiplas opções sejam selecionadas. Considere um formulário que permita ao usuário escolher apenas um dos cursos de uma lista. Como vimos, esse formulário pode ter a seguinte estrutura:

```
1  <h:selectOneListbox value="#{cursosBean.siglaDoCursoEscolhido}">
2    <f:selectItems
3      value="#{cursosBean.cursos}"
4      var="curso"
5      itemValue="#{curso.sigla}"
6      itemLabel="#{curso.nome}" />
7  </h:selectOneListbox>
```

Código XHTML 6.15: Seleção de um único curso

```
1  @ManagedBean
2  public class CursosBean {
3    private String siglaDoCursoEscolhido;
4    private List<Curso> cursos;
5
6    // GETTERS E SETTERS
7  }
```

Código Java 6.2: CursosBean.java

```
1  public class Curso {
2    private String sigla;
3    private String nome;
4
5    // GETTERS E SETTERS
6  }
```

Código Java 6.3: Curso.java

A propriedade associada à caixa de seleção deve ser compatível com o tipo de dado utilizado no atributo `itemValue`. No exemplo acima, as siglas dos cursos são do tipo `String`. Consequentemente, a propriedade que armazenará a sigla do curso escolhido também deve ser to tipo `String`.

Agora, suponha que o usuário possa escolher mais do que um curso. Nesse caso, o managed bean deve estar preparado para guardar uma lista de siglas de cursos ao invés de uma única sigla. Para isso, devemos adicionar uma propriedade no managed bean `CursosBean`, como mostra o código abaixo.

```
1  @ManagedBean
2  public class CursosBean {
```

```
3   private List<String> siglasDosCursosEscolhidos;
4   private List<Curso> cursos;
5
6   // GETTERS E SETTERS
7 }
```

Código Java 6.4: CursosBean.java

Para permitir a seleção múltipla, devemos utilizar uma caixa de seleção do tipo <h:selectManyListbox> ou <h:selectManyMenu>. Essa caixa deve então ser vinculada à nova propriedade do managed bean CursoBean. Por exemplo, se optarmos pela <h:selectManyListbox>, poderíamos ter o seguinte código.

```
1 <h:selectManyListbox value="#{cursosBean.siglasDosCursosEscolhidos}">
2   <f:selectItems
3     value="#{cursosBean.cursos}"
4     var="curso"
5     itemValue="#{curso.sigla}"
6     itemLabel="#{curso.nome}" />
7 </h:selectManyListbox>
```

Código XHTML 6.16: Seleção de diversos cursos

Caixa de Seleção com Pseudo-opção

Por padrão, a caixa de seleção do tipo <h:selectOneMenu> sempre tem uma opção selecionada. Contudo, em alguns casos, faz sentido para o usuário não selecionar nenhuma das opções disponíveis. Para resolver esse problema, podemos adicionar uma pseudo-opção na caixa de seleção. Considere o código abaixo.

```
1 <h:selectOneMenu value="#{cursosBean.siglaDoCursoEscolhido}">
2   <f:selectItem itemLabel="Nenhum" noSelectionOption="true"/>
3   <f:selectItems
4     value="#{cursosBean.cursos}"
5     var="curso"
6     itemValue="#{curso.sigla}"
7     itemLabel="#{curso.nome}" />
8 </h:selectOneMenu>
```

Código XHTML 6.17: Pseudo-opção

Para adicionar uma pseudo-opção, basta utilizar a tag <f:selectItem> com o atributo noSelectionOption igual a ‘‘true’’. Se essa pseudo-opção estiver selecionada no momento em que o formulário for submetido, a propriedade correspondente receberá o valor null.

Muitas vezes, a pseudo-opção é utilizada simplesmente para exibir uma mensagem ao usuário, como “Escolha uma opção” ou “Selecione um item”.

Exemplos

A seguir, mostraremos exemplos de cada tipo de caixa de seleção.

1. XHTML:

```
1 <h:outputLabel value="Ex-aluno" for="exaluno"/>
2 <h:selectBooleanCheckbox value="#{cursosBean.exAluno}" id="exaluno"/>
```

HTML gerado:

```
<label for="j_idt6:exaluno">Ex-aluno</label>
<input id="j_idt6:exaluno" type="checkbox" name="j_idt6:exaluno" />
```

Resultado:

Ex-aluno ☑

2. XHTML:

```
<h:selectManyCheckbox value="#{cursosBean.siglasDosCursosEscolhidos}"
  layout="pageDirection">
  <f:selectItems
    value="#{cursosBean.cursos}"
    var="curso"
    itemValue="#{curso.sigla}"
    itemLabel="#{curso.nome}" />
</h:selectManyCheckbox>
```

HTML gerado:

```
<table>
  <tr>
    <td>
      <input name="j_idt6:j_idt27" id="j_idt6:j_idt27:0" value="K11" type="checkbox"/>
      >
      <label for="j_idt6:j_idt27:0" class="">Orientação a Objetos em Java</label>
    </td>
  </tr>
  <tr>
    <td>
      <input name="j_idt6:j_idt27" id="j_idt6:j_idt27:1" value="K12" type="checkbox"/>
      >
      <label for="j_idt6:j_idt27:1" class="">
        Desenvolvimento Web com JSF2 e JPA2
      </label>
    </td>
  </tr>
  <tr>
    <td>
      <input name="j_idt6:j_idt27" id="j_idt6:j_idt27:2" value="K51" type="checkbox"/>
      >
      <label for="j_idt6:j_idt27:2" class="">Design Patterns em Java</label>
    </td>
  </tr>
</table>
```

Resultado:

☑ Orientação a Objetos em Java
☐ Desenvolvimento Web com JSF2 e JPA2
☑ Design Patterns em Java

3. XHTML:

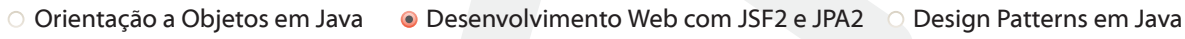
```
<h:selectOneRadio value="#{cursosBean.siglasDosCursosEscolhidos}" layout="
    lineDirection">
  <f:selectItems
    value="#{cursosBean.cursos}"
    var="curso"
    itemValue="#{curso.sigla}"
```

```
6       itemLabel="#{curso.nome}" />
7   </h:selectOneRadio>
```

HTML gerado:

```
1   <table>
2     <tr>
3       <td>
4         <input type="radio" name="j_idt6:j_idt30" id="j_idt6:j_idt30:0" value="K11" />
5         <label for="j_idt6:j_idt30:0">Orientação a Objetos em Java</label>
6       </td>
7       <td>
8         <input type="radio" name="j_idt6:j_idt30" id="j_idt6:j_idt30:1" value="K12" />
9         <label for="j_idt6:j_idt30:1">Desenvolvimento Web com JSF2 e JPA2</label>
10      </td>
11      <td>
12        <input type="radio" name="j_idt6:j_idt30" id="j_idt6:j_idt30:2" value="K51" />
13        <label for="j_idt6:j_idt30:2">Design Patterns em Java</label>
14      </td>
15    </tr>
16  </table>
```

Resultado:



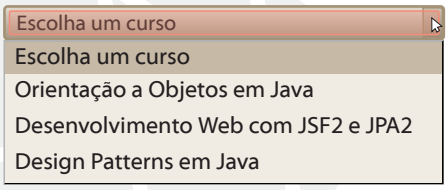
4. XHTML:

```
1   <h:selectOneMenu value="#{cursosBean.siglasDosCursosEscolhidos}">
2     <f:selectItem itemLabel="Escolha um curso" noSelectionOption="true" />
3     <f:selectItems
4       value="#{cursosBean.cursos}"
5       var="curso"
6       itemValue="#{curso.sigla}"
7       itemLabel="#{curso.nome}" />
8   </h:selectOneMenu>
```

HTML gerado:

```
1   <select name="j_idt6:j_idt33" size="1">
2     <option value="">Escolha um curso</option>
3     <option value="K11">Orientação a Objetos em Java</option>
4     <option value="K12">Desenvolvimento Web com JSF2 e JPA2</option>
5     <option value="K51">Design Patterns em Java</option>
6   </select>
```

Resultado:



5. XHTML:

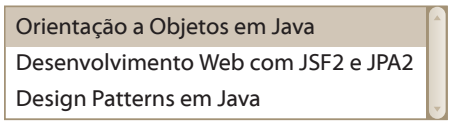
```
1   <h:selectOneListbox value="#{cursosBean.siglasDosCursosEscolhidos}">
2     <f:selectItems
3       value="#{cursosBean.cursos}"
4       var="curso"
5       itemValue="#{curso.sigla}"
```

```
6       itemLabel="#{curso.nome}" />
7   </h:selectOneListbox>
```

HTML gerado:

```
1   <select name="j_idt6:j_idt37" size="3">
2     <option value="K11">Orientação a Objetos em Java</option>
3     <option value="K12">Desenvolvimento Web com JSF2 e JPA2</option>
4     <option value="K51">Design Patterns em Java</option>
5   </select>
```

Resultado:



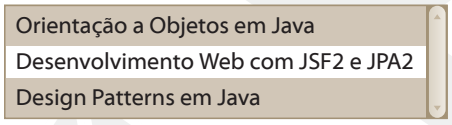
6. XHTML:

```
1   <h:selectManyListbox value="#{cursosBean.siglasDosCursosEscolhidos}">
2     <f:selectItems
3       value="#{cursosBean.cursos}"
4       var="curso"
5       itemValue="#{curso.sigla}"
6       itemLabel="#{curso.nome}" />
7   </h:selectManyListbox>
```

HTML gerado:

```
1   <select name="j_idt6:j_idt40" multiple="multiple" size="3">
2     <option value="K11">Orientação a Objetos em Java</option>
3     <option value="K12">Desenvolvimento Web com JSF2 e JPA2</option>
4     <option value="K51">Design Patterns em Java</option>
5   </select>
```

Resultado:



7. XHTML:

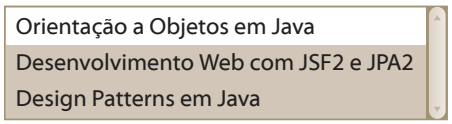
```
1   <h:selectManyMenu value="#{cursosBean.siglasDosCursosEscolhidos}" style="height: 70px↩
        ;">
2     <f:selectItem itemLabel="Escolha um curso" noSelectionOption="true" />
3     <f:selectItems
4       value="#{cursosBean.cursos}"
5       var="curso"
6       itemValue="#{curso.sigla}"
7       itemLabel="#{curso.nome}" />
8   </h:selectManyMenu>
```

HTML gerado:

```
1   <select name="j_idt6:j_idt43" multiple="multiple" size="1" style="height: 70px;">
2     <option value="" selected="selected">Escolha um curso</option>
3     <option value="K11">Orientação a Objetos em Java</option>
4     <option value="K12">Desenvolvimento Web com JSF2 e JPA2</option>
```

```
5   <option value="K51">Design Patterns em Java</option>
6 </select>
```

Resultado:





Botões e Links

O JSF define cinco tipos de botões e links:

- <h:commandButton> e <h:commandLink>
  Enviam os dados de um formulário HTML para o servidor através do método POST do HTTP.
- <h:button> e <h:link>
  Realizam requisições HTTP do tipo GET. As URLs das requisições são geradas pelo JSF a partir do atributo `outcome`.
- <h:outputLink>
  Também realiza requisições HTTP do tipo GET, mas exige que a URL de destino seja explicitamente especificada.

Os componentes <h:commandButton> e <h:commandLink> são usados para submeter formulários HTML por meio do método POST do HTTP. Esses dois componentes podem ser associados a métodos de ação de um managed bean através dos atributos `action` e `actionListener`. Esses atributos serão detalhados em outro capítulo.

```
1 <h:commandButton value="Adiciona curso" action="#{cursosBean.adicionaCurso}"/>
2 <h:commandLink value="Remove curso" action="#{cursosBean.removeCurso}"/>
```

Código XHTML 6.25: Exemplos de utilização do <h:commandButton> *e do* <h:commandLink>

A diferença básica entre o <h:commandButton> e o <h:commandLink> é o modo como eles são apresentados visualmente.

Os componentes <h:button> e <h:link>, por sua vez, são usados para realizar requisições através do método GET. O JSF utiliza o atributo `outcome` de um <h:button> ou <h:link> para definir a página de destino. Esses componentes não são utilizados para submeter formulários HTML.

```
1 <h:button value="Lista de cursos" outcome="lista-cursos" />
2 <h:link value="Home" outcome="home" />
```

Código XHTML 6.26: Exemplos de utilização do <h:button> *e do* <h:link>

Novamente, a diferença fundamental entre o <h:button> e o <h:link> é o modo como eles são exibidos visualmente.

O componente <h:outputLink> permite apenas a criação de links HTML que realizam requisições do tipo GET. Diferentemente dos componentes <h:button> e <h:link>, a URL da requisição é definida explicitamente no atributo `value`.

```
1 <h:outputLink value="http://www.k19.com.br"/>
```

Código XHTML 6.27: Um link para a página da K19

Exemplos

1. XHTML:

```
1 <h:commandButton value="Adiciona curso" action="#{cursosBean.adicionaCurso}" />
```

HTML gerado:

```
1 <input type="submit" name="j_idt59:j_idt60" value="Adiciona curso" />
```

Resultado:



2. XHTML:

```
1 <h:commandLink value="Remove curso" action="#{cursosBean.removeCurso}"/>
```

HTML gerado:

```
1 <a href="#" onclick="mojarra.jsfcljs(document.getElementById('j_idt62'),{'j_idt62:↩
      j_idt66':'j_idt62:j_idt66'},'');return false">
2   Remove curso
3 </a>
```

Resultado:

Remove curso

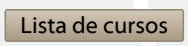
3. XHTML:

```
1 <h:button value="Lista de cursos" outcome="lista-cursos" />
```

HTML gerado:

```
1 <input
2   type="button"
3   onclick="window.location.href='/K19-Componentes-Visuais/lista-cursos.xhtml'; return↩
        false;"
4   value="Lista de cursos" />
```

Resultado:



4. XHTML:

```
1 <h:link value="Home" outcome="home" />
```

HTML gerado:

```
1 <a href="/K19-Componentes-Visuais/home.xhtml">Home</a>
```

Resultado:

Home

5. XHTML:

```
1 <h:outputLink value="http://www.k19.com.br">K19</h:outputLink>
```

HTML gerado:

```
1 <a href="http://www.k19.com.br">K19</a>
```

Resultado:

K19



Exercícios de Fixação

1 Crie um projeto chamado **K19-Componentes-Visuais** seguindo os passos vistos nos exercícios do Capítulo 5.

2 Crie uma classe chamada **UsuarioBean** em um pacote chamado **br.com.k19.controle**. Essa classe deve possuir dois atributos: um atributo do tipo **String** para armazenar o nome do usuário e um atributo do tipo **int** para armazenar a idade do usuário.

```
1  package br.com.k19.controle;
2
3  import javax.faces.bean.ManagedBean;
4
5  @ManagedBean
6  public class UsuarioBean {
7    private String nome;
8    private int idade;
9
10   // GETTERS E SETTERS
11 }
```

Código Java 6.5: www.k19.com.br/exercicios/k12/06/002/UsuarioBean.java

3 Crie um arquivo XHTML chamado **usuario.xhtml**. Nesse arquivo, crie um formulário para o usuário digitar e enviar o seu nome e a sua idade.

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4 <html xmlns="http://www.w3.org/1999/xhtml"
5   xmlns:h="http://java.sun.com/jsf/html">
6
7 <h:head>
```

```
8   <title>K19 Treinamentos</title>
9 </h:head>
10 <h:body>
11   <h:form>
12     <h:outputLabel value="Nome: " for="campo-nome" />
13     <h:inputText value="#{usuarioBean.nome}" id="campo-nome" />
14
15     <h:outputLabel value="Idade: " for="campo-idade" />
16     <h:inputText value="#{usuarioBean.idade}" id="campo-idade" />
17
18     <h:commandButton value="Enviar" />
19   </h:form>
20 </h:body>
21 </html>
```

Código XHTML 6.33: www.k19.com.br/exercicios/k12/06/003/usuario.xhtml

4 Remova o projeto **K19-Visao-Geral** do GlassFish e do Wildfly. Implante o projeto **K19-Componentes-Visuais** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Componentes-Visuais/usuario.xhtml`.

5 Implante o projeto **K19-Componentes-Visuais** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Componentes-Visuais/usuario.xhtml`.

6 No arquivo **usuario.xhtml**, acrescente um trecho de código para exibir os dados do usuário.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html">
6
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10 <h:body>
11   <h:form>
12     <h:outputLabel value="Nome: " for="campo-nome" />
13     <h:inputText value="#{usuarioBean.nome}" id="campo-nome" />
14
15     <h:outputLabel value="Idade: " for="campo-idade" />
16     <h:inputText value="#{usuarioBean.idade}" id="campo-idade" />
17
18     <h:commandButton value="Enviar" />
19   </h:form>
20
21   O nome do usuário é <b>#{usuarioBean.nome}</b>
22   e sua idade é <b>#{usuarioBean.idade}</b>.
23 </h:body>
24 </html>
```

Código XHTML 6.34: www.k19.com.br/exercicios/k12/06/006/usuario.xhtml

7 Observe o resultado das alterações realizadas acessando novamente a aplicação no endereço abaixo.

```
http://localhost:8080/K19-Componentes-Visuais/usuario.xhtml
```

8 Agora, crie um conversor monetário. Esse conversor deve receber três dados de entrada: as moedas de origem e destino, e o valor a ser convertido. Primeiramente, crie uma classe chamada **ConversorMonetarioBean** para implementar esse conversor.

```
1  package br.com.k19.controle;
2
3  import javax.faces.bean.ManagedBean;
4
5  @ManagedBean
6  public class ConversorMonetarioBean {
7    private String de;
8    private String para;
9    private Double valor;
10
11   // GETTERS E SETTERS
12 }
```

Código Java 6.6: www.k19.com.br/exercicios/k12/06/008/ConversorMonetarioBean.java

9 Na classe **ConversorMonetarioBean** adicione um método para realizar a conversão e um atributo para armazenar o resultado.

```
1  package br.com.k19.controle;
2
3  import java.util.LinkedHashMap;
4  import java.util.Map;
5  import javax.faces.bean.ManagedBean;
6
7  @ManagedBean
8  public class ConversorMonetarioBean {
9    private String de;
10   private String para;
11   private Double valor;
12
13   private Double resultado;
14
15   private Map<String, Double> taxas = new LinkedHashMap<String, Double>();
16
17   public ConversorMonetarioBean() {
18     this.taxas.put("Real", 1.0);
19     this.taxas.put("Euro", 2.33);
20     this.taxas.put("Peso argentino", 0.42);
21     this.taxas.put("Dolar americano", 1.84);
22   }
23
24   public void converte() {
25     this.resultado = this.valor * this.taxas.get(this.de) / this.taxas.get(this.para);
26   }
27
28   // GETTERS E SETTERS
29 }
```

Código Java 6.7: www.k19.com.br/exercicios/k12/06/009/ConversorMonetarioBean.java

10 Crie um arquivo XHTML chamado **conversor-monetario.xhtml**. Nesse arquivo, implemente um formulário para o usuário digitar os dados de entrada e ver o resultado da conversão.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:form>
    <h:commandButton value="Converter" action="#{conversorMonetarioBean.converte}"/>

    <h:inputText value="#{conversorMonetarioBean.valor}"/>

    <h:outputLabel value="de " for="de"/>

    <h:selectOneMenu value="#{conversorMonetarioBean.de}" id="de">
      <f:selectItems
        value="#{conversorMonetarioBean.taxas.keySet()}"
        var="moeda"
        itemValue="#{moeda}"
        itemLabel="#{moeda}" />
    </h:selectOneMenu>

    <h:outputLabel value="para " for="para"/>

    <h:selectOneMenu value="#{conversorMonetarioBean.para}" id="para">
      <f:selectItems
        value="#{conversorMonetarioBean.taxas.keySet()}"
        var="moeda"
        itemValue="#{moeda}"
        itemLabel="#{moeda}" />
    </h:selectOneMenu>
  </h:form>

  Resultado: #{conversorMonetarioBean.resultado}

</h:body>
</html>
```

Código XHTML 6.35: www.k19.com.br/exercicios/k12/06/010/conversor-monetario.xhtml

11 Acesse a aplicação no endereço abaixo.

http://localhost:8080/K19-Componentes-Visuais/conversor-monetario.xhtml



Exercícios Complementares

1 Considere o serviço de atendimento ao consumidor de uma loja virtual. Você deve criar um formulário para que o consumidor possa se comunicar com a loja. Esse formulário deve ter campos para o usuário informar o seu nome, e-mail, telefone, sexo e número do pedido. Além disso, deve existir um campo para o usuário selecionar um assunto e outro para ele escrever o seu comentário.



Textos e Imagens

Para inserir textos em uma página, podemos usar os componentes <h:outputText> e <h:outputFormat>. O texto exibido por tais componentes é definido pelo atributo value. Em particular, o <h:outputFormat> permite diversos tipos de formatação do texto que será exibido.

```
1 <h:outputFormat value="Preço do produto {0}: R$ {1}">
2   <f:param value="#{lojaBean.produto.nome}"/>
3   <f:param value="#{lojaBean.produto.preco}"/>
4 </h:outputFormat>
```

Código XHTML 6.37: Exemplo de utilização do <h:outputFormat>

O componente <h:graphicImage> é usado para inserir imagens. Podemos indicar o caminho da imagem através dos atributos value ou url. Esses dois atributos são exatamente iguais.

```
1 <h:graphicImage value="/imagens/k19-treinamentos.png" />
```

Código XHTML 6.38: Exemplo de inserção de imagem

Para padronizar a organização dos recursos da sua aplicação, o JSF 2 permite a criação de bibliotecas de imagens. Para criar uma biblioteca, basta adicionar um diretório na pasta /resources (na raiz da aplicação). Por exemplo, se criarmos o diretório /resources/imagens-k19/ e adicionarmos a imagem k19-treinamentos.png nesse diretório, podemos inseri-la em uma página da seguinte forma:

```
1 <h:graphicImage library="imagens-k19" name="k19-treinamentos.png" />
```

Código XHTML 6.39: Exemplo de inserção de imagem usando o atributo library



Mais Sobre

Todos os componentes do JSF possuem um atributo booleano chamado rendered. Esse atributo indica se o componente deve ser renderizado durante a etapa *Render Response* do processamento de uma requisição. O valor padrão desse atributo é true, o que indica que o componente deve ser exibido.

O preço de alguns produtos vendidos pela Amazon.com, por exemplo, não são exibidos na página de apresentação do produto. Para um produto desse tipo, o usuário precisa adicioná-lo ao carrinho de compras para então poder ver o seu preço.

Como exemplo, suponha que um objeto do tipo Produto possua uma propriedade booleana chamada mostraPreco. Ela indica se o preço daquele produto deve ser exibido ou não. Assim, podemos associar o atributo rendered do componente responsável pela exibição do preço do produto a essa propriedade. Isso pode ser feito usando *expression language*, como no exemplo abaixo.

```
1 <h:outputFormat value="Preço do produto {0}: R$ {1}"
2   rendered="#{lojaBean.produto.mostraPreco}">
3   <f:param value="#{lojaBean.produto.nome}"/>
4   <f:param value="#{lojaBean.produto.preco}"/>
5 </h:outputFormat>
```

Código XHTML 6.40: O preço do produto é exibido somente se o valor da propriedade `mostraPreco` *for* `true`

Exemplos

1. XHTML:

```
<h:outputText value="Curso: #{curso.sigla} - #{curso.descricao}"/>
```

HTML gerado:

```
Curso: K12 - Desenvolvimento Web com JSF2 e JPA2
```

2. XHTML:

```
<h:outputFormat value="{0} amava {1} que amava {2} que amava {3}
  que amava {4} que amava {5} que não amava ninguém. {0} foi
  para os Estados Unidos, {1} para o convento, {2} morreu de
  desastre, {3} ficou para tia, {4} suicidou-se e {5} casou
  com J. Pinto Fernandes que não tinha entrado na história.">

  <f:param value="João"/>
  <f:param value="Teresa"/>
  <f:param value="Raimundo"/>
  <f:param value="Maria"/>
  <f:param value="Joaquim"/>
  <f:param value="Lili"/>
</h:outputFormat>
```

HTML gerado:

```
João amava Teresa que amava Raimundo que amava Maria que amava Joaquim que amava
Lili que não amava ninguém. João foi para os Estados Unidos, Teresa para o
convento, Raimundo morreu de desastre, Maria ficou para tia, Joaquim suicidou-se
e Lili casou com J. Pinto Fernandes que não tinha entrado na história.
```

3. XHTML:

```
<h:graphicImage value="/imagens/k19-treinamentos.png" />
```

HTML gerado:

```
<img src="/K19-Componentes-Visuais/imagens/k19-treinamentos.png" />
```

Resultado:





Exercícios de Fixação

12 No arquivo **conversor-monetario.xhtml** do projeto **K19-Componentes-Visuais**, use a tag **<h:outputFormat>** para exibir o resultado do conversor monetário.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:form>
    <h:commandButton value="Converter" action="#{conversorMonetarioBean.converte}"/>

    <h:inputText value="#{conversorMonetarioBean.valor}"/>

    <h:outputLabel value="de " for="de"/>

    <h:selectOneMenu value="#{conversorMonetarioBean.de}" id="de">
      <f:selectItems
        value="#{conversorMonetarioBean.taxas.keySet()}"
        var="moeda"
        itemValue="#{moeda}"
        itemLabel="#{moeda}" />
    </h:selectOneMenu>

    <h:outputLabel value="para " for="para"/>

    <h:selectOneMenu value="#{conversorMonetarioBean.para}" id="para">
      <f:selectItems
        value="#{conversorMonetarioBean.taxas.keySet()}"
        var="moeda"
        itemValue="#{moeda}"
        itemLabel="#{moeda}" />
    </h:selectOneMenu>
  </h:form>

  <h:outputFormat value="{0} em {1} equivale a {2} em {3}"
    rendered="#{conversorMonetarioBean.resultado != null}">
    <f:param value="#{conversorMonetarioBean.valor}"/>
    <f:param value="#{conversorMonetarioBean.de}"/>
    <f:param value="#{conversorMonetarioBean.resultado}"/>
    <f:param value="#{conversorMonetarioBean.para}"/>
  </h:outputFormat>

</h:body>
</html>
```

Código XHTML 6.44: www.k19.com.br/exercicios/k12/06/012/conversor-monetario.xhtml

13 Acesse novamente a aplicação em
http://localhost:8080/K19-Componentes-Visuais/conversor-monetario.xhtml



Componentes de Organização

O JSF define dois componentes que nos ajudam a organizar visualmente os elementos de uma página JSF.

- <h:panelGrid>

Organiza os elementos em uma grade.

- <h:panelGroup>
  Permite que diversos componentes sejam tratados como um único componente.

O <h:panelGrid> é usado para organizar componentes em forma de uma grade. O uso de um <h:panelGrid> é bastante simples. Basicamente, devemos configurar a quantidade de colunas que desejamos. Para isso, utilizamos o atributo columns.

```
<h:panelGrid columns="2">
  <h:outputLabel value="Nome do curso: " for="nome"/>
  <h:inputText value="#{cursosBean.curso.nome}" id="nome"/>

  <h:outputLabel value="Descrição: " for="descricao"/>
  <h:inputTextarea value="#{cursosBean.curso.descricao}" id="descricao"/>

  <h:outputLabel value="Carga horária: " for="carga-horaria"/>
  <h:inputText value="#{cursosBean.curso.cargaHoraria}" id="carga-horaria"/>

  <h:commandButton action="#{cursosBean.cadastraCurso}" value="Cadastrar"/>
</h:panelGrid>
```

Código XHTML 6.45: Organização de um formulário com o <h:panelGrid>

Os componentes são inseridos na grade de acordo com a ordem em que estão apresentados no código. O primeiro elemento é inserido na primeira coluna da primeira linha. O segundo elemento é inserido na segunda coluna da primeira linha. Uma vez que uma linha foi completamente preenchida, o próximo componente é inserido na primeira coluna da próxima linha, e o processo se repete até o último componente.

Suponha que seja necessário inserir mais de um componente em uma mesma célula de um <h:panelGrid>. No exemplo acima, podemos adicionar um ícone de um relógio ao lado do rótulo "Carga horária:". No entanto, queremos que a imagem e o rótulo sejam colocados na mesma célula. Para isso, podemos agrupá-los usando um <h:panelGroup>.

```
<h:panelGroup>
  <h:outputLabel value="Carga horária: " for="carga-horaria"/>
  <h:graphicImage library="imagens" name="relogio.png" />
</h:panelGroup>
```

Código XHTML 6.46: Agrupando componentes com o <h:panelGroup>

Exemplos

1. XHTML:

```
<h:panelGrid columns="2">
  <h:outputLabel value="Nome do curso: " for="nome"/>
  <h:inputText value="#{cursosBean.curso.nome}" id="nome"/>

  <h:outputLabel value="Descrição: " for="descricao"/>
  <h:inputTextarea value="#{cursosBean.curso.descricao}" id="descricao"/>

  <h:panelGroup>
    <h:outputLabel value="Carga horária: " for="carga-horaria"/>
    <h:graphicImage library="imagens" name="relogio.png" />
  </h:panelGroup>
  <h:inputText value="#{cursosBean.curso.cargaHoraria}" id="carga-horaria"/>

```

```
14   <h:commandButton action="#{cursosBean.cadastraCurso}" value="Cadastrar"/>
15 </h:panelGrid>
```

HTML gerado:

```
1  <table>
2    <tbody>
3      <tr>
4        <td><label for="nome"> Nome do curso: </label></td>
5        <td><input id="nome" type="text" name="nome" /></td>
6      </tr>
7      <tr>
8        <td><label for="descricao"> Descrição: </label></td>
9        <td><textarea id="descricao" name="descricao"></textarea></td>
10     </tr>
11     <tr>
12       <td>
13         <label for="carga-horaria"> Carga horária: </label>
14         <img src="/K19-Componentes-Visuais/javax.faces.resource/relogio.png.xhtml?ln=↩
               imagens" />
15       </td>
16       <td><input id="carga-horaria" type="text" name="carga-horaria" /></td>
17     </tr>
18     <tr>
19       <td><input type="submit" name="j_idt98" value="Cadastrar" /></td>
20     </tr>
21   </tbody>
22 </table>
```

Resultado:

Nome do curso:

Descrição:

Carga horária:

Cadastrar



Tabelas

O JSF fornece o componente <h:dataTable> para a criação de tabelas. Podemos associar uma lista de elementos a um <h:dataTable> através do atributo value. Automaticamente, esse componente gera uma linha para cada item da lista. Os itens da lista podem ser acessados através de uma variável definida pelo atributo var.

As colunas da tabela são definidas pelo componente <h:column>. Podemos acrescentar cabeçalhos e rodapés à tabela e a cada coluna usando o componente <f:facet>.

Veja um exemplo a seguir.

```
1  <h:dataTable value="#{cursosBean.cursos}" var="curso">
2
3    <f:facet name="header">Lista de cursos</f:facet>
4
```

```
5   <h:column>
6     <f:facet name="header">Sigla</f:facet>
7     #{curso.sigla}
8   </h:column>
9
10  <h:column>
11    <f:facet name="header">Nome</f:facet>
12    #{curso.nome}
13  </h:column>
14
15  <h:column>
16    <f:facet name="header">Descrição</f:facet>
17    #{curso.descricao}
18  </h:column>
19
20  <h:column>
21    <f:facet name="header">Adicionar turma</f:facet>
22    <h:commandLink
23      value="Adicionar turma"
24      action="#{cursosBean.adicionarTurma(curso)}" />
25  </h:column>
26</h:dataTable>
```

Código XHTML 6.48: Exemplo de criação de tabela com o componente `<h:dataTable>`

Observe, no exemplo acima, que a tabela está associada à propriedade `cursos` do managed bean `cursosBean`. A variável `curso` é utilizada para acessar cada um dos elementos da propriedade `cursos`.

O cabeçalho "Lista de cursos" da tabela está definido com o componente `<f:facet>` assim como o cabeçalho de cada coluna ("Sigla", "Nome", "Descrição" e "Adicionar turma").

O HTML gerado pelo JSF a partir do código acima seria mais ou menos assim:

```
1  <table>
2    <thead>
3      <tr>
4        <th colspan="4" scope="colgroup">Lista de cursos</th>
5      </tr>
6      <tr>
7        <th scope="col">Sigla</th>
8        <th scope="col">Nome</th>
9        <th scope="col">Descrição</th>
10       <th scope="col">Adicionar turma</th>
11     </tr>
12   </thead>
13   <tbody>
14     <tr>
15       <td>K11</td>
16       <td>Orientação a Objetos em Java</td>
17       <td>
18         Com este curso você vai obter uma base sólida de
19         conhecimentos de Java e de Orientação a Objetos
20       </td>
21       <td>
22         <input
23           type="submit" name="j_idt100:j_idt101:0:j_idt114"
24           value="Adicionar turma" />
25       </td>
26     </tr>
27     <tr>
28       <td>K12</td>
29       <td>Desenvolvimento Web com JSF2 e JPA2</td>
30       <td>
31         Depois deste curso, você estará apto a desenvolver
32         aplicações Web com os padrões da plataforma Java
```

```
33        </td>
34        <td>
35          <input
36            type="submit" name="j_idt100:j_idt101:1:j_idt114"
37            value="Adicionar turma" />
38        </td>
39      </tr>
40      <tr>
41        <td>K21</td>
42        <td>Persistência com JPA2 e Hibernate</td>
43        <td>
44          Neste curso de Java Avançado, abordamos de maneira profunda
45          os recursos de persistência do JPA2 e do Hibernate
46        </td>
47        <td>
48          <input
49            type="submit" name="j_idt100:j_idt101:2:j_idt114"
50            value="Adicionar turma" />
51        </td>
52      </tr>
53    </tbody>
54  </table>
```

Código HTML 6.24: Código HTML gerado pelo JSF

A figura abaixo mostra como essa tabela é apresentada no navegador.

Lista de cursos

Sigla	Nome	Descrição	Adicionar Turma
K11	Orientação a Objetos em Java	Com este curso você vai obter uma base sólida de conhecimentos de Java e de Orientação a Objetos.	Adiciona turma
K12	Desenvolvimento Web com JSF2 e JPA2	Depois deste curso, você estará apto a desenvolver aplicações Web com os padrões da plataforma Java	Adiciona turma
K21	Persistência com JPA2 e Hibernate	Neste curso de Java Avançado, abordamos de maneira profunda os recursos de persistência do JPA2 e do Hibernate	Adiciona turma

Figura 6.1: Representação do HTML no navegador



Exercícios de Fixação

14 No projeto **K19-Componentes-Visuais**, crie uma página que contenha um formulário para adicionar cursos e que exiba os cursos já adicionados. Primeiramente, crie uma classe chamada **Curso** em um pacote chamado **br.com.k19.modelo** para representar um curso. Essa classe deve ter dois atributos do tipo **String**: um para armazenar o nome e outro para armazenar a sigla do curso.

```
1 package br.com.k19.modelo;
2
3 public class Curso {
```

```
4   private String nome;
5   private String sigla;
6
7   // GETTERS E SETTERS
8 }
```

Código Java 6.9: www.k19.com.br/exercicios/k12/06/014/Curso.java

15 Crie uma classe chamada **CursosBean** para armazenar uma lista de cursos. Para que os cursos adicionados sejam mantidos nessa variável entre uma requisição e outra, marque a classe com a anotação **@SessionScoped**. No Capítulo 9 discutiremos sobre escopos de managed beans.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Curso;
4  import java.util.ArrayList;
5  import java.util.List;
6  import javax.faces.bean.ManagedBean;
7  import javax.faces.bean.SessionScoped;
8
9  @ManagedBean
10 @SessionScoped
11 public class CursosBean {
12   private List<Curso> cursos = new ArrayList<Curso>();
13
14   // GETTERS E SETTERS
15 }
```

Código Java 6.10: www.k19.com.br/exercicios/k12/06/015/CursosBean.java

16 Na classe **CursosBean**, adicione um atributo do tipo **Curso** para armazenar os dados de um curso. Além disso, crie um método para adicionar o curso na lista de cursos.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Curso;
4  import java.util.ArrayList;
5  import java.util.List;
6  import javax.faces.bean.ManagedBean;
7  import javax.faces.bean.SessionScoped;
8
9  @ManagedBean
10 @SessionScoped
11 public class CursosBean {
12   private List<Curso> cursos = new ArrayList<Curso>();
13   private Curso curso = new Curso();
14
15   public void adicionaCurso() {
16     this.cursos.add(this.curso);
17     this.curso = new Curso();
18   }
19
20   // GETTERS E SETTERS
21 }
```

Código Java 6.11: www.k19.com.br/exercicios/k12/06/016/CursosBean.java

17 Crie um arquivo XHTML chamado **cursos.xhtml** e implemente um formulário para adicionar cursos à lista de cursos do managed bean **CursosBean**. Use a tag **<h:panelGrid>** para organizar o

formulário.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:form>
    <h:panelGrid columns="2">
      <h:outputLabel value="Nome: " for="campo-nome" />
      <h:inputText value="#{cursosBean.curso.nome}" id="campo-nome" />

      <h:outputLabel value="Sigla: " for="campo-sigla" />
      <h:inputText value="#{cursosBean.curso.sigla}" id="campo-sigla" />

      <h:commandButton value="Adicionar" action="#{cursosBean.adicionaCurso}" />
    </h:panelGrid>
  </h:form>
</h:body>
</html>
```

Código XHTML 6.49: www.k19.com.br/exercicios/k12/06/017/cursos.xhtml

18 No arquivo **cursos.xhtml**, insira um trecho de código para exibir os cursos já adicionados. Use a tag **<h:dataTable>** para apresentar os cursos. Você pode usar o atributo **rendered** desse componente para exibir a tabela apenas quando a lista de cursos não está vazia.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:form>
    <h:panelGrid columns="2">
      <h:outputLabel value="Sigla: " for="campo-sigla" />
      <h:inputText value="#{cursosBean.curso.sigla}" id="campo-sigla" />

      <h:outputLabel value="Nome: " for="campo-nome" />
      <h:inputText value="#{cursosBean.curso.nome}" id="campo-nome" />

      <h:commandButton value="Adicionar" action="#{cursosBean.adicionaCurso}" />
    </h:panelGrid>
  </h:form>

  <h:dataTable value="#{cursosBean.cursos}" var="curso"
    rendered="#{not empty cursosBean.cursos}">

    <f:facet name="header">Lista de Cursos</f:facet>

    <h:column>
      <f:facet name="header">Sigla</f:facet>
      #{curso.sigla}
    </h:column>

    <h:column>
```

```
35        <f:facet name="header">Nome</f:facet>
36        #{curso.nome}
37      </h:column>
38    </h:dataTable>
39  </h:body>
40  </html>
```

Código XHTML 6.50: www.k19.com.br/exercicios/k12/06/018/cursos.xhtml

Acesse a aplicação no endereço:
http://localhost:8080/K19-Componentes-Visuais/cursos.xhtml

Exercícios Complementares

2 Usando o componente **<h:dataTable>**, liste os produtos de uma loja virtual. A tabela deve mostrar o nome e o preço de cada produto. No projeto **K19-Componentes-Visuais**, crie uma classe chamada **Produto** em um pacote chamado chamado **br.com.k19.modelo**. Essa classe deve ter um atributo do tipo **String** para armazenar o nome do produto e um atributo do tipo **Double** para guardar o preço do produto. Crie uma classe chamada **ProdutosBean** em um pacote chamado **br.com.k19.controle** para guardar uma lista de produtos. Implemente a listagem de produtos em um arquivo XHTML chamado `lista-de-produtos.xhtml`.

3 Na classe **Produto**, acrescente uma propriedade do tipo **Boolean** para indicar se o preço do produto deve ser exibido. Modifique a tabela de listagem dos produtos para considerar essa nova restrição. Se o preço de um produto não puder ser exibido, então o texto "Adicione o produto ao carrinho para ver o preço" deve aparecer no lugar do preço do produto.

Mensagens

Durante o processamento de uma requisição, podemos adicionar mensagens que podem ser exibidas na página de resposta. Uma mensagem pode ser adicionada, por exemplo, da seguinte forma:

```
1 FacesMessage mensagem = new FacesMessage("Turma adicionada");
2 FacesContext.getCurrentInstance().addMessage(null, mensagem);
```

Código Java 6.16: Adicionando uma mensagem

O primeiro parâmetro do método `addMessage()` define qual componente será associado à mensagem adicionada. Quando o valor `null` é passado nesse parâmetro, a mensagem é considerada global, ou seja, ela não será associada a nenhum componente específico.

Na tela, podemos exibir todas as mensagens geradas no processamento da requisição através do componente `<h:messages>`. Se desejarmos exibir apenas as mensagens globais, devemos utilizar o atributo `globalOnly` da seguinte forma:

```
1 <h:messages globalOnly="true" />
```

Código XHTML 6.53: Exibindo as mensagens globais

As mensagens são comumente utilizadas para indicar erros de conversão e validação. Veremos mais detalhes sobre isso no Capítulo 10.



Adicionando JavaScript e CSS

Podemos adicionar recursos como scripts e arquivos de estilos (CSS) usando diretamente as tags `<script>` e `<link>` do HTML. Contudo, o JSF define outra maneira de inserir esses recursos nas páginas da aplicação. Tais recursos podem ser inseridos com as tags `<h:outputScript>` e `<h:outputStylesheet>`. A utilização dessas tags facilita a criação de componentes visuais personalizados ou o gerenciamento de recursos que dependem de regionalidade.

```
1 <h:outputScript name="k19.js" library="javascript" target="head" />
2 <h:outputStylesheet name="k19.css" library="css" />
```

Código XHTML 6.54: Exemplo de adição de código JavaScript e arquivo CSS na página da aplicação



Outros Componentes

<ui:remove>

Para comentar (ou excluir) partes de código XHTML de uma aplicação, o JSF provê a tag `<ui:remove>`. Qualquer trecho de código dentro dessa tag é removido durante o processamento de uma página JSF. No exemplo abaixo, a caixa para inserir o sobrenome do usuário não será processada e, portanto, não será exibida na página gerada.

```
1 <h:outputText value="Nome: #{testeBean.nome}" />
2 <ui:remove>
3   <h:outputText value="Sobrenome: #{testeBean.sobrenome}" />
4 </ui:remove>
```

Código XHTML 6.55: Excluindo parte do código de uma página JSF

O trecho de código HTML correspondente, gerado pelo JSF, é o seguinte:

```
1 Nome: Jonas
```

Código HTML 6.25: Trecho de código HTML gerado pelo JSF

Alternativamente, podemos usar os delimitadores `<!--` e `-->` do XML para comentar o código. Nesse caso, contudo, o trecho de código comentado será processado pelo JSF. Por exemplo, a partir do código

```
1 <h:outputText value="Nome: #{testeBean.nome}" />
2 <!--
3 <h:outputText value="Sobrenome: #{testeBean.sobrenome}" />
4 -->
```

Código XHTML 6.56: Uso de delimitadores do XML para comentar código

o JSF produzirá o seguinte código HTML:

```
Nome: Jonas
<!--
<h:outputText value="Sobrenome: Hirata" />
-->
```

Código HTML 6.26: Trecho de código HTML gerado pelo JSF

Note que o código dentro do comentário foi processado pelo JSF. A string "Hirata" foi obtida da propriedade sobrenome do managed bean testeBean.

<ui:repeat>

A tag <ui:repeat> é usada para iterar sobre coleções. Ela possui dois atributos obrigatórios: value e var. O atributo value deve ser associado a uma coleção de objetos e o atributo var deve definir o nome da variável. Essa variável será usada para referenciar cada um dos elementos da coleção. No exemplo abaixo, temos um código para gerar uma lista HTML não ordenada a partir de uma coleção de cursos.

```
<h:outputText value="Alguns cursos da K19:" />
<ul>
  <ui:repeat value="#{testeBean.cursos}" var="curso">
    <li>
      <h:outputText value="#{curso.sigla}: #{curso.nome}" />
    </li>
  </ui:repeat>
</ul>
```

Código XHTML 6.57: Iterando sobre uma lista de cursos

A partir desse código, o JSF gera o seguinte código HTML:

```
Alguns cursos da K19:
<ul>
  <li>K11: Orientação a Objetos em Java</li>
  <li>K12: Desenvolvimento Web com JSF2 e JPA2</li>
  <li>K21: Persistência com JPA2 e Hibernate</li>
  <li>K51: Design Patterns em Java</li>
</ul>
```

Código HTML 6.27: Código HTML gerado pelo JSF



Exercícios de Fixação

19 No projeto **K19-Componentes-Visuais**, crie um arquivo CSS para formatar a página que lista os cursos. Na raiz da aplicação crie um diretório chamado **resources** contendo um outro diretório chamado **css**. Dentro do diretório **css** crie um arquivo CSS chamado **k19.css** com o conteúdo abaixo.

```
body {
  font-family: arial, helvetica, sans-serif;
  font-size: 14px;
}

h1 {
  color: #006699;
  font-size: 18px;
```

```
}

ul {
  list-style-type: square;
}

input {
  background-color: #E6E6FA;
  border: solid 1px #000000;
}
```

Código CSS 6.1: www.k19.com.br/exercicios/k12/06/019/k19.css

20 Use a tag **<h:outputStylesheet>** para incluir o arquivo de estilos na página definida por **cursos.xhtml**. Use a tag <ui:repeat> para exibir os cursos.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core"
  xmlns:ui="http://java.sun.com/jsf/facelets">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>

  <h:outputStylesheet name="k19.css" library="css"/>

  <h:form>
    <h:panelGrid columns="2">
      <h:outputLabel value="Sigla: " for="campo-sigla" />
      <h:inputText value="#{cursosBean.curso.sigla}" id="campo-sigla" />

      <h:outputLabel value="Nome: " for="campo-nome" />
      <h:inputText value="#{cursosBean.curso.nome}" id="campo-nome" />

      <h:commandButton value="Adicionar" action="#{cursosBean.adicionaCurso}" />
    </h:panelGrid>
  </h:form>

  <h1>Alguns cursos da K19:</h1>
  <ul>
    <ui:repeat value="#{cursosBean.cursos}" var="curso">
      <li>
        <h:outputText value="#{curso.sigla}: #{curso.nome}" />
      </li>
    </ui:repeat>
  </ul>
</h:body>
</html>
```

Código XHTML 6.58: www.k19.com.br/exercicios/k12/06/020/cursos.xhtml

21 Acesse novamente a aplicação em http://localhost:8080/K19-Componentes-Visuais/cursos.xhtml e observe o resultado das alterações realizas.



Exercícios Complementares

4 Considere novamente a página de adição e listagem de cursos. Modifique sua aplicação de forma que uma mensagem de confirmação seja exibida na tela após o usuário adicionar um novo curso.

CAPÍTULO 7

TEMPLATES E MODULARIZAÇÃO

Certamente, você já ouviu alguém falar da importância da reutilização de código no desenvolvimento de software. Os objetivos principais dessa reutilização são diminuir o tempo e o custo do desenvolvimento e da manutenção das aplicações.

Com essa ideia em mente, um projeto chamado **Facelets** foi desenvolvido com o objetivo principal de facilitar todo o processo de desenvolvimento e manutenção das telas de uma aplicação JSF. O Facelets já faz parte do JSF 2 e é a engine padrão para o gerenciamento das telas de aplicações web JSF 2.



Templates

A reutilização do código das telas é realizada principalmente pelo uso de templates. A ideia é identificar um padrão em um determinado conjunto de telas e defini-lo através de um esqueleto (template). Esse esqueleto é formado por trechos estáticos e dinâmicos. O posicionamento desses trechos é sempre fixo. Contudo, ao contrário do conteúdo de um trecho estático, o conteúdo de um trecho dinâmico pode diferir de tela para tela.

Por exemplo, considere uma aplicação na qual todas as telas são divididas em três partes: cabeçalho, corpo e rodapé. Essas partes estão sempre posicionadas da mesma forma. O cabeçalho é posicionado no topo da página, o rodapé, no final e o corpo, entre os dois anteriores. O conteúdo do cabeçalho e do rodapé é sempre o mesmo em todas as telas da aplicação. Por outro lado, o conteúdo do corpo varia de tela para tela.

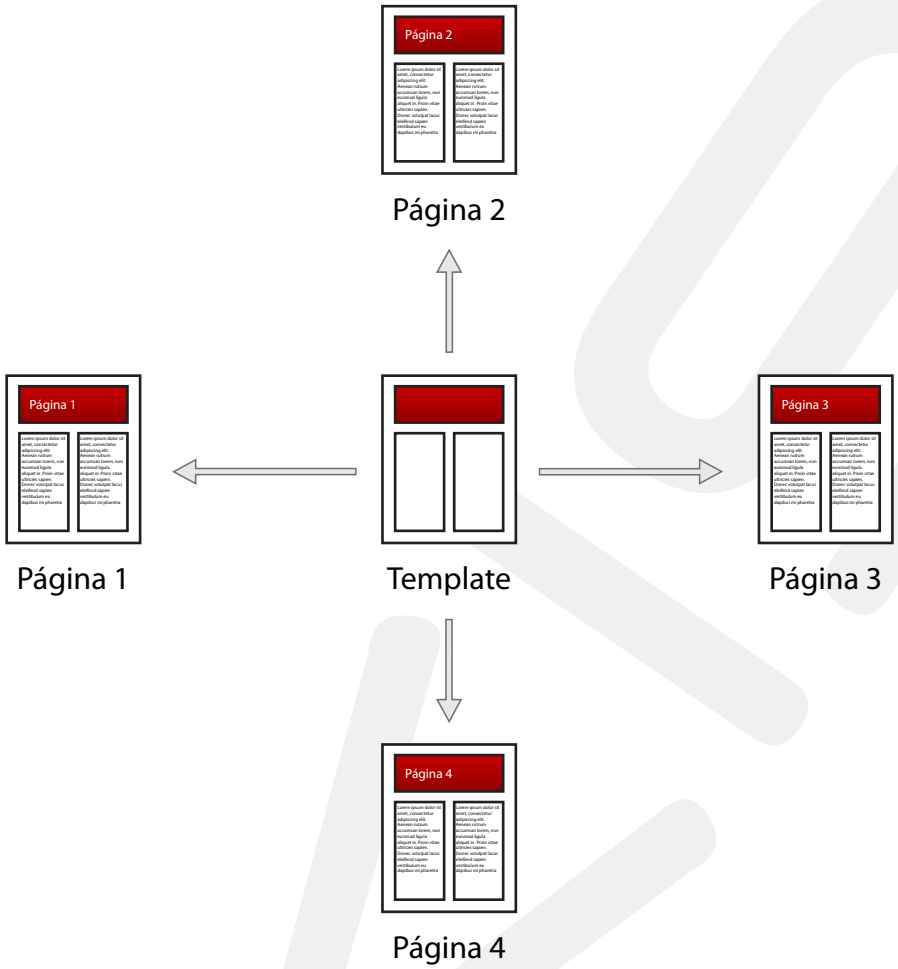


Figura 7.1: Template utilizado por diversas páginas

Criando um template

A criação de um template é simples. Basta criar um arquivo XHTML e definir o posicionamento dos trechos estáticos e dinâmicos. O conteúdo dos trechos estáticos também deve ser definido da forma usual dentro do template. Por outro lado, o conteúdo dos trechos dinâmicos só será definido nas telas. Para indicar o posicionamento dos trechos dinâmicos, devemos utilizar a tag `<ui:insert>`.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://java.sun.com/jsf/html"
    xmlns:ui="http://java.sun.com/jsf/facelets">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <div id="header">
    <img src="k19-logo.png" />
    <hr />
  </div>

  <ui:insert name="corpo-da-pagina"> Espaço para o conteúdo da tela </ui:insert>
```

```
19
20    <div id="footer" style="text-align: center">
21      <hr />
22      &copy; 2010 K19. Todos os direitos reservados.
23    </div>
24  </h:body>
25  </html>
```

Código XHTML 7.1: template.xhtml

O componente <ui:insert> é utilizado para posicionar os trechos dinâmicos. O atributo name desse componente é utilizado para nomear os trechos dinâmicos.

Normalmente, a declaração <!DOCTYPE> não varia de tela para tela dentro de uma aplicação. Dessa forma, podemos adicioná-la de forma estática no template, conforme o exemplo acima.

Vimos no Capítulo 6 que é fundamental a utilização dos componentes <h:head> e <h:body>. No exemplo acima, esses componentes foram definidos estaticamente no template. Assim, todas as telas que utilizam esse template não correm mais o risco de ficarem sem esses componentes.

Utilizando templates

Para criar uma tela que usa um determinado template, devemos criar um arquivo XHTML e adicionar a tag <ui:composition> a esse arquivo. O arquivo do template desejado deve ser indicado através do atributo template da tag <ui:composition>.

```
1  <ui:composition template="/template.xhtml"
2    xmlns="http://www.w3.org/1999/xhtml"
3    xmlns:ui="http://java.sun.com/jsf/facelets">
4  </ui:composition>
```

Código XHTML 7.2: tela.xhtml

Todo conteúdo não contido na tag <ui:composition> será descartado pelo JSF no processo de construção da tela.

O conteúdo de um trecho dinâmico pode ser definido através do componente <ui:define>. Esse componente possui o atributo name que é utilizado para indicar qual trecho dinâmico do template queremos definir.

No exemplo acima, podemos definir o conteúdo do trecho dinâmico corpo-da-pagina da seguinte forma:

```
1  <ui:composition template="/template.xhtml"
2    xmlns="http://www.w3.org/1999/xhtml"
3    xmlns:h="http://java.sun.com/jsf/html"
4    xmlns:ui="http://java.sun.com/jsf/facelets">
5
6    <ui:define name="corpo-da-pagina">
7      <h1>Lista de treinamentos</h1>
8      <h2>K11 - Orientação a Objetos em Java</h2>
9      <h2>K12 - Desenvolvimento Web com JSF2 e JPA2</h2>
10   </ui:define>
11 </ui:composition>
```

Código XHTML 7.3: tela.xhtml

Se o conteúdo de um trecho dinâmico não for definido, o JSF utilizará o conteúdo existente no corpo da tag `<ui:insert>` definido no template. No exemplo acima, caso o conteúdo do trecho dinâmico `corpo-da-pagina` não fosse definido no arquivo `tela.xhtml`, o JSF utilizaria o texto "Espaço para o conteúdo da tela", que foi definido no arquivo `template.xhtml` como conteúdo padrão para esse trecho.



Pare para pensar...
Em geral, os templates não definem uma tela concreta da aplicação. Eles funcionam como uma base para a criação de telas concretas. Dessa forma, não seria interessante que os navegadores acessassem diretamente a tela correspondente a um template, pois, provavelmente, essa tela estaria incompleta. Para evitar esse comportamento indesejado, podemos colocar todos os templates dentro do diretório `/WEB-INF`.



Exercícios de Fixação

1 Crie um projeto chamado **K19-Templates-e-Modularizacao** seguindo os passos vistos no exercício do Capítulo 5.

2 Adicione um diretório chamado **templates** na pasta **WEB-INF**. Crie um template chamado **template.xhtml** na pasta **WEB-INF/templates**.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
        "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <div id="header"><img src="imagens/k19-logo.png" />
  <hr />
  </div>

  <ui:insert name="conteudo">
    Espaço para o conteúdo da tela
  </ui:insert>

  <div id="footer" style="text-align: center">
  <hr />
  &copy; 2012 K19. Todos os direitos reservados.</div>
</h:body>
</html>
```

Código XHTML 7.4: www.k19.com.br/exercicios/k12/07/002/template.xhtml

3 Crie uma pasta chamada **imagens** no projeto **K19-Templates-e-Modularizacao**.

4 Copie o arquivo **k19-logo.png** da pasta **K19-Arquivos/imagens** para o diretório **imagens**.



Importante
Você também pode obter o arquivo **k19-logo.png** através do site da K19: www.k19.com.br/arquivos.

5 Monte uma página que use o template criado anteriormente. O nome do arquivo deve ser **formulario.xhtml**.

```
<ui:composition template="/WEB-INF/templates/template.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="conteudo">
    <h:form>
      <h:outputLabel value="Nome: " for="campo-nome" />
      <h:inputText id="campo-nome" />
      <h:commandButton value="Enviar" />
    </h:form>
  </ui:define>
</ui:composition>
```

Código XHTML 7.5: www.k19.com.br/exercicios/k12/07/005/formulario.xhtml

6 Remova o projeto **K19-Componentes-Visuais** do GlassFish e do Wildfly. Implante o projeto **K19-Templates-e-Modularizacao** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

http://localhost:8080/K19-Templates-e-Modularizacao/formulario.xhtml

7 Implante o projeto **K19-Templates-e-Modularizacao** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

http://localhost:8080/K19-Templates-e-Modularizacao/formulario.xhtml



Modularização

Os trechos estáticos ou dinâmicos definidos em um template possuem posição fixa. Em determinadas situações, é necessário tornar flexível o posicionamento de um determinado trecho.

Por exemplo, considere uma aplicação com 50 páginas diferentes. Os usuários dessa aplicação podem enviar mensagens para o administrador do sistema quando identificarem algum problema. Essas mensagens são enviadas através de um formulário HTML. Esse formulário de contato deve ser exibido em todas as telas da aplicação.

Nesse caso, poderíamos pensar em criar um template e definir o formulário de contato como um trecho estático desse template. Contudo, há uma restrição importante que descarta essa abordagem.

O formulário de contato deve ser exibido em posições diferentes nas telas da aplicação. Por exemplo, em algumas telas o formulário aparecerá no topo e em outras ele aparecerá no centro.

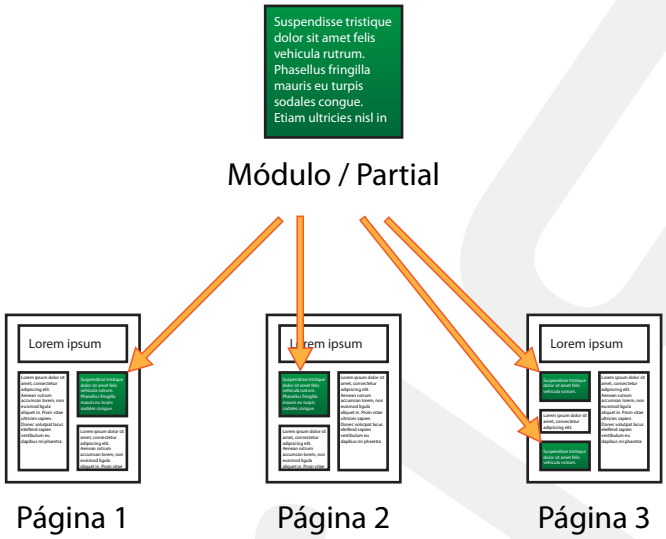


Figura 7.2: Módulo reaproveitado em diversas páginas

Nessa situação, a melhor abordagem é definir o formulário de contato, separadamente, em um arquivo XHTML. O código XHTML que define o formulário de contato deve ser inserido no corpo da tag `<ui:composition>`. Observe o código abaixo.

```
<ui:composition
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <h:form>
    <h:panelGrid>
      <h:outputLabel value="Mensagem: " for="mensagem"/>
      <h:inputTextarea id="mensagem" value="#{mensagensBean.mensagem}"/>
      <h:commandButton value="Enviar" action="#{mensagensBean.envia}"/>
    </h:panelGrid>
  </h:form>
</ui:composition>
```

Código XHTML 7.6: formulario-de-contato.xhtml

Com o formulário de contato definido em um arquivo separado, podemos utilizar a tag `<ui:include>` para adicioná-lo onde for necessário.

```
...
<ui:include src="/formulario-de-contato.xhtml"/>
...
```

Código XHTML 7.7: Inserindo o formulário de contato

Parâmetros

Em algumas situações é necessário passar parâmetros de um arquivo XHTML para outro. Por exemplo, considere a aplicação de uma livraria. Na primeira página dessa aplicação, uma lista com

livros mais vendidos é exibida no canto direito. Em outra página, uma lista com os livros mais caros é exibida no canto esquerdo.

Podemos criar um arquivo XHTML e atribuir a ele a tarefa de exibir uma lista qualquer de livros independentemente de posicionamento.

```
<ui:composition
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <h:dataTable value="#{livros}" var="livro">
    ...
  </h:dataTable>
</ui:composition>
```

Código XHTML 7.8: lista-livros.xhtml

Observe, no código acima, que o componente `<h:dataTable>` foi vinculado a uma variável chamada `livros`. Essa variável é um parâmetro do arquivo `lista-livros.xhtml`.

Na primeira página da aplicação da livraria, podemos adicionar o arquivo `lista-livros.xhtml` com a tag `<ui:include>`, passando como parâmetro a lista dos livros mais vendidos. Para isso, devemos utilizar o componente `<ui:param>`.

```
...
<ui:include src="/lista-livros.xhtml">
  <ui:param name="livros" value="#{livrosBean.listaDosLivrosMaisVendidos}"/>
</ui:include>
...
```

Código XHTML 7.9: Inserindo a lista dos livros mais vendidos

Em outra página da aplicação da livraria, podemos adicionar o arquivo `lista-livros.xhtml` passando como parâmetro a lista dos livros mais caros.

```
...
<ui:include src="/lista-livros.xhtml">
  <ui:param name="livros" value="#{livrosBean.listaDosLivrosMaisCaros}"/>
</ui:include>
...
```

Código XHTML 7.10: Inserindo a lista dos livros mais caros



Exercícios de Fixação

8 Vamos implementar uma listagem de instrutores no projeto **K19-Templates-e-Modularizacao**. O primeiro passo é criar uma classe para modelar os instrutores. Crie um pacote chamado **br.com.k19.modelo** no projeto **K19-Templates-e-Modularizacao** e adicione nele uma classe chamada **Instrutor** com seguinte código.

```
package br.com.k19.modelo;

public class Instrutor {
  private String nome;
```

```
5   private String dataDeNascimento;
6
7   // GETTERS E SETTERS
8 }
```

Código Java 7.1: www.k19.com.br/exercicios/k12/07/008/Instrutor.java

9 Faça um managed bean que forneça uma lista de instrutores para uma tela de listagem de instrutores. Crie um pacote chamado **br.com.k19.controle** no projeto **K19-Templates-e-Modularizacao** e adicione nele uma classe chamada **InstrutorBean** com seguinte código:

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Instrutor;
4  import java.util.ArrayList;
5  import java.util.List;
6  import javax.faces.bean.ManagedBean;
7
8  @ManagedBean
9  public class InstrutorBean {
10
11   private List<Instrutor> instrutores = new ArrayList<Instrutor>();
12
13   public InstrutorBean() {
14     Instrutor rafael = new Instrutor();
15     rafael.setNome("Rafael Cosentino");
16     rafael.setDataDeNascimento("30/10/1984");
17
18     Instrutor marcelo = new Instrutor();
19     marcelo.setNome("Marcelo Martins");
20     marcelo.setDataDeNascimento("02/04/1985");
21
22     this.instrutores.add(rafael);
23     this.instrutores.add(marcelo);
24   }
25
26   public List<Instrutor> getInstrutores() {
27     return instrutores;
28   }
29
30   public void setInstrutores(List<Instrutor> instrutores) {
31     this.instrutores = instrutores;
32   }
33 }
```

Código Java 7.2: www.k19.com.br/exercicios/k12/07/009/InstrutorBean.java

10 Crie um diretório chamado **partials** na pasta **WEB-INF**. Adicione nesse diretório um arquivo chamado **instrutor-info.xhtml** com o seguinte conteúdo.

```
1  <ui:composition xmlns="http://www.w3.org/1999/xhtml"
2    xmlns:h="http://java.sun.com/jsf/html"
3    xmlns:ui="http://java.sun.com/jsf/facelets">
4
5  <li>
6    <h:outputText value="Nome: #{instrutor.nome}"/>
7    <br/>
8    <h:outputText value="Data Nascimento: #{instrutor.dataDeNascimento}"/>
9  </li>
10
11 </ui:composition>
```

Código XHTML 7.11: www.k19.com.br/exercicios/k12/07/010/instrutor-info.xhtml

11 Crie uma página no projeto **K19-Templates-e-Modularizacao** com o nome **listagem-de-instrutores.xhtml** e com o seguinte código.

```
<ui:composition template="/WEB-INF/templates/template.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://java.sun.com/jsf/html"
    xmlns:f="http://java.sun.com/jsf/core"
    xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="conteudo">
    <ul>
      <ui:repeat var="instrutor" value="#{instrutorBean.instrutores}">
        <ui:include src="/WEB-INF/partials/instrutor-info.xhtml">
          <ui:param name="instrutor" value="#{instrutor}"/>
        </ui:include>
      </ui:repeat>
    </ul>
  </ui:define>
</ui:composition>
```

Código XHTML 7.12: www.k19.com.br/exercicios/k12/07/011/listagem-de-instrutores.xhtml

Veja o resultado acessando a url:

http://localhost:8080/K19-Templates-e-Modularizacao/listagem-de-instrutores.xhtml



Exercícios Complementares

1 Usando templates, você deve criar uma página para exibir os detalhes dos produtos de uma loja virtual. A página de apresentação dos produtos deve ter o seguinte formato. Uma imagem do produto deve ser apresentada do lado esquerdo da página. O nome e o preço do produto devem ser exibidos do lado direito da imagem. Uma descrição do produto deve aparecer abaixo de tudo. Além disso, o logotipo da loja deve aparecer no topo da página e, logo abaixo, deve haver uma caixa de seleção para escolher o produto cujos detalhes serão exibidos.

NAVEGAÇÃO

CAPÍTULO 8

Navegar entre as telas de uma aplicação web é preciso. O mecanismo de navegação do JSF é bem sofisticado e permite vários tipos de transições entre telas. A ideia é muito simples: no clique de um botão ou link, muda-se a tela apresentada ao usuário.

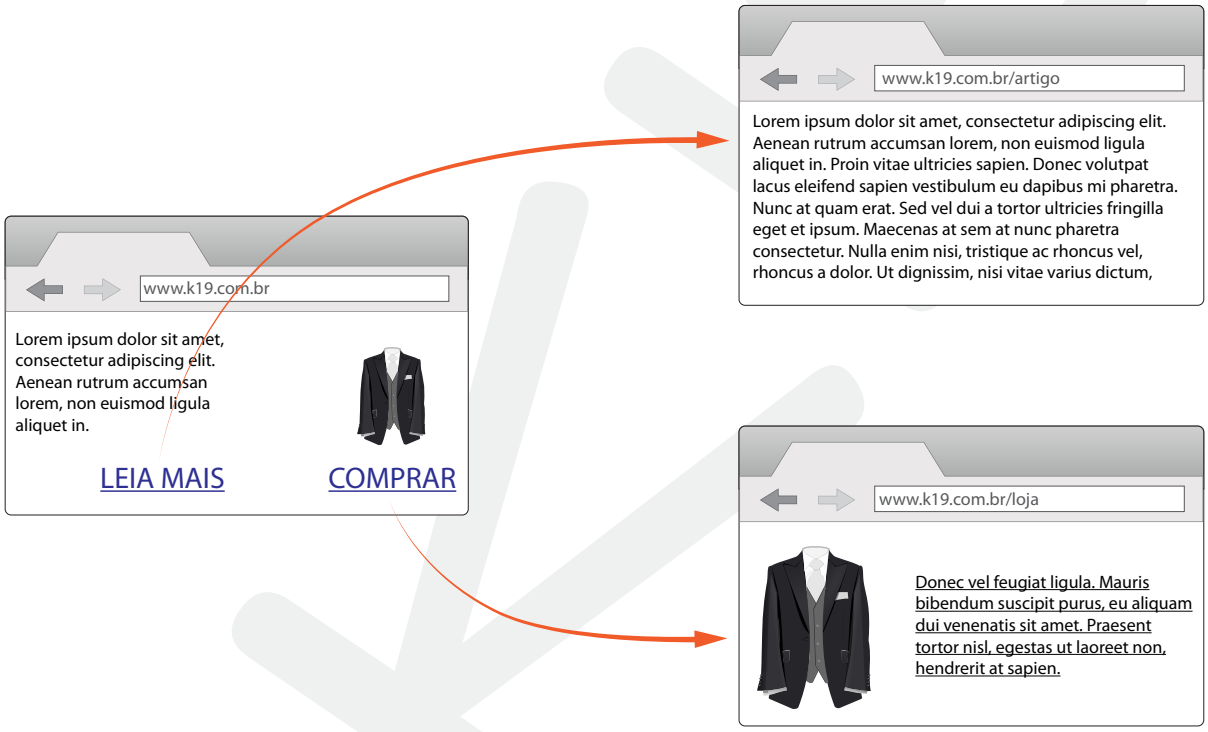


Figura 8.1: Navegação entre páginas

Navegação Implícita

Na navegação implícita, quando o usuário clica em algum botão ou link, um sinal (*outcome*) é enviado para o JSF. Esse sinal é uma string que será utilizada pelo tratador de navegação do JSF para definir a próxima tela que será apresentada ao usuário.

Considere, por exemplo, um link ou botão de uma tela (definida pelo arquivo `pagina1.xhtml`) que envia o outcome "`pagina2`". Quando um usuário clicar nesse link ou botão, ele será redirecionado para a tela definida pelo arquivo `pagina2.xhtml`. Esse arquivo deve estar no mesmo diretório do arquivo `pagina1.xhtml`. Nesse caso, o outcome é igual ao nome do arquivo de resposta sem o sufixo `.xhtml`.

Quando utilizamos os componentes `<h:commandButton>` e `<h:commandLink>` para criar botões

e links, devemos definir os outcomes através do atributo `action`. Por outro lado, quando utilizamos os componentes <h:button> e <h:link>, devemos definir os outcomes através do atributo `outcome`.

Veja como seria o código da `pagina1.xhtml` e `pagina2.xhtml`.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h1>K19 Página 1</h1>
  <h:form>
    <h:commandButton value="Página 2" action="pagina2"/>
  </h:form>
</h:body>
</html>
```

Código XHTML 8.1: pagina1.xhtml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h1>K19 Página 2</h1>
  <h:link outcome="pagina1">
    <h:outputText value="Página 1"/>
  </h:link>
</h:body>
</html>
```

Código XHTML 8.2: pagina2.xhtml

Além do nome, os outcomes também podem determinar o caminho do arquivo de resposta. Se o outcome começar com caractere "/", esse caminho será definido a partir do diretório raiz da aplicação web . Caso contrário, será definido a partir do diretório atual.

Considere, por exemplo, um arquivo chamado `pagina1.xhtml` no diretório `/diretorio1` e um arquivo chamado `pagina2.xhtml` no diretório `/diretorio1/diretorio2`. No arquivo `pagina1.xhtml`, podemos adicionar um botão para navegar para o arquivo `pagina2.xhtml` da seguintes formas:

```
...
<h:commandButton value="Página 2" action="/diretorio1/diretorio2/pagina2"/>
...
```

Código XHTML 8.3: pagina1.xhtml

```
...
<h:commandButton value="Página 2" action="diretorio2/pagina2"/>
...
```

Código XHTML 8.4: pagina1.xhtml

Note que, no primeiro código, o caminho é definido a partir do diretório raiz da aplicação web. Já no segundo código, o caminho é definido relativamente ao diretório atual.



Navegação Explícita

Na navegação explícita, podemos associar um outcome a um arquivo de resposta independentemente do nome ou do caminho desse arquivo. Essa associação deve ser registrada no arquivo de configuração do JSF, o `faces-config.xml`.

Para registrar uma navegação explícita, podemos adicionar três informações no arquivo de configuração do JSF:

- O caminho do arquivo que define a tela de origem.
- O outcome.
- O caminho do arquivo que define a tela de resposta.

Veja o exemplo abaixo.

```
<navigation-rule>
  <from-view-id>/pagina1.xhtml</from-view-id>

  <navigation-case>
    <from-outcome>proxima</from-outcome>
    <to-view-id>/pagina2.xhtml</to-view-id>
  </navigation-case>
</navigation-rule>
```

Código XML 8.1: Registrando uma navegação explícita

Após o registro da navegação acima, podemos adicionar um botão ou link no arquivo `pagina1.xhtml` com o outcome `proxima` para navegar para o arquivo `pagina2.xhtml`.

```
...
<h:commandButton value="Próxima tela" action="proxima"/>
...
```

Código XHTML 8.5: pagina1.xhtml



Importante
O JSF utiliza a seguinte lógica para determinar a página de resposta. Primeiro, ele verifica se o outcome é compatível com alguma regra de navegação registrada no arquivo `faces-config.xml`. Caso seja, o JSF realizará uma navegação explícita processando essa regra. Caso contrário, o JSF tentará realizar uma navegação implícita, procurando um arquivo compatível com o outcome. Se esse arquivo não existir, a tela atual será reapresentada.



Mais Sobre
Podemos registrar uma regra de navegação para diversos arquivos utilizando `wildcards`. Considere o exemplo abaixo.

```
<navigation-rule>
  <from-view-id>/K19/*</from-view-id>

  <navigation-case>
    <from-outcome>home</from-outcome>
    <to-view-id>/home.xhtml</to-view-id>
  </navigation-case>
</navigation-rule>
```

Código XML 8.2: Registrando uma navegação explícita

O caractere "*" na tag `<from-view-id>` indica que essa regra de navegação será aplicada para todo arquivo do diretório `/K19`.



Mais Sobre

Podemos criar navegações condicionais. Considere, por exemplo, uma aplicação em que os usuários podem escolher a versão da interface que desejam utilizar. As regras de navegação podem então considerar a preferência do usuário. Para isso, podemos adicionar a tag `<if>` na definição de uma regra de navegação. Veja o exemplo abaixo.

```
<navigation-rule>
  <from-view-id>/login.xhtml</from-view-id>

  <navigation-case>
    <from-outcome>sucesso</from-outcome>
    <if>
      #{usuarioBean.preferencias['versao-da-interface'] == 'simples'}
    </if>
    <to-view-id>/home-simples.xhtml</to-view-id>
  </navigation-case>

  <navigation-case>
    <from-outcome>sucesso</from-outcome>
    <if>
      #{usuarioBean.preferencias['versao-da-interface'] == 'avancada'}
    </if>
    <to-view-id>/home-avancada.xhtml</to-view-id>
  </navigation-case>
</navigation-rule>
```

Código XML 8.3: Registrando uma navegação explícita condicional



Exercícios de Fixação

1. Crie um projeto chamado **K19-Navegacao** seguindo os passos vistos no exercício do Capítulo 5.

2. No projeto **K19-Navegacao**, deve existir três páginas: **componentes-visuais.xhtml**, **templates-e-modularizacao.xhtml** e **navegacao.xhtml**. A partir de qualquer uma delas, deve ser possível navegar para qualquer outra. Podemos criar uma espécie de menu para navegar entre essas páginas:

```
...
```

```
<h:link value="Componentes Visuais" outcome="componentes-visuais"/> -
<h:link value="Templates e Modularização" outcome="templates-e-modularizacao"/> -
<h:link value="Navegação" outcome="navegacao"/>
...
```

Código XHTML 8.6: Menu para navegar entre as páginas.

Para facilitar, podemos definir um template para criar essas três páginas. Crie um arquivo chamado **template.xhtml** em um diretório chamado **templates** dentro de **WEB-INF**.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>

  <div id="header" style="text-align: center">
    <h1>K19 - Desenvolvimento Web com JSF2 e JPA2</h1>
  </div>

  <h:link value="Componentes Visuais" outcome="componentes-visuais"/> -
  <h:link value="Templates e Modularização" outcome="templates-e-modularizacao"/> -
  <h:link value="Navegação" outcome="navegacao"/>

  <ui:insert name="corpo-da-pagina"> Espaço para o conteúdo da tela </ui:insert>

  <div id="footer" style="text-align: center">
    <hr />
    &copy; 2012 K19. Todos os direitos reservados.
  </div>

</h:body>
</html>
```

Código XHTML 8.7: www.k19.com.br/exercicios/k12/08/002/template.xhtml

3 Agora, use o template para criar as páginas **componentes-visuais.xhtml**, **templates-e-modularizacao.xhtml** e **navegacao.xhtml**

```
<ui:composition template="/WEB-INF/templates/template.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="corpo-da-pagina">
    <h1>Componentes Visuais</h1>

    No JSF 2, as telas são definidas em arquivos xhtml. Os componentes visuais que
    constituem as telas são adicionados por meio de tags. A especificação do JSF
    define uma grande quantidade de tags e as classifica em bibliotecas.
    As principais bibliotecas de tags do JSF são:
    <ul>
      <li>Core (http://java.sun.com/jsf/core)</li>
      <li>HTML (http://java.sun.com/jsf/html)</li>
      <li>Facelets (http://java.sun.com/jsf/facelets)</li>
    </ul>
  </ui:define>
</ui:composition>
```

Código XHTML 8.8: www.k19.com.br/exercicios/k12/08/003/componentes-visuais.xhtml

```
<ui:composition template="/WEB-INF/templates/template.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="corpo-da-pagina">
    <h1>Templates e Modularização</h1>

    A criação de um template é simples. Basta criar um arquivo XHTML e definir o
    posicionamento dos trechos estáticos e dinâmicos. O conteúdo dos trechos
    estáticos também deve ser definido da forma usual dentro do template. Por outro
    lado, o conteúdo dos trechos dinâmicos só será definido nas telas. Para indicar
    o posicionamento dos trechos dinâmicos, devemos utilizar a tag ui:insert.
  </ui:define>
</ui:composition>
```

Código XHTML 8.9: www.k19.com.br/exercicios/k12/08/003/templates-e-modularizacao.xhtml

```
<ui:composition template="/WEB-INF/templates/template.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="corpo-da-pagina">
    <h1>Navegação</h1>

    Navegar entre as telas de uma aplicação web é preciso. O mecanismo de
    navegação do JSF é bem sofisticado e permite vários tipos de transições entre
    telas. A ideia é muito simples: no clique de um botão ou link, muda-se a tela
    apresentada ao usuário.
  </ui:define>
</ui:composition>
```

Código XHTML 8.10: www.k19.com.br/exercicios/k12/08/003/navegacao.xhtml

4 Remova o projeto **K19-Templates-e-Modularizacao** do GlassFish e do Wildfly. Implante o projeto **K19-Navegacao** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

```
http://localhost:8080/K19-Navegacao/componentes-visuais.xhtml
```

5 Implante o projeto **K19-Navegacao** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

```
http://localhost:8080/K19-Navegacao/componentes-visuais.xhtml
```

6 Suponha que a sequência dos tópicos seja "Componentes Visuais", "Templates e Modularização" e "Navegação". Queremos adicionar dois botões às páginas: um para avançar para o próximo tópico e outro para voltar para o tópico anterior. Para isso, configure uma navegação explícita no arquivo **faces-config.xml**.

```
...
<navigation-rule>
  <from-view-id>componentes-visuais.xhtml</from-view-id>
```

```
4
5     <navigation-case>
6       <from-outcome>proximo</from-outcome>
7       <to-view-id>templates-e-modularizacao.xhtml</to-view-id>
8     </navigation-case>
9   </navigation-rule>
10
11  <navigation-rule>
12    <from-view-id>templates-e-modularizacao.xhtml</from-view-id>
13
14    <navigation-case>
15      <from-outcome>proximo</from-outcome>
16      <to-view-id>navegacao.xhtml</to-view-id>
17    </navigation-case>
18
19    <navigation-case>
20      <from-outcome>anterior</from-outcome>
21      <to-view-id>componentes-visuais.xhtml</to-view-id>
22    </navigation-case>
23  </navigation-rule>
24
25  <navigation-rule>
26    <from-view-id>navegacao.xhtml</from-view-id>
27
28    <navigation-case>
29      <from-outcome>anterior</from-outcome>
30      <to-view-id>templates-e-modularizacao.xhtml</to-view-id>
31    </navigation-case>
32  </navigation-rule>
33  ...
```

Código XML 8.4: www.k19.com.br/exercicios/k12/08/006/faces-config.xml

7 No arquivo **/WEB-INF/templates/template.xhtml**, adicione dois botões. O primeiro deve ser usado para navegar para o tópico anterior (logo, deve emitir o sinal "`anterior`"). O segundo botão deve ser usado para navegar para o próximo tópico (portanto, deve emitir o sinal "`proximo`").

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:ui="http://java.sun.com/jsf/facelets">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11 <h:body>
12
13   <div id="header" style="text-align: center">
14     <h1>K19 - Desenvolvimento Web com JSF2 e JPA2</h1>
15   </div>
16
17   <h:form>
18     <h:commandButton value="Anterior" action="anterior"/>
19     <h:commandButton value="Próximo" action="proximo"/>
20   </h:form>
21
22   <h:link value="Componentes Visuais" outcome="componentes-visuais"/> -
23   <h:link value="Templates e Modularização" outcome="templates-e-modularizacao"/> -
24   <h:link value="Navegação" outcome="navegacao"/>
25
26   <ui:insert name="corpo-da-pagina"> Espaço para o conteúdo da tela </ui:insert>
27
28   <div id="footer" style="text-align: center">
29     <hr />
```

```
30      &copy; 2010 K19. Todos os direitos reservados.
31    </div>
32
33  </h:body>
34  </html>
```

Código XHTML 8.11: www.k19.com.br/exercicios/k12/08/007/template.xhtml

8 Navegue através dos links e botões da url:

http://localhost:8080/K19-Navegacao/componentes-visuais.xhtml



Navegações Estática e Dinâmica

Até agora, utilizamos apenas navegações estáticas, pois os outcomes foram definidos nos botões e links. Dessa forma, toda vez que um botão ou link é clicado, ele emite sempre o mesmo outcome. Por exemplo, o botão abaixo sempre emite o outcome “k19”.

```
1  <h:commandButton value="K19" action="k19"/>
```

Código XHTML 8.12: Botão com navegação estática

Podemos tornar esse comportamento dinâmico, fazendo com que os outcomes sejam diferentes a cada clique. Para isso, os botões e links devem ser associados a métodos de ação de managed beans através de expression language. Veja o exemplo abaixo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5      xmlns:h="http://java.sun.com/jsf/html">
6
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10 <h:body>
11   <h1>K19 Cara ou Coroa</h1>
12   <h:form>
13     <h:commandButton value="Lançar Moeda" action="#{caraOuCoroaBean.lanca}" />
14   </h:form>
15 </h:body>
16 </html>
```

Código XHTML 8.13: cara-ou-coroa.xhtml

```
1  @ManagedBean
2  public class CaraOuCoroaBean {
3    public String lanca() {
4      if(Math.random() < 0.5) {
5        return "cara";
6      } else {
7        return "coroa";
8      }
9    }
10 }
```

Código Java 8.1: CaraOuCoroaBean.java

Toda vez que o botão “Lançar Moeda” for clicado, o método `lanca()` do managed bean `caraOuCoroaBean` será chamado. A string devolvida por esse método será considerada o outcome a ser utilizado na navegação.



Mais Sobre

Vimos que a página de resposta depende tanto da página de origem quanto do outcome. No caso da navegação dinâmica, a página de resposta também pode depender do método de ação que produziu o outcome. Para isso, basta acrescentar a tag `<from-action>` na definição da regra de navegação. Veja o exemplo abaixo.

```
<navigation-rule>
  <from-view-id>/index.xhtml</from-view-id>

  <navigation-case>
    <from-outcome>lista</from-outcome>
    <from-action>#{cursosBean.lista}</from-action>
    <to-view-id>/lista-cursos.xhtml</to-view-id>
  </navigation-case>

  <navigation-case>
    <from-outcome>lista</from-outcome>
    <from-action>#{alunosBean.lista}</from-action>
    <to-view-id>/lista-alunos.xhtml</to-view-id>
  </navigation-case>
</navigation-rule>
```

Código XML 8.5: faces-config.xml

```
...
<h:commandLink value="Lista cursos" action="#{cursosBean.lista}"/>
<h:commandLink value="Lista alunos" action="#{alunosBean.lista}"/>
...
```

Código XHTML 8.14: index.xhtml

Se o link “Lista cursos” for clicado, o método `lista()` do managed bean `cursosBean` será acionado. Suponha que esse método devolva “`lista`”. De acordo com a regra de navegação definida no `faces-config.xml`, o usuário será direcionado à página definida pelo arquivo `/lista-cursos.xhtml`.

Por outro lado, se o usuário clicar no link “Lista alunos”, o método `lista()` do managed bean `alunosBean` será invocado. Se esse método devolver “`lista`”, o usuário será direcionado à página definida pelo arquivo `/lista-alunos.xhtml`, conforme a regra de navegação definida no `faces-config.xml`.



Exercícios de Fixação

9 Implemente um managed bean que, de forma aleatória, escolha entre dois outcomes. Crie um pacote chamado **br.com.k19.controle** no projeto **K19-Navegacao** e adicione uma classe chamada **CaraOuCoroaBean**.

```java
package br.com.k19.controle;

import javax.faces.bean.ManagedBean;

@ManagedBean
public class CaraOuCoroaBean {
  public String proxima() {
    if(Math.random() < 0.5) {
      return "cara";
    } else {
      return "coroa";
    }
  }
}
```

Código Java 8.2: www.k19.com.br/exercicios/k12/08/009/CaraOuCoroaBean.java

10 Crie uma tela principal com um botão que chama o managed bean do exercício anterior para escolher o outcome que deve ser emitido para o JSF. Para isso, faça uma página chamada **cara-ou-coroa.xhtml** no projeto K19-Navegacao.

```xml
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h1>K19 Cara ou Coroa</h1>
  <h:form>
    <h:commandButton value="Lançar Moeda" action="#{caraOuCoroaBean.proxima}"/>
  </h:form>
</h:body>
</html>
```

Código XHTML 8.15: www.k19.com.br/exercicios/k12/08/010/cara-ou-coroa.xhtml

11 Crie as páginas de saída. O arquivo **cara.xhtml**:

```xml
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h1>Deu Cara!</h1>
  <h:form>
    <h:commandButton value="voltar" action="cara-ou-coroa"/>
  </h:form>
</h:body>
</html>
```

Código XHTML 8.16: www.k19.com.br/exercicios/k12/08/011/cara.xhtml

O arquivo **coroa.xhtml**:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h1>Deu Coroa!</h1>
  <h:form>
    <h:commandButton value="voltar" action="cara-ou-coroa"/>
  </h:form>
</h:body>
</html>
```

Código XHTML 8.17: www.k19.com.br/exercicios/k12/08/011/coroa.xhtml

12 Verifique o funcionamento acessando o endereço:

`http://localhost:8080/K19-Navegacao/cara-ou-coroa.xhtml`



Exercícios Complementares

1 Considere uma aplicação em que o usuário pode escolher entre acessar uma versão simples ou avançada da aplicação. O usuário pode alterar essa preferência em uma página específica da aplicação. Crie uma aplicação web que tenha três páginas. Uma das páginas deve permitir que o usuário altere suas preferências. As outras duas devem ser a página principal da aplicação, sendo uma para a versão simples e outra para a versão avançada. Cada uma das páginas deve ter um menu para que o usuário possa navegar entre a página principal e a página de configurações. O link para a página principal deve levar em consideração a preferência do usuário. Use navegação **implícita**.

Dica: Para armazenar a preferência do usuário, você pode usar um managed bean. Anote a classe que implementará esse managed bean com `@SessionScoped` para que a preferência do usuário seja mantida entre uma requisição e outra da aplicação.

2 Repita o exercício anterior usando navegação **explícita**.

Escopos

CAPÍTULO 9

Os managed beans são instanciados pelo JSF, ou seja, os desenvolvedores definem as classes e o JSF cuida do "new". Porém, podemos determinar quando os managed beans devem ser criados e descartados. O tempo de vida de uma instância afeta principalmente a durabilidade dos dados que ela armazena. Por isso, precisamos escolher qual escopo queremos utilizar em cada managed bean. Eis alguns dos escopos do JSF 2:

- Request.
- View.
- Session.
- Application.



Request

No escopo *request*, as instâncias dos managed beans são criadas durante o processamento de uma requisição assim que forem necessárias e descartadas no final desse mesmo processamento. Assim, os dados não são mantidos de uma requisição para outra.

A partir do JSF 2, os managed beans podem ser registrados através da anotação `@ManagedBean`. O JSF utiliza o escopo *request* como padrão quando managed beans são registrados usando essa anotação. Mesmo sendo o padrão, podemos deixar explícito a escolha do escopo *request* através da anotação `@RequestScoped`.

```
1 package br.com.k19;
2
3 @ManagedBean
4 @RequestScoped
5 class TesteBean {
6   ...
7 }
```

Código Java 9.1: TesteBean.java

Por outro lado, se o managed bean for registrado da maneira tradicional, no arquivo `faces-config.xml`, a escolha de um escopo é obrigatória e realizada através da tag `managed-bean-scope`.

```
1 ...
2 <managed-bean>
3   <managed-bean-name>testeBean</managed-bean-name>
4   <managed-bean-class>br.com.k19.TesteBean</managed-bean-class>
5   <managed-bean-scope>request</managed-bean-scope>
6 </managed-bean>
7 ...
```

Código XML 9.1: faces-config.xml

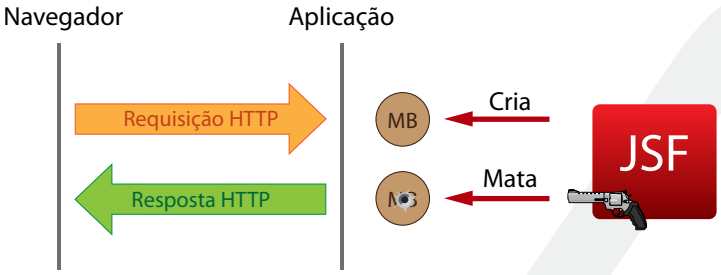


Figura 9.1: Escopo de request

View

O escopo *view* foi adicionado no JSF 2. A ideia é manter determinados dados enquanto o usuário não mudar de tela. As instância dos managed beans em escopo *view* são eliminadas somente quando há uma navegação entre telas.

Analogamente, para escolher o escopo *view*, devemos utilizar a anotação `@ViewScoped` ou a tag `managed-bean-scope`.

```
1 package br.com.k19;
2
3 @ManagedBean
4 @ViewScoped
5 class TesteBean {
6   ...
7 }
```

Código Java 9.2: TesteBean.java

```
1 ...
2 <managed-bean>
3   <managed-bean-name>testeBean</managed-bean-name>
4   <managed-bean-class>br.com.k19.TesteBean</managed-bean-class>
5   <managed-bean-scope>view</managed-bean-scope>
6 </managed-bean>
7 ...
```

Código XML 9.2: faces-config.xml

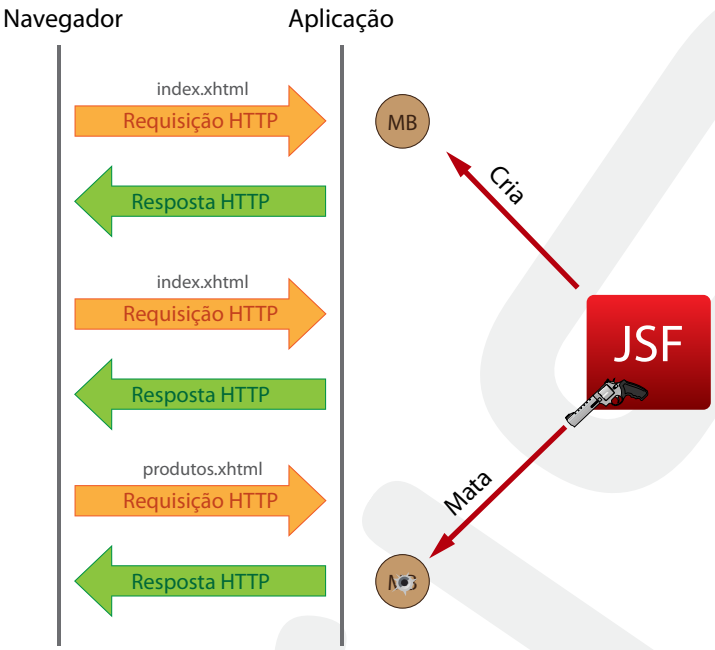


Figura 9.2: Escopo de view



Session

Certas informações devem ser mantidas entre as requisições de um determinado usuário em um determinado navegador. Por exemplo, considere uma aplicação que utiliza carrinho de compras. Um usuário faz diversas requisições para escolher os produtos e colocá-los no seu carrinho. Durante todo esse tempo, a aplicação deve manter a informação de quais produtos já foram escolhidos por esse usuário.

Daí surge o escopo *session*. Cada usuário possui um espaço na memória do servidor que é chamado de *session*. Tecnicamente, é possível existir duas ou mais *sessions* de um mesmo usuário, por exemplo, se ele estiver utilizando dois ou mais navegadores.

As instâncias dos managed beans configurados com o escopo *session* são criadas quando necessárias durante o processamento de uma requisição e armazenadas na *session* do usuário que fez a requisição.

Essas instâncias são eliminadas basicamente em duas situações: a própria aplicação decide por algum motivo específico apagar a *session* de um usuário (por exemplo, quando o usuário faz logout) ou o servidor decide apagar a *session* de um usuário quando esse usuário não faz requisições por um determinado período de tempo. Esse tempo pode ser configurado com o Web Container.

Para escolher o escopo *session*, devemos utilizar a anotação `@SessionScoped` ou a tag `managed-bean-scope`.

```
1 package br.com.k19;
2
3 @ManagedBean
4 @SessionScoped
5 class TesteBean {
```

```
6     . . .
7 }
```

Código Java 9.3: TesteBean.java

```
1 . . .
2 <managed-bean>
3   <managed-bean-name>testeBean</managed-bean-name>
4   <managed-bean-class>br.com.k19.TesteBean</managed-bean-class>
5   <managed-bean-scope>session</managed-bean-scope>
6 </managed-bean>
7 . . .
```

Código XML 9.3: faces-config.xml

Temos que tomar um cuidado maior ao utilizar o escopo *session*, pois podemos acabar sobrecarregando o servidor. Portanto, a dica é evitar utilizar o escopo *session* quando possível. Para não consumir excessivamente os recursos de memória do servidor, o escopo *request* é mais apropriado.

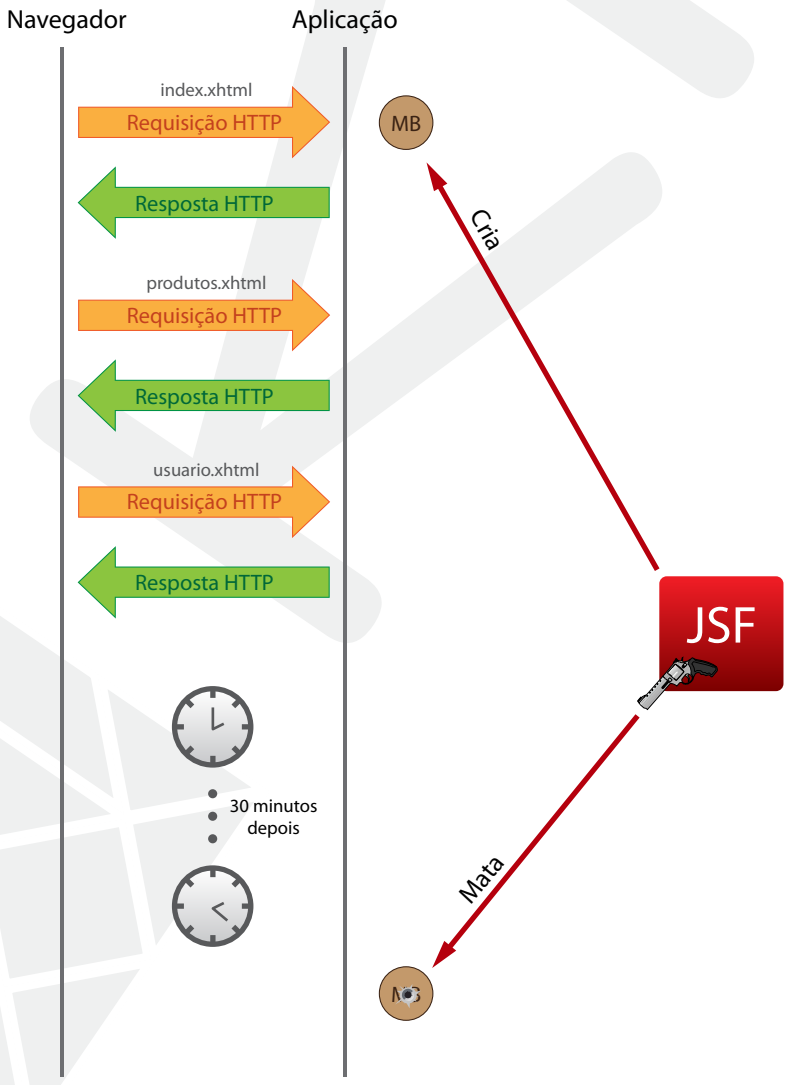


Figura 9.3: Escopo de session



Application

As instâncias dos managed beans configurados com escopo *application* são criadas no primeiro momento em que elas são utilizadas e mantidas até a aplicação ser finalizada.

Ao contrário do que ocorre com escopos discutidos anteriormente, as instâncias dos managed beans registrados com escopo *application* são compartilhadas por todos os usuários da aplicação. O JSF cria apenas uma instância de cada managed bean em escopo de *application*.

Analogamente, para escolher o escopo *application*, devemos utilizar a anotação `@ApplicationScoped` ou a tag `managed-bean-scope`.

```
1 package br.com.k19;
2
3 @ManagedBean
4 @ApplicationScoped
5 class TesteBean {
6   ...
7 }
```

Código Java 9.4: TesteBean.java

```
1 ...
2 <managed-bean>
3   <managed-bean-name>testeBean</managed-bean-name>
4   <managed-bean-class>br.com.k19.TesteBean</managed-bean-class>
5   <managed-bean-scope>application</managed-bean-scope>
6 </managed-bean>
7 ...
```

Código XML 9.4: faces-config.xml



Mais Sobre

Por padrão, a instância de um managed bean em escopo *application* é criada no momento em que ela é usada pela primeira vez. Podemos alterar esse comportamento, fazendo com que essa instância seja criada quando a aplicação é inicializada. Para isso, basta acrescentar a propriedade `eager` com o valor `true` na anotação `@ManagedBean`. Veja o exemplo abaixo.

```
1 package br.com.k19;
2
3 @ManagedBean(eager=true)
4 @ApplicationScoped
5 class TesteBean {
6   ...
7 }
```

Código Java 9.5: TesteBean.java



Exercícios de Fixação

1 Crie um projeto chamado **K19-Escopos** seguindo os passos vistos no exercício do Capítulo 5.

2 Vamos criar uma página para adicionar carros e visualizá-los. Adicione um pacote chamado **br.com.k19.modelo** no projeto **K19-Escopos**.

3 No pacote **br.com.k19.modelo**, crie uma classe chamada **Carro** para modelar os carros. Essa classe deve ter dois atributos do tipo `String`: um para armazenar a marca e outro para guardar o modelo do carro.

```
1  package br.com.k19.modelo;
2
3  public class Carro {
4    private String marca;
5    private String modelo;
6
7    // GETTERS E SETTERS
8  }
```

Código Java 9.6: www.k19.com.br/exercicios/k12/09/003/Carro.java

4 Crie um outro pacote com o nome **br.com.k19.controle**.

5 No pacote **br.com.k19.controle**, crie uma classe chamada **CarroBean** para armazenar uma lista de carros.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Carro;
4  import java.util.ArrayList;
5  import java.util.List;
6  import javax.faces.bean.ManagedBean;
7
8  @ManagedBean
9  public class CarroBean {
10   private List<Carro> carros = new ArrayList<Carro>();
11
12   // GETTERS E SETTERS
13 }
```

Código Java 9.7: www.k19.com.br/exercicios/k12/09/005/CarroBean.java

6 Na classe **CarroBean**, acrescente um atributo do tipo **Carro** para armazenar o carro a ser adicionado. Implemente também um método chamado **adicionaCarro()** para inserir um novo carro na lista.

```
1  package br.com.k19.controle;
2
3  import java.util.List;
4  import java.util.ArrayList;
5  import javax.faces.bean.ManagedBean;
6
7  @ManagedBean
8  public class CarroBean {
9    private List<Carro> carros = new ArrayList<Carro>();
10   private Carro carro = new Carro();
```

```
11
12    public void adicionaCarro() {
13      this.carros.add(this.carro);
14      this.carro = new Carro();
15    }
16
17    // GETTERS E SETTERS
18  }
```

Código Java 9.8: www.k19.com.br/exercicios/k12/09/006/CarroBean.java

Não esqueça dos métodos de acesso do atributo **carro**.

7 Defina o escopo do managed bean **carroBean** como sendo **request**. Para isso, use a anotação **@RequestScoped**. Lembre-se esse é o escopo padrão.

```
1  ...
2  @RequestScoped
3  public class CarroBean {
4    ...
5  }
```

Código Java 9.9: www.k19.com.br/exercicios/k12/09/007/CarroBean.java

Não esqueça de importar a anotação **@RequestScoped**.

8 Crie um página no projeto **K19-Escopos** chamada **carros.xhtml**. Implemente um formulário para adicionar um novo carro.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:ui="http://java.sun.com/jsf/facelets">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11 <h:body>
12   <h:form>
13     <h:panelGrid columns="2">
14       <h:outputLabel value="Marca: " for="campo-marca" />
15       <h:inputText value="#{carroBean.carro.marca}" id="campo-marca" />
16
17       <h:outputLabel value="Modelo: " for="campo-modelo" />
18       <h:inputText value="#{carroBean.carro.modelo}" id="campo-modelo" />
19
20       <h:commandButton value="Adicionar" action="#{carroBean.adicionaCarro}" />
21     </h:panelGrid>
22   </h:form>
23 </h:body>
24 </html>
```

Código XHTML 9.1: www.k19.com.br/exercicios/k12/09/008/carros.xhtml

9 No arquivo **carros.xhtml**, adicione um trecho de código para exibir os carros adicionados.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
```

```
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4 <html xmlns="http://www.w3.org/1999/xhtml"
5   xmlns:h="http://java.sun.com/jsf/html"
6   xmlns:ui="http://java.sun.com/jsf/facelets">
7
8 <h:head>
9   <title>K19 Treinamentos</title>
10 </h:head>
11 <h:body>
12   <h:form>
13     <h:panelGrid columns="2">
14       <h:outputLabel value="Marca: " for="campo-marca" />
15       <h:inputText value="#{carroBean.carro.marca}" id="campo-marca" />
16
17       <h:outputLabel value="Modelo: " for="campo-modelo" />
18       <h:inputText value="#{carroBean.carro.modelo}" id="campo-modelo" />
19
20       <h:commandButton value="Adicionar" action="#{carroBean.adicionaCarro}" />
21     </h:panelGrid>
22   </h:form>
23
24   <h:panelGroup rendered="#{not empty carroBean.carros}">
25     <h1>Lista de carros:</h1>
26     <ul>
27       <ui:repeat value="#{carroBean.carros}" var="carro">
28         <li><h:outputText value="#{carro.marca} #{carro.modelo}" /></li>
29       </ui:repeat>
30     </ul>
31   </h:panelGroup>
32 </h:body>
33 </html>
```

Código XHTML 9.2: www.k19.com.br/exercicios/k12/09/009/carros.xhtml

10 Remova o projeto **K19-Navegacao** do GlassFish e do Wildfly. Implante o projeto **K19-Escopos** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Escopos/carros.xhtml`

Adicione alguns carros e observe a lista de carros. O que acontece?

11 Implante o projeto **K19-Escopos** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Escopos/carros.xhtml`

Novamente, adicione alguns carros e observe a lista de carros. O que acontece?

12 Mude o escopo do managed bean `carroBean` de *request* para *view*. Para isso, use a anotação `@ViewScoped`.

```
1 @ViewScoped
2 public class CarroBean {
3   ...
4 }
```

Código Java 9.10: www.k19.com.br/exercicios/k12/09/012/CarroBean.java

Não esqueça de importar a anotação **@ViewScoped**.

13 Novamente, acesse a aplicação no endereço:

http://localhost:8080/K19-Escopos/carros.xhtml

Adicione alguns carros e observe a lista de carros. O que acontece?

14 Crie uma página chamada **menu.xhtml**. Essa página deve possuir um link para a página **carros.xhtml**.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:link value="Adicionar carros" outcome="carros" />
</h:body>
</html>
```

Código XHTML 9.3: www.k19.com.br/exercicios/k12/09/014/menu.xhtml

15 No arquivo **carros.xhtml**, adicione um link para a página **menu.xhtml**.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core"
  xmlns:ui="http://java.sun.com/jsf/facelets">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:link value="Menu" outcome="menu"/>

  <h:form>
    <h:panelGrid columns="2">
      <h:outputLabel value="Marca: " for="campo-marca" />
      <h:inputText value="#{carroBean.carro.marca}" id="campo-marca" />

      <h:outputLabel value="Modelo: " for="campo-modelo" />
      <h:inputText value="#{carroBean.carro.modelo}" id="campo-modelo" />

      <h:commandButton value="Adicionar" action="#{carroBean.adicionaCarro}" />
    </h:panelGrid>
  </h:form>

  <h:panelGroup rendered="#{not empty carroBean.carros}">
    <h1>Lista de carros:</h1>
    <ul>
      <ui:repeat value="#{carroBean.carros}" var="carro">
        <li><h:outputText value="#{carro.marca} #{carro.modelo}" /></li>
      </ui:repeat>
    </ul>
```

```
34    </h:panelGroup>
35  </h:body>
36  </html>
```

Código XHTML 9.4: www.k19.com.br/exercicios/k12/09/015/carros.xhtml

16 Acesse a aplicação no endereço:

`http://localhost:8080/K19-Escopos/carros.xhtml`

Certifique-se de que o escopo do managed bean **carroBean** seja **view**. Mais uma vez, adicione alguns carros e observe a lista de carros. Use os links criados para navegar entre as páginas **carros.xhtml** e **menu.xhtml**. O que acontece?

17 Mude o escopo do managed bean **carroBean** de **view** para **session**. Para isso, use a anotação **@SessionScoped**.

```
1  @SessionScoped
2  public class CarroBean {
3    ...
4  }
```

Código Java 9.11: www.k19.com.br/exercicios/k12/09/017/CarroBean.java

Não esqueça de importar a anotação **@SessionScoped**.

18 Acesse a aplicação no endereço:

`http://localhost:8080/K19-Escopos/carros.xhtml`

Novamente, adicione alguns carros e observe a lista de carros. Use os links que você criou para navegar entre as páginas definidas por **carros.xhtml** e **menu.xhtml**. O que acontece?

19 Agora, acesse a aplicação a partir de navegadores diferentes. Adicione alguns carros a partir dos diferentes navegadores. O que você observa?

20 Mude o escopo do managed bean **carroBean** de **session** para **application**. Para isso, use a anotação **@ApplicationScoped**.

```
1  @ApplicationScoped
2  public class CarroBean {
3    ...
4  }
```

Código Java 9.12: www.k19.com.br/exercicios/k12/09/020/CarroBean.java

Não esqueça de importar a anotação **@ApplicationScoped**.

21 Novamente, usando navegadores diferentes, adicione alguns carros. O que você pode observar?

Conversão e Validação

CAPÍTULO 10



Conversão

Quando um usuário preenche um formulário, os valores preenchidos são enviados para uma aplicação. De acordo com o HTTP, protocolo de comunicação utilizado entre os navegadores e as aplicações web, esses dados não possuem tipagem. Eles são tratados como texto puro. Dessa forma, quando uma aplicação recebe valores preenchidos em formulários HTML, ela precisa realizar a conversão dos dados que deseja tratar de forma específica.

Por exemplo, considere um formulário que possui um campo para os usuários digitarem a sua idade. A informação digitada nesse campo é tratada como texto puro até chegar na aplicação, que deve converter esse dado para algum tipo adequado do Java como `int` ou `long`.

Eventualmente, os dados que são enviados para a aplicação não podem ser convertidos, pois não estão no formato esperado. Por exemplo, se o texto preenchido em um campo numérico possui caracteres não numéricos, a conversão falhará.

Podemos observar o processo de conversão de outro ponto de vista. Nem sempre o formato das informações que estão em uma aplicação web Java corresponde ao formato que desejamos que seja apresentado aos usuários. Novamente, os dados devem ser convertidos antes de serem enviados para os navegadores.

Felizmente, o JSF oferece um mecanismo automatizado de conversão de dados. Veremos a seguir o funcionamento desse mecanismo.



Conversores Padrão

Para facilitar o trabalho de desenvolvimento de uma aplicação, o JSF define um conjunto de conversores padrão. Alguns desses conversores são aplicados automaticamente. Outros conversores são aplicados apenas se forem explicitamente indicados.

O JSF aplica automaticamente conversores padrão para os seguintes tipos fundamentais do Java:

- BigDecimal
- BigInteger
- Boolean e boolean
- Byte e byte
- Character e char
- Double e double

- Float e float
- Integer e int
- Long e long
- Short e short

No exemplo abaixo, o conteúdo digitado na caixa de texto será convertido para o tipo `double` automaticamente, pois a propriedade `numero` é do tipo `double`.

```
@ManagedBean
public class TesteBean {

  private double numero;

  // GETTERS E SETTERS
}
```

Código Java 10.1: TesteBean.java

```
<!-- O valor digitado nesse campo será convertido para double -->
<h:inputText value="#{testeBean.numero}" />
```

Código XHTML 10.1: Caixa de texto vinculada à propriedade `numero`

Personalizando Conversores Padrão

Alguns conversores padrão podem ou precisam usar informações adicionais para realizar a conversão de dados. As tags `<f:convertNumber>` e `<f:convertDateTime>` são usadas para transmitir informações aos conversores de números (`java.lang.Number`) e datas (`java.util.Date`), respectivamente.

<f:convertNumber>

A tag `<f:convertNumber>` permite que conversões mais sofisticadas sejam feitas em valores numéricos.

Para estipular que um valor numérico seja exibido com um número máximo de casas decimais, podemos usar o atributo `maxFractionDigits` da tag `<f:convertNumber>`. No exemplo abaixo, a propriedade `numero` do managed bean `testeBean` é exibida com no máximo duas casas decimais.

```
<h:outputText value="#{testeBean.numero}" >
  <f:convertNumber maxFractionDigits="2" />
</h:outputText>
```

Analogamente, podemos definir o número mínimo de casas decimais com o atributo `minFractionDigits`.

Podemos também definir a formatação de um número por meio de uma espécie de expressão regular. Isso pode ser feito usando o atributo `pattern`, como no exemplo abaixo.

```
<h:outputText value="#{testeBean.numero}" >
  <f:convertNumber pattern="#0.000" />
</h:outputText>
```

Para apresentar dados na forma de porcentagem, podemos usar o atributo `type` e defini-lo com o valor `percent`.

```
<h:outputText value="#{testeBean.numero}" >
  <f:convertNumber type="percent" />
</h:outputText>
```

Para personalizar a exibição de valores monetários, a tag `<f:convertNumber>` provê o atributo `currencySymbol` e `locale`. No código abaixo, exemplificamos o uso desses dois atributos.

```
<h:outputText value="#{testeBean.numero}" >
  <f:convertNumber currencySymbol="R$" type="currency" />
</h:outputText>

<h:outputText value="#{testeBean.numero}" >
  <f:convertNumber locale="pt_BR" type="currency" />
</h:outputText>
```

<f:convertDateTime>

A tag `<f:convertDateTime>` permite que conversões de datas sejam realizadas. Esse conversor pode ser aplicado em dados do tipo `java.util.Date`. Essa tag possui o atributo `pattern`, que permite a definição do formato da data que desejamos utilizar.

```
<h:outputText value="#{testeBean.data}">
  <f:convertDateTime pattern="dd/MM/yyyy" />
</h:outputText>
```

Para mais detalhes sobre o uso do atributo `pattern`, consulte a página http://docs.oracle.com/javase/7/docs/api/java/text/SimpleDateFormat.html.

Uma outra maneira de escolher o formato de data a ser utilizado é por meio do atributo `locale`. No exemplo abaixo, ao definir o valor do atributo `locale` como `pt_BR`, o padrão de data utilizado será "dd/MM/yyyy".

```
<h:outputText value="#{testeBean.data}">
  <f:convertDateTime locale="pt_BR" />
</h:outputText>
```



Exercícios de Fixação

1. Crie um projeto chamado **K19-Conversao-e-Validacao** seguindo os passos vistos no exercício do Capítulo 5.

2. Crie uma classe chamada `Funcionario` para modelar funcionários em um pacote chamado **br.com.k19.modelo**.

```
package br.com.k19.modelo;

import java.util.Date;

```

```
5  public class Funcionario {
6    private Double salario;
7    private Integer codigo;
8    private Date aniversario;
9
10   // GETTERS E SETTERS
11 }
```

Código Java 10.2: www.k19.com.br/exercicios/k12/10/002/Funcionario.java

3 Crie uma class chamada **FuncionarioBean** contendo um atributo do tipo **Funcionario** em um pacote chamado **br.com.k19.controle**.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Funcionario;
4  import javax.faces.bean.ManagedBean;
5
6  @ManagedBean
7  public class FuncionarioBean {
8    private Funcionario funcionario = new Funcionario();
9
10   // GETTERS E SETTERS
11 }
```

Código Java 10.3: www.k19.com.br/exercicios/k12/10/003/FuncionarioBean.java

4 Crie uma página chamada **cadastro.xhtml** para cadastrar funcionários.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10
11 <h:body>
12   <h1>Cadastro de Funcionário</h1>
13   <h:form>
14     <h:panelGrid columns="3">
15       <h:outputLabel value="Salário: R$ " for="campo-salario"/>
16       <h:inputText id="campo-salario" value="#{funcionarioBean.funcionario.salario}">
17         <f:convertNumber locale="pt_BR"/>
18       </h:inputText>
19       <h:message for="campo-salario"/>
20
21       <h:outputLabel value="Código: " for="campo-codigo"/>
22       <h:inputText id="campo-codigo" value="#{funcionarioBean.funcionario.codigo}"/>
23       <h:message for="campo-codigo"/>
24
25       <h:outputLabel value="Data: (dd/MM/yyyy)" for="campo-aniversario"/>
26       <h:inputText id="campo-aniversario"
27         value="#{funcionarioBean.funcionario.aniversario}">
28         <f:convertDateTime pattern="dd/MM/yyyy"/>
29       </h:inputText>
30       <h:message for="campo-aniversario"/>
31
32       <h:commandButton value="Cadastrar"/>
33     </h:panelGrid>
34     <h:messages/>
35   </h:form>
```

```
36 </h:body>
37 </html>
```

Código XHTML 10.8: www.k19.com.br/exercicios/k12/10/004/cadastro.xhtml

5 Remova o projeto **K19-Escopos** do GlassFish e do Wildfly. Implante o projeto **K19-Conversao-e-Validacao** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Conversao-e-Validacao/cadastro.xhtml`

Preencha o formulário com valores inadequados diversas vezes e observe as mensagens de erros.

6 Implante o projeto **K19-Conversao-e-Validacao** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Conversao-e-Validacao/cadastro.xhtml`

Preencha o formulário com valores inadequados diversas vezes e observe as mensagens de erros.



Exercícios Complementares

1 Crie um formulário para registrar a cotação do dólar. Esse formulário deve ter três campos obrigatórios. No primeiro, o usuário deve digitar o valor do dólar em reais. Esse campo deve estar associado a um conversor de números cujo tipo seja a moeda brasileira (use os atributos `locale` e `type` da tag `<f:convertNumber>`). O segundo campo deve guardar a variação do dólar, em porcentagem, com no máximo cinco dígitos na parte fracionária. No terceiro campo, o usuário deve colocar o horário e a data da cotação, cujo formato deve ser "HH:mm dd-MM-yyyy".

2 Exiba na tela os valores que foram submetidos pelo formulário e convertidos.



Mensagens de Erro

Eventualmente, as informações preenchidas pelos usuários em formulários não são adequadas, impedindo a conversão dos dados. Nesses casos, geralmente, desejamos apresentar mensagens relacionadas aos erros no preenchimento das informações.

<h:message>

Para exibir erros relacionados a um determinado campo, podemos utilizar o componente `<h:message>`. Esse componente deve ser associado ao campo correspondente aos erros que desejamos exibir. Para estabelecer essa ligação, devemos definir o valor do atributo `for` do componente `<h:message>` igual ao valor do atributo `id` do campo em questão.

```
1 @ManagedBean
```

```
2 public class TesteBean {
3
4   private Double numero;
5
6   // GETTERS E SETTERS
7 }
```

Código Java 10.6: TesteBean.java

```
1 <h:inputText value="#{testeBean.numero}" id="campo-numero"/>
2 <h:message for="campo-numero"/>
```

Código XHTML 10.11: Exibindo mensagens de erro relacionadas a um campo de texto

As imagens abaixo exibem o campo de texto com o valor digitado pelo usuário e após o processamento da requisição.



Figura 10.1: Campo de texto associado a uma propriedade numérica de um managed bean

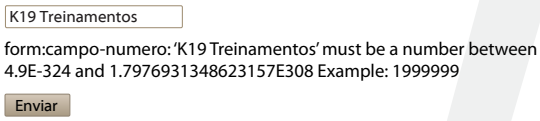


Figura 10.2: Mensagem de erro de conversão



Mais Sobre

Os textos padrão das mensagens de erro são definidos no *resource bundle* `javax.faces.Messages`.



Mais Sobre

Cada mensagem de erro possui duas versões: uma detalhada e outra resumida. Por padrão, apenas a mensagem detalhada é exibida quando a tag `<h:message>` é aplicada. Para modificar esse comportamento, podemos alterar os valores dos atributos `showSummary` e `showDetail`. O primeiro atributo indica se a mensagem resumida deve ser apresentada, enquanto o segundo indica se a mensagem detalhada deve ser exibida. Esses atributos devem assumir valores `true` ou `false`. Veja o exemplo abaixo.

```
1 <h:inputText value="#{testeBean.numero}" id="campo-numero"/>
2 <h:message for="campo-numero" showSummary="true" showDetail="false"/>
```

Código XHTML 10.12: Definindo o tipo de mensagem de erro a ser apresentada

Note que as mensagens detalhada e resumida podem ser exibidas ao mesmo tempo. Para isso, basta que esses dois atributos possuam o valor `true`.

<h:messages>

Como vimos, a tag <h:message> permite que os erros relacionados a um determinado campo sejam exibidos. Para exibir os erros de todos os campos, podemos incluir uma tag <h:message> para cada campo. Entretanto, podemos fazer o mesmo de uma forma mais prática. A tag <h:messages> exibe todas as mensagens.

```
1 <h:messages/>
```

Código XHTML 10.13: Exibindo todas as mensagens

Ao contrário da tag <h:message>, os valores padrão dos atributos showSummary e showDetail da tag <h:messages> são true e false, respectivamente.

Alterando as Mensagens de Erro

O texto de cada mensagem de erro de conversão ou validação está definido na especificação do JSF 2 que pode ser obtida através da url http://jcp.org/en/jsr/detail?id=314. Normalmente, queremos personalizar essas mensagens. Para isso, são necessários dois passos:

1. Criar um arquivo de mensagens.
2. Registrar esse arquivo no faces-config.xml.

Criando um Arquivo de Mensagens

Um arquivo de mensagens é um conjunto de chaves e valores. As chaves são utilizadas para recuperar as mensagens, enquanto os valores definem os textos que serão exibidos.

O maior problema para definir um arquivo de mensagens no JSF é saber quais são as chaves que podemos utilizar. Para conhecer as chaves, devemos consultar a especificação do JSF que pode ser obtida através da url http://jcp.org/en/jsr/detail?id=314. Veja um exemplo de arquivo de mensagens:

```
1 javax.faces.converter.BooleanConverter.BOOLEAN={1}: ''{0}'' must be 'true' or 'false'↩
    .
```

Código XML 10.1: Exemplo de arquivo de mensagens

Os nomes dos arquivos de mensagens devem possuir o sufixo .properties.

Registrando um Arquivo de Mensagens

Suponha que você tenha criado um arquivo de mensagens chamado Messages.properties num pacote chamado resources. Para registrá-lo, você deve acrescentar uma configuração no arquivo faces-config.xml.

```
1 ...
2 <application>
3   <message-bundle>resources.Messages</message-bundle>
4 </application>
5 ...
```

Código XML 10.2: faces-config.xml

Note que o sufixo `.properties` é omitido no `faces-config.xml`.



Mais Sobre

Suponha que uma aplicação tenha um arquivo de mensagens registrado no `faces-config.xml`. Para obter uma mensagem de erro, o JSF buscará primeiramente nesse arquivo. Caso não a encontre, ele utilizará a mensagem padrão.



Mais Sobre

Podemos definir, de maneira específica, a mensagem que deve ser utilizada quando um erro de conversão ocorrer em um determinado campo. Para isso, basta utilizar o atributo `converterMessage`. Observe o seguinte exemplo.

```
1 <h:inputText value="#{testeBean.numero}" id="campo-numero"
2   converterMessage="Por favor, digite um número"/>
3 <h:message for="campo-numero"/>
```

Código XHTML 10.14: Definindo uma mensagem para um erro de conversão

No exemplo acima, se o usuário submeter o formulário com um valor não numérico no campo `campo-numero`, então a mensagem de erro "Por favor, digite um número" será exibida na tela de resposta.



Exercícios de Fixação

7 No projeto **K19-Conversao-e-Validacao**, crie um pacote chamado **resources**. Adicione a esse pacote um arquivo de mensagens chamado **Messages.properties** com o seguinte conteúdo.

```
1 javax.faces.converter.NumberConverter.NUMBER=O valor {0} não é adequado.
2 javax.faces.converter.NumberConverter.NUMBER_detail={0} não é número ou é inadequado.
3 javax.faces.converter.IntegerConverter.INTEGER=O valor {0} não é adequado.
4 javax.faces.converter.IntegerConverter.INTEGER_detail={0} não é um número inteiro.
5 javax.faces.converter.DateTimeConverter.DATE=A data {0} não está correta.
6 javax.faces.converter.DateTimeConverter.DATE_detail= {0} não parece uma data.
```

Código XML 10.3: www.k19.com.br/exercicios/k12/10/007/Messages.properties

8 Adicione a configuração necessária no **faces-config.xml** para utilizar o arquivo de mensagens criado no exercício anterior.

```
1 ...
2 <application>
3   <message-bundle>resources.Messages</message-bundle>
4 </application>
5 ...
```

Código XML 10.4: www.k19.com.br/exercicios/k12/10/008/faces-config.xml

Observação: a tag **<application>** deve ser colocada dentro de da tag **<faces-config>**.

9 Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/cadastro.xhtml.
Preencha o formulário várias vezes com valores inadequados e observe as novas mensagens.



Validação

Como vimos, um dado enviado através de uma requisição HTTP chega na aplicação no formato de texto. O processo de conversão consiste em transformar esse texto em algum tipo específico do Java. Após a conversão, podemos verificar se os valores obtidos respeitam determinadas restrições impostas pelas regras da aplicação.

Por exemplo, considere um campo para o usuário digitar sua idade. Como discutido anteriormente, o valor digitado será tratado como texto até chegar na aplicação. Em seguida, esse dado será convertido para um tipo numérico do Java (como `int` ou `long` por exemplo).

Obviamente, a idade de um usuário deve ser um valor não-negativo. Contudo, essa condição não é verificada na etapa de conversão. Dessa forma, depois da conversão dos dados, mais uma etapa é necessária para que essa verificação seja realizada. Essa etapa é conhecida como validação.



Validadores Padrão

Novamente, para facilitar o trabalho de desenvolvimento de uma aplicação, o JSF define um conjunto de validadores padrão. A seguir, veremos alguns detalhes sobre esses validadores.

Campo Obrigatório (Required)

A validação mais comum de todas é a de verificar se um determinado campo foi preenchido. Podemos aplicar essa validação utilizando o atributo `required`.

```
1 <h:inputText value="#{testeBean.nome}" id="campo-nome" required="true"/>
2 <h:message for="campo-nome"/>
```

Código XHTML 10.15: Campo de texto obrigatório

Uma outra forma de registrar um validador desse tipo é por meio da tag `<f:validateRequired>`, conforme o exemplo abaixo.

```
1 <h:inputText value="#{testeBean.nome}" id="campo-nome">
2   <f:validateRequired/>
3 </h:inputText>
4 <h:message for="campo-nome"/>
```

Código XHTML 10.16: Outra forma de exigir que um campo seja obrigatório

<f:validateLongRange>

O validador `<f:validateLongRange>` é utilizado para verificar se um valor numérico inteiro pertence a um determinado intervalo de números. No exemplo abaixo, o validador irá verificar se o valor da propriedade `idade` está entre 10 e 130.

```
1 <h:inputText value="#{testeBean.idade}" id="campo-idade">
2   <f:validateLongRange minimum="10" maximum="130" />
3 </h:inputText>
4 <h:message for="campo-idade"/>
```

Código XHTML 10.17: A propriedade `idade` *deve receber um número entre 10 e 130*

<f:validateDoubleRange>

O validador `<f:validateDoubleRange>` funciona de forma análoga ao `<f:validateLongRange>`. Ele é utilizado para verificar se um valor numérico real pertence a um determinado intervalo de números. No exemplo abaixo, o validador verificará se o valor da propriedade `preco` do managed bean `testeBean` está entre 20.57 e 200.56.

```
1 <h:inputText value="#{testeBean.preco}" id="campo-preco">
2   <f:validateDoubleRange minimum="20.57" maximum="200.56" />
3 </h:inputText>
4 <h:message for="campo-preco"/>
```

Código XHTML 10.18: A propriedade `preco` *deve receber um número entre 20.57 e 200.56*

<f:validateLength>

O validador `<f:validateLength>` verifica se uma string possui uma quantidade mínima ou máxima de caracteres. Veja o exemplo abaixo.

```
1 <h:inputText value="#{testeBean.nome}" id="campo-nome">
2     <f:validateLength minimum = "6" maximum = "20"/>
3 </h:inputText>
4 <h:message for="campo-nome"/>
```

Código XHTML 10.19: Restrição no número de caracteres da propriedade `nome`

Nesse exemplo, a restrição imposta é a de que a propriedade `nome` do managed bean `testeBean` seja uma string com pelo menos 6 e no máximo 20 caracteres.

<f:validateRegex>

O validador `<f:validateRegex>` é usado para verificar se um texto respeita uma determinada expressão regular. No exemplo abaixo, o validador verificará se a propriedade `codigo` tem pelo menos 6 e no máximo 20 caracteres, além de ser formada apenas por letras.

```
1 <h:inputText value="#{testeBean.codigo}" id="campo-codigo">
2   <f:validateRegex pattern="[a-zA-Z]{6,20}"/>
3 </h:inputText>
4 <h:message for="campo-codigo"/>
```

Código XHTML 10.20: Expressão regular para restringir o dado de entrada de um campo de texto



Exercícios de Fixação

10 Vamos acrescentar alguns validadores para os campos do formulário de cadastro de funcionários do projeto **K19-Conversao-e-Validacao**. Com base nas restrições abaixo, acrescente validadores apropriados.

- O salário de um funcionário deve ser não-negativo.
- O código de um funcionário deve ser um número inteiro maior ou igual a 5 e estritamente menor do que 20.
- Todos os campos são obrigatórios.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">
<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h1>Cadastro de Funcionário</h1>
  <h:form>
    <h:panelGrid columns="3">
      <h:outputLabel value="Salário: R$ " for="campo-salario"/>
      <h:inputText id="campo-salario" value="#{funcionarioBean.funcionario.salario}"
        required="true">
        <f:convertNumber locale="pt_BR"/>
        <f:validateDoubleRange minimum="0"/>
      </h:inputText>
      <h:message for="campo-salario"/>

      <h:outputLabel value="Código: " for="campo-codigo"/>
      <h:inputText id="campo-codigo" value="#{funcionarioBean.funcionario.codigo}"
        required="true">
        <f:validateLongRange minimum="5" maximum="19"/>
      </h:inputText>
      <h:message for="campo-codigo"/>

      <h:outputLabel value="Data: (dd/MM/yyyy)" for="campo-aniversario"/>
      <h:inputText id="campo-aniversario"
        value="#{funcionarioBean.funcionario.aniversario}"
        required="true">
        <f:convertDateTime pattern="dd/MM/yyyy"/>
      </h:inputText>
      <h:message for="campo-aniversario"/>

      <h:commandButton value="Cadastrar"/>
    </h:panelGrid>
    <h:messages/>
  </h:form>
</h:body>
</html>
```

Código XHTML 10.21: www.k19.com.br/exercicios/k12/10/010/cadastro.xhtml

11 Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/cadastro.xhtml

Teste o formulário com diversos valores e observe o que acontece.



Exercícios Complementares

3 Crie um formulário para o cadastro de produtos. Um produto tem quatro atributos: nome, código, tamanho e volume. O nome deve ser formado apenas por letras e espaços. O código deve ter pelo menos quatro e no máximo dez caracteres. O primeiro caractere desse código deve ser uma letra maiúscula e os demais devem ser números. O tamanho deve ser um número inteiro entre 1 e 32. O volume deve ser um número real maior ou igual a 0.1. Todos os campos do formulário devem ser obrigatórios.



Bean Validation

Uma nova abordagem para definir validações foi adicionada no JSF 2. A ideia é declarar as regras de validação nas classes do modelo ao invés de inseri-las nos arquivos XHTML que definem as telas. A grande vantagem das validações definidas nas classes do modelo é que elas podem ser utilizadas em diversas partes da aplicação. Esse novo recurso é chamado **Bean Validation** e foi definido pela especificação JSR 303, que pode ser obtida através da url `http://jcp.org/en/jsr/detail?id=303`.

Para definir as validações com bean validation, basta adicionar anotações nas classes do modelo. No código abaixo, o atributo `nome` foi marcado com a anotação `@NotNull`. Com essa anotação, o JSF verificará se o valor do atributo `nome` não é nulo.

```
1 public class Funcionario {
2   @NotNull
3   private String nome;
4   ...
5 }
```

Código Java 10.9: Funcionario.java



Mais Sobre

Considere uma caixa de texto vinculada a uma propriedade do tipo `String`. Por padrão, quando essa caixa não é preenchida, o JSF considera o seu conteúdo como sendo a string vazia (“”) e não como o valor `null`. Assim, a anotação `@NotNull` não pode ser utilizada para obrigar que o usuário preencha essa caixa de texto.

Podemos alterar o comportamento padrão do JSF fazendo com que ele considere o conteúdo de uma caixa de texto não preenchida como `null`. Para isso, devemos acrescentar o seguinte trecho no arquivo de configuração da aplicação web (`web.xml`).

```
1 ...
2 <context-param>
3     <param-name>
4       javax.faces.INTERPRET_EMPTY_STRING_SUBMITTED_VALUES_AS_NULL
5     </param-name>
6     <param-value>true</param-value>
7 </context-param>
8 ...
```

Código XML 10.5: web.xml

Veja abaixo as anotações disponíveis e suas respectivas funções.

- `@AssertFalse`
  Verifica se uma propriedade booleana possui valor `false`.
- `@AssertTrue`
  Verifica se uma propriedade booleana possui valor `true`.
- `@DecimalMax`
  Define o valor real máximo que uma propriedade pode armazenar.
- `@DecimalMin`
  Define o valor real mínimo que uma propriedade pode assumir.
- `@Digits`
  Define a quantidade máxima de dígitos da parte inteira (através do atributo `integer`) ou da parte fracionária (através do atributo `fraction`) de um número.
- `@Future`
  Verifica se uma data é posterior ao instante atual.
- `@Max`
  Define o valor inteiro máximo que uma propriedade pode assumir.
- `@Min`
  Define o valor inteiro mínimo que uma propriedade pode armazenar.
- `@NotNull`
  Verifica se o valor de uma propriedade não é `null`.
- `@Null`
  Verifica se o valor de uma propriedade é `null`.
- `@Past`
  Verifica se uma data é anterior ao instante atual.
- `@Pattern`
  Verifica se o valor de uma propriedade respeita uma expressão regular.
- `@Size`
  Define os tamanhos mínimo (através do atributo `min`) e máximo (através do atributo `max`) para uma `Collection`, array ou `String`.

As validações definidas através das anotações da especificação bean validation são aplicadas automaticamente pelo JSF. As mensagens de erro referentes a essas validações são automaticamente acrescentadas no contexto do processamento da requisição correspondente.



Mais Sobre

Para desabilitar bean validations em um componente particular de um formulário, podemos usar a tag `<f:validateBean>`. Isso pode ser feito atribuindo o valor "true" ao atributo `disabled` dessa tag. Considere o seguinte managed bean.

```
1 public class FuncionarioBean {
2
3   private Funcionario funcionario = new Funcionario();
4
5   // GETTER E SETTER
6 }
```

Código Java 10.10: FuncionarioBean.java

No exemplo abaixo, a validação introduzida pela anotação `@NotNull` do atributo `nome` da classe `Funcionario` é desabilitada.

```
1 <h:inputText value="#{funcionarioBean.funcionario.nome}">
2   <f:validateBean disabled="true" />
3 </h:inputText>
```

Código XHTML 10.23: Desabilitando validações introduzidas por anotações

Personalizando as mensagens de erro

Podemos personalizar as mensagens de erro através do atributo `message` das anotações do bean validation. Veja o seguinte código.

```
1 public class Funcionario {
2   @NotNull(message="O nome não pode ser nulo")
3   private String nome;
4   ...
5 }
```

Código Java 10.11: Funcionario.java

No exemplo acima, a mensagem de erro de validação está definida de maneira fixa. Pode ser mais apropriado defini-la em um arquivo de mensagens. Nesse caso, devemos criar um arquivo chamado `ValidationMessages.properties` no classpath da aplicação. No exemplo abaixo, a mensagem "O nome do funcionário não pode ser nulo" está associada à chave `br.com.k19.Funcionario.nome`.

```
1 br.com.k19.Funcionario.nome=O nome do funcionário não pode ser nulo
```

Arquivo de Propriedades 10.1: ValidationMessages.properties

Finalmente, devemos definir o valor do atributo `message` com a chave associada à mensagem que desejamos exibir. O identificador da mensagem deve estar entre chaves, como mostrado a seguir.

```
1 public class Funcionario {
2   @NotNull(message="{br.com.k19.Funcionario.nome}")
3   private String nome;
4   ...
5 }
```

Código Java 10.12: Funcionario.java

A mensagem "O nome do funcionário não pode ser nulo" é específica para o caso acima. Agora, suponha que desejamos criar uma mensagem comum associada ao validador `@NotNull`. Como o valor padrão da propriedade `message` desse validador é `"{javax.validation.constraints.NotNull-`

.message}", basta definir uma mensagem com essa chave no arquivo ValidationMessages.properties.

```
1 javax.validation.constraints.NotNull.message=O valor não pode ser nulo
```

Arquivo de Propriedades 10.2: ValidationMessages.properties

Podemos também alterar as mensagens dos demais validadores do bean validation. As chaves das mensagens associadas a esses validadores são listadas abaixo:

- @AssertFalse: "{javax.validation.constraints.AssertFalse.message}"
- @AssertTrue: "{javax.validation.constraints.AssertTrue.message}"
- @DecimalMax: "{javax.validation.constraints.DecimalMax.message}"
- @DecimalMin: "{javax.validation.constraints.DecimalMin.message}"
- @Digits: "{javax.validation.constraints.Digits.message}"
- @Future: "{javax.validation.constraints.Future.message}"
- @Max: "{javax.validation.constraints.Max.message}"
- @Min: "{javax.validation.constraints.Min.message}"
- @Null: "{javax.validation.constraints.Null.message}"
- @Past: "{javax.validation.constraints.Past.message}"
- @Pattern: "{javax.validation.constraints.Pattern.message}"
- @Size: "{javax.validation.constraints.Size.message}"



Exercícios de Fixação

12 Remova os validadores do arquivo **cadastro.xhtml** que você adicionou no exercício anterior.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10
11 <h:body>
12   <h1>Cadastro de Funcionário</h1>
13   <h:form>
14     <h:panelGrid columns="3">
15       <h:outputLabel value="Salário: R$ " for="campo-salario"/>
16       <h:inputText id="campo-salario" value="#{funcionarioBean.funcionario.salario}">
17         <f:convertNumber locale="pt_BR"/>
18       </h:inputText>
19       <h:message for="campo-salario"/>
20
21       <h:outputLabel value="Código: " for="campo-codigo"/>
22       <h:inputText id="campo-codigo" value="#{funcionarioBean.funcionario.codigo}"/>
23       <h:message for="campo-codigo"/>
24
```

```
25        <h:outputLabel value="Data: (dd/MM/yyyy)" for="campo-aniversario"/>
26        <h:inputText id="campo-aniversario"
27          value="#{funcionarioBean.funcionario.aniversario}">
28          <f:convertDateTime pattern="dd/MM/yyyy"/>
29        </h:inputText>
30        <h:message for="campo-aniversario"/>
31
32        <h:commandButton value="Cadastrar"/>
33      </h:panelGrid>
34      <h:messages/>
35    </h:form>
36  </h:body>
37  </html>
```

Código XHTML 10.24: www.k19.com.br/exercicios/k12/10/012/cadastro.xhtml

13 Usando bean validation, modifique a classe **Funcionario** para considerar as restrições do exercício anterior, isto é:

- O salário de um funcionário deve ser não-negativo.
- O código de um funcionário deve ser um número inteiro maior ou igual a 5 e estritamente menor do que 20.
- Todos os campos são obrigatórios.

```
1  package br.com.k19.modelo;
2
3  import java.util.Date;
4  import javax.validation.constraints.Max;
5  import javax.validation.constraints.Min;
6  import javax.validation.constraints.NotNull;
7
8  public class Funcionario {
9
10   @NotNull
11   @Min(value = 0)
12   private Double salario;
13
14   @NotNull
15   @Min(value = 5)
16   @Max(value = 19)
17   private Integer codigo;
18
19   @NotNull
20   private Date aniversario;
21
22   // GETTERS E SETTERS
23 }
```

Código Java 10.13: www.k19.com.br/exercicios/k12/10/013/Funcionario.java

14 Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/cadastro.xhtml

Teste o formulário com diversos valores e observe o que acontece.



Criando o seu Próprio Conversor

Se os conversores padrão não forem suficientes para atender as necessidades de uma aplicação, podemos criar os nossos próprios conversores. Considere uma aplicação que armazena números de telefones internacionais. Esses números serão modelados pela seguinte classe.

```
1 public class Telefone {
2   private String codigoDoPais;
3   private String codigoDeArea;
4   private String numeroLocal;
5
6   // GETTERS E SETTERS
7 }
```

Código Java 10.14: Telefone.java

Um número de telefone é divido em três partes: o código do país, o código da área e o número local. A interface da nossa aplicação utilizará o seguinte formato para apresentar ou receber números de telefone:

```
CodigoDoPais  CodigoDeArea  NumeroLocal
```

As três partes são separadas por espaços. Tanto o código do país quanto o código de área são formados apenas por dígitos. O número local, por outro lado, é formado por dígitos e, opcionalmente, por hifens. No exemplo abaixo, o número de telefone da K19 é mostrado nesse formato. Veja abaixo alguns exemplos de como o número de telefone da K19 poderia ser apresentado seguindo esse formato.

- `55 11 2387-3791`
- `55 11 23873791`
- `55 11 23-87-37-91`

Para automatizar o processo de transformação das strings digitadas pelos usuários em objetos da classe `Telefone` e vice-e-versa, podemos implementar um conversor JSF.

O primeiro passo para implementar um conversor JSF é criar uma classe que implementa a interface `javax.faces.convert.Converter`. Nessa classe, devemos adicionar a anotação `@FacesConverter` para indicar a classe associada a esse conversor, como no código abaixo.

```
1 @FacesConverter(forClass=Telefone.class)
2 public class ConversorDeTelefone implements Converter {
3   ...
4 }
```

Código Java 10.15: ConversorDeTelefone.java

A interface `Converter` exige a implementação de dois métodos: `getAsObject()` e `getAsString()`. O primeiro método deve transformar um objeto do tipo `String` em um objeto Java (em nosso caso, um objeto do tipo `Telefone`). Já o método `getAsString()` deve converter um objeto Java em um objeto do tipo `String` (em nosso caso, criar uma `String` a partir de um `Telefone`).

Uma possível implementação para o conversor de números de telefone é apresentada a seguir.

```
1 package br.com.k19;
2
```

```
3  @FacesConverter(forClass=Telefone.class)
4  public class ConversorDeTelefone implements Converter {
5
6    @Override
7    public Object getAsObject(FacesContext context, UIComponent component,
8        String value) {
9
10     value = value.trim();
11
12     if (!Pattern.matches("[0-9]+\\s+[0-9]+\\s+[0-9-]+", value)) {
13       FacesMessage mensagem = new FacesMessage("Número de telefone inválido");
14       mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
15       throw new ConverterException(mensagem);
16     }
17
18     String campos[] = value.split("\\s+");
19
20     String codigoDoPais = campos[0];
21     String codigoDeArea = campos[1];
22     String numeroLocal = campos[2].replaceAll("-", "");
23
24     Telefone telefone = new Telefone(codigoDoPais, codigoDeArea, numeroLocal);
25     return telefone;
26   }
27
28   @Override
29   public String getAsString(FacesContext context, UIComponent component,
30       Object value) {
31     Telefone telefone = (Telefone) value;
32     return telefone.getCodigoDoPais()
33         + " " + telefone.getCodigoDeArea()
34         + " " + telefone.getNumeroLocal();
35   }
36 }
```

Código Java 10.16: ConversorDeTelefone.java

Se a string recebida pelo método `getAsObject()` não respeita o formato de números de telefone definido acima, uma mensagem de erro é criada e uma exceção do tipo `javax.faces.convert.ConverterException` guardando essa mensagem é lançada.



Exercícios de Fixação

15 Considere um formulário que possua uma caixa de texto para receber um número de CPF (Cadastro de Pessoas Físicas). Um número de CPF é dividido em duas partes. A primeira parte do número é formada por nove dígitos. Chamaremos essa parte de número de identificação. A segunda parte é formada por dois dígitos, que são chamados de dígitos verificadores. Esses dois últimos dígitos são usados para verificar o número de identificação e prevenir erros. Um número de CPF é apresentado com a primeira parte separada da segunda por um hífen.

Implemente um conversor de números de CPF. Primeiramente, crie uma classe chamada **CPF** dentro do pacote **br.com.k19.modelo** para representar um número de CPF. Essa classe deve ter três atributos do tipo **int**: um para armazenar o número de identificação e outros dois para guardar os dígitos verificadores.

```
1  package br.com.k19.modelo;
2
3  public class CPF {
```

```
4   private int numeroDeIdentificacao;
5   private int primeiroDigitoVerificador;
6   private int segundoDigitoVerificador;
7
8   // GETTERS E SETTERS
9 }
```

Código Java 10.17: www.k19.com.br/exercicios/k12/10/015/CPF.java

16 Crie um pacote chamado **br.com.k19.converters**.

17 Agora, dentro do pacote **br.com.k19.converters**, crie uma classe chamada **ConversorDeCPF** para implementar o conversor de números de CPF. Um número de CPF deve respeitar o seguinte formato: `XXXXXXXXX-XX`. Lembre que essa classe deve ser anotada com **@FacesConverter** e implementar a interface **javax.faces.convert.Converter**.

```
1  package br.com.k19.converters;
2
3  import br.com.k19.modelo.CPF;
4  import java.util.regex.Pattern;
5  import javax.faces.application.FacesMessage;
6  import javax.faces.component.UIComponent;
7  import javax.faces.context.FacesContext;
8  import javax.faces.convert.Converter;
9  import javax.faces.convert.ConverterException;
10 import javax.faces.convert.FacesConverter;
11
12 @FacesConverter(forClass=CPF.class)
13 public class ConversorDeCPF implements Converter {
14
15   @Override
16   public Object getAsObject(FacesContext context, UIComponent component,
17       String value) {
18
19     value = value.trim();
20
21     if (!Pattern.matches("[0-9]{9}-[0-9]{2}", value)) {
22       FacesMessage mensagem = new FacesMessage("Número de CPF inválido");
23       mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
24       throw new ConverterException(mensagem);
25     }
26
27     String partesDoCPF[] = value.split("-");
28     int numeroDeIdentificacao = Integer.parseInt(partesDoCPF[0]);
29     int primeiroDigitoVerificador = Integer.parseInt(partesDoCPF[1].substring(0, 1));
30     int segundoDigitoVerificador = Integer.parseInt(partesDoCPF[1].substring(1, 2));
31
32     CPF cpf = new CPF();
33     cpf.setNumeroDeIdentificacao(numeroDeIdentificacao);
34     cpf.setPrimeiroDigitoVerificador(primeiroDigitoVerificador);
35     cpf.setSegundoDigitoVerificador(segundoDigitoVerificador);
36
37     return cpf;
38   }
39
40   @Override
41   public String getAsString(FacesContext context, UIComponent component,
42       Object value) {
43     CPF cpf = (CPF) value;
44     return cpf.getNumeroDeIdentificacao() + "-" + cpf.getPrimeiroDigitoVerificador()
45         + cpf.getSegundoDigitoVerificador();
46   }
47 }
```

Código Java 10.18: www.k19.com.br/exercicios/k12/10/017/ConversorDeCPF.java

18 No pacote **br.com.k19.controle**, crie uma classe chamada **CPFBean** para implementar o managed bean que dará suporte ao formulário.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.CPF;
4  import javax.faces.bean.ManagedBean;
5
6  @ManagedBean
7  public class CPFBean {
8    private CPF cpf;
9
10   // GETTERS E SETTERS
11 }
```

Código Java 10.19: www.k19.com.br/exercicios/k12/10/018/CPFBean.java

19 Finalmente, crie uma página chamada `formulario.xhtml` e implemente um formulário para receber um número de CPF.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html">
6  <h:head>
7    <title>K19 Treinamentos</title>
8  </h:head>
9
10 <h:body>
11   <h1>Formulário</h1>
12   <h:form>
13     <h:outputLabel value="CPF: " for="cpf"/>
14     <h:inputText id="cpf" value="#{cPFBean.cpf}"/>
15     <h:commandButton value="Enviar"/>
16   </h:form>
17
18   <h:message for="cpf" style="color: red" />
19 </h:body>
20 </html>
```

Código XHTML 10.25: www.k19.com.br/exercicios/k12/10/019/formulario.xhtml

20 Acesse a aplicação no endereço:
`http://localhost:8080/K19-Conversao-e-Validacao/formulario.xhtml`

Teste o formulário com diversos valores para campo de CPF. Observe as mensagens de erro de conversão para valores inválidos.



Exercícios Complementares

4 Em 2010, foi lançado um novo de documento de identificação no Brasil, o Registro de Identidade Civil (RIC). Esse documento tem um número que identifica unicamente cada cidadão. O número RIC tem duas partes. A primeira parte é formada por dez dígitos e a segunda, o dígito verificador, é formada por um único dígito. A primeira parte é opcionalmente separada da segunda por um hífen. Primeiro, você deve criar uma classe para modelar esse documento. Em seguida, você deve implementar um conversor para esse tipo de objeto. Por fim, você deve implementar um formulário para testar o seu conversor.



Criando o seu Próprio Validador

A criação de um validador é análoga à criação de um conversor. Precisamos criar uma classe que implemente a interface `javax.faces.validator.Validator`. Essa interface define um único método, o `validate()`. Além disso, a classe também precisa ser anotada com `@FacesValidator`.

Considere um formulário HTML que possui uma caixa de texto para o usuário digitar um número primo. Um número primo é um número natural maior do que 1 cujos únicos divisores positivos são 1 e ele mesmo. Essa caixa de texto está associada a uma propriedade do tipo `long` em um managed bean.

Após o usuário submeter esse formulário e uma vez que a etapa de conversão foi concluída, precisamos validar os dados do formulário. Em particular, precisamos verificar se o número digitado pelo usuário é primo. Criaremos uma classe chamada `ValidadorDeNumerosPrimos` para implementar um validador que execute essa tarefa.

```
1  package br.com.k19;
2
3  @FacesValidator("br.com.k19.ValidadorDeNumerosPrimos")
4  public class ValidadorDeNumerosPrimos implements Validator {
5    ...
6  }
```

Código Java 10.23: ValidadorDeNumerosPrimos.java

Uma forma simples de verificar se determinado número é primo é testar se ele é divisível por algum número maior do que 1 e menor ou igual à raiz quadrada desse número. Se ele for divisível por algum desses números, então ele não é primo. Caso contrário, ele é um número primo. No código abaixo, apresentamos uma implementação para esse validador.

```
1  package br.com.k19;
2
3  @FacesValidator("br.com.k19.ValidadorDeNumerosPrimos")
4  public class ValidadorDeNumerosPrimos implements Validator {
5
6    @Override
7    public void validate(FacesContext context, UIComponent component, Object value) {
8
9      long numero = (Long) value;
10     boolean isPrimo = false;
11
12     if (numero > 1) {
13       double raizQuadrada = Math.sqrt((double) numero);
14       isPrimo = true;
15
16       for (long i = 2; i <= raizQuadrada; i++) {
17         if (numero % i == 0) {
18           isPrimo = false;
```

```
19            break;
20          }
21        }
22      }
23
24      if (!isPrimo) {
25        FacesMessage mensagem = new FacesMessage("O número " + numero + " não é primo")↩
              ;
26        mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
27        throw new ValidatorException(mensagem);
28      }
29    }
30  }
```

Código Java 10.24: ValidadorDeNumerosPrimos.java

Se o número que estamos validando não for primo então precisamos lançar uma exceção do tipo `javax.faces.validator.ValidatorException`. Analogamente ao que fizemos na elaboração de um conversor, lançamos uma exceção contendo uma `FacesMessage` com uma mensagem de erro.

Para associar um validador a um determinado campo de um formulário, podemos usar a tag `<f:validator>`. Essa tag possui um atributo chamado `validatorId` cujo valor deve ser igual ao valor registrado na anotação `@FacesValidator`. Em nosso caso particular, esse valor é `"br.com.k19.ValidadorDeNumerosPrimos"`.

O código XHTML abaixo define um formulário com uma caixa de texto e vincula essa caixa ao validador de números primos.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11 <h:body>
12   <h:form>
13     <h:outputLabel value="Digite um número primo: " for="numero-primo" />
14     <h:inputText value="#{testeBean.numeroPrimo}" id="numero-primo">
15       <f:validator validatorId="br.com.k19.ValidadorDeNumerosPrimos" />
16     </h:inputText>
17     <h:commandButton value="Enviar" />
18     <br />
19     <h:message for="numero-primo" style="color: red" />
20   </h:form>
21 </h:body>
22 </html>
```

Código XHTML 10.27: Associando um validador a uma propriedade

Passando Parâmetros para o Validador

Para realizar a validação de um determinado dado, informações adicionais podem ser necessárias. Por exemplo, considere o caso em que precisamos verificar se uma data escolhida por um usuário pertence a um determinado intervalo. Nesse caso, precisamos de duas informações adicionais: as datas de início e fim do intervalo.

Da mesma forma que criamos um validador de números primos, podemos criar um validador

para realizar essa verificação. A diferença é que agora precisamos das datas que determinam o intervalo. Por exemplo, se tivermos uma caixa de texto para receber a data escolhida pelo usuário, então as datas de início e fim do intervalo podem ser definidas como atributos dessa caixa.

Para definir um atributo de um componente visual, podemos usar a tag `<f:attribute>`. Essa tag possui dois parâmetros (`name` e `value`), que são usados para definir o nome e o valor desse atributo.

Em nosso exemplo, criaremos dois atributos, que chamaremos de `"inicio"` e `"fim"`, para definir as datas que delimitam o intervalo a ser considerado. No código abaixo, criamos esses atributos com valores `"01/05/2014"` e `"30/08/2014"`, respectivamente.

```
<h:inputText value="#{testeBean.data}" id="campo-data">
  <f:attribute name="inicio" value="01/05/2014" />
  <f:attribute name="fim" value="30/08/2014"/>
</h:inputText>
```

Código XHTML 10.28: Adicionando atributos a um componente visual

Esses atributos podem ser recuperados no método `validade()` do validador da seguinte forma:

```
@Override
public void validate(FacesContext context, UIComponent component, Object value) {

  String inicio = (String) component.getAttributes().get("inicio");
  String fim = (String) component.getAttributes().get("fim");
  ...
}
```

Código Java 10.25: Recuperando atributos de um componente visual



Exercícios de Fixação

21 Agora, crie um validador de números de CPF. Como dissemos no exercício anterior, os dois últimos dígitos do CPF (dígitos verificadores) são usados para validar os demais dígitos (número de identificação). Os dígitos verificadores são gerados por um algoritmo chamado *Módulo 11* a partir do número de identificação.

O primeiro dígito verificador é obtido da seguinte forma. Suponha que o número de identificação seja $x_9x_8x_7x_6x_5x_4x_3x_2x_1$, onde x_i representa o i-ésimo dígito da direita para a esquerda do número de identificação. Seja y o valor dado por

$$y = 2x_1 + 3x_2 + 4x_3 + 5x_4 + 6x_5 + 7x_6 + 8x_7 + 9x_8 + 10x_9.$$

Se o valor do resto da divisão de y por 11 (y%11 em Java) for menor do que 2, então o primeiro dígito verificador é $d_1 = 0$. Caso contrário, o primeiro dígito verificador d_1 é igual a 11 menos o resto da divisão de y por 11.

O segundo dígito verificador é obtido de forma análoga, mas aplicando o algoritmo acima ao número $x_9x_8x_7x_6x_5x_4x_3x_2x_1d_1$. Nesse caso, o novo valor de y será

$$y = 2d_1 + 3x_1 + 4x_2 + 5x_3 + 6x_4 + 7x_5 + 8x_6 + 9x_7 + 10x_8 + 11x_9.$$

Crie um pacote chamado **br.com.k19.validators** no projeto **K19-Conversao-e-Validacao**.

22 No pacote **br.com.k19.validators**, crie uma classe chamada **ValidadorDeCPF**. Lembre que essa classe deve ser anotada com **@FacesValidator** e implementar a interface **javax.faces.validator.Validator**.

```
1  package br.com.k19.validators;
2
3  import br.com.k19.modelo.CPF;
4  import javax.faces.application.FacesMessage;
5  import javax.faces.component.UIComponent;
6  import javax.faces.context.FacesContext;
7  import javax.faces.validator.FacesValidator;
8  import javax.faces.validator.Validator;
9  import javax.faces.validator.ValidatorException;
10
11 @FacesValidator(value = "br.com.k19.validators.ValidadorDeCPF")
12 public class ValidadorDeCPF implements Validator {
13
14   @Override
15   public void validate(FacesContext context, UIComponent component,
16       Object value) throws ValidatorException {
17
18     CPF cpf = (CPF) value;
19     int numeroDeIdentificacao = cpf.getNumeroDeIdentificacao();
20     int primeiroDigitoVerificador = cpf.getPrimeiroDigitoVerificador();
21     int segundoDigitoVerificador = cpf.getSegundoDigitoVerificador();
22
23     if (!this.validaCPF(numeroDeIdentificacao, primeiroDigitoVerificador,
24         segundoDigitoVerificador)) {
25
26       String numero = numeroDeIdentificacao + "-"
27           + primeiroDigitoVerificador + segundoDigitoVerificador;
28
29       FacesMessage mensagem = new FacesMessage("O número " + numero
30           + " não é um CPF válido");
31       mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
32       throw new ValidatorException(mensagem);
33     }
34   }
35
36   private boolean validaCPF(int numeroDeIdentificacao,
37       int primeiroDigitoVerificador, int segundoDigitoVerificador) {
38
39     long primeiroDigito = this.modulo11((long) numeroDeIdentificacao);
40     long segundoDigito = this.modulo11((long) numeroDeIdentificacao * 10
41         + primeiroDigito);
42     return primeiroDigitoVerificador == primeiroDigito
43         && segundoDigitoVerificador == segundoDigito;
44   }
45
46   private long modulo11(long numero) {
47     long soma = 0;
48     long multiplicador = 2;
49     while (numero > 0) {
50       long digito = numero % 10;
51       soma += multiplicador * digito;
52       numero /= 10;
53       multiplicador++;
54     }
55     long resto = soma % 11;
56     if (resto < 2)
57       return 0;
58     else
59       return 11 - resto;
60   }
61 }
```

Código Java 10.26: www.k19.com.br/exercicios/k12/10/022/ValidadorDeCPF.java

23 No arquivo **formulario.xhtml**, associe a caixa de texto ao validador de números de CPF criado.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h1>Formulário</h1>
  <h:form>
    <h:outputLabel value="CPF: " for="cpf"/>
    <h:inputText id="cpf" value="#{cPFBean.cpf}">
      <f:validator validatorId="br.com.k19.validators.ValidadorDeCPF" />
    </h:inputText>
    <h:commandButton value="Enviar"/>
  </h:form>
  <h:message for="cpf" style="color: red" />
</h:body>
</html>
```

Código XHTML 10.29: www.k19.com.br/exercicios/k12/10/023/formulario.xhtml

24 Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/formulario.xhtml

Submeta o formulário com valores inválidos para o número de CPF e observe as mensagens de erro de validação.

Exercícios Complementares

5 Considere um formulário que possua um campo de texto para o usuário digitar uma data. Essa data deve pertencer a um determinado intervalo. Implemente um validador que considere essa restrição. As datas que definem o intervalo devem ser passadas como parâmetro para o validador com o uso da tag `<f:attribute>`.

Criando o seu Próprio Bean Validator

Vimos que os bean validators podem ser registrados através de anotações. Agora, mostraremos como criar o seu próprio bean validator e uma anotação para aplicá-lo. Para exemplificar, considere a validação de números primos. No exemplo abaixo, definimos a anotação `@Primo`.

```
package br.com.k19;

import static java.lang.annotation.ElementType.ANNOTATION_TYPE;
import static java.lang.annotation.ElementType.FIELD;
import static java.lang.annotation.ElementType.METHOD;
import static java.lang.annotation.RetentionPolicy.RUNTIME;

```

```
8  import java.lang.annotation.Retention;
9  import java.lang.annotation.Target;
10
11 import javax.validation.Constraint;
12 import javax.validation.Payload;
13
14 @Target({ METHOD, FIELD, ANNOTATION_TYPE })
15 @Retention(RUNTIME)
16 @Constraint(validatedBy = ValidadorDePrimo.class)
17 public @interface Primo {
18   String message() default "{br.com.k19.Primo.message}";
19   Class<?>[] groups() default {};
20   Class<? extends Payload>[] payload() default {};
21 }
```

Código Java 10.29: Primo.java

Note que a anotação `@Primo` é vinculada ao validador definido pela classe `ValidadorDePrimo`. Esse vínculo é estabelecido através da propriedade `validatedBy` da anotação `@Constraint`.

Observe também que, de acordo com o código acima, a mensagem definida pela chave `br.com.k19.Primo.message` será utilizada por padrão quando um erro for identificado na validação.

A classe que define o validador de números primos deve implementar a interface `javax.validation.ConstraintValidator`. Essa interface possui dois métodos: `initialize()` e `isValid()`. O primeiro é utilizado para extrair, se necessário, os valores dos atributos da anotação. O segundo deve implementar a validação propriamente dita.

```
1  public class ValidadorDePrimo implements ConstraintValidator<Primo, Long> {
2
3    @Override
4    public void initialize(Primo constraintAnnotation) {
5
6    }
7
8    @Override
9    public boolean isValid(Long value, ConstraintValidatorContext context) {
10     if (value <= 1) {
11       return false;
12     }
13     double raizQuadrada = Math.sqrt((double) value);
14
15     for (long i = 2; i <= raizQuadrada; i++) {
16       if (value % i == 0) {
17         return false;
18       }
19     }
20     return true;
21   }
22 }
```

Código Java 10.30: ValidadorDePrimo.java

Por fim, devemos definir o valor da mensagem associada à chave `br.com.k19.Primo.message` no arquivo `ValidationMessages.properties`.

```
1  br.com.k19.Primo.message=O número não é primo
```

Arquivo de Propriedades 10.3: ValidationMessages.properties



Exercícios de Fixação

25 Seguindo o exercício anterior, crie um bean validator para validar números de CPF. Primeiro, crie um pacote chamado **br.com.k19.beanvalidators**.

26 No pacote **br.com.k19.beanvalidators**, defina a anotação **@CPF** como no código abaixo.

```
1  package br.com.k19.beanvalidators;
2
3  import static java.lang.annotation.ElementType.ANNOTATION_TYPE;
4  import static java.lang.annotation.ElementType.FIELD;
5  import static java.lang.annotation.ElementType.METHOD;
6  import static java.lang.annotation.RetentionPolicy.RUNTIME;
7
8  import java.lang.annotation.Retention;
9  import java.lang.annotation.Target;
10
11 import javax.validation.Constraint;
12 import javax.validation.Payload;
13
14 @Target({ METHOD, FIELD, ANNOTATION_TYPE })
15 @Retention(RUNTIME)
16 @Constraint(validatedBy = br.com.k19.beanvalidators.CPFBeanValidator.class)
17 public @interface CPF {
18   String message() default "O número de CPF não é válido";
19   Class<?>[] groups() default {};
20   Class<? extends Payload>[] payload() default {};
21 }
```

Código Java 10.31: www.k19.com.br/exercicios/k12/10/026/CPF.java

27 No pacote **br.com.k19.beanvalidators**, crie uma classe chamada **CPFBeanValidator** para implementar o validador associado à anotação **@CPF**. Essa classe deve implementar a interface **javax.validation.ConstraintValidator**.

```
1  package br.com.k19.beanvalidators;
2
3  import javax.validation.ConstraintValidator;
4  import javax.validation.ConstraintValidatorContext;
5
6  public class CPFBeanValidator implements
7      ConstraintValidator<CPF, br.com.k19.modelo.CPF> {
8
9    @Override
10   public void initialize(CPF constraintAnnotation) {
11   }
12
13   @Override
14   public boolean isValid(br.com.k19.modelo.CPF cpf, ConstraintValidatorContext ↩
         context) {
15
16     int numeroDeIdentificacao = cpf.getNumeroDeIdentificacao();
17     int primeiroDigitoVerificador = cpf.getPrimeiroDigitoVerificador();
18     int segundoDigitoVerificador = cpf.getSegundoDigitoVerificador();
19
20     return this.validaCPF(numeroDeIdentificacao, primeiroDigitoVerificador,
21         segundoDigitoVerificador);
22   }
23
```

```
24    private boolean validaCPF(int numeroDeIdentificacao,
25        int primeiroDigitoVerificador, int segundoDigitoVerificador) {
26
27      long primeiroDigito = this.modulo11((long) numeroDeIdentificacao);
28      long segundoDigito = this.modulo11((long) numeroDeIdentificacao * 10
29          + primeiroDigito);
30      return primeiroDigitoVerificador == primeiroDigito
31          && segundoDigitoVerificador == segundoDigito;
32    }
33
34    private long modulo11(long numero) {
35      long soma = 0;
36      long multiplicador = 2;
37      while (numero > 0) {
38        long digito = numero % 10;
39        soma += multiplicador * digito;
40        numero /= 10;
41        multiplicador++;
42      }
43      long resto = soma % 11;
44      if (resto < 2)
45        return 0;
46      else
47        return 11 - resto;
48    }
49  }
```

Código Java 10.32: www.k19.com.br/exercicios/k12/10/027/CPFBeanValidator.java

28 Na classe **CPFBean**, adicione a anotação **@CPF** ao atributo **cpf** para usar o validador que você acabou de criar.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.CPF;
4  import javax.faces.bean.ManagedBean;
5
6  @ManagedBean
7  public class CPFBean {
8
9    @br.com.k19.beanvalidators.CPF
10   private CPF cpf;
11
12   // GETTERS E SETTERS
13 }
```

Código Java 10.33: www.k19.com.br/exercicios/k12/10/028/CPFBean.java

29 Modifique no arquivo **formulario.xhtml** a linha em destaque.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html">
6  <h:head>
7    <title>K19 Treinamentos</title>
8  </h:head>
9
10 <h:body>
11   <h1>Formulário</h1>
12   <h:form>
13     <h:outputLabel value="CPF: " for="cpf"/>
14     <h:inputText id="cpf" value="#{cPFBean.cpf}"/>
```

```
15      <h:commandButton value="Enviar"/>
16    </h:form>
17    <h:message for="cpf" style="color: red" />
18  </h:body>
19  </html>
```

Código XHTML 10.32: www.k19.com.br/exercicios/k12/10/029/formulario.xhtml

30 Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/formulario.xhtml

EVENTOS

CAPÍTULO 11

Aplicações JSF são fortemente baseadas em eventos. Esses eventos podem ser gerados pelos usuários da aplicação ou pelo próprio JSF. Veja abaixo alguns exemplos de eventos gerados pelos usuários.

- Um clique em um botão ou link.
- A troca do valor preenchido em uma caixa de texto.
- A troca da opção escolhida em uma caixa de seleção.

Veja abaixo alguns exemplos de eventos gerados pelo JSF.

- Inicialização da aplicação.
- Finalização da aplicação.
- Erro no processamento de uma requisição.

Um evento pode ser associado a procedimentos que serão executados quando esse evento ocorrer. No JSF, os eventos são divididos em três categorias: `FacesEvent`, `PhaseEvent` e `SystemEvent`.



FacesEvent

Há dois tipos de `FacesEvents`: `ActionEvent` e `ValueChangeEvent`. Apresentaremos a seguir o funcionamento desses tipos de eventos.

ActionEvent

Um `ActionEvent` é gerado quando um botão ou link é pressionado pelo usuário. Métodos de um managed bean podem ser associados a esses eventos. Toda vez que um evento ocorrer os métodos associados a ele serão executados. Para associar métodos de um managed bean a um `ActionEvent`, podemos utilizar os atributos `action` ou `actionListener` dos componentes `<h:commandButton>` e `<h:commandLink>`

O atributo `action` deve ser associado a um método público do tipo `void` ou `String` de um managed bean. Se esse método devolver uma `String`, ela será utilizada para determinar a navegação das páginas (veja Capítulo 8). Conceitualmente, o atributo `action` deve ser associado a métodos que representam alguma regra de negócio da aplicação.

No exemplo abaixo, o botão "Salva" está associado ao método `salva()` do managed bean `produtoBean` através do atributo `action` da tag `<h:commandButton>`. Esse método implementa a lógica de negócio para cadastrar produtos. A `String ''lista-produtos''` devolvida pelo método `salva()` será utilizada para determinar a navegação das páginas.

```
<h:commandButton value="Salva" action="#{produtoBean.salva}" />
```

Código XHTML 11.1: Botão associado ao método `salva()` *do managed bean* `produtoBean`

```
@ManagedBean
public class ProdutoBean {
  private Produto produto = new Produto();
  private List<Produto> produtos = new ArrayList<Produto>();

  public String salva() {
    this.produtos.add(this.produto);
    this.produto = new Produto();
    return "lista-produtos";
  }

  // GETTERS E SETTERS
}
```

Código Java 11.1: ProdutoBean.java

O atributo `actionListener`, por sua vez, deve ser associado a um método público do tipo `void` que receba um `ActionEvent` como parâmetro. Conceitualmente, o atributo `actionListener` deve ser associado a métodos que implementam alguma lógica associada à interface do usuário.

No exemplo abaixo, o botão "Clique Aqui" está associado ao método `mudaTexto()` do managed bean `cliqueBean` através do atributo `actionListener` da tag `<h:commandButton>`. Esse método altera o texto do botão para "Clicado".

```
<h:commandButton value="Clique Aqui" actionListener="#{cliqueBean.mudaTexto}"/>
```

Código XHTML 11.2: Botão associado ao método `mudaTexto()` *do managed bean* `cliqueBean`

```
@ManagedBean
public class CliqueBean {
  public void mudaTexto(ActionEvent e) {
    UICommand c = (UICommand) e.getComponent();
    c.setValue("Clicado");
  }
}
```

Código Java 11.2: CliqueBean.java

Os atributos `action` e `actionListener` permitem que no máximo dois métodos sejam associados a um evento de ação. Para associar mais do que dois métodos a um evento de ação, podemos usar a tag `<f:actionListener>`. O valor do atributo `type` dessa tag deve ser o nome de uma classe que implemente a interface `javax.faces.event.ActionListener`. Essa interface exige a criação de um método chamado `processAction()`. Veja o exemplo abaixo.

```
<h:commandLink value="Enviar" action="..." actionListener="...">
  <f:actionListener type="br.com.k19.MudaCorDaFonte"/>
</h:commandLink>
```

Código XHTML 11.3: Link associado ao método `processAction()` *da classe* `MudaCorDaFonte`

```
package br.com.k19;

public class MudaCorDaFonte implements ActionListener {
  @Override
  public void processAction(ActionEvent e) {
```

```
6     UICommand c = (UICommand) e.getComponent();
7     c.getAttributes().put("style", "color: red;");
8   }
9 }
```

Código Java 11.3: MudaCorDaFonte.java

Nesse exemplo, usamos a tag `<f:actionListener>` para associar o link ao método `processAction()` da classe `br.com.k19.MudaCorDaFonte`.



Importante
Os métodos associados a um evento de ação são executados na seguinte ordem:

1. O método associado com o atributo `actionListener`.
2. Métodos associados com tag `<f:actionListener>` de acordo com a ordem em que elas aparecem no arquivo XHTML.
3. O método indicado pelo atributo `action`.



Importante
Por padrão, os métodos associados a eventos de ação são executados na fase *Invoke Application* do processamento de uma requisição (veja o Capítulo 5 para mais informações).

ValueChangeEvent

Um `ValueChangeEvent` é produzido quando o valor de uma caixa de texto ou a opção de uma caixa de seleção são alterados. Podemos associar métodos de um managed bean a esses eventos. Tais métodos devem receber um parâmetro do tipo `ValueChangeEvent` e serem públicos. Para estabelecer essa associação, podemos utilizar o atributo `valueChangeListener` das caixas de texto ou de seleção ou a tag `<f:valueChangeListener>`.

Considere um formulário usado para editar as informações dos produtos de uma loja. No exemplo abaixo, associamos uma caixa de texto ao método `mudaPreco()` do managed bean `produtoBean` através do atributo `valueChangeListener` da tag `<h:inputText>`. Esse método registra a mudança do preço de um produto.

```
1 <h:outputLabel value="Preço: " for="preco" />
2 <h:inputText valueChangeListener="#{produtoBean.mudaPreco}" id="preco" />
```

Código XHTML 11.4: Caixa de texto associada ao método `mudaPreco()` do managed bean `produtoBean`

```
1 @ManagedBean
2 public class ProdutoBean {
3   public void mudaPreco(ValueChangeEvent e) {
4     System.out.println("Preço antigo: " + e.getOldValue());
5     System.out.println("Preço novo: " + e.getNewValue());
6   }
7 }
```

Código Java 11.4: ProdutoBean.java

O mesmo pode ser feito com o uso da tag <f:valueChangeListener>. No exemplo abaixo, criamos uma classe chamada RegistraAlteracao, que implementa a interface ValueChangeListener, e associamos a caixa de texto ao método processValueChange() dessa classe.

```
1 <h:outputLabel value="Preço: " for="preco" />
2 <h:inputText id="preco">
3   <f:valueChangeListener type="br.com.k19.RegistraAlteracao" />
4 </h:inputText>
```

Código XHTML 11.5: Caixa de texto associada ao método processValueChange() *da classe* RegistraAlteracao

```
1 package br.com.k19;
2
3 public class RegistraAlteracao implements ValueChangeListener {
4   @Override
5   public void processValueChange(ValueChangeEvent e) {
6     System.out.println("Preço antigo: " + e.getOldValue());
7     System.out.println("Preço novo: " + e.getNewValue());
8   }
9 }
```

Código Java 11.5: RegistraAlteracao.java



Importante
Os métodos associados a um evento de mudança de valor são executados na seguinte ordem:

1. O método associado com o atributo valueChangeListener.
2. Métodos associados com tag <f:valueChangeListener> na ordem em que elas aparecem no arquivo XHTML.



Importante
Os métodos associados a eventos de mudança de valor são executados na fase *Process Validations* do processamento de uma requisição. Veja o Capítulo 5 para mais informações sobre o processamento de uma requisição.



Exercícios de Fixação

1 Crie um projeto chamado **K19-Eventos** seguindo os passos vistos no exercício do Capítulo 5.

2 Criaremos uma página com três botões. Um dos botões estará habilitado e os outros dois desabilitados. Quando o usuário pressionar o botão habilitado, a aplicação deve escolher aleatoriamente qual dos três botões estará habilitado da próxima vez e desabilitar os outros dois.

Adicione uma página chamada **botoes.xhtml** no projeto **K19-Eventos** com o seguinte conteúdo.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 - Eventos</title>
</h:head>

<h:body>
  <h:form>
    <h:commandButton
      id="botao-jonas"
      value="Jonas"
      disabled="false"
      actionListener="#{botaoBean.sorteiaBotao}"/>

    <h:commandButton
      id="botao-marcelo"
      value="Marcelo"
      disabled="true"
      actionListener="#{botaoBean.sorteiaBotao}"/>

    <h:commandButton
      id="botao-rafael"
      value="Rafael"
      disabled="true"
      actionListener="#{botaoBean.sorteiaBotao}"/>
  </h:form>
</h:body>
</html>
```

Código XHTML 11.6: www.k19.com.br/exercicios/k12/11/002/botoes.xhtml

3 Crie um managed bean para implementar o tratamento dos botões. Adicione em um pacote chamado **br.com.k19.controle** uma classe com o nome **BotaoBean** e com o seguinte conteúdo.

```
package br.com.k19.controle;

import javax.faces.bean.ManagedBean;
import javax.faces.component.UIComponent;
import javax.faces.event.ActionEvent;

@ManagedBean
public class BotaoBean {
  public void sorteiaBotao(ActionEvent event){
    UIComponent formulario = event.getComponent().getParent();

    UIComponent botaoJonas = formulario.findComponent("botao-jonas");
    UIComponent botaoMarcelo = formulario.findComponent("botao-marcelo");
    UIComponent botaoRafael = formulario.findComponent("botao-rafael");

    botaoJonas.getAttributes().put("disabled", true);
    botaoMarcelo.getAttributes().put("disabled", true);
    botaoRafael.getAttributes().put("disabled", true);

    double numero = Math.random();

    if(numero < 1.0/3.0){
      botaoJonas.getAttributes().put("disabled", false);
    } else if(numero < 2.0/3.0){
      botaoMarcelo.getAttributes().put("disabled", false);
    } else {
      botaoRafael.getAttributes().put("disabled", false);
    }
```

```
29    }
30 }
```

Código Java 11.6: www.k19.com.br/exercicios/k12/11/003/BotaoBean.java

4 Remova o projeto **K19-Conversao-e-Validacao** do GlassFish e do Wildfly. Implante o projeto **K19-Eventos** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

```
http://localhost:8080/K19-Eventos/botoes.xhtml
```

5 Implante o projeto **K19-Eventos** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

```
http://localhost:8080/K19-Eventos/botoes.xhtml
```



PhaseEvent

Os `PhaseEvents` são eventos disparados automaticamente pelo JSF antes e depois de cada uma das fases do processamento de uma requisição. Esses eventos são úteis basicamente para monitorar a execução dessas fases.

Para tratar um `PhaseEvent`, podemos criar uma classe que implemente a interface `javax.faces.event.PhaseListener`. Além disso, precisamos registrar essa classe no arquivo `faces-config.xml`. A interface `PhaseListener` obriga a implementação de três métodos: `afterPhase()`, `beforePhase()` e `getPhaseId()`.

O método `getPhaseId()` deve devolver a referência de um objeto do tipo `javax.faces.event.PhaseId`, que indica em quais fases do processamento de uma requisição estamos interessados. Se estivermos interessados nos eventos da fase *Apply Request Values*, por exemplo, podemos fazer esse método devolver `PhaseId.APPLY_REQUEST_VALUES`. Os valores que podem ser utilizados são:

- `PhaseId.RESTORE_VIEW`
- `PhaseId.APPLY_REQUEST_VALUES`
- `PhaseId.INVOKE_APPLICATION`
- `PhaseId.PROCESS_VALIDATIONS`
- `PhaseId.UPDATE_MODEL_VALUES`
- `PhaseId.RENDER_RESPONSE`
- `PhaseId.ANY_PHASE`

O método `beforePhase()` recebe a referência de um `PhaseEvent` como parâmetro e é executado antes das fases especificadas pelo método `getPhaseId()`. O método `afterPhase()` é análogo ao `beforePhase()` e é executado após as fases especificadas pelo método `getPhaseId()`.

No exemplo abaixo, criamos a classe `MonitorPhaseListener` que implementa a interface `PhaseListener`. No método `getPhaseId()`, devolvemos `PhaseId.ANY_PHASE`, que indica que estamos interessados nos eventos relacionados a todas as fases.

```java
package br.com.k19;

public class MonitorPhaseListener implements PhaseListener {

  @Override
  public void afterPhase(PhaseEvent event) {
    System.out.println("MonitorPhaseListener.afterPhase()");
    System.out.println(event.getPhaseId());
  }

  @Override
  public void beforePhase(PhaseEvent event) {
    System.out.println("MonitorPhaseListener.beforePhase()");
    System.out.println(event.getPhaseId());
  }

  @Override
  public PhaseId getPhaseId() {
    return PhaseId.ANY_PHASE;
  }
}
```

Código Java 11.7: MonitorPhaseListener.java

No arquivo `faces-config.xml`, registramos essa classe da seguinte forma:

```
...
<lifecycle>
  <phase-listener>br.com.k19.MonitorPhaseListener</phase-listener>
</lifecycle>
...
```

Código XML 11.1: faces-config.xml



Exercícios de Fixação

6 Crie uma classe chamada **MonitorPhaseListener** em um pacote chamado `br.com.k19.phaselisteners` no projeto **K19-Eventos**.

```java
package br.com.k19.phaselisteners;

import javax.faces.event.PhaseEvent;
import javax.faces.event.PhaseId;
import javax.faces.event.PhaseListener;

public class MonitorPhaseListener implements PhaseListener {

  @Override
  public void afterPhase(PhaseEvent event) {
    System.out.println("MonitorPhaseListener.afterPhase() - " + event.getPhaseId());
  }

  @Override
  public void beforePhase(PhaseEvent event) {
    System.out.println("MonitorPhaseListener.beforePhase() - " + event.getPhaseId());
  }

  @Override
  public PhaseId getPhaseId() {
    return PhaseId.ANY_PHASE;
  }
```

```
23 }
```

Código Java 11.8: www.k19.com.br/exercicios/k12/11/006/MonitorPhaseListener.java

7 Registre a classe **MonitorPhaseListener** no arquivo **faces-config.xml**.

```
1 ...
2 <lifecycle>
3   <phase-listener>br.com.k19.phaselisteners.MonitorPhaseListener</phase-listener>
4 </lifecycle>
5 ...
```

Código XML 11.2: www.k19.com.br/exercicios/k12/11/007/faces-config.xml

8 Acesse a página **botoes.xhtml**, clique nos botões e verifique as mensagens exibidas no Console .



SystemEvent

`SystemEvents` são similares aos `PhaseEvents` no sentido de que ambos estão relacionados a pontos específicos de execução de uma aplicação JSF. Contudo, os eventos de sistema são gerados em diversos pontos não cobertos pelos eventos de fase. Todas as classes que definem eventos de sistema são subclasses de `javax.faces.event.SystemEvent`. Apresentamos abaixo algumas dessas classes.

- `PostConstructApplicationEvent`
  É gerado imediatamente após o início da aplicação, depois que todas as configurações são processadas.
- `PreDestroyApplicationEvent`
  Esse tipo de evento é gerado imediatamente antes da aplicação ser finalizada.
- `ExceptionQueuedEvent`
  Esse evento é gerado assim que uma exceção não esperada é lançada durante o processamento de uma requisição. Alguns exemplos de exceções esperadas durante o processamento de uma requisição são aquelas relacionadas à conversão e validação dos dados.
- `PreValidateEvent` e `PostValidateEvent`
  Esses eventos são gerados imediatamente antes e logo após um componente ser validado, respectivamente.

As três primeiras classes listadas acima são subclasses diretas de `SystemEvent`, enquanto que as duas últimas são subclasses de `javax.faces.event.ComponentSystemEvent`.

Uma das formas para se registrar um interessado em eventos de sistema é usando a tag `<f:event>`. Essa tag possui dois atributos: `type` e `listener`. O atributo `type` é usado para indicar o tipo de evento. Alguns dos possíveis valores para esse atributo são `preValidate` e `postValidate`. O atributo `listener` deve indicar um método que será chamado quando o evento for processado. Esse método deve ser público, do tipo `void`, e receber como parâmetro uma referência para um objeto do tipo `javax.faces.event.ComponentSystemEvent`.

O evento `PostValidateEvent` pode ser útil na validação de múltiplos campos, por exemplo. Considere uma aplicação bancária que permite gerar extratos da conta corrente. Os dados de entrada são

duas datas e o resultado é o extrato referente ao período determinado por essas datas. Uma restrição que devemos impor é que a data final não pode vir antes da data inicial.

No formulário do código abaixo, usamos a tag `<f:event>` para indicar que o método `validaDatas()` do managed bean `extratoBean` deve ser chamado assim que o evento `postValidate` for processado.

```
<h:form>
  <f:event type="postValidate" listener="#{extratoBean.validaDatas}" />
  <h:messages/>

  <h:outputLabel value="Data inicial: " for="data-inicial"/>
  <h:inputText value="#{extratoBean.dataInicial}" id="data-inicial" required="true">
    <f:convertDateTime pattern="dd/MM/yyyy" />
  </h:inputText>

  <h:outputLabel value="Data final: " for="data-final"/>
  <h:inputText value="#{extratoBean.dataFinal}" id="data-final" required="true">
    <f:convertDateTime pattern="dd/MM/yyyy" />
  </h:inputText>

  <h:commandButton value="Ver extrato" action="#{extratoBean.geraExtrato}" />
</h:form>
```

Código XHTML 11.7: O método `validaDatas()` *do managed bean* `extratoBean` *é registrado para receber eventos do tipo* `PostValidateEvent`

Na classe `ExtratoBean`, definimos o método `validaDatas()` que recebe uma referência para um `ComponentSystemEvent` como parâmetro. Após verificar se as datas inicial e final são validas (tarefa executada pelo validador de datas padrão de cada um dos componentes), prosseguimos com a verificação da nossa restrição adicional, isto é, a data inicial deve vir antes da data final.

Se essa restrição não for respeitada, não devemos exibir extrato algum. Dessa forma, adicionamos uma mensagem no contexto do processamento da requisição e redirecionamos o fluxo para a última fase do processamento, isto é, para a fase `Render Response`.

```
package managedbeans;

@ManagedBean
public class ExtratoBean {

  private Date dataInicial;
  private Date dataFinal;

  public void validaDatas(ComponentSystemEvent event) {

    UIComponent source = event.getComponent();
    UIInput dataInicialInput = (UIInput) source.findComponent("data-inicial");
    UIInput dataFinalInput = (UIInput) source.findComponent("data-final");

    if (dataInicialInput.isValid() && dataFinalInput.isValid()) {

      Date dataInicialEscolhida = (Date) dataInicialInput.getLocalValue();
      Date dataFinalEscolhida = (Date) dataFinalInput.getLocalValue();

      if (dataFinalEscolhida.before(dataInicialEscolhida)) {
        FacesMessage message =
          new FacesMessage("A data final não pode vir antes da data inicial");
        message.setSeverity(FacesMessage.SEVERITY_ERROR);
        FacesContext context = FacesContext.getCurrentInstance();
        context.addMessage(source.getClientId(), message);
        context.renderResponse();
      }
    }
```

```
29    }
30
31    public void geraExtrato() {
32      FacesMessage message = new FacesMessage("extrato gerado com sucesso");
33      message.setSeverity(FacesMessage.SEVERITY_INFO);
34      FacesContext context = FacesContext.getCurrentInstance();
35      context.addMessage(null, message);
36    }
37
38    // GETTERS E SETTERS
39  }
```

Código Java 11.9: ExtratoBean.java



Exercícios de Fixação

9 Adicione uma página chamada **extrato.xhtml** no projeto **K19-Eventos** com o seguinte conteúdo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:ui="http://java.sun.com/jsf/facelets"
6    xmlns:h="http://java.sun.com/jsf/html"
7    xmlns:f="http://java.sun.com/jsf/core">
8
9  <h:head>
10   <title>K19 - Eventos</title>
11 </h:head>
12
13 <h:body>
14   <h:form>
15     <f:event type="postValidate" listener="#{extratoBean.validaDatas}" />
16         <h:messages/>
17     <h:outputLabel value="Data inicial: " for="data-inicial" />
18     <h:inputText
19       value="#{extratoBean.dataInicial}"
20       id="data-inicial"
21       required="true">
22       <f:convertDateTime pattern="dd/MM/yyyy" />
23     </h:inputText>
24
25     <h:outputLabel value="Data final: " for="data-final" />
26     <h:inputText
27       value="#{extratoBean.dataFinal}"
28       id="data-final"
29       required="true">
30       <f:convertDateTime pattern="dd/MM/yyyy" />
31     </h:inputText>
32
33     <h:commandButton value="Ver extrato" action="#{extratoBean.geraExtrato}" />
34   </h:form>
35 </h:body>
36 </html>
```

Código XHTML 11.8: www.k19.com.br/exercicios/k12/11/009/extrato.xhtml

10 Crie uma classe chamada **ExtratoBean** no pacote **br.com.k19.controle** com o seguinte conteúdo:

```java
package br.com.k19.controle;

import java.util.Date;
import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.component.UIComponent;
import javax.faces.component.UIInput;
import javax.faces.context.FacesContext;
import javax.faces.event.ComponentSystemEvent;

@ManagedBean
public class ExtratoBean {

  private Date dataInicial;
  private Date dataFinal;

  public void validaDatas(ComponentSystemEvent event) {

    UIComponent source = event.getComponent();
    UIInput dataInicialInput = (UIInput) source.findComponent("data-inicial");
    UIInput dataFinalInput = (UIInput) source.findComponent("data-final");

    if (dataInicialInput.isValid() && dataFinalInput.isValid()) {

      Date dataInicialEscolhida = (Date) dataInicialInput.getLocalValue();
      Date dataFinalEscolhida = (Date) dataFinalInput.getLocalValue();

      if (dataFinalEscolhida.before(dataInicialEscolhida)) {
        FacesMessage message =
          new FacesMessage("A data final não pode vir antes da data inicial");
        message.setSeverity(FacesMessage.SEVERITY_ERROR);
        FacesContext context = FacesContext.getCurrentInstance();
        context.addMessage(source.getClientId(), message);
        context.renderResponse();
      }
    }
  }

  public void geraExtrato() {
    FacesMessage message = new FacesMessage("extrato gerado com sucesso");
    message.setSeverity(FacesMessage.SEVERITY_INFO);
    FacesContext context = FacesContext.getCurrentInstance();
    context.addMessage(null, message);
  }

  // GETTERS E SETTERS
}
```

Código Java 11.10: www.k19.com.br/exercicios/k12/11/010/ExtratoBean.java

11 Acesse a página **extrato.xhtml** e teste a validação das datas.

Immediate

Por padrão, a conversão e a validação dos dados de um componente de entrada (`<h:inputText>`, por exemplo) são realizadas na fase *Process Validation*. Além disso, os eventos de mudança de valor (`ValueChangeEvent`) também ocorrem nessa fase.

Também por padrão, os eventos de ação (`ActionEvent`) associados aos componentes `<h:commandButton>` ou `<h:commandLink>` são disparados no final da fase `Invoke Application` do processamento de uma requisição.

Porém, esse comportamento pode ser alterado através do atributo `immediate` desses componentes. Se o valor do atributo `immediate` de um componente de entrada for `true`, então a conversão e validação dos dados desse componente irão ocorrer durante a fase *Apply Request Values*, onde também irão ser disparados possíveis eventos de mudança de valor. No caso dos links e botões, os eventos de ação serão disparados no final da fase *Apply Request Values*.



Exercícios de Fixação

Crie um formulário para o cadastro de pessoas. Esse formulário deve possuir três campos obrigatórios. O primeiro deve ser uma caixa de texto para receber o nome de uma pessoa. O segundo deve ser uma caixa de seleção para o usuário escolher um estado. O terceiro campo também deve ser uma caixa de seleção, que permitirá ao usuário escolher uma cidade. A caixa de seleção de cidade deve exibir apenas as cidades do estado escolhido. Ou seja, quando o usuário selecionar determinado estado, a lista de cidades deve ser atualizada de acordo.

12 Crie um pacote chamado **br.com.k19.modelo** no projeto `K19-Eventos`.

13 No pacote **br.com.k19.modelo**, crie uma classe chamada **Estado** para modelar um estado. Essa classe deve ter como atributos o nome e a sigla do estado, bem como uma lista de cidades.

```
1  package br.com.k19.modelo;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class Estado {
7    private String nome;
8    private String sigla;
9    private List<String> cidades = new ArrayList<String>();
10
11   // GETTERS E SETTERS
12 }
```

Código Java 11.11: www.k19.com.br/exercicios/k12/11/013/Estado.java

14 No pacote **br.com.k19.controle**, crie uma classe chamada **CadastroBean**, que dará suporte ao formulário de cadastro. Essa classe também deve armazenar uma lista de estados que poderão ser escolhidos no formulário. Anote essa classe com **@SessionScoped** para que os dados do formulário sejam mantidos entre uma requisição e outra.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Estado;
4  import java.util.ArrayList;
5  import java.util.List;
6  import javax.faces.bean.ManagedBean;
7  import javax.faces.bean.SessionScoped;
8  import javax.faces.event.ValueChangeEvent;
9
10 @ManagedBean
11 @SessionScoped
12 public class CadastroBean {
13
```

```
14   private String nome;
15   private String cidade;
16   private String siglaDoEstadoEscolhido;
17
18   private Estado estadoSelecionado = new Estado();
19   private List<Estado> estados = new ArrayList<Estado>();
20
21   public CadastroBean() {
22     Estado sp = new Estado();
23     sp.setSigla("SP");
24     sp.setNome("São Paulo");
25     sp.getCidades().add("São Paulo");
26     sp.getCidades().add("Campinas");
27
28     Estado rj = new Estado();
29     rj.setSigla("RJ");
30     rj.setNome("Rio de Janeiro");
31     rj.getCidades().add("Rio de Janeiro");
32     rj.getCidades().add("Niterói");
33
34     Estado rn = new Estado();
35     rn.setSigla("RN");
36     rn.setNome("Rio Grande do Norte");
37     rn.getCidades().add("Natal");
38     rn.getCidades().add("Mossoró");
39
40     this.estados.add(sp);
41     this.estados.add(rj);
42     this.estados.add(rn);
43   }
44
45   public void mudaEstado(ValueChangeEvent event) {
46     this.siglaDoEstadoEscolhido = event.getNewValue().toString();
47     for (Estado e : this.estados) {
48       if (e.getSigla().equals(this.siglaDoEstadoEscolhido)) {
49         this.estadoSelecionado = e;
50         break;
51       }
52     }
53   }
54
55   // GETTERS E SETTERS
56 }
```

Código Java 11.12: www.k19.com.br/exercicios/k12/11/014/CadastroBean.java

15 Adicione uma página chamada **cadastro.xhtml** no projeto **K19-Eventos** com o seguinte conteúdo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10
11 <h:body>
12   <h1>Cadastro</h1>
13   <h:form>
14     <h:panelGrid columns="3">
15
16       <h:outputLabel value="Nome: " for="nome" />
17       <h:inputText value="#{cadastroBean.nome}" required="true" id="nome" />
18       <h:message for="nome" />
```

```
19
20        <h:outputLabel value="Estado: " for="estado" />
21        <h:selectOneMenu id="estado"
22          value="#{cadastroBean.siglaDoEstadoEscolhido}"
23          valueChangeListener="#{cadastroBean.mudaEstado}"
24          onchange="this.form.submit();"
25          required="true">
26
27          <f:selectItem itemLabel="Escolha um estado" noSelectionOption="true" />
28          <f:selectItems
29            value="#{cadastroBean.estados}"
30            var="estado"
31            itemLabel="#{estado.nome}"
32            itemValue="#{estado.sigla}" />
33
34        </h:selectOneMenu>
35        <h:message for="estado" />
36
37        <h:outputLabel value="Cidade: " for="cidade" />
38        <h:selectOneMenu id="cidade" value="#{cadastroBean.cidade}" required="true">
39          <f:selectItem itemLabel="Escolha uma cidade" noSelectionOption="true" />
40          <f:selectItems value="#{cadastroBean.estadoSelecionado.cidades}" />
41        </h:selectOneMenu>
42        <h:message for="cidade" />
43
44      </h:panelGrid>
45      <h:commandButton value="Cadastrar" />
46    </h:form>
47  </h:body>
48  </html>
```

Código XHTML 11.9: www.k19.com.br/exercicios/k12/11/015/cadastro.xhtml

Para que a lista de cidades seja atualizada assim que o usuário selecionar um novo estado, usamos os atributos **onchange** e **valueChangeListener** do componente **<h:selectOneMenu>**. Definimos o valor do atributo **onchange** como sendo **"this.form.submit();"**, o que significa que o formulário será submetido toda vez que um novo estado for selecionado. Além disso, o atributo **valueChangeListener** está associado ao método **mudaEstado()**, o que significa que esse método será chamado quando o evento de mudança de valor for gerado no processamento da requisição. Nesse método, alteramos o valor da propriedade **estadoSelecionado** para refletir a escolha atual do usuário, o que implica que a caixa de seleção de cidade exibirá as cidades desse estado. Veja as linhas em destaque no código acima.

16 Acesse a aplicação no endereço:

```
http://localhost:8080/K19-Eventos/cadastro.xhtml
```

Note que, quando selecionamos um estado, o formulário é submetido automaticamente e os dados de todos os campos são convertidos e validados.

17 Para atualizar a lista de cidades, precisamos que o dado presente na caixa de seleção de estado passe pelas etapas de conversão e validação. Por outro lado, queremos evitar que os demais campos sejam convertidos e validados, bem como evitar que o cadastro seja efetivado sem que o usuário tenha submetido o formulário.

Assim, precisamos (i) antecipar a conversão e validação dessa caixa de seleção e, logo em seguida, (ii) desviar o fluxo do processamento da requisição para a fase *Render Response*. A primeira tarefa pode ser feita definindo como **true** o valor do atributo **immediate** do componente **<h:selectOneMe-**

nu>. O fluxo do processamento da requisição pode ser desviado para a fase *Render Response* se chamarmos o método **renderResponse()** da instância da classe **FacesContext** logo após atualizarmos o valor do atributo **estadoSelecionado** do managed bean **cadastroBean**.

```
1  ...
2  <h:selectOneMenu id="estado"
3    value="#{cadastroBean.siglaDoEstadoEscolhido}"
4    valueChangeListener="#{cadastroBean.mudaEstado}"
5    onchange="this.form.submit();"
6    required="true"
7    immediate="true">
8
9    <f:selectItem itemLabel="Escolha um estado" noSelectionOption="true" />
10   <f:selectItems value="#{cadastroBean.estados}" var="estado"
11     itemLabel="#{estado.nome}" itemValue="#{estado.sigla}" />
12 </h:selectOneMenu>
13 ...
```

Código XHTML 11.10: www.k19.com.br/exercicios/k12/11/017/cadastro.xhtml

```
1  ...
2  public void mudaEstado(ValueChangeEvent event) {
3    this.siglaDoEstadoEscolhido = event.getNewValue().toString();
4    for (Estado e : this.estados) {
5      if (e.getSigla().equals(this.siglaDoEstadoEscolhido)) {
6        this.estadoSelecionado = e;
7        break;
8      }
9    }
10   FacesContext.getCurrentInstance().renderResponse();
11 }
12 ...
```

Código Java 11.13: www.k19.com.br/exercicios/k12/11/017/CadastroBean.java

Não esqueça de importar a classe **FacesContext**.

AJAX

CAPÍTULO 12

Quando as aplicações possuem telas complexas, com grande quantidade de conteúdo, não é interessante recarregar uma página inteira só para modificar uma pequena parte da tela.

Com o intuito de melhorar a interatividade entre as aplicações e os usuários, podemos aplicar o conceito do AJAX (Asynchronous Javascript And XML). Aplicando esse conceito, obtemos duas capacidades muito uteis. A primeira é a possibilidade de atualizar trechos de uma página ao invés da página inteira. A segunda é poder realizar requisições sem interromper a navegação dos usuários.

Por exemplo, considere uma aplicação web para a visualização de fotos. As fotos são exibidas individualmente em uma tela que também apresenta outros conteúdos. Os usuários podem, através do link "Próxima", avançar para a próxima foto, ou voltar para a foto anterior através do link "Anterior". Quando o usuário decide visualizar a próxima foto ou a anterior, não é necessário recarregar a página inteira, já que o único conteúdo alterado da página é a foto.

Outro exemplo, considere uma aplicação de mensagens instantâneas (GTalk, MSN, ...). A listagem de contatos de um usuário pode ser atualizada frequentemente sem que ele tenha que pressionar qualquer botão ou link, sem que a página tenha de ser recarregada e sem interromper a navegação do usuário.

A versão 2 do JSF, diferentemente das anteriores, oferece suporte nativo a AJAX. Neste capítulo, veremos como utilizar esse suporte.



Fazendo requisições AJAX

As requisições AJAX são realizadas quando determinados eventos ocorrem. Esses eventos estão fortemente relacionados aos componentes visuais adicionados às páginas. Devemos indicar para o JSF quais componentes e eventos devem realizar requisições AJAX. Para fazer isso, utilizamos a tag **<f:ajax>**.

```
1 <h:inputText>
2   <f:ajax/>
3 </h:inputText>
```

Código XHTML 12.1: Aplicando a tag `<f:ajax>`

No exemplo acima, uma requisição AJAX será realizada toda vez que o valor da caixa de texto for modificado, já que o evento padrão associado ao componente `<h:inputText>` é o **onchange**.

Por outro lado, podemos explicitar o evento que deve disparar as requisições AJAX através do atributo **event** da tag `<f:ajax>`. Devemos tomar cuidado pois nem todos os eventos são aceitos por todos os componentes.

```
1 <h:inputText>
```

```
2   <f:ajax event="keyup"/>
3 </h:inputText>
```

Código XHTML 12.2: Definindo o tipo de evento que deve disparar requisições AJAX

Quando temos vários componentes para os quais desejamos oferecer o suporte do AJAX, podemos agrupá-los através da tag <f:ajax>.

```
1 <f:ajax>
2   <h:inputText/>
3   <h:inputSecret/>
4   <h:commandButton value="OK"/>
5 </f:ajax>
```

Código XHTML 12.3: Aplicando a tag <f:ajax> a vários componentes

Novamente, se não escolhermos explicitamente o evento que vai disparar as requisições AJAX, o JSF assumirá o evento padrão de cada componente. O padrão dos componentes <h:inputText> e <h:inputSecret> é onchange. Já o padrão do componente <h:commandButton> é **onclick**.

Podemos explicitar o evento que deve disparar as requisições AJAX para um determinado grupo de componentes da mesma forma como fizemos anteriormente.

```
1 <f:ajax event="mouseout">
2   <h:inputText/>
3   <h:inputSecret/>
4   <h:commandButton value="OK"/>
5 </f:ajax>
```

Código XHTML 12.4: Definindo o tipo de evento que deve disparar requisições AJAX



Processando uma parte específica da tela

Quando uma requisição AJAX é feita, podemos determinar quais componentes da tela devem ser avaliados pelo JSF. Por exemplo, quando enviamos um formulário, talvez seja necessário avaliar apenas os componentes que estão no próprio formulário.

Podemos definir quais componentes devem ser avaliados no servidor através do atributo **execute** da tag <f:ajax>. O valor desse atributo deve ser uma lista contendo os identificadores dos componentes que precisam ser avaliados no servidor. Quando um componente é avaliado, todos os componentes definidos em seu corpo também serão avaliados.

```
1 <h:form id="formulario-login">
2   <h:inputText/>
3
4   <h:inputSecret/>
5
6   <h:commandButton value="Enviar">
7     <f:ajax event="click" execute="formulario-login"/>
8   </h:commandButton>
9 </h:form>
```

Código XHTML 12.5: Definindo quais componentes serão avaliados

No exemplo acima, quando o botão "Enviar" for pressionado, uma requisição AJAX será realizada. No servidor, o formulário e todos os componentes definidos dentro dele serão avaliados.



Recarregando parte da tela

Podemos definir quais componentes devem ser atualizados quando a resposta de uma requisição AJAX chega no navegador. Para isso, devemos utilizar o atributo `render` da tag `<f:ajax>`. O valor desse atributo deve ser uma lista contendo os identificadores dos componentes que precisam ser atualizados quando a resposta de uma requisição AJAX chegar no navegador.

```
1 <h:commandButton value="Gera Número">
2   <f:ajax event="click" render="numero"/>
3 </h:commandButton>
4 <h:outputText id="numero" value="#{geradorDeNumeroBean.numero}"/>
```

Código XHTML 12.6: Definindo quais componentes serão atualizados

No exemplo acima, uma requisição AJAX é realizada quando o botão "Gera Número" é pressionado. Quando a resposta dessa requisição chega no navegador, apenas o componente `<h:outputText>` com o identificador `numero` será atualizado. Já no código abaixo, dois componentes `<h:outputText>` são atualizados. Mais especificamente, os componentes com os identificadores `numero1` e `numero2` são atualizados.

```
1 <h:commandButton value="Gera Números">
2   <f:ajax event="click" render="numero1 numero2"/>
3 </h:commandButton>
4 <h:outputText id="numero1" value="#{geradorDeNumeroBean.numero}"/>
5 <h:outputText id="numero2" value="#{geradorDeNumeroBean.numero}"/>
```

Código XHTML 12.7: Definindo dois componentes para serem atualizados



Associando um procedimento a uma requisição AJAX

Podemos associar um método a uma requisição AJAX. Esse método será executado durante o processamento dessa requisição no servidor. Mais especificamente, ele será executado na fase *Invoke Application*. Essa associação é realizada através do atributo **listener** da tag `<f:ajax>`.

```
1 <h:commandButton value="Salva">
2   <f:ajax
3     event="click"
4     execute="formulario"
5     render="formulario"
6     listener="#{produtoBean.salva}"/>
7 </h:commandButton>
```

Código XHTML 12.8: Definindo quais componentes serão atualizados

No exemplo acima, as requisições AJAX realizadas através do botão "Salva" foram associadas ao método `salva()` do managed bean `produtoBean`.



Palavras especiais

Como podemos passar uma lista de componentes para os atributos `render` e `execute`, o JSF criou

palavras chaves associadas a grupos especiais de componentes.

@all refere-se a todos os componentes da tela.

@none refere-se a nenhum componente.

@this refere-se ao componente que disparou a requisição AJAX.

@form refere-se aos componentes do formulário que contém o componente que disparou a requisição AJAX.

Podemos alterar o código do formulário anterior para utilizar a palavra especial `@form` no lugar do identificador do formulário.

```
<h:form id="formulario-login">
  <h:inputText/>

  <h:inputSecret/>

  <h:commandButton value="Enviar">
    <f:ajax event="click" execute="@form"/>
  </h:commandButton>
</h:form>
```

Código XHTML 12.9: Utilizando a palavra especial `@form`



Exercícios de Fixação

1 Crie um projeto chamado **K19-Ajax** seguindo os passos vistos no exercício do Capítulo 5.

2 Criaremos um formulário com a validação dos campos realizadas com AJAX. Crie um arquivo chamado **formulario.xhtml** com o seguinte conteúdo:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">

<h:head>
  <title>K19 - Ajax</title>
</h:head>

<h:body>
  <h:form>
    <h:panelGrid columns="2">
      <h:inputText id="caixa">
        <f:ajax event="keyup" execute="caixa" render="mensagem" />
        <f:validateLength minimum="5" maximum="10" />
      </h:inputText>

      <h:message id="mensagem" for="caixa" style="color: red"/>

      <h:commandButton value="Enviar" />
```

```
23      </h:panelGrid>
24    </h:form>
25  </h:body>
26  </html>
```

Código XHTML 12.10: www.k19.com.br/exercicios/k12/12/002/formulario.xhtml

3 Remova o projeto **K19-Eventos** do GlassFish e do Wildfly. Implante o projeto **K19-Ajax** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Ajax/formulario.xhtml`

Observe a validação ocorrendo ao preenchimento do formulário.

4 Implante o projeto **K19-Ajax** no Wildfly. Certifique-se que o GlassFish esteja parado. Inicie o Wildfly e acesse o endereço abaixo para testar a aplicação.

`http://localhost:8080/K19-Ajax/formulario.xhtml`

Observe a validação ocorrendo ao preenchimento do formulário.

INTEGRAÇÃO JSF E JPA

CAPÍTULO 13

Como vimos nos primeiros capítulos, os SGDBs são utilizados para armazenar os dados manipulados pelas aplicações. Até agora, os dados manipulados pelas aplicações JSF desenvolvidas nos exercícios não foram armazenados em um SGDB. Adicionaremos essa capacidade às aplicações JSF neste capítulo. Para isso, mostraremos uma maneira de integrar os recursos do JSF e do JPA.

Bibliotecas

Para utilizar os recursos do JPA em uma aplicação JSF, os jars do provedor JPA e do driver JDBC que serão utilizados devem estar no classpath da aplicação. Nos capítulos anteriores, as aplicações JSF desenvolvidas nos exercícios foram implantadas no GlassFish 3.0.1 que é um servidor de aplicação Java EE 6.

Por padrão, a versão 3.0.1 do GlassFish possui os jars do provedor JPA EclipseLink. Dessa forma, as aplicações JSF implantadas nessa versão do GlassFish utilizarão o EclipseLink como implementação do JPA. Contudo, queremos utilizar o provedor JPA Hibernate. Podemos facilmente substituir os jars do EclipseLink pelos jars do Hibernate através da interface de administração do GlassFish.



Mais Sobre
Consulte o artigo da K19 sobre a substituição dos jars do provedor JPA EclipseLink pelos jars do provedor JPA Hibernate.

http://www.k19.com.br/artigos/configurando-hibernate-no-glassfish-3-1/

Utilizaremos o MySQL Server como SGDB. Dessa forma, devemos adicionar o driver JDBC do MySQL Server no classpath das aplicações JSF. O GlassFish 3.0.1 não possui os jars desse driver JDBC. Contudo, podemos adicioná-los manualmente. Para isso, basta acrescentar os jars do driver JDBC do MySQL em uma pasta apropriada do GlassFish. Nos exercícios deste capítulo, mostraremos como realizar tal tarefa.

Configuração

Como vimos no Capítulo 3, devemos configurar as unidades de persistência utilizadas através do arquivo `persistence.xml` da pasta `META-INF` do classpath da aplicação.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
5   http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
```

```
6   version="2.1">
7   <persistence-unit name="K19-PU" transaction-type="RESOURCE_LOCAL">
8     <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
9     <exclude-unlisted-classes>false</exclude-unlisted-classes>
10    <properties>
11      <property
12        name="javax.persistence.schema-generation.database.action"
13        value="drop-and-create"/>
14      <property
15        name="javax.persistence.jdbc.driver"
16        value="com.mysql.jdbc.Driver" />
17      <property
18        name="javax.persistence.jdbc.user"
19        value="root" />
20      <property
21        name="javax.persistence.jdbc.password"
22        value="root" />
23      <property
24        name="javax.persistence.jdbc.url"
25        value="jdbc:mysql://localhost:3306/K19-DB" />
26    </properties>
27  </persistence-unit>
28 </persistence>
```

Código XML 13.1: persistence.xml



Mapeamento

Também devemos definir o mapeamento das entidades. No Capítulo 3, vimos como utilizar as anotações do JPA para estabelecer esse mapeamento.

```
1  @Entity
2  public class Produto {
3    @Id @GeneratedValue
4    private Long id;
5
6    private String nome;
7
8    private Double preco;
9
10   // GETTERS e SETTERS
11 }
```

Código Java 13.1: Produto.java



Inicialização e Finalização

As unidades de persistência devem ser inicializadas antes de serem utilizadas, e finalizadas quando não forem mais necessárias. A inicialização e a finalização de uma unidade de persistência devem ser realizadas apenas uma vez durante a execução da aplicação.

Para implementar essa característica em aplicações web Java, podemos utilizar um filtro. Os filtros de uma aplicação web Java são inicializados automaticamente depois que a aplicação é implantada no Web Container e antes da primeira requisição HTTP. Além disso, eles são finalizados ao término da execução da aplicação.

Para adicionar um filtro em uma aplicação web Java, é necessário criar uma classe que imple-

mente a interface `javax.servlet.Filter`.

```
@WebFilter(servletNames={"Faces Servlet"})
public class JPAFilter implements Filter {

  private EntityManagerFactory factory;

  @Override
  public void init(FilterConfig filterConfig) throws ServletException {
    this.factory = Persistence.createEntityManagerFactory("K19-PU");
  }

  @Override
  public void destroy() {
    this.factory.close();
  }

  @Override
  public void doFilter(ServletRequest request, ServletResponse response,
      FilterChain chain) throws IOException, ServletException {

    // por enquanto vazio
  }
}
```

Código Java 13.2: JPAFilter.java

Um filtro pode ser registrado no Web Container através da anotação **@WebFilter**. Com essa anotação, podemos definir qual servlet será associada ao filtro. No exemplo acima, o filtro definido pela classe `JPAFilter` foi associado a servlet *Faces Servlet*.

O método **init()** é chamado automaticamente na inicialização do filtro. No exemplo acima, esse método inicializa a unidade de persistência *K19-PU*. O método **destroy()** é chamado automaticamente para desativar o filtro no encerramento da aplicação. No exemplo acima, finalizamos a unidade de persistência *K19-PU*.



Transações

Como vimos no Capítulo 3, para atualizar as informações armazenadas no SGDB de acordo com os dados da memória da aplicação, devemos abrir uma transação e confirmá-la através do método `commit()`.

O filtro criado anteriormente para controlar a inicialização e finalização das unidades de persistência pode também gerenciar a abertura e a confirmação das transações da aplicação. Para isso, utilizaremos o método `doFilter()` desse filtro.

```
@WebFilter(servletNames={"Faces Servlet"})
public class JPAFilter implements Filter {

  private EntityManagerFactory factory;

  @Override
  public void init(FilterConfig filterConfig) throws ServletException {
    this.factory = Persistence.createEntityManagerFactory("K19-PU");
  }

  @Override
  public void destroy() {
    this.factory.close();
  }
```

```
15
16   @Override
17   public void doFilter(ServletRequest request, ServletResponse response,
18       FilterChain chain) throws IOException, ServletException {
19
20     // CHEGADA
21     EntityManager manager = this.factory.createEntityManager();
22     request.setAttribute("EntityManager", manager);
23     entityManager.getTransaction().begin();
24     // CHEGADA
25
26     // FACES SERVLET
27     chain.doFilter(request, response);
28     // FACES SERVLET
29
30     // SAÍDA
31     try {
32       entityManager.getTransaction().commit();
33     } catch (Exception e) {
34       entityManager.getTransaction().rollback();
35     } finally {
36       entityManager.close();
37     }
38     // SAÍDA
39   }
40 }
```

Código Java 13.3: JPAFilter.java

No exemplo acima, o método `doFilter()` é chamado toda vez que uma requisição é realizada para a servlet *Faces Servlet*. Antes de repassar a requisição para a *Faces Servlet*, o método `doFilter()` cria um `EntityManager`, armazena-o na requisição e abre uma transação. Depois que a *Faces Servlet* processou a requisição, o método `doFilter()` tenta confirmar a transação através do método `commit()`. Se um erro ocorrer nessa tentativa, o método `rollback()` é chamado para cancelar a transação.



Recuperando o EntityManager da Requisição

O `EntityManager` armazenado dentro da requisição pelo filtro pode ser recuperado a qualquer momento durante o processamento da requisição. Veja o código abaixo.

```
1 FacesContext fc = FacesContext.getCurrentInstance();
2 ExternalContext ec = fc.getExternalContext();
3 HttpServletRequest request = (HttpServletRequest) ec.getRequest();
4 EntityManager manager =  (EntityManager) request.getAttribute("EntityManager");
```

Código Java 13.4: Recuperando o `EntityManager` da requisição

O `EntityManager` será utilizado pela aplicação para realizar as operações de persistência.



Exercícios de Fixação

1 Entre na pasta **K19-Arquivos/mysql-connector-java-VERSAO** da Área de Trabalho e copie o arquivo **mysql-connector-java-VERSAO-bin.jar** para pasta **glassfish4/glassfish/lib** também da sua Área de Trabalho. OBS: O GlassFish deve ser reiniciado para reconhecer o driver JDBC do MySQL.



Importante
Você pode obter o arquivo **mysql-connector-java-VERSAO-bin.jar** no site da K19: www.k19.com.br/arquivos

2 Crie um projeto chamado **K19-Integracao-JSF-JPA** seguindo os passos vistos no exercício do Capítulo 5.

3 Adicione uma pasta chamada **META-INF** no classpath do projeto **K19-Integracao-JSF-JPA**. A pasta **META-INF** deve ser salva na pasta **src**.

4 Configure o JPA adicionando o arquivo **persistence.xml** na pasta **META-INF**.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
  http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
  version="2.1">
  <persistence-unit name="K19-PU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties>
      <property
        name="javax.persistence.schema-generation.database.action"
        value="drop-and-create"/>
      <property
        name="javax.persistence.jdbc.driver"
        value="com.mysql.jdbc.Driver" />
      <property
        name="javax.persistence.jdbc.user"
        value="root" />
      <property
        name="javax.persistence.jdbc.password"
        value="root" />
      <property
        name="javax.persistence.jdbc.url"
        value="jdbc:mysql://localhost:3306/K19-DB" />
    </properties>
  </persistence-unit>
</persistence>
```

Código XML 13.2: www.k19.com.br/exercicios/k12/13/004/persistence.xml

5 Abra um terminal; entre no cliente do MySQL Server; apague se existir a base de dados **K19-DB**; e crie uma base de dados nova chamada **K19-DB**.

```
k19@k19-11:~/rafael$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 36
Server version: 5.1.58-1ubuntu1 (Ubuntu)

Copyright (c) 2000, 2010, Oracle and/or its affiliates. All rights reserved.
This software comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to modify and redistribute it under the GPL v2 license

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql> DROP DATABASE IF EXISTS `K19-DB`;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE `K19-DB`;
Query OK, 1 row affected (0.02 sec)
```

6 Crie um pacote chamado **br.com.k19.filters** no projeto **K19-Integracao-JSF-JPA**.

7 No pacote **br.com.k19.filters**, crie uma classe chamada **JPAFilter** com o seguinte conteúdo:

```
1  package br.com.k19.filters;
2
3  import java.io.IOException;
4  import javax.persistence.EntityManager;
5  import javax.persistence.EntityManagerFactory;
6  import javax.persistence.Persistence;
7  import javax.servlet.Filter;
8  import javax.servlet.FilterChain;
9  import javax.servlet.FilterConfig;
10 import javax.servlet.ServletException;
11 import javax.servlet.ServletRequest;
12 import javax.servlet.ServletResponse;
13 import javax.servlet.annotation.WebFilter;
14
15 @WebFilter(servletNames={"Faces Servlet"})
16 public class JPAFilter implements Filter {
17
18   private EntityManagerFactory factory;
19
20   @Override
21   public void init(FilterConfig filterConfig) throws ServletException {
22     this.factory = Persistence.createEntityManagerFactory("K19-PU");
23   }
24
25   @Override
26   public void destroy() {
27     this.factory.close();
28   }
29
30   @Override
31   public void doFilter(ServletRequest request, ServletResponse response,
32       FilterChain chain) throws IOException, ServletException {
33
34     // CHEGADA
35     EntityManager manager = this.factory.createEntityManager();
36     request.setAttribute("EntityManager", manager);
37     manager.getTransaction().begin();
38     // CHEGADA
39
40     // FACES SERVLET
41     chain.doFilter(request, response);
42     // FACES SERVLET
43
44     // SAÍDA
45     try {
46       manager.getTransaction().commit();
47     } catch (Exception e) {
48       manager.getTransaction().rollback();
49     } finally {
50       manager.close();
51     }
52     // SAÍDA
53   }
54 }
```

Código Java 13.5: www.k19.com.br/exercicios/k12/13/007/JPAFilter.java

8 Crie um pacote chamado **br.com.k19.modelo** no projeto **K19-Integracao-JSF-JPA**.

9 No pacote **br.com.k19.modelo**, crie uma classe chamada **Carro** com o seguinte conteúdo:

```
1  package br.com.k19.modelo;
2
3  import java.io.Serializable;
4  import javax.persistence.Entity;
5  import javax.persistence.GeneratedValue;
6  import javax.persistence.Id;
7
8  @Entity
9  public class Carro implements Serializable {
10
11   @Id @GeneratedValue
12   private Long id;
13   private String marca;
14   private String modelo;
15
16   // GETTERS E SETTERS
17 }
```

Código Java 13.6: www.k19.com.br/exercicios/k12/13/009/Carro.java

10 No pacote **br.com.k19.modelo**, crie uma classe chamada **CarroRepository** com o seguinte conteúdo:

```
1  package br.com.k19.modelo;
2
3  import java.util.List;
4  import javax.persistence.EntityManager;
5  import javax.persistence.Query;
6
7  public class CarroRepository {
8
9    private EntityManager manager;
10
11   public CarroRepository(EntityManager manager) {
12     this.manager = manager;
13   }
14
15   public void adiciona(Carro carro) {
16     this.manager.persist(carro);
17   }
18
19   public List<Carro> buscaTodos() {
20     Query query = this.manager.createQuery("select x from Carro x");
21     return query.getResultList();
22   }
23 }
```

Código Java 13.7: www.k19.com.br/exercicios/k12/13/010/CarroRepository.java

11 Crie um pacote chamado **br.com.k19.controle** no projeto **K19-Integracao-JSF-JPA**.

12 No pacote **br.com.k19.controle**, crie uma classe chamada **CarroBean** com o seguinte código:

```
1  package br.com.k19.controle;
2
```

```
3  import br.com.k19.modelo.Carro;
4  import br.com.k19.modelo.CarroRepository;
5  import java.util.List;
6  import javax.faces.bean.ManagedBean;
7  import javax.faces.context.ExternalContext;
8  import javax.faces.context.FacesContext;
9  import javax.persistence.EntityManager;
10 import javax.servlet.http.HttpServletRequest;
11
12 @ManagedBean
13 public class CarroBean {
14
15   private Carro carro = new Carro();
16
17   public void adicionaCarro() {
18     EntityManager manager = this.getEntityManager();
19     CarroRepository repository = new CarroRepository(manager);
20
21     repository.adiciona(this.carro);
22     this.carro = new Carro();
23   }
24
25   public List<Carro> getCarros() {
26     EntityManager manager = this.getEntityManager();
27     CarroRepository repository = new CarroRepository(manager);
28     return repository.buscaTodos();
29   }
30
31   private EntityManager getEntityManager() {
32     FacesContext fc = FacesContext.getCurrentInstance();
33     ExternalContext ec = fc.getExternalContext();
34     HttpServletRequest request = (HttpServletRequest) ec.getRequest();
35     EntityManager manager =  (EntityManager)request.getAttribute("EntityManager");
36
37     return manager;
38   }
39   // GETTERS E SETTERS
40 }
```

Código Java 13.8: www.k19.com.br/exercicios/k12/13/012/CarroBean.java

13 Crie uma página chamada **carros.xhtml** no projeto **K19-Integracao-JSF-JPA** com o seguinte conteúdo:

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core"
7    xmlns:ui="http://java.sun.com/jsf/facelets">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12 <h:body>
13   <h:form>
14     <h:panelGrid columns="2">
15       <h:outputLabel value="Marca: " for="campo-marca" />
16       <h:inputText value="#{carroBean.carro.marca}" id="campo-marca" />
17
18       <h:outputLabel value="Modelo: " for="campo-modelo" />
19       <h:inputText value="#{carroBean.carro.modelo}" id="campo-modelo" />
20
21       <h:commandButton value="Adicionar" action="#{carroBean.adicionaCarro}" />
22     </h:panelGrid>
23   </h:form>
```

```
24 </h:body>
25 </html>
```

Código XHTML 13.1: www.k19.com.br/exercicios/k12/13/013/carros.xhtml

14 No arquivo **carros.xhtml**, adicione também um trecho de código para exibir os carros adicionados.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core"
7    xmlns:ui="http://java.sun.com/jsf/facelets">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12 <h:body>
13   <h:form>
14     <h:panelGrid columns="2">
15       <h:outputLabel value="Marca: " for="campo-marca" />
16       <h:inputText value="#{carroBean.carro.marca}" id="campo-marca" />
17
18       <h:outputLabel value="Modelo: " for="campo-modelo" />
19       <h:inputText value="#{carroBean.carro.modelo}" id="campo-modelo" />
20
21       <h:commandButton value="Adicionar" action="#{carroBean.adicionaCarro}" />
22     </h:panelGrid>
23   </h:form>
24
25   <h:panelGroup rendered="#{not empty carroBean.carros}">
26     <h1>Lista de carros:</h1>
27     <ul>
28       <ui:repeat value="#{carroBean.carros}" var="carro">
29         <li><h:outputText value="#{carro.marca} #{carro.modelo}" /></li>
30       </ui:repeat>
31     </ul>
32   </h:panelGroup>
33 </h:body>
34 </html>
```

Código XHTML 13.2: www.k19.com.br/exercicios/k12/13/014/carros.xhtml

15 Remova o projeto **K19-Ajax** do GlassFish e do Wildfly. Implante o projeto **K19-Integracao-JSF-JPA** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

```
http://localhost:8080/K19-Integracao-JSF-JPA/carros.xhtml
```

Adicione alguns carros e verifique se eles foram adicionados no banco de dados.



Otimizando o número de consultas ao SGDB

Durante o processamento de uma requisição, os métodos *getters* são chamados pelo JSF. A quantidade de chamadas não pode ser controlada pela aplicação. A lógica implementada nos *getters* pode comprometer a performance da aplicação. Por exemplo, nos exercícios anteriores, toda vez que o

método `getCarros()` do managed bean `carroBean` é chamado, uma consulta é realizada no SGBD através do repositório de carros.

Podemos diminuir a quantidade de consultas armazenando o resultado da primeira chamada em um atributo da classe `CarroBean`. Esse resultado pode ser mantido até que alguma alteração na listagem de carros ocorra. Por exemplo, se um carro for adicionado ou removido, a listagem armazenada deve ser descartada e recuperada novamente do SGDB. Veja o código abaixo.

```
@ManagedBean
public class CarroBean {

  private Carro carro = new Carro();
  private List<Carro> carros;

  public void adicionaCarro() {
    EntityManager manager = this.getEntityManager();
    CarroRepository repository = new CarroRepository(manager);

    repository.adiciona(this.carro);
    this.carro = new Carro();
    this.carros = null;
  }

  public List<Carro> getCarros() {
    if(this.carros == null) {
    EntityManager manager = this.getEntityManager();
    CarroRepository repository = new CarroRepository(manager);
      this.carros = repository.buscaTodos();
    }

    return this.carros;
  }

  private EntityManager getEntityManager() {
    FacesContext fc = FacesContext.getCurrentInstance();
    ExternalContext ec = fc.getExternalContext();
    HttpServletRequest request = (HttpServletRequest) ec.getRequest();
    EntityManager manager =  (EntityManager)request.getAttribute("EntityManager");

    return manager;
  }
  // GETTERS E SETTERS
}
```

Código Java 13.9: CarroBean.java

No código acima, o atributo `carros` armazena a última lista carregada do SGDB. No método `getCarros()`, verificamos se já existe uma lista carregada no atributo `carros`. Se não existir uma lista carregada então utilizamos o repositório de carros para recuperar os dados do SGDB e carregar a lista. Caso contrário, simplesmente utilizamos a lista que já estava carregada. No método `adicionaCarro()`, depois que o repositório de carros é acionado para persistir um carro, descartamos o conteúdo do atributo `carros`. Dessa forma, na próxima chamada do método `getCarros()`, a lista será carregada novamente e conterá o carro que acabou de ser adicionado.



Exercícios de Fixação

16 Vamos monitorar a quantidade de chamadas ao método **getCarros()** do managed bean **carroBean**. Altere a classe desse managed bean da seguinte forma:

```java
package br.com.k19.controle;

import br.com.k19.modelo.Carro;
import br.com.k19.modelo.CarroRepository;
import java.util.List;
import javax.faces.bean.ManagedBean;
import javax.faces.context.ExternalContext;
import javax.faces.context.FacesContext;
import javax.persistence.EntityManager;
import javax.servlet.http.HttpServletRequest;

@ManagedBean
public class CarroBean {

  private Carro carro = new Carro();

  public void adicionaCarro() {
    EntityManager manager = this.getEntityManager();
    CarroRepository repository = new CarroRepository(manager);

    repository.adiciona(this.carro);
    this.carro = new Carro();
  }

  public List<Carro> getCarros() {
    EntityManager manager = this.getEntityManager();
    CarroRepository repository = new CarroRepository(manager);

    System.out.println("CHAMANDO O REPOSITORIO");
    return repository.buscaTodos();
  }

  private EntityManager getEntityManager() {
    FacesContext fc = FacesContext.getCurrentInstance();
    ExternalContext ec = fc.getExternalContext();
    HttpServletRequest request = (HttpServletRequest) ec.getRequest();
    EntityManager manager =  (EntityManager)request.getAttribute("EntityManager");

    return manager;
  }

  // GETTERS E SETTERS
}
```

Código Java 13.10: www.k19.com.br/exercicios/k12/13/016/CarroBean.java

17 Acesse a página http://localhost:8080/K19-Integracao-JSF-JPA/carros.xhtml e depois verifique as mensagens impressas.

18 Altere o comportamento do managed bean **carroBean** para diminuir o número de chamadas ao repositório de carros.

```java
package br.com.k19.controle;

import br.com.k19.modelo.Carro;
import br.com.k19.modelo.CarroRepository;
import java.util.List;
import javax.faces.bean.ManagedBean;
import javax.faces.context.ExternalContext;
import javax.faces.context.FacesContext;
import javax.persistence.EntityManager;
import javax.servlet.http.HttpServletRequest;

@ManagedBean
```

```
13 public class CarroBean {
14
15   private Carro carro = new Carro();
16
17   private List<Carro> carros;
18
19   public void adicionaCarro() {
20     EntityManager manager = this.getEntityManager();
21     CarroRepository repository = new CarroRepository(manager);
22
23     repository.adiciona(this.carro);
24     this.carro = new Carro();
25     this.carros = null;
26   }
27
28   public List<Carro> getCarros() {
29     if(this.carros == null) {
30       EntityManager manager = this.getEntityManager();
31       CarroRepository repository = new CarroRepository(manager);
32       System.out.println("CHAMANDO O REPOSITORIO");
33       this.carros = repository.buscaTodos();
34     }
35
36     return this.carros;
37   }
38
39   private EntityManager getEntityManager() {
40     FacesContext fc = FacesContext.getCurrentInstance();
41     ExternalContext ec = fc.getExternalContext();
42     HttpServletRequest request = (HttpServletRequest) ec.getRequest();
43     EntityManager manager =  (EntityManager)request.getAttribute("EntityManager");
44
45     return manager;
46   }
47
48   // GETTERS E SETTERS
49 }
```

Código Java 13.11: www.k19.com.br/exercicios/k12/13/018/CarroBean.java

19 Acesse novamente a página http://localhost:8080/K19-Integracao-JSF-JPA/carros.xhtml e depois verifique as mensagens impressas.

AUTENTICAÇÃO

APÊNDICE A

Neste capítulo, apresentaremos uma maneira de implementar o processo de autenticação dos usuários de uma aplicação JSF.



Exercícios de Fixação

1 Crie um projeto chamado **K19-Autenticacao** seguindo os passos vistos no exercício do Capítulo 5.

2 Por simplicidade, utilizaremos um atributo de um managed bean para armazenar os usuários da aplicação e suas respectivas senhas. A implementação que será apresentada a seguir pode ser alterada para que esses dados sejam armazenadas em um arquivo ou em um banco de dados.



Importante
Por motivos de segurança, as senhas dos usuários não devem ser armazenadas literalmente. Ao invés disso, as senhas dos usuários devem passar por um processo de transformação (criptografia) antes de serem armazenadas.

Quando um usuário tenta logar no sistema, ele digita o seu nome de usuário e sua senha. Para garantir que o usuário tenha acesso ao sistema, precisamos verificar se o nome de usuário digitado está cadastrado no sistema e se sua senha está correta. Como nós não armazenamos a senha do usuário, o que fazemos é aplicar a mesma transformação feita anteriormente e comparar o valor obtido com aquele armazenado no servidor. Se esses valores forem iguais, então permitimos que o usuário acesse o sistema. Caso contrário, o acesso ao sistema é negado.

Crie uma classe chamada **AutenticadorBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-Autenticacao** com o seguinte conteúdo:

```
1  package br.com.k19.controle;
2
3  import java.util.HashMap;
4  import java.util.Map;
5  import javax.faces.bean.ManagedBean;
6
7  @ManagedBean
8  public class AutenticadorBean {
9
10   private static Map<String, String> mapa = new HashMap<String, String>();
11
12 }
```

Código Java A.1: AutenticadorBean.java

3 Acrescente alguns usuários e suas respectivas senhas no atributo **mapa**.

```
1  package br.com.k19.controle;
2
3  import java.util.HashMap;
4  import java.util.Map;
5  import javax.faces.bean.ManagedBean;
6
7  @ManagedBean
8  public class AutenticadorBean {
9
10   private static Map<String, String> mapa = new HashMap<String, String>();
11
12   static {
13     AutenticadorBean.mapa.put("k19", "k19");
14     AutenticadorBean.mapa.put("jonas.hirata", "jonas.hirata");
15     AutenticadorBean.mapa.put("marcelo.martins", "marcelo.martins");
16     AutenticadorBean.mapa.put("rafael.cosentino", "rafael.cosentino");
17   }
18 }
```

Código Java A.2: AutenticadorBean.java

4 Crie propriedades para armazenar os dados enviados através do formulário de identificação, um método para implementar o processo de autenticação e outro método para registrar a saída do usuário.

```
1  package br.com.k19.controle;
2
3  import java.util.HashMap;
4  import java.util.Map;
5  import javax.faces.application.FacesMessage;
6  import javax.faces.bean.ManagedBean;
7  import javax.faces.context.ExternalContext;
8  import javax.faces.context.FacesContext;
9  import javax.servlet.http.HttpSession;
10
11 @ManagedBean
12 public class AutenticadorBean {
13
14   private static Map<String, String> mapa = new HashMap<String, String>();
15
16   private String usuario;
17
18   private String senha;
19
20   static {
21     AutenticadorBean.mapa.put("k19", "k19");
22     AutenticadorBean.mapa.put("jonas.hirata", "jonas.hirata");
23     AutenticadorBean.mapa.put("marcelo.martins", "marcelo.martins");
24     AutenticadorBean.mapa.put("rafael.cosentino", "rafael.cosentino");
25   }
26
27   public String autentica() {
28     FacesContext fc = FacesContext.getCurrentInstance();
29
30     if (AutenticadorBean.mapa.containsKey(this.usuario)
31         && AutenticadorBean.mapa.get(this.usuario).equals(this.senha)) {
32
33       ExternalContext ec = fc.getExternalContext();
34       HttpSession session = (HttpSession)ec.getSession(false);
35       session.setAttribute("usuario", this.usuario);
36
37       return "/home";
38     } else {
39       FacesMessage fm = new FacesMessage("usuário e/ou senha inválidos");
```

```
40        fm.setSeverity(FacesMessage.SEVERITY_ERROR);
41        fc.addMessage(null, fm);
42        return "/login";
43      }
44    }
45
46    public String registraSaida() {
47      FacesContext fc = FacesContext.getCurrentInstance();
48      ExternalContext ec = fc.getExternalContext();
49      HttpSession session = (HttpSession)ec.getSession(false);
50      session.removeAttribute("usuario");
51
52      return "/login";
53    }
54
55    // GETTERS E SETTERS
56  }
```

Código Java A.3: AutenticadorBean.java

O método `autentica()` verifica se os dados enviados pelo formulário de autenticação estão cadastrados no `mapa`. Se esses dados estiverem cadastrados, registramos o nome do usuário na sessão HTTP e navegamos para a página principal da aplicação. Caso contrário, adicionamos no contexto do processamento da requisição uma mensagem de erro e navegamos para a página do formulário de autenticação. O método `registraSaida()` simplesmente retira da sessão HTTP o nome do usuário anteriormente autenticado.

5 Para testar, crie a tela do formulário de autenticação e a tela principal da aplicação.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html">
6
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10
11 <h:body>
12   <h:messages/>
13   <h:form>
14     <h:outputLabel value="Usuário: " for="campo-usuario" />
15     <h:inputText value="#{autenticadorBean.usuario}" id="campo-usuario" />
16
17     <h:outputLabel value="Senha: " for="campo-senha" />
18     <h:inputSecret value="#{autenticadorBean.senha}" id="campo-senha" />
19
20     <h:commandButton value="Entrar" action="#{autenticadorBean.autentica}"/>
21   </h:form>
22 </h:body>
23 </html>
```

Código XHTML A.1: login.xhtml

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:ui="http://java.sun.com/jsf/facelets"
6    xmlns:h="http://java.sun.com/jsf/html"
7    xmlns:f="http://java.sun.com/jsf/core">
8
9  <h:head>
```

```
10   <title>K19 Treinamentos</title>
11 </h:head>
12
13 <h:body>
14   Olá #{sessionScope.usuario}!
15   Você está logado.
16
17   <h:form>
18     <h:commandLink value="Sair" action="#{autenticadorBean.registraSaida}"/>
19   </h:form>
20 </h:body>
21 </html>
```

Código XHTML A.2: home.xhtml

Na tela principal da aplicação, utilizamos a variável **sessionScope** para recuperar o nome do usuário autenticado.

6 Implante o projeto **K19-Autenticacao** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

```
http://localhost:8080/K19-Autenticacao/login.xhtml
```

7 Somente usuários autenticados podem acessar a página principal da aplicação. Para controlar o acesso às páginas da aplicação, implemente um filtro para interceptar todas as requisições HTTP direcionadas à "Faces Servlet".

Crie uma classe chamada **ControleDeAcesso** em um pacote chamado **br.com.k19.filters** no projeto **K19-Autenticacao** com o seguinte conteúdo:

```
1  package br.com.k19.filters;
2
3  import java.io.IOException;
4  import javax.servlet.Filter;
5  import javax.servlet.FilterChain;
6  import javax.servlet.FilterConfig;
7  import javax.servlet.ServletException;
8  import javax.servlet.ServletRequest;
9  import javax.servlet.ServletResponse;
10 import javax.servlet.annotation.WebFilter;
11 import javax.servlet.http.HttpServletRequest;
12 import javax.servlet.http.HttpServletResponse;
13 import javax.servlet.http.HttpSession;
14
15 @WebFilter(servletNames = {"Faces Servlet"})
16 public class ControleDeAcesso implements Filter {
17
18   @Override
19   public void doFilter(ServletRequest request, ServletResponse response,
20       FilterChain chain) throws IOException, ServletException {
21
22     HttpServletRequest req = (HttpServletRequest) request;
23     HttpSession session = req.getSession();
24
25     if (session.getAttribute("usuario") != null
26         || req.getRequestURI().endsWith("login.xhtml")) {
27       chain.doFilter(request, response);
28     } else {
29       HttpServletResponse res = (HttpServletResponse) response;
30       res.sendRedirect("login.xhtml");
31     }
32   }
```

```
33
34    @Override
35    public void init(FilterConfig filterConfig) throws ServletException {
36    }
37
38    @Override
39    public void destroy() {
40    }
41  }
```

Código Java A.4: ControleDeAcesso.java

A anotação `@WebServlet` registra o filtro no Web Container. O atributo `servletNames` é utilizado para definir quais servlets serão interceptadas pelo filtro. O método `doFilter()` é chamado toda vez que uma requisição HTTP à servlet "Faces Servlet" é realizada. Esse método verifica se existe um usuário registrado na sessão HTTP ou se a página requisitada é a do formulário de autenticação. Se uma dessas condições for satisfeita, o filtro permite que o fluxo prossiga para a "Faces Servlet". Caso contrário, ele redireciona o usuário para a página do formulário de autenticação.

8 Tente acessar diretamente a página principal da aplicação acessando `http://localhost:8080/K19-Autenticacao/home.xhtml`. Observe que a aplicação redireciona o navegador para a página do formulário de autenticação.

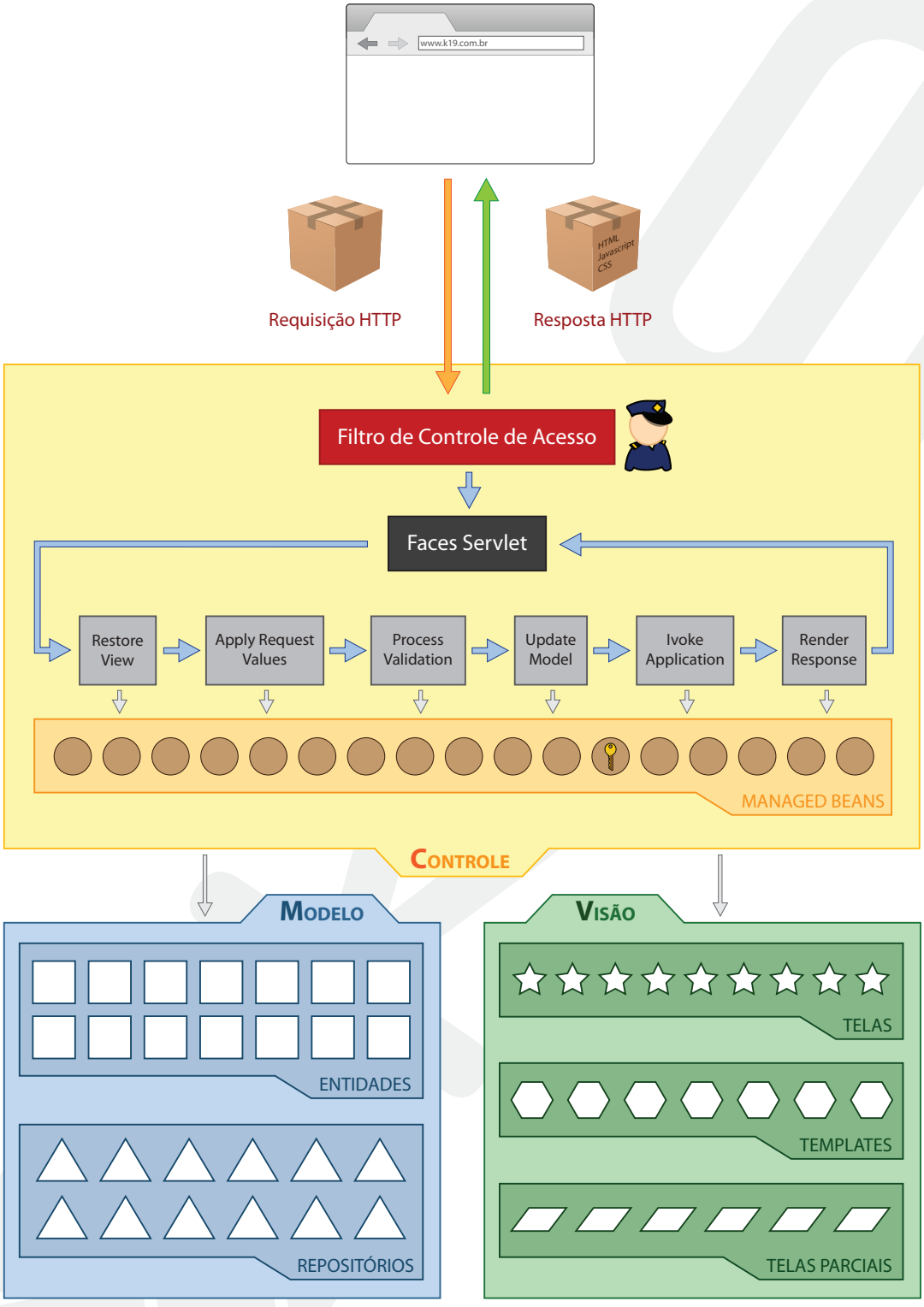


Figura A.1: Filtro e Managed Bean para implementar o controle de autenticação

Páginas de Erro

APÊNDICE B

Por padrão, quando determinados erros ocorrem no processamento de uma requisição, páginas com informações técnicas sobre o problema que ocorreu são geradas e enviadas para os usuários. Na fase de desenvolvimento, essas páginas são úteis para os desenvolvedores. Por outro lado, na fase de produção, essas páginas podem confundir os usuários da aplicação e revelar a estrutura do sistema, expondo possíveis falhas de segurança.



Figura B.1: Expondo possíveis falhas de segurança

Neste capítulo, apresentaremos uma maneira de personalizar as páginas de erro da aplicação.



Exercícios de Fixação

1 Crie um projeto chamado **K19-Paginas-de-Erro** seguindo os passos vistos no exercício do Capítulo 5.

2 Criaremos uma página de erro padrão. Adicione uma página chamada **pagina-de-erro.xhtml** no projeto **K19-Paginas-de-Erro**.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h1>Ocorreu um erro interno no sistema.</h1>
```

```
13   <h3>Tente novamente em alguns instantes.</h3>
14 </h:body>
15 </html>
```

Código XHTML B.1: pagina-de-erro.xhtml

3 Criaremos uma página com um botão que sempre produzirá um erro ao ser clicado. Adicione uma página chamada **gerador-de-erro.xhtml** no projeto **K19-Paginas-de-Erro**.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5        xmlns:h="http://java.sun.com/jsf/html">
6
7      <h:head>
8          <title>K19 Treinamentos</title>
9      </h:head>
10
11     <h:body>
12         <h:form>
13             <h:commandButton value="Gera Erro" action="#{erroBean.geraErro}" />
14         </h:form>
15     </h:body>
16 </html>
```

Código XHTML B.2: gerador-de-erro.xhtml

4 Crie um managed bean que provoque propositalmente um erro. Adicione em um pacote chamado **br.com.k19.controle** uma classe chamada **ErroBean** com o seguinte código:

```
1  package br.com.k19.controle;
2
3  import javax.faces.bean.ManagedBean;
4
5  @ManagedBean
6  public class ErroBean {
7    public void geraErro(){
8      System.out.println(10/0);
9    }
10 }
```

Código Java B.1: ErroBean.java

5 Implante o projeto **K19-Paginas-de-Erro** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

```
http://localhost:8080/K19-Paginas-de-Erro/gerador-de-erro.xhtml
```

6 Configure o Web Container para direcionar todas as exceptions para a página de erro padrão. Adicione o seguinte código no arquivo **web.xml**.

```
1  ...
2  <error-page>
3    <exception-type>java.lang.Exception</exception-type>
4    <location>/pagina-de-erro.xhtml</location>
5  </error-page>
```

```
6 . . .
```

Código XML B.1: web.xml

7 Teste a aplicação novamente acessando o seguinte endereço:

http://localhost:8080/K19-Paginas-de-Erro/gerador-de-erro.xhtml

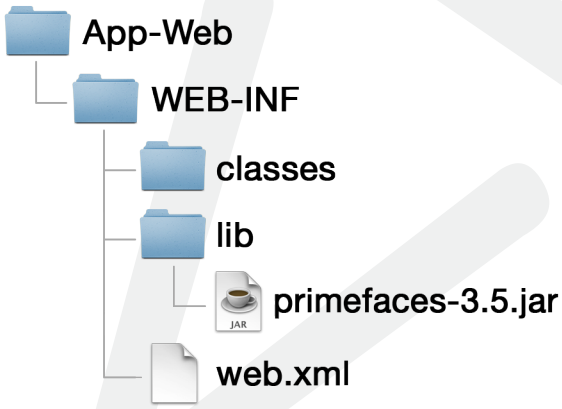
PRIMEFACES

APÊNDICE C

Atualmente, a principal biblioteca de componentes visuais JSF é o PrimeFaces. Visite o site do PrimeFaces http://www.primefaces.org/ para maiores informações.

Instalação

Para ter acesso aos recursos do PrimeFaces, é necessário adicionar o JAR dessa biblioteca no classpath de uma aplicação JSF.



Nas páginas, devemos adicionar o namespace do PrimeFaces para ter acesso aos componentes dessa biblioteca.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:p="http://primefaces.org/ui">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11
12 <h:body>
13   . . .
14 </h:body>
15 </html>
```

Código XHTML C.1: Namespace do PrimeFaces

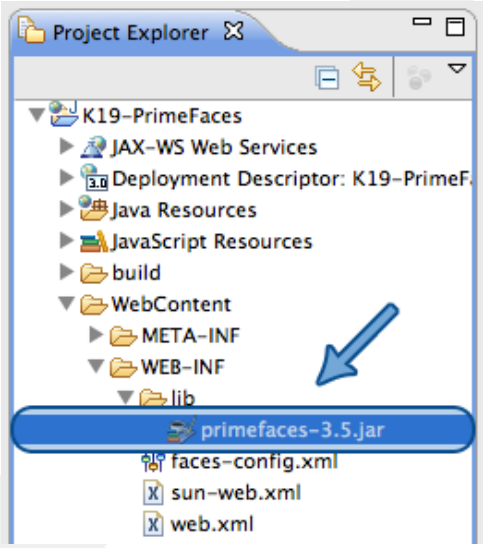
Exercícios de Fixação

1 Crie um projeto chamado **K19-PrimeFaces** seguindo os passos vistos no exercício do Capítulo 5.

2 Adicione o JAR do PrimeFaces no projeto **K19-PrimeFaces**. Você pode encontrar esse arquivo na pasta **K19-Arquivos/primefaces**.

> **Importante**
> Você também pode obter o JAR do PrimeFaces através do site da K19: www.k19.com.br/arquivos.

Copie o arquivo **primefaces-VERSAO.jar** da pasta **K19-Arquivos/primefaces** para a pasta **WEB-INF/lib** do projeto **K19-PrimeFaces**.



3 Analogamente ao exercício anterior, adicione os arquivos **itext-2.1.7.jar** e **poi-3.9-20121203.jar** no projeto **K19-PrimeFaces**. O primeiro arquivo corresponde à biblioteca **iText** (http://itextpdf.com/). O segundo arquivo corresponde à biblioteca **Apache POI** (http://poi.apache.org/). Essas bibliotecas são utilizadas pelo PrimeFaces para exportar dados em PDF e XLS respectivamente.

> **Importante**
> Você também pode obter os arquivos **itext-2.1.7.jar** e **poi-3.9-20121203.jar** através do site da K19: www.k19.com.br/arquivos.

4 Adicione uma página chamada **spinner.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
```

```
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:p="http://primefaces.org/ui">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11
12 <h:body>
13   <p:spinner/>
14 </h:body>
15 </html>
```

Código XHTML C.2: spinner.xhtml

5 Implante o projeto **K19-PrimeFaces** no GlassFish. Certifique-se que o Wildfly esteja parado. Inicie o GlassFish e acesse o endereço abaixo para testar a aplicação.

http://localhost:8080/K19-PrimeFaces/spinner.xhtml



AutoComplete

O componente **AutoComplete** permite que as aplicações ofereçam aos usuários sugestões para o preenchimento de campos.



Exercícios de Fixação

6 Adicione uma página chamada **auto-complete.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:p="http://primefaces.org/ui">
7
8    <h:head>
9      <title>K19 Treinamentos</title>
10   </h:head>
11
12   <h:body>
13   <h:form>
14     <h:panelGrid columns="2">
15
16       <h:outputLabel value="Valor:"/>
17       <p:autoComplete
18         value="#{autoCompleteBean.value}"
19         completeMethod="#{autoCompleteBean.completeMethod}"
20         minQueryLength="3"
21         effect="fade"
22         effectDuration="400"
23         queryDelay="1000" />
24
```

```
25        <h:outputLabel value="Valores:"/>
26        <p:autoComplete
27          value="#{autoCompleteBean.values}"
28          completeMethod="#{autoCompleteBean.completeMethod}"
29          minQueryLength="4"
30          maxResults="5"
31          effect="blind"
32          effectDuration="600"
33          queryDelay="500"
34          multiple="true" />
35        </h:panelGrid>
36      </h:form>
37    </h:body>
38  </html>
```

Código XHTML C.3: auto-complete.xhtml

7 Adicione uma classe chamada **AutoCompleteBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  package br.com.k19.controle;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import javax.faces.bean.ManagedBean;
6
7  @ManagedBean
8  public class AutoCompleteBean {
9
10   private String value;
11   private List<String> values;
12
13   public List<String> completeMethod(String query) {
14     List<String> lista = new ArrayList<String>();
15
16     for (int i = 0; i < 10; i++) {
17       lista.add(query + Math.random());
18     }
19     return lista;
20   }
21
22   // GETTERS E SETTERS
23 }
```

Código Java C.1: AutoCompleteBean.java

Execute o projeto **K19-PrimeFaces** e acesse o endereço:

http://localhost:8080/K19-PrimeFaces/auto-complete.xhtml



Poll

O componente **Poll** é utilizado quando desejamos que os navegadores realizem requisições AJAX periodicamente.



Exercícios de Fixação

8 Adicione uma página chamada **poll.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:p="http://primefaces.org/ui">

  <h:head>
    <title>K19 Treinamentos</title>
  </h:head>

  <h:body>
    <h:form>
      <h:outputText
        id="placar"
        value="#{pollBean.placarTimeA} X #{pollBean.placarTimeB}" />

      <p:poll
        interval="1"
        listener="#{pollBean.update}"
        update="placar" />
    </h:form>
  </h:body>
</html>
```

Código XHTML C.4: poll.xhtml

9 Adicione uma classe chamada **PollBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
package br.com.k19.controle;

import javax.faces.bean.ManagedBean;
import javax.faces.bean.ViewScoped;

@ManagedBean
@ViewScoped
public class PollBean {

  private int placarTimeA;
  private int placarTimeB;

  public void update() {
    if (Math.random() > 0.5) {
      this.placarTimeA++;
    } else {
      this.placarTimeB++;
    }
  }

  // GETTERS E SETTERS
}
```

Código Java C.2: PollBean.java

Teste a aplicação acessando o endereço:

http://localhost:8080/K19-PrimeFaces/poll.xhtml



Calendar

O componente **Calendar** oferece uma interface sofisticada para a seleção de uma data.



Exercícios de Fixação

10 Adicione uma página chamada **calendar.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:p="http://primefaces.org/ui">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h:outputScript name="calendar.js" library="js" target="head" />
  <h:form>
    <h:panelGrid columns="2">
      <h:outputLabel value="Data de nascimento:" />
      <p:calendar
        value="#{calendarBean.dataDeNascimento}"
        locale="pt_BR"
        mode="inline" />

      <h:outputLabel value="Data de nascimento:" />
      <p:calendar
        value="#{calendarBean.dataDeNascimento}"
        locale="pt_BR"
        effect="slide" />

      <h:outputLabel value="Data de nascimento:" />
      <p:calendar
        value="#{calendarBean.dataDeNascimento}"
        locale="pt_BR"
        showOn="button"
        effect="fold" />

      <h:outputLabel value="Data de nascimento:" />
      <p:calendar
        value="#{calendarBean.dataDeNascimento}"
        locale="pt_BR"
        navigator="true"
        effect="slideDown" />
    </h:panelGrid>
  </h:form>
</h:body>
</html>
```

Código XHTML C.5: calendar.xhtml

11 Adicione um arquivo chamado **calendar.js** em uma pasta chamada **resources/js** na raiz da aplicação **K19-PrimeFaces** com o seguinte conteúdo. A pasta **resources/js** deve ser ficar em **WebContent**

```
PrimeFaces.locales['pt_BR'] = {
    closeText: 'Fechar',
    prevText: 'Anterior',
    nextText: 'Próximo',
    currentText: 'Hoje',
    monthNames: ['Janeiro','Fevereiro','Março','Abril','Maio','Junho',
        'Julho','Agosto','Setembro','Outubro','Novembro','Dezembro'],
    monthNamesShort: ['Jan','Fev','Mar','Abr','Mai','Jun',
        'Jul','Ago','Set','Out','Nov','Dez'],
    dayNames: ['Domingo','Segunda','Terça','Quarta','Quinta','Sexta','Sábado'],
    dayNamesShort: ['Dom','Seg','Ter','Qua','Qui','Sex','Sáb'],
    dayNamesMin: ['D','S','T','Q','Q','S','S'],
    weekHeader: 'Semana',
    firstDay: 0,
    isRTL: false,
    showMonthAfterYear: false,
    yearSuffix: '',
    timeOnlyTitle: 'Só Horas',
    timeText: 'Tempo',
    hourText: 'Hora',
    minuteText: 'Minuto',
    secondText: 'Segundo',
    ampm: false,
    month: 'Mês',
    week: 'Semana',
    day: 'Dia',
    allDayText : 'Todo o Dia'
};
```

Código Javascript C.1: calendar.js

12 Adicione uma classe chamada **CalendarBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
package br.com.k19.controle;

import java.util.Date;

import javax.faces.bean.ManagedBean;

@ManagedBean
public class CalendarBean {
  private Date dataDeNascimento;

  // GETTERS E SETTERS
}
```

Código Java C.3: Calendar.java

Teste a aplicação acessando o endereço:

http://localhost:8080/K19-PrimeFaces/calendar.xhtml



InputMask

O componente **InputMask** permite a definição de máscaras de preenchimento para caixas de texto.



Exercícios de Fixação

13 Adicione uma página chamada **input-mask.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:p="http://primefaces.org/ui">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h:form>
    <h:panelGrid columns="2">
      <h:outputLabel value="Data:" />
      <p:inputMask mask="99/99/9999"/>

      <h:outputLabel value="Telefone:" />
      <p:inputMask mask="(99) 9999-9999?9"/>

      <h:outputLabel value="CPF:" />
      <p:inputMask mask="999.999.999-99"/>

      <h:outputLabel value="Código" />
      <p:inputMask mask="a*-99-*a"/>
    </h:panelGrid>
  </h:form>
</h:body>
</html>
```

Código XHTML C.6: input-mask.xhtml

Teste a aplicação acessando o endereço:

http://localhost:8080/K19-PrimeFaces/input-mask.xhtml



DataTable

O componente **DataTable** é utilizado para apresentar dados de forma tabular. Ele possui recursos como: paginação, filtragem e ordenação.



Exercícios de Fixação

14 Adicione uma página chamada **data-table.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
```

```
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4   <html xmlns="http://www.w3.org/1999/xhtml"
5     xmlns:h="http://java.sun.com/jsf/html"
6     xmlns:f="http://java.sun.com/jsf/core"
7     xmlns:ui="http://java.sun.com/jsf/facelets"
8     xmlns:p="http://primefaces.org/ui">
9
10  <h:head>
11    <title>K19 Treinamentos</title>
12  </h:head>
13
14  <h:body>
15    <h:form id="form">
16      <p:dataTable
17        value="#{dataTableBean.cursos}"
18        var="curso"
19        rowKey="#{curso.sigla}"
20        paginator="true"
21        rows="4"
22        selectionMode="multiple"
23        filteredValue="#{dataTableBean.cursosFiltrados}"
24        selection="#{dataTableBean.cursosSelecionados}">
25
26        <f:facet name="header">Cursos da K19</f:facet>
27
28        <p:column
29          headerText="ID"
30          sortBy="#{curso.id}"
31          filterBy="#{curso.id}">
32          #{curso.id}
33        </p:column>
34
35        <p:column
36          headerText="Logo">
37          <h:graphicImage value="#{curso.logo}" />
38        </p:column>
39
40        <p:column
41          headerText="Sigla"
42          sortBy="#{curso.sigla}"
43          filterBy="#{curso.sigla}">
44          #{curso.sigla}
45        </p:column>
46
47        <p:column
48          headerText="Nome"
49          sortBy="#{curso.nome}"
50          filterBy="#{curso.nome}">
51          #{curso.nome}
52        </p:column>
53
54        <f:facet name="footer">
55          <p:commandButton
56            value="Escolher Curso"
57            update=":form:curso-escolhido" />
58        </f:facet>
59      </p:dataTable>
60
61      <p:panel id="curso-escolhido" header="Cursos Escolhidos">
62        <ul>
63          <ui:repeat
64            var="curso"
65            value="#{dataTableBean.cursosSelecionados}"
66            rendered="#{not empty dataTableBean.cursosSelecionados}">
67            <li>
68              <h:outputText value="#{curso.nome}" />
69            </li>
70          </ui:repeat>
71
```

```
72          <h:panelGroup
73            rendered="#{dataTableBean.cursosSelecionados == null or empty dataTableBean↩
                .cursosSelecionados}">
74            <li>Nenhum curso escolhido</li>
75          </h:panelGroup>
76        </ul>
77      </p:panel>
78    </h:form>
79  </h:body>
80  </html>
```

Código XHTML C.7: data-table.xhtml

15 Adicione uma classe chamada **Curso** em um pacote chamado **br.com.k19.modelo** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  package br.com.k19.modelo;
2
3  public class Curso {
4    private Long id;
5    private String sigla;
6    private String nome;
7    private String logo;
8
9    public Curso(Long id, String sigla, String nome, String logo) {
10     this.id = id;
11     this.sigla = sigla;
12     this.nome = nome;
13     this.logo = logo;
14   }
15
16   // GETTERS E SETTERS
17 }
```

Código Java C.4: Curso.java

16 Adicione uma classe chamada **CursoRepository** em um pacote chamado **br.com.k19.modelo** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  package br.com.k19.modelo;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class CursoRepository {
7    private static List<Curso> cursos = new ArrayList<Curso>();
8
9    static {
10     cursos.add(new Curso(1L, "K01",
11       "Lógica de Programação",
12       "http://www.k19.com.br/css/img/k01-logo-small.png"));
13
14     cursos.add(new Curso(2L, "K02",
15       "Desenvolvimento Web com HTML, CSS e JavaScript",
16       "http://www.k19.com.br/css/img/k02-logo-small.png"));
17
18     cursos.add(new Curso(3L, "K03",
19       "SQL e Modelo Relacional",
20       "http://www.k19.com.br/css/img/k03-logo-small.png"));
21
22     cursos.add(new Curso(4L, "K11",
23       "Orientação a Objetos em Java",
24       "http://www.k19.com.br/css/img/k11-logo-small.png"));
```

```
25
26      cursos.add(new Curso(5L, "K12",
27        "Desenvolvimento Web com JSF2 e JPA2",
28        "http://www.k19.com.br/css/img/k12-logo-small.png"));
29
30      cursos.add(new Curso(6L, "K21",
31        "Persistência com JAP2 e Hibernate",
32        "http://www.k19.com.br/css/img/k21-logo-small.png"));
33
34      cursos.add(new Curso(7L, "K22",
35        "Desenvolvimento Web Avançado com EJB, JSF e CDI",
36        "http://www.k19.com.br/css/img/k22-logo-small.png"));
37
38      cursos.add(new Curso(8L, "K23",
39        "Integração de Sistemas com Webservices, JMS e EJB",
40        "http://www.k19.com.br/css/img/k23-logo-small.png"));
41
42      cursos.add(new Curso(9L, "K31",
43        "C# e Orientação a Objetos",
44        "http://www.k19.com.br/css/img/k31-logo-small.png"));
45
46      cursos.add(new Curso(10L, "K32",
47        "Desenvolvimento Web com ASP.NET MVC",
48        "http://www.k19.com.br/css/img/k32-logo-small.png"));
49
50      cursos.add(new Curso(11L, "K41",
51        "Desenvolvimento Mobile com Android",
52        "http://www.k19.com.br/css/img/k41-logo-small.png"));
53
54      cursos.add(new Curso(12L, "K51",
55        "Design Patterns em Java",
56        "http://www.k19.com.br/css/img/k51-logo-small.png"));
57
58      cursos.add(new Curso(13L, "K52",
59        "Desenvolvimento Web com Struts",
60        "http://www.k19.com.br/css/img/k52-logo-small.png"));
61    }
62
63    public List<Curso> getCursos() {
64      return cursos;
65    }
66  }
```

Código Java C.5: CursoRepository.java

17 Adicione uma classe chamada **DataTableBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  package br.com.k19.controle;
2
3  import java.util.List;
4
5  import javax.faces.bean.ManagedBean;
6
7  import br.com.k19.modelo.Curso;
8  import br.com.k19.modelo.CursoRepository;
9
10 @ManagedBean
11 public class DataTableBean {
12   private List<Curso> cursosSelecionados;
13   private List<Curso> cursosFiltrados;
14
15   public List<Curso> getCursos() {
16     return new CursoRepository().getCursos();
17   }
18
19   // GETTERS E SETTERS
```

```
20 }
```

Código Java C.6: DataTableBean.java

Teste a aplicação acessando o endereço:

http://localhost:8080/K19-PrimeFaces/data-table.xhtml



DataExporter

O componente **DataExporter** permite que os dados apresentados em um **DataTable** sejam exportados para diversos formatos como: xls, pdf, cvs e xml.



Exercícios de Fixação

18 Adicione uma página chamada **data-exporter.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core"
7    xmlns:ui="http://java.sun.com/jsf/facelets"
8    xmlns:p="http://primefaces.org/ui">
9
10 <h:head>
11   <title>K19 Treinamentos</title>
12 </h:head>
13
14 <h:body>
15   <h:form id="form">
16     <p:dataTable
17       id="tabela"
18       value="#{dataTableBean.cursos}"
19       var="curso"
20       rowKey="#{curso.sigla}"
21       paginator="true"
22       rows="4"
23       selectionMode="multiple"
24       filteredValue="#{dataTableBean.cursosFiltrados}"
25       selection="#{dataTableBean.cursosSelecionados}">
26
27       <f:facet name="header">Cursos da K19</f:facet>
28
29       <p:column sortBy="#{curso.id}" filterBy="#{curso.id}">
30         <f:facet name="header">
31           <h:outputText value="ID" />
32         </f:facet>
33         <h:outputText value="#{curso.id}" />
34       </p:column>
35
36       <p:column exportable="false">
37         <f:facet name="header">
38           <h:outputText value="Logo" />
39         </f:facet>
40         <h:graphicImage value="#{curso.logo}" />
```

```
41        </p:column>
42
43        <p:column sortBy="#{curso.sigla}" filterBy="#{curso.sigla}">
44          <f:facet name="header">
45            <h:outputText value="Sigla" />
46          </f:facet>
47          <h:outputText value="#{curso.sigla}" />
48        </p:column>
49
50        <p:column sortBy="#{curso.nome}" filterBy="#{curso.nome}">
51          <f:facet name="header">
52            <h:outputText value="Nome" />
53          </f:facet>
54          <h:outputText value="#{curso.nome}" />
55        </p:column>
56
57        <f:facet name="footer">
58          <h:panelGrid columns="3">
59            <h:panelGroup>
60              Exportar toda a tabela para:
61
62              <h:commandButton value="XLS">
63                <p:dataExporter
64                  type="xls"
65                  target="tabela"
66                  fileName="cursos" />
67              </h:commandButton>
68
69              <h:commandButton value="PDF">
70                <p:dataExporter
71                  type="pdf"
72                  target="tabela"
73                  fileName="cursos" />
74              </h:commandButton>
75
76              <h:commandButton value="CSV">
77                <p:dataExporter
78                  type="csv"
79                  target="tabela"
80                  fileName="cursos" />
81              </h:commandButton>
82
83              <h:commandButton value="XML">
84                <p:dataExporter
85                  type="xml"
86                  target="tabela"
87                  fileName="cursos" />
88              </h:commandButton>
89            </h:panelGroup>
90            <h:panelGroup>
91              Exportar apenas esta página para:
92
93              <h:commandButton value="XLS">
94                <p:dataExporter
95                  type="xls"
96                  target="tabela"
97                  fileName="cursos"
98                  pageOnly="true" />
99              </h:commandButton>
100
101             <h:commandButton value="PDF">
102               <p:dataExporter
103                 type="pdf"
104                 target="tabela"
105                 fileName="cursos"
106                 pageOnly="true" />
107             </h:commandButton>
108
109             <h:commandButton value="CSV">
110               <p:dataExporter
```

```
111                 type="csv"
112                 target="tabela"
113                 fileName="cursos"
114                 pageOnly="true" />
115             </h:commandButton>
116
117             <h:commandButton value="XML">
118               <p:dataExporter
119                 type="xml"
120                 target="tabela"
121                 fileName="cursos"
122                 pageOnly="true" />
123             </h:commandButton>
124           </h:panelGroup>
125         </h:panelGrid>
126       </f:facet>
127     </p:dataTable>
128   </h:form>
129 </h:body>
130 </html>
```

Código XHTML C.8: data-exporter.xhtml



PickList

O componente **PickList** oferece uma interface simples para seleção de elementos de um conjunto.



Exercícios de Fixação

19 Adicione uma página chamada **pick-list.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core"
7    xmlns:p="http://primefaces.org/ui">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12
13 <h:body>
14 <h:form>
15   <p:growl id="msg" showDetail="true" escape="false" />
16
17   <p:pickList
18     value="#{pickListBean.cursos}"
19     var="curso"
20     effect="bounce"
21     itemValue="#{curso.id}"
22     itemLabel="#{curso.sigla}"
23     showSourceControls="true"
24     showTargetControls="true"
25     showCheckbox="true"
26     showSourceFilter="true"
```

```
27      showTargetFilter="true"
28      filterMatchMode="contains">
29
30      <f:facet name="sourceCaption">Cursos</f:facet>
31      <f:facet name="targetCaption">Selecionados</f:facet>
32
33      <p:ajax
34        event="transfer"
35        listener="#{pickListBean.onTransfer}"
36        update="msg" />
37
38      <p:column>
39        <p:graphicImage value="#{curso.logo}"/>
40      </p:column>
41
42      <p:column>
43        <h:outputText value="#{curso.sigla}"/>
44      </p:column>
45      </p:pickList>
46  </h:form>
47  </h:body>
48  </html>
```

Código XHTML C.9: pick-list.xhtml

20 Adicione uma classe chamada **PickListBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1   package br.com.k19.controle;
2
3   import java.util.ArrayList;
4   import java.util.List;
5
6   import javax.faces.application.FacesMessage;
7   import javax.faces.bean.ManagedBean;
8   import javax.faces.context.FacesContext;
9
10  import org.primefaces.event.TransferEvent;
11  import org.primefaces.model.DualListModel;
12
13  import br.com.k19.modelo.Curso;
14  import br.com.k19.modelo.CursoRepository;
15
16  @ManagedBean
17  public class PickListBean {
18
19    private DualListModel<Curso> cursos;
20
21    public PickListBean() {
22      List<Curso> source = new CursoRepository().getCursos();
23      List<Curso> target = new ArrayList<Curso>();
24
25      this.cursos = new DualListModel<Curso>(source, target);
26    }
27
28    public void onTransfer(TransferEvent event) {
29      StringBuilder builder = new StringBuilder();
30
31      for (Object obj : event.getItems()) {
32        Integer id = Integer.valueOf(obj.toString());
33        Curso c = this.cursos.getSource().get(id - 1);
34        builder.append(c.getNome()).append("<br/>");
35      }
36
37      FacesMessage msg = new FacesMessage();
38      msg.setSeverity(FacesMessage.SEVERITY_INFO);
39
```

```
40      if(event.isAdd()) {
41        msg.setSummary("Cursos Selecionados");
42      } else {
43        msg.setSummary("Cursos Removidos");
44      }
45
46      msg.setDetail(builder.toString());
47
48      FacesContext.getCurrentInstance().addMessage(null, msg);
49    }
50
51    // GETTERS E SETTERS
52  }
```

Código Java C.7: PickListBean.java

Teste a aplicação acessando o endereço:

http://localhost:8080/K19-PrimeFaces/pick-list.xhtml



MegaMenu

O componente **MegaMenu** é utilizado para criação dos menus de uma aplicação JSF.



Exercícios de Fixação



Exercícios de Fixação

21 Adicione uma página chamada **mega-menu.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:p="http://primefaces.org/ui">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11
12 <h:body>
13   <p:megaMenu model="#{megaMenuBean.menuModel}" />
14 </h:body>
15 </html>
```

Código XHTML C.10: mega-menu.xhtml

22 Adicione uma classe chamada **MegaMenuBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  package br.com.k19.controle;
2
3  import javax.faces.bean.ManagedBean;
4
5  import org.primefaces.component.column.Column;
6  import org.primefaces.component.menuitem.MenuItem;
7  import org.primefaces.component.submenu.Submenu;
8  import org.primefaces.model.DefaultMenuModel;
9  import org.primefaces.model.MenuModel;
10
11 @ManagedBean
12 public class MegaMenuBean {
13   private MenuModel menuModel;
14
15   public MegaMenuBean() {
16     this.menuModel = new DefaultMenuModel();
17
18     Submenu esporteMenu = new Submenu();
19     esporteMenu.setLabel("Esportes");
20
21     Submenu futebolMenu = new Submenu();
22     futebolMenu.setLabel("Futebol");
23
24     MenuItem item = new MenuItem();
25     item.setValue("Brasileiro");
26     item.setUrl("#");
27     futebolMenu.getChildren().add(item);
28
29     item = new MenuItem();
30     item.setValue("Europeu");
31     item.setUrl("#");
32     futebolMenu.getChildren().add(item);
33
34     item = new MenuItem();
35     item.setValue("Japonês");
36     item.setUrl("#");
37     futebolMenu.getChildren().add(item);
38
39     Submenu voleiMenu = new Submenu();
40     voleiMenu.setLabel("Volei");
41
42     item = new MenuItem();
43     item.setValue("Liga Mundial");
44     item.setUrl("#");
45     voleiMenu.getChildren().add(item);
46
47     item = new MenuItem();
48     item.setValue("Eliminatórias da Copa");
49     item.setUrl("#");
50     voleiMenu.getChildren().add(item);
51
52     Submenu automobilismoMenu = new Submenu();
53     automobilismoMenu.setLabel("Automobilismo");
54
55     item = new MenuItem();
56     item.setValue("Fórmula 1");
57     item.setUrl("#");
58     automobilismoMenu.getChildren().add(item);
59
60     item = new MenuItem();
61     item.setValue("Fórmula 3");
62     item.setUrl("#");
63     automobilismoMenu.getChildren().add(item);
64
65     item = new MenuItem();
66     item.setValue("Fotos");
67     item.setUrl("#");
68     automobilismoMenu.getChildren().add(item);
69
70     Column esporteColuna1 = new Column();
```

```
71      esporteColuna1.getChildren().add(futebolMenu);
72      esporteColuna1.getChildren().add(voleiMenu);
73      esporteMenu.getChildren().add(esporteColuna1);
74
75      Column esporteColuna2 = new Column();
76      esporteColuna2.getChildren().add(automobilismoMenu);
77      esporteMenu.getChildren().add(esporteColuna2);
78
79      this.menuModel.addSubmenu(esporteMenu);
80
81      Submenu noticiaMenu = new Submenu();
82      noticiaMenu.setLabel("Notícias");
83
84      Submenu economiaMenu = new Submenu();
85      economiaMenu.setLabel("Economia");
86
87      item = new MenuItem();
88      item.setValue("Brasil");
89      item.setUrl("#");
90      economiaMenu.getChildren().add(item);
91
92      item = new MenuItem();
93      item.setValue("Mundo");
94      item.setUrl("#");
95      economiaMenu.getChildren().add(item);
96
97      Submenu tecnologiaMenu = new Submenu();
98      tecnologiaMenu.setLabel("Tecnologia");
99
100     item = new MenuItem();
101     item.setValue("Segurança");
102     item.setUrl("#");
103     tecnologiaMenu.getChildren().add(item);
104
105     item = new MenuItem();
106     item.setValue("Negócios em TI");
107     item.setUrl("#");
108     tecnologiaMenu.getChildren().add(item);
109
110     Submenu educacaoMenu = new Submenu();
111     educacaoMenu.setLabel("Educação");
112
113     item = new MenuItem();
114     item.setValue("Vestibular");
115     item.setUrl("#");
116     educacaoMenu.getChildren().add(item);
117
118     item = new MenuItem();
119     item.setValue("ENEM");
120     item.setUrl("#");
121     educacaoMenu.getChildren().add(item);
122
123     item = new MenuItem();
124     item.setValue("ProUni");
125     item.setUrl("#");
126     educacaoMenu.getChildren().add(item);
127
128     Column noticiaColuna1 = new Column();
129     noticiaColuna1.getChildren().add(economiaMenu);
130     noticiaColuna1.getChildren().add(tecnologiaMenu);
131     noticiaMenu.getChildren().add(noticiaColuna1);
132
133     Column noticiaColuna2 = new Column();
134     noticiaColuna2.getChildren().add(educacaoMenu);
135     noticiaMenu.getChildren().add(noticiaColuna2);
136
137     this.menuModel.addSubmenu(noticiaMenu);
138   }
139
140   public MenuModel getMenuModel() {
```

```
141     return menuModel;
142   }
143 }
```

Código Java C.8: MegaMenuBean.java

Teste a aplicação acessando o endereço:

http://localhost:8080/K19-PrimeFaces/mega-menu.xhtml



Chart

Os componentes de "charts" permitem que gráficos sejam criados de forma extremamente simples.



Exercícios de Fixação

23 Adicione uma página chamada **chart.xhtml** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2       "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core"
7    xmlns:p="http://primefaces.org/ui">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12
13 <h:body>
14   <h:form>
15     <p:lineChart
16       value="#{chartBean.linearModel}"
17       legendPosition="e"
18       animate="true"
19       title="Gráfico de Linhas" />
20
21     <p:barChart
22       value="#{chartBean.categoryModel}"
23       legendPosition="ne"
24       animate="true"
25       title="Gráfico de Barras" />
26
27     <p:pieChart
28       value="#{chartBean.pieModel}"
29       legendPosition="w"
30       title="Libertadores" />
31   </h:form>
32 </h:body>
33 </html>
```

Código XHTML C.11: chart.xhtml

24 Adicione uma classe chamada **ChartBean** em um pacote chamado **br.com.k19.controle** no projeto **K19-PrimeFaces** com o seguinte conteúdo.

```
1  package br.com.k19.controle;
2
3  @ManagedBean
4  public class ChartBean {
5
6    private PieChartModel pieModel = new PieChartModel();
7    private CartesianChartModel categoryModel = new CartesianChartModel();;
8    private CartesianChartModel linearModel = new CartesianChartModel();;
9
10   public ChartBean() {
11     this.pieModel.set("São Paulo", 3);
12     this.pieModel.set("Santos", 3);
13     this.pieModel.set("Palmeiras", 1);
14     this.pieModel.set("Corinthians", 1);
15
16     ChartSeries serieA = new ChartSeries();
17     serieA.setLabel("A");
18
19     serieA.set("2004", 10);
20     serieA.set("2005", 200);
21     serieA.set("2006", 77);
22     serieA.set("2007", 13);
23     serieA.set("2008", 177);
24
25     ChartSeries serieB = new ChartSeries();
26     serieB.setLabel("B");
27
28     serieB.set("2004", 100);
29     serieB.set("2005", 95);
30     serieB.set("2006", 115);
31     serieB.set("2007", 105);
32     serieB.set("2008", 111);
33
34     categoryModel.addSeries(serieA);
35     categoryModel.addSeries(serieB);
36
37     LineChartSeries series1 = new LineChartSeries();
38     series1.setLabel("1");
39
40     series1.set(1, 1);
41     series1.set(2, 2);
42     series1.set(3, 2);
43     series1.set(4, 7);
44     series1.set(5, 1);
45
46     LineChartSeries series2 = new LineChartSeries();
47     series2.setLabel("1");
48
49     series2.set(1, 5);
50     series2.set(2, 6);
51     series2.set(3, 7);
52     series2.set(4, 1);
53     series2.set(5, 7);
54
55     linearModel.addSeries(series1);
56     linearModel.addSeries(series2);
57   }
58
59   // GETTERS E SETTERS
60 }
```

Código Java C.9: ChartBean.java

Teste a aplicação acessando o endereço:

```
http://localhost:8080/K19-PrimeFaces/chart.xhtml
```



RESPOSTAS

APÊNDICE D

Exercício Complementar 2.1

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.PreparedStatement;
6
7  public class CriaTabelaLivro {
8    public static void main(String[] args) throws Exception {
9      String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
10     String usuario = "root";
11     String senha = "root";
12
13     System.out.println("Abrindo conexão...");
14     Connection conexao =
15       DriverManager.getConnection(stringDeConexao, usuario, senha);
16
17     System.out.println("Criando a tabela Livro...");
18     String sql =
19       "CREATE TABLE Livro ("
20         + "id BIGINT NOT NULL AUTO_INCREMENT, "
21         + "titulo VARCHAR(255) NOT NULL, "
22         + "preco DOUBLE NOT NULL, "
23         + "editora_id BIGINT NOT NULL, "
24         + "PRIMARY KEY(id), "
25         + "FOREIGN KEY fk_editora(editora_id) REFERENCES Editora(id)"
26       + ") "
27       + "ENGINE = InnoDB";
28     PreparedStatement comando = conexao.prepareStatement(sql);
29     comando.execute();
30     comando.close();
31
32     System.out.println("Fechando conexão...");
33     conexao.close();
34   }
35 }
```

Código Java 2.14: www.k19.com.br/exercicios/k12/02/C001/CriaTabelaLivro.java

Exercício Complementar 2.2

```
1  package br.com.k19.jdbc;
2
3  public class Livro {
4    private Long id;
5    private String titulo;
6    private Double preco;
7    private Long editoraId;
8
```

```
9   // GETTERS E SETTERS
10 }
```

Código Java 2.15: www.k19.com.br/exercicios/k12/02/C002/Livro.java

Exercício Complementar 2.3

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.PreparedStatement;
6  import java.sql.ResultSet;
7  import java.sql.Statement;
8  import java.util.Scanner;
9
10 public class InsereLivro {
11   public static void main(String[] args) throws Exception {
12     String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
13     String usuario = "root";
14     String senha = "root";
15
16     System.out.println("Abrindo conexão...");
17     Connection conexao =
18       DriverManager.getConnection(stringDeConexao, usuario, senha);
19
20     Scanner entrada = new Scanner(System.in);
21
22     Livro livro = new Livro();
23     System.out.println("Digite o título do livro: ");
24     livro.setTitulo(entrada.nextLine());
25
26     System.out.println("Digite o preço do livro: ");
27     livro.setPreco(Double.parseDouble(entrada.nextLine()));
28
29     System.out.println("Digite o id da editora do livro: ");
30     livro.setEditoraId(Long.parseLong(entrada.nextLine()));
31
32     entrada.close();
33
34     String sql = "INSERT INTO Livro (titulo, preco, editora_id) " +
35       "VALUES ('" + livro.getTitulo() + "', " + livro.getPreco() +
36             ", " + livro.getEditoraId() + ")";
37
38     PreparedStatement comando =
39       conexao.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
40
41     System.out.println("Executando comando...");
42     comando.execute();
43
44     System.out.println("Recuperando o ID gerado pelo SGBD...");
45     ResultSet generatedKeys = comando.getGeneratedKeys();
46     generatedKeys.next();
47     livro.setId(generatedKeys.getLong(1));
48     System.out.println("ID: " + livro.getId());
49
50     System.out.println("Fechando conexão...");
51     conexao.close();
52   }
53 }
```

Código Java 2.16: www.k19.com.br/exercicios/k12/02/C003/InsereLivro.java

Exercício Complementar 2.5

```java
package br.com.k19.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Scanner;

public class InsereLivro {
  public static void main(String[] args) throws Exception {
    String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
    String usuario = "root";
    String senha = "root";

    System.out.println("Abrindo conexão...");
    Connection conexao =
      DriverManager.getConnection(stringDeConexao, usuario, senha);

    Scanner entrada = new Scanner(System.in);

    Livro livro = new Livro();
    System.out.println("Digite o título do livro: ");
    livro.setTitulo(entrada.nextLine());

    System.out.println("Digite o preço do livro: ");
    livro.setPreco(Double.parseDouble(entrada.nextLine()));

    System.out.println("Digite o id da editora do livro: ");
    livro.setEditoraId(Long.parseLong(entrada.nextLine()));

    entrada.close();

    String sql = "INSERT INTO Livro (titulo, preco, editora_id) " +
      "VALUES (?, ?, ?)";

    PreparedStatement comando =
      conexao.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
    comando.setString(1, livro.getTitulo());
    comando.setDouble(2, livro.getPreco());
    comando.setLong(3, livro.getEditoraId());

    System.out.println("Executando comando...");
    comando.execute();

    System.out.println("Recuperando o ID gerado pelo SGBD...");
    ResultSet generatedKeys = comando.getGeneratedKeys();
    generatedKeys.next();
    livro.setId(generatedKeys.getLong(1));
    System.out.println("ID: " + livro.getId());

    System.out.println("Fechando conexão...");
    conexao.close();
  }
}
```

Código Java 2.21: www.k19.com.br/exercicios/k12/02/C005/InsereLivro.java

Exercício Complementar 2.7

```java
package br.com.k19.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;
import java.util.List;

public class ListaLivros {
  public static void main(String[] args) throws Exception {
    String stringDeConexao = "jdbc:mysql://localhost:3306/livraria";
    String usuario = "root";
    String senha = "root";

    System.out.println("Abrindo conexão...");
    Connection conexao =
      DriverManager.getConnection(stringDeConexao, usuario, senha);

    String sql = "SELECT * FROM Livro;";

    PreparedStatement comando = conexao.prepareStatement(sql);

    System.out.println("Executando comando...");
    ResultSet resultado = comando.executeQuery();

    List<Livro> lista = new ArrayList<Livro>();
    while (resultado.next()) {
      Livro livro = new Livro();
      livro.setId(resultado.getLong("id"));
      livro.setTitulo(resultado.getString("titulo"));
      livro.setPreco(resultado.getDouble("preco"));
      livro.setEditoraId(resultado.getLong("editora_id"));
      lista.add(livro);
    }

    System.out.println("Resultados encontrados: \n");
    for (Livro livro : lista) {
      System.out.println("Id: " + livro.getId());
      System.out.println("Título: " + livro.getTitulo());
      System.out.println("Preço: " + livro.getPreco());
      System.out.println("Editora Id: " + livro.getEditoraId());
      System.out.println("----------------------------");
    }
    System.out.println("\nFechando conexão...");
    conexao.close();
  }
}
```

Código Java 2.27: www.k19.com.br/exercicios/k12/02/C007/ListaLivros.java

Exercício Complementar 2.8

```java
package br.com.k19.jdbc;

import java.sql.Connection;
import java.sql.PreparedStatement;

public class CriaTabelaLivro {
  public static void main(String[] args) throws Exception {
    System.out.println("Abrindo conexão...");
    Connection conexao = ConnectionFactory.createConnection();

```

```
11      System.out.println("Criando a tabela Livro...");
12      String sql =
13        "CREATE TABLE Livro ("
14          + "id BIGINT NOT NULL AUTO_INCREMENT, "
15          + "titulo VARCHAR(255) NOT NULL, "
16          + "preco DOUBLE NOT NULL, "
17          + "editora_id BIGINT NOT NULL, "
18          + "PRIMARY KEY(id), "
19          + "FOREIGN KEY fk_editora(editora_id) REFERENCES Editora(id)"
20        + ") "
21        + "ENGINE = InnoDB";
22      PreparedStatement comando = conexao.prepareStatement(sql);
23      comando.execute();
24      comando.close();
25
26      System.out.println("Fechando conexão...");
27      conexao.close();
28    }
29  }
```

Código Java 2.36: www.k19.com.br/exercicios/k12/02/C008/CriaTabelaLivro.java

```
1   package br.com.k19.jdbc;
2
3   import java.sql.Connection;
4   import java.sql.PreparedStatement;
5   import java.sql.ResultSet;
6   import java.sql.Statement;
7   import java.util.Scanner;
8
9   public class InsereLivro {
10    public static void main(String[] args) throws Exception {
11      System.out.println("Abrindo conexão...");
12      Connection conexao = ConnectionFactory.createConnection();
13
14      Scanner entrada = new Scanner(System.in);
15
16      Livro livro = new Livro();
17      System.out.println("Digite o título do livro: ");
18      livro.setTitulo(entrada.nextLine());
19
20      System.out.println("Digite o preço do livro: ");
21      livro.setPreco(Double.parseDouble(entrada.nextLine()));
22
23      System.out.println("Digite o id da editora do livro: ");
24      livro.setEditoraId(Long.parseLong(entrada.nextLine()));
25
26      entrada.close();
27
28      String sql = "INSERT INTO Livro (titulo, preco, editora_id) " +
29        "VALUES (?, ?, ?)";
30
31      PreparedStatement comando =
32        conexao.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
33      comando.setString(1, livro.getTitulo());
34      comando.setDouble(2, livro.getPreco());
35      comando.setLong(3, livro.getEditoraId());
36
37      System.out.println("Executando comando...");
38      comando.execute();
39
40      System.out.println("Recuperando o ID gerado pelo SGBD...");
41      ResultSet generatedKeys = comando.getGeneratedKeys();
42      generatedKeys.next();
43      livro.setId(generatedKeys.getLong(1));
44      System.out.println("ID: " + livro.getId());
45
46      System.out.println("Fechando conexão...");
47      conexao.close();
```

```
48    }
49 }
```

Código Java 2.37: www.k19.com.br/exercicios/k12/02/C008/InsereLivro.java

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5  import java.sql.ResultSet;
6  import java.util.ArrayList;
7  import java.util.List;
8
9  public class ListaLivros {
10   public static void main(String[] args) throws Exception {
11     System.out.println("Abrindo conexão...");
12     Connection conexao = ConnectionFactory.createConnection();
13
14     String sql = "SELECT * FROM Livro;";
15
16     PreparedStatement comando = conexao.prepareStatement(sql);
17
18     System.out.println("Executando comando...");
19     ResultSet resultado = comando.executeQuery();
20
21     List<Livro> lista = new ArrayList<Livro>();
22     while (resultado.next()) {
23       Livro livro = new Livro();
24       livro.setId(resultado.getLong("id"));
25       livro.setTitulo(resultado.getString("titulo"));
26       livro.setPreco(resultado.getDouble("preco"));
27       livro.setEditoraId(resultado.getLong("editora_id"));
28       lista.add(livro);
29     }
30
31     System.out.println("Resultados encontrados: \n");
32     for (Livro livro : lista) {
33       System.out.println("Id: " + livro.getId());
34       System.out.println("Título: " + livro.getTitulo());
35       System.out.println("Preço: " + livro.getPreco());
36       System.out.println("Editora Id: " + livro.getEditoraId());
37       System.out.println("----------------------------");
38     }
39     System.out.println("\nFechando conexão...");
40     conexao.close();
41   }
42 }
```

Código Java 2.38: www.k19.com.br/exercicios/k12/02/C008/ListaLivros.java

Exercício Complementar 2.9

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5  import java.util.Scanner;
6
7  public class AlteraLivro {
8    public static void main(String[] args) throws Exception {
9      System.out.println("Abrindo conexão...");
10     Connection conexao = ConnectionFactory.createConnection();
11
```

```
12      Scanner entrada = new Scanner(System.in);
13      Livro livro = new Livro();
14
15      System.out.println("Digite o id do livro que deseja alterar: ");
16      livro.setId(Long.parseLong(entrada.nextLine()));
17
18      System.out.println("Digite o novo título do livro: ");
19      livro.setTitulo(entrada.nextLine());
20
21      System.out.println("Digite o novo preço do livro: ");
22      livro.setPreco(Double.parseDouble(entrada.nextLine()));
23
24      System.out.println("Digite o id da nova editora do livro: ");
25      livro.setEditoraId(Long.parseLong(entrada.nextLine()));
26
27      entrada.close();
28
29      String sql =
30        "UPDATE Livro SET titulo = ?, preco = ?, editora_id = ? WHERE id = ?";
31
32      PreparedStatement comando = conexao.prepareStatement(sql);
33      comando.setString(1, livro.getTitulo());
34      comando.setDouble(2, livro.getPreco());
35      comando.setLong(3, livro.getEditoraId());
36      comando.setLong(4, livro.getId());
37
38      System.out.println("Executando comando...");
39      comando.execute();
40
41      System.out.println("Fechando conexão...");
42      conexao.close();
43    }
44  }
```

Código Java 2.39: www.k19.com.br/exercicios/k12/02/C009/AlteraLivro.java

Exercício Complementar 2.10

```
1  package br.com.k19.jdbc;
2
3  import java.sql.Connection;
4  import java.sql.PreparedStatement;
5  import java.util.Scanner;
6
7  public class RemoveLivro {
8    public static void main(String[] args) throws Exception {
9      System.out.println("Abrindo conexão...");
10     Connection conexao = ConnectionFactory.createConnection();
11
12     Scanner entrada = new Scanner(System.in);
13
14     System.out.println("Digite o id do livro que deseja remover: ");
15     Long id = Long.parseLong(entrada.nextLine());
16
17     entrada.close();
18
19     String sql = "DELETE FROM Livro WHERE id = ?";
20
21     PreparedStatement comando = conexao.prepareStatement(sql);
22     comando.setLong(1, id);
23
24     System.out.println("Executando comando...");
25     comando.execute();
26
```

```
27      System.out.println("Fechando conexão...");
28      conexao.close();
29    }
30  }
```

Código Java 2.40: www.k19.com.br/exercicios/k12/02/C010/RemoveLivro.java

Exercício Complementar 6.1

No projeto **K19-Componentes-Visuais**, crie uma classe chamada **ContatoSACBean** em um pacote chamado **br.com.k19.controle** para implementar o managed bean que dará suporte ao formulário.

```
1  package br.com.k19.controle;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import javax.faces.bean.ManagedBean;
6
7  @ManagedBean
8  public class ContatoSACBean {
9    private String nome;
10   private String email;
11   private String codigoDeArea;
12   private String numeroDoTelefone;
13
14   private Integer numeroDoPedido;
15
16   private String assunto;
17   private String comentario;
18
19   private Boolean sexoFeminino;
20
21   private List<String> assuntos = new ArrayList<String>();
22
23   public ContatoSACBean() {
24     assuntos.add("Entrega");
25     assuntos.add("Pagamento");
26     assuntos.add("Trocas ou devoluções");
27     assuntos.add("Dúvidas gerais");
28     assuntos.add("Comentários");
29   }
30
31   // GETTERS E SETTERS
32 }
```

Código Java 6.8: ContatoSACBean.java

Crie um arquivo XHTML chamado `contato-sac.xhtml` e implemente o formulário de contato.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11 <h:body>
12   <h1>Serviço de Atendimento ao Consumidor</h1>
13
14   <h:form>
15     <h:panelGrid columns="2">
```

```
16        <h:outputLabel value="Nome: " for="campo-nome" />
17        <h:inputText value="#{contatoSACBean.nome}" id="campo-nome" />
18
19        <h:outputLabel value="E-mail: " for="campo-email" />
20        <h:inputText value="#{contatoSACBean.email}" id="campo-email" />
21
22        <h:outputLabel value="Telefone: " for="campo-telefone" />
23        <h:panelGroup>
24          (<h:inputText value="#{contatoSACBean.codigoDeArea}" id="campo-telefone"
25            size="2" />)
26          <h:inputText value="#{contatoSACBean.numeroDoTelefone}" /> (DDD) XXXX-XXXX
27        </h:panelGroup>
28
29        <h:outputLabel value="Sexo: " for="campo-sexo" />
30        <h:selectOneRadio value="#{contatoSACBean.sexoFeminino}" id="campo-sexo">
31          <f:selectItem itemLabel="Feminino" itemValue="true" />
32          <f:selectItem itemLabel="Masculino" itemValue="false" />
33        </h:selectOneRadio>
34
35        <h:outputLabel value="Número do pedido: " for="campo-pedido" />
36        <h:inputText value="#{contatoSACBean.numeroDoPedido}" id="campo-pedido" />
37
38        <h:outputLabel value="Assunto" for="campo-assunto" />
39        <h:selectOneMenu value="#{contatoSACBean.assunto}">
40          <f:selectItem noSelectionOption="true" itemLabel="Escolha um assunto" />
41          <f:selectItems
42            value="#{contatoSACBean.assuntos}"
43            var="assunto"
44            itemLabel="#{assunto}"
45            itemValue="#{assunto}" />
46        </h:selectOneMenu>
47
48        <h:outputLabel value="Comentário: " for="campo-comentario" />
49        <h:inputTextarea value="#{contatoSACBean.comentario}" id="campo-comentario" />
50
51        <h:commandButton value="Enviar" />
52      </h:panelGrid>
53    </h:form>
54  </h:body>
55  </html>
```

Código XHTML 6.36: contato-sac.xhtml

Acesse a aplicação em
http://localhost:8080/K19-Componentes-Visuais/contato-sac.xhtml

Exercício Complementar 6.2

Primeiramente, crie a classe **Produto** em um pacote chamado chamado **br.com.k19.modelo** do projeto **K19-Componentes-Visuais**.

```
1  package br.com.k19.modelo;
2
3  public class Produto {
4    private String nome;
5    private Double preco;
6
7    public Produto(String nome, Double preco) {
8      this.nome = nome;
9      this.preco = preco;
10   }
11
12   // GETTERS E SETTERS
13 }
```

Código Java 6.12: Produto.java

Agora, crie um managed bean que guardará a lista de produtos disponíveis.

```java
package br.com.k19.controle;

import br.com.k19.modelo.Produto;
import java.util.ArrayList;
import java.util.List;
import javax.faces.bean.ManagedBean;

@ManagedBean
public class ProdutosBean {
  private List<Produto> produtos = new ArrayList<Produto>();

  public ProdutosBean() {
    this.produtos.add(new Produto("Camiseta branca", 29.90));
    this.produtos.add(new Produto("Camiseta preta", 39.99));
    this.produtos.add(new Produto("Calça jeans", 95.99));
    this.produtos.add(new Produto("Gravata", 19.90));
    this.produtos.add(new Produto("Terno", 289.95));
  }

  // GETTER E SETTER
}
```

Código Java 6.13: ProdutosBean.java

Finalmente, crie o arquivo XHTML **lista-de-produtos.xhtml** e implemente a listagem dos produtos.

```xml
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:dataTable value="#{produtosBean.produtos}" var="produto">
    <f:facet name="header">Lista de produtos</f:facet>

    <h:column>
      <f:facet name="header">Nome</f:facet>
      #{produto.nome}
    </h:column>

    <h:column>
      <f:facet name="header">Preço</f:facet>
      <h:outputFormat value="{0, number, R$ .00}" >
        <f:param value="#{produto.preco}" />
      </h:outputFormat>
    </h:column>
  </h:dataTable>
</h:body>
</html>
```

Código XHTML 6.51: lista-de-produtos.xhtml

Acesse a aplicação no endereço:
http://localhost:8080/K19-Componentes-Visuais/lista-de-produtos.xhtml

Exercício Complementar 6.3

Na classe **Produto**, adicione um atributo booleano chamado **mostraPreco**.

```
1  package br.com.k19.modelo;
2
3  public class Produto {
4    private String nome;
5    private Double preco;
6    private Boolean mostraPreco;
7
8    public Produto(String nome, Double preco) {
9      this.nome = nome;
10     this.preco = preco;
11     this.mostraPreco = true;
12   }
13
14   // GETTERS E SETTERS
15 }
```

Código Java 6.14: Produto.java

No arquivo **lista-de-produtos.xhtml** modifique a tabela de forma a exibir somente os preços dos produtos para os quais a propriedade **mostraPreco** seja verdadeira. Use o atributo **rendered** para determinar quando um componente deve ser exibido.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7
8  <h:head>
9    <title>K19 Treinamentos</title>
10 </h:head>
11 <h:body>
12   <h:dataTable value="#{produtosBean.produtos}" var="produto">
13     <f:facet name="header">Lista de produtos</f:facet>
14
15     <h:column>
16       <f:facet name="header">Nome</f:facet>
17       #{produto.nome}
18     </h:column>
19
20     <h:column>
21       <f:facet name="header">Preço</f:facet>
22       <h:outputFormat value="{0, number, R$ .00}" rendered="#{produto.mostraPreco}" >
23         <f:param value="#{produto.preco}" />
24       </h:outputFormat>
25       <h:outputText value="Adicione o produto ao carrinho para ver o preço"
26         rendered="#{!produto.mostraPreco}" />
27     </h:column>
28   </h:dataTable>
29 </h:body>
30 </html>
```

Código XHTML 6.52: lista-de-produtos.xhtml

Defina como **false** o atributo **mostraPreco** de alguns produtos.

```java
package br.com.k19.controle;

import br.com.k19.modelo.Produto;
import java.util.ArrayList;
import java.util.List;
import javax.faces.bean.ManagedBean;

@ManagedBean
public class ProdutosBean {
  private List<Produto> produtos = new ArrayList<Produto>();

  public ProdutosBean() {
    this.produtos.add(new Produto("Camiseta branca", 29.90));
    this.produtos.add(new Produto("Camiseta preta", 39.99));
    this.produtos.add(new Produto("Calça jeans", 95.99));
    this.produtos.add(new Produto("Gravata", 19.90));
    Produto produto = new Produto("Terno", 289.95);
    produto.setMostraPreco(false);
    this.produtos.add(produto);
  }

  // GETTER E SETTER
}
```

Código Java 6.15: ProdutosBean.java

Acesse a aplicação no endereço:
http://localhost:8080/K19-Componentes-Visuais/lista-de-produtos.xhtml

Exercício Complementar 6.4

Essa mensagem pode ser criada no método **adicionaCurso()** do managed bean **CursosBean**. Nesse método, adicione uma mensagem usando o método **addMessage()** da classe **FacesContext**.

```java
package br.com.k19.controle;

import br.com.k19.modelo.Curso;
import java.util.ArrayList;
import java.util.List;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.SessionScoped;

@ManagedBean
@SessionScoped
public class CursosBean {
  private List<Curso> cursos = new ArrayList<Curso>();
  private Curso curso = new Curso();

  public void adicionaCurso() {
    this.cursos.add(this.curso);
    this.adicionaMensagemDeConfirmacao(curso);
    this.curso = new Curso();
  }

  private void adicionaMensagemDeConfirmacao(Curso curso) {
    FacesMessage mensagem = new FacesMessage("O curso '" + curso.getSigla()
        + " - " + curso.getNome() + "' foi adicionado");
    FacesContext.getCurrentInstance().addMessage(
        "form-adiciona:botao-adiciona", mensagem);
  }

  // GETTERS E SETTERS
}
```

Código Java 6.17: CursosBean.java

Modifique o arquivo **cursos.xhtml** para exibir a mensagem. Faça isso usando a tag **<h:message>**, como mostrado abaixo.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core"
  xmlns:ui="http://java.sun.com/jsf/facelets">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <h:form id="form-adiciona">
    <h:panelGrid columns="2">
      <h:outputLabel value="Sigla: " for="campo-sigla" />
      <h:inputText value="#{cursosBean.curso.sigla}" id="campo-sigla" />

      <h:outputLabel value="Nome: " for="campo-nome" />
      <h:inputText value="#{cursosBean.curso.nome}" id="campo-nome" />

      <h:commandButton value="Adicionar" action="#{cursosBean.adicionaCurso}"
        id="botao-adiciona"/>
      </h:panelGrid>
  </h:form>

  <p><h:message for="botao-adiciona" style="color: green" /></p>

  <h1>Alguns cursos da K19:</h1>
  <ul>
      <ui:repeat value="#{cursosBean.cursos}" var="curso">
    <li>
        <h:outputText value="#{curso.sigla}: #{curso.nome}" />
    </li>
      </ui:repeat>
  </ul>
</h:body>
</html>
```

Código XHTML 6.59: cursos.xhtml

Acesse a aplicação em:
http://localhost:8080/K19-Componentes-Visuais/cursos.xhtml

Exercício Complementar 7.1

No pacote **br.com.k19.modelo**, crie uma classe chamada **Produto** para representar os produtos da loja.

```
package br.com.k19.modelo;

public class Produto {

  private String nome;
  private String descricao;
  private String caminhoDaFigura;
  private double preco;
```

```
9   private int id;
10
11  public Produto(String nome, String descricao, String caminhoDaFigura,
12      double preco, int id) {
13    this.nome = nome;
14    this.descricao = descricao;
15    this.caminhoDaFigura = caminhoDaFigura;
16    this.preco = preco;
17    this.id = id;
18  }
19
20  // GETTERS E SETTERS
21 }
```

Código Java 7.3: Produto.java

Copie os arquivos da pasta **K19-Arquivos/imagens** para o diretório **imagens** do projeto **K19-Templates-e-Modularizacao**.



Importante
Você também pode obter esses arquivos através do site da K19: www.k19.com.br/arquivos.

No pacote **br.com.k19.controle**, crie uma classe chamada **ProdutosBean**. Essa classe deve armazenar uma lista de produtos e deve ter um atributo para guardar o produto cujos detalhes devem ser exibidos.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Produto;
4  import java.util.ArrayList;
5  import java.util.List;
6  import javax.faces.bean.ManagedBean;
7
8  @ManagedBean
9  public class ProdutosBean {
10
11   private Produto produtoSelecionado;
12   private int idDoProdutoSelecionado;
13   private List<Produto> produtos = new ArrayList<Produto>();
14
15   public ProdutosBean() {
16     this.produtos.add(new Produto(
17             "Camiseta branca",
18             "Esta é uma camiseta básica na cor branca. Ela é feita de algodão e " +
19             "está disponível nos tamanhos P, M e G.",
20             "/imagens/camiseta-branca.jpg", 29.90, 1));
21
22     this.produtos.add(new Produto(
23             "Camiseta preta",
24             "Esta é uma camiseta na cor preta, que está disponível " +
25             "nos tamanhos P, M e G. Seu material é 100% algodão.",
26             "/imagens/camiseta-preta.jpg", 39.90, 2));
27
28     this.produtos.add(new Produto(
29             "Calça jeans",
30             "Calça jeans disponível nos tamanhos 38 a 50.",
31             "/imagens/calca-jeans.jpg", 95.99, 3));
32
33     this.produtos.add(new Produto(
34             "Terno",
35             "Terno de seda na cor cinza.",
```

```
            "/imagens/terno.jpg", 589.95, 4));

    this.produtos.add(new Produto(
            "Gravata",
            "Gravata nas cores vermelha, azul e verde.",
            "/imagens/gravata.jpg", 19.90, 5));

    this.produtoSelecionado = this.produtos.get(0);
  }

  public void carregaProduto() {
    for (Produto produto : produtos) {
      if (this.idDoProdutoSelecionado == produto.getId()) {
        this.produtoSelecionado = produto;
        break;
      }
    }
  }

  // GETTERS E SETTERS
}
```

Código Java 7.4: ProdutosBean.java

No diretório **WEB-INF/templates**, crie um arquivo chamado **template-loja-virtual.xhtml** para definir um template para as páginas da loja.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
        "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

<h:head>
  <title>K19 Treinamentos</title>
</h:head>
<h:body>
  <div id="header">
    <h1>Loja virtual</h1>
    <hr />
  </div>

  <ui:insert name="conteudo" />

  <div id="footer" style="text-align: center">
    <hr />
    &copy; 2012 K19. Todos os direitos reservados.
  </div>
</h:body>
</html>
```

Código XHTML 7.13: template-loja-virtual.xhtml

No diretório **partials**, crie um arquivo chamado **produto-info.xhtml** e implemente o pedaço da tela responsável por exibir os detalhes de um produto. O produto particular a ser exibido deve ser passado como parâmetro para esse arquivo.

```
<ui:composition xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <h:panelGrid columns="2" cellspacing="5">
    <h:graphicImage value="#{produto.caminhoDaFigura}" width="100" />
    <h:panelGroup>
```

```
9       <h2>#{produto.nome}</h2>
10
11      Preço:
12      <h:outputFormat value="{0, number, R$ .00}" style="color: red">
13        <f:param value="#{produto.preco}" />
14      </h:outputFormat>
15    </h:panelGroup>
16  </h:panelGrid>
17
18  <h3>Descrição</h3>
19  #{produto.descricao}
20
21 </ui:composition>
```

Código XHTML 7.14: produto-info.xhtml

Note que o parâmetro para o arquivo **produto-info.xhtml** deve se chamar **produto**. Crie uma página chamada **loja.xhtml**. Esse arquivo usará o template que você acabou de criar. O conteúdo da página será formado pela caixa de seleção de produto e pelo pedaço de tela de informações do produto.

```
1  <ui:composition
2    template="/WEB-INF/templates/template-loja-virtual.xhtml"
3    xmlns="http://www.w3.org/1999/xhtml"
4    xmlns:h="http://java.sun.com/jsf/html"
5    xmlns:f="http://java.sun.com/jsf/core"
6    xmlns:ui="http://java.sun.com/jsf/facelets">
7
8    <ui:define name="conteudo">
9
10     <h:form>
11       <h:selectOneMenu value="#{produtosBean.idDoProdutoSelecionado}">
12         <f:selectItems
13           value="#{produtosBean.produtos}"
14           var="produto"
15           itemValue="#{produto.id}"
16           itemLabel="#{produto.nome}">
17         </f:selectItems>
18       </h:selectOneMenu>
19       <h:commandButton value="Detalhes" action="#{produtosBean.carregaProduto}" />
20     </h:form>
21
22     <ui:include src="/WEB-INF/partials/produto-info.xhtml">
23       <ui:param name="produto" value="#{produtosBean.produtoSelecionado}" />
24     </ui:include>
25
26   </ui:define>
27 </ui:composition>
```

Código XHTML 7.15: loja.xhtml

Acesse a aplicação no endereço:

http://localhost:8080/K19-Templates-e-Modularizacao/loja.xhtml

Exercício Complementar 8.1

No pacote **br.com.k19.controle**, do projeto **K19-Navegacao** crie uma classe chamada **UsuarioBean** e implemente um managed bean para armazenar a preferência do usuário.

```
1 package br.com.k19.controle;
2
```

```
3  import javax.faces.bean.ManagedBean;
4  import javax.faces.bean.SessionScoped;
5
6  @SessionScoped
7  @ManagedBean
8  public class UsuarioBean {
9    private String versaoPreferida = "Simples";
10
11   public String principal() {
12     if (this.versaoPreferida.equals("Avançada")) {
13       return "principal-avancada.xhtml";
14     } else {
15       return "principal-simples.xhtml";
16     }
17   }
18
19   // GETTERS E SETTERS
20 }
```

Código Java 8.3: UsuarioBean.java

O método `principal()` da classe `UsuarioBean` será usado para determinar a página de resposta quando o link para a página principal for clicado.

Na pasta **/WEB-INF/templates**, crie um arquivo chamado **template-aplicacao.xhtml** para implementar um template para as páginas da aplicação.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core"
7    xmlns:ui="http://java.sun.com/jsf/facelets">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12 <h:body>
13   <div id="header" style="text-align: center">
14     <h:outputText value="Loja Virtual" style="font-size: 36px; font-weight: bold" />
15     <h:form>
16       <h:commandLink value="Página principal" action="#{usuarioBean.principal}"/>
17       -
18       <h:link value="Configurações" outcome="configuracoes"/>
19     </h:form>
20   </div>
21
22   <ui:insert name="conteudo" />
23
24   <div id="footer" style="text-align: center">
25     <hr />
26     &copy; 2012 K19. Todos os direitos reservados.
27   </div>
28
29 </h:body>
30 </html>
```

Código XHTML 8.18: template-aplicacao.xhtml

Note que o menu para navegar entre as páginas da aplicação foi definido nesse arquivo. Observe que a navegação para a página de configurações foi definida de forma estática, enquanto que a navegação para a página principal foi definida de forma dinâmica.

Crie duas páginas no projeto **K19-Navegacao** chamadas **principal-simples.xhtml** e `principal-a-`

vancada.xhtml para implementar as versões simples e avançada da página principal, respectivamente.

```
<ui:composition template="/WEB-INF/templates/template-aplicacao.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="conteudo">
    <h1>Página Principal Simples</h1>
    <ul>
      <li><h:link>Visualizar produtos cadastrados</h:link></li>
    </ul>
  </ui:define>
</ui:composition>
```

Código XHTML 8.19: principal-simples.xhtml

```
<ui:composition template="/WEB-INF/templates/template-aplicacao.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="conteudo">
    <h1>Página Principal Avançada</h1>
    <ul>
      <li><h:link>Visualizar produtos cadastrados</h:link></li>
      <li><h:link>Editar produtos cadastrados</h:link></li>
      <li><h:link>Adicionar ou remover produtos</h:link></li>
      <li><h:link>Adicionar ou remover usuários</h:link></li>
    </ul>
  </ui:define>
</ui:composition>
```

Código XHTML 8.20: principal-avancada.xhtml

Finalmente, crie uma página chamada **configuracoes.xhtml** para implementar a página da aplicação em que o usuário pode alterar suas preferências.

```
<ui:composition template="/WEB-INF/templates/template-aplicacao.xhtml"
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <ui:define name="conteudo">
    <h1>Configurações</h1>

    <p>Você está usando a versão <b>#{usuarioBean.versaoPreferida}</b>
    da aplicação. Se desejar, escolha outra versão abaixo e clique no
    botão "Salvar".</p>

    <h:form>
      <h:panelGrid columns="3">
        <h:outputLabel value="Versão da aplicação: " for="versao" />

        <h:selectOneMenu value="#{usuarioBean.versaoPreferida}" id="versao">
          <f:selectItem itemLabel="Simples" itemValue="Simples" />
          <f:selectItem itemLabel="Avançada" itemValue="Avançada" />
        </h:selectOneMenu>

        <h:commandButton value="Salvar" />
      </h:panelGrid>
    </h:form>
  </ui:define>
</ui:composition>
```

Código XHTML 8.21: configuracoes.xhtml

Acesse a aplicação no endereço:

http://localhost:8080/K19-Navegacao/principal-simples.xhtml

Exercício Complementar 8.2

No arquivo **template-aplicacao.xhtml**, altere o link para a página principal para que ele emita o sinal `"principal"`.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core"
7    xmlns:ui="http://java.sun.com/jsf/facelets">
8
9  <h:head>
10   <title>K19 Treinamentos</title>
11 </h:head>
12 <h:body>
13   <div id="header" style="text-align: center">
14     <h:outputText value="Loja Virtual" style="font-size: 36px; font-weight: bold" />
15     <h:form>
16       <h:link value="Página principal" outcome="principal"/>
17       -
18       <h:link value="Configurações" outcome="configuracoes"/>
19     </h:form>
20   </div>
21
22   <ui:insert name="conteudo" />
23
24   <div id="footer" style="text-align: center">
25     <hr />
26     &copy; 2012 K19. Todos os direitos reservados.
27   </div>
28
29 </h:body>
30 </html>
```

Código XHTML 8.22: template-aplicacao.xhtml

No arquivo **faces-config.xhtml**, defina as regras de navegação para a página principal. Se o usuário prefere a versão simples da aplicação, então o sinal `"principal"` deve direcionar o usuário para a página definida pelo arquivo **principal-simples.xhtml**. Por outro lado, se o usuário prefere a versão avançada da aplicação, então o sinal `"principal"` deve direcioná-lo para a página definida pelo arquivo **principal-avancada.xhtml**.

```
1  ...
2  <navigation-rule>
3    <from-view-id>*</from-view-id>
4
5    <navigation-case>
6      <from-outcome>principal</from-outcome>
7      <if>#{usuarioBean.versaoPreferida == 'Simples'}</if>
8      <to-view-id>principal-simples.xhtml</to-view-id>
9    </navigation-case>
10
```

```
11    <navigation-case>
12      <from-outcome>principal</from-outcome>
13      <if>#{usuarioBean.versaoPreferida == 'Avançada'}</if>
14      <to-view-id>principal-avancada.xhtml</to-view-id>
15    </navigation-case>
16
17  </navigation-rule>
18  ...
```

Código XML 8.6: faces-config.xml

Acesse a aplicação no endereço:

http://localhost:8080/K19-Navegacao/principal-simples.xhtml

Exercício Complementar 10.1

No projeto **K19-Conversao-e-Validacao**, crie uma classe chamada **Cotacao** para modelar a cotação de uma moeda. Essa classe deve estar no pacote **br.com.k19.modelo**.

```
1  package br.com.k19.modelo;
2
3  import java.util.Date;
4
5  public class Cotacao {
6    private Double valor;
7    private Double variacao;
8    private Date horario;
9
10   // GETTERS E SETTERS
11 }
```

Código Java 10.4: Cotacao.java

Crie uma classe chamada **CotacaoBean** no pacote **br.com.k19.controle** para armazenar os valores submetidos pelo formulário.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.Cotacao;
4  import javax.faces.bean.ManagedBean;
5
6  @ManagedBean
7  public class CotacaoBean {
8    private Cotacao cotacao = new Cotacao();
9
10   // GETTER E SETTER
11 }
```

Código Java 10.5: CotacaoBean.java

Crie uma página chamada `registrar-cotacao.xhtml` e implemente o formulário para registrar uma cotação.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7  <h:head>
```

```
8   <title>K19 Treinamentos</title>
9 </h:head>
10
11 <h:body>
12   <h1>Registrar cotação do dólar</h1>
13   <h:form>
14     <h:panelGrid columns="2">
15       <h:outputLabel value="Valor: " for="valor" />
16       <h:panelGroup>
17         <h:inputText value="#{cotacaoBean.cotacao.valor}" id="valor" required="true">
18           <f:convertNumber locale="pt_BR" type="currency" />
19         </h:inputText>
20         <h:message for="valor" style="color: red" />
21       </h:panelGroup>
22
23       <h:outputLabel value="Variação: " for="variacao" />
24       <h:panelGroup>
25         <h:inputText value="#{cotacaoBean.cotacao.variacao}" id="variacao" required="↩
               true">
26           <f:convertNumber maxFractionDigits="5" type="percent" />
27         </h:inputText>
28         <h:message for="variacao" style="color: red" />
29       </h:panelGroup>
30
31       <h:outputLabel value="Horário: " for="horario" />
32       <h:panelGroup>
33         <h:inputText value="#{cotacaoBean.cotacao.horario}" id="horario" required="↩
               true">
34           <f:convertDateTime pattern="HH:mm dd-MM-yyyy" />
35         </h:inputText>
36         <h:message for="horario" style="color: red" />
37       </h:panelGroup>
38
39       <h:commandButton value="Registrar" />
40     </h:panelGrid>
41   </h:form>
42 </h:body>
43 </html>
```

Código XHTML 10.9: registrar-cotacao.xhtml

Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/registrar-cotacao.xhtml

Exercício Complementar 10.2

No arquivo **registrar-cotacao.xhtml**, adicione as linhas em destaque do código abaixo.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html"
6    xmlns:f="http://java.sun.com/jsf/core">
7  <h:head>
8    <title>K19 Treinamentos</title>
9  </h:head>
10
11 <h:body>
12   <h1>Registrar cotação do dólar</h1>
13   <h:form>
14     <h:panelGrid columns="2">
15       <h:outputLabel value="Valor: " for="valor" />
16       <h:panelGroup>
17         <h:inputText value="#{cotacaoBean.cotacao.valor}" id="valor" required="true">
```

```
18            <f:convertNumber locale="pt_BR" type="currency" />
19          </h:inputText>
20          <h:message for="valor" style="color: red" />
21        </h:panelGroup>
22
23        <h:outputLabel value="Variação: " for="variacao" />
24        <h:panelGroup>
25          <h:inputText value="#{cotacaoBean.cotacao.variacao}" id="variacao" required="↩
              true">
26            <f:convertNumber maxFractionDigits="5" type="percent" />
27          </h:inputText>
28          <h:message for="variacao" style="color: red" />
29        </h:panelGroup>
30
31        <h:outputLabel value="Horário: " for="horario" />
32        <h:panelGroup>
33          <h:inputText value="#{cotacaoBean.cotacao.horario}" id="horario" required="↩
              true">
34            <f:convertDateTime pattern="HH:mm dd-MM-yyyy" />
35          </h:inputText>
36          <h:message for="horario" style="color: red" />
37        </h:panelGroup>
38
39        <h:commandButton value="Registrar" />
40      </h:panelGrid>
41    </h:form>
42
43    <h:panelGroup rendered="#{cotacaoBean.cotacao.valor != null}">
44      <h2>Cotação registrada:</h2>
45      Valor: #{cotacaoBean.cotacao.valor}
46      <br />
47      Variação: #{cotacaoBean.cotacao.variacao * 100}%
48      <br />
49      Horário: #{cotacaoBean.cotacao.horario}
50    </h:panelGroup>
51  </h:body>
52  </html>
```

Código XHTML 10.10: registrar-cotacao.xhtml

Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/registrar-cotacao.xhtml

Altere o valor do atributo **timeZone** do componente **<f:convertDateTime>**. Use, por exemplo, os valores "GMT-2" e "GMT-3". Teste o formulário novamente.

Exercício Complementar 10.3

Crie uma classe chamada **Produto** no pacote **br.com.k19.modelo**.

```
1  package br.com.k19.modelo;
2
3  public class Produto {
4    private String nome;
5    private String codigo;
6    private Integer tamanho;
7    private Double volume;
8
9    // GETTERS E SETTERS
10 }
```

Código Java 10.7: Produto.java

Crie uma classe chamada **ProdutoBean** no pacote **br.com.k19.controle**.

```
package br.com.k19.controle;

import br.com.k19.modelo.Produto;
import javax.faces.bean.ManagedBean;

@ManagedBean
public class ProdutoBean {
  private Produto produto = new Produto();

  // GETTER E SETTER
}
```

Código Java 10.8: ProdutoBean.java

Crie uma página XHTML chamada **cadastro-de-produtos.xhtml** e implemente o formulário.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">
<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h1>Cadastro de Produtos</h1>
  <h:form>
    <h:panelGrid columns="2">
      <h:outputLabel value="Nome: " for="nome" />
      <h:panelGroup>
        <h:inputText value="#{produtoBean.produto.nome}" id="nome" required="true">
          <f:validateRegex pattern="[a-zA-Z\s]+" />
        </h:inputText>
        <h:message for="nome" style="color: red" />
      </h:panelGroup>

      <h:outputLabel value="Código: " for="codigo" />
      <h:panelGroup>
        <h:inputText value="#{produtoBean.produto.codigo}" id="codigo" required="true↩
            ">
          <f:validateRegex pattern="[A-Z][0-9]+" />
          <f:validateLength minimum="4" maximum="10" />
        </h:inputText>
        <h:message for="codigo" style="color: red" />
      </h:panelGroup>

      <h:outputLabel value="Tamanho: " for="tamanho" />
      <h:panelGroup>
        <h:inputText value="#{produtoBean.produto.tamanho}" id="tamanho">
          <f:validateRequired />
          <f:validateLongRange minimum="1" maximum="32" />
        </h:inputText>
        <h:message for="tamanho" style="color: red" />
      </h:panelGroup>

      <h:outputLabel value="Volume: " for="volume" />
      <h:panelGroup>
        <h:inputText value="#{produtoBean.produto.volume}" id="volume">
          <f:validateRequired />
          <f:validateDoubleRange minimum="0.1" />
        </h:inputText>
        <h:message for="volume" style="color: red" />
      </h:panelGroup>

```

```
50        <h:commandButton value="Enviar" />
51      </h:panelGrid>
52    </h:form>
53  </h:body>
54  </html>
```

Código XHTML 10.22: cadastro-de-produtos.xhtml

Teste a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/cadastro-de-produtos.xhtml

Exercício Complementar 10.4

Na pacote **br.com.k19.modelo**, crie uma classe chamada **RIC**.

```
1  package br.com.k19.modelo;
2
3  public class RIC {
4    private String numeroDeIdentificacao;
5    private String digitoVerificador;
6
7    // GETTERS E SETTERS
8  }
```

Código Java 10.20: RIC.java

Na pacote **br.com.k19.converters**, crie uma classe chamada **ConversorDeRIC** e implemente o conversor.

```
1  package br.com.k19.converters;
2
3  import br.com.k19.modelo.RIC;
4  import java.util.regex.Pattern;
5  import javax.faces.application.FacesMessage;
6  import javax.faces.component.UIComponent;
7  import javax.faces.context.FacesContext;
8  import javax.faces.convert.Converter;
9  import javax.faces.convert.ConverterException;
10 import javax.faces.convert.FacesConverter;
11
12 @FacesConverter(forClass = RIC.class)
13 public class ConversorDeRIC implements Converter {
14
15   @Override
16   public Object getAsObject(FacesContext context, UIComponent component,
17       String value) {
18
19     String ricString = value.trim();
20
21     if (!Pattern.matches("[0-9]{10}[-]?[0-9]", ricString)) {
22       FacesMessage mensagem = new FacesMessage("Número RIC inválido");
23       mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
24       throw new ConverterException(mensagem);
25     }
26
27     ricString = ricString.replaceAll("-", "");
28
29     RIC ric = new RIC();
30     ric.setNumeroDeIdentificacao(ricString.substring(0, 10));
31     ric.setDigitoVerificador(ricString.substring(10, 11));
32
33     return ric;
34   }
```

```
35
36   @Override
37   public String getAsString(FacesContext context, UIComponent component,
38       Object value) {
39     RIC ric = (RIC) value;
40     return ric.getNumeroDeIdentificacao() + "-" + ric.getDigitoVerificador();
41   }
42 }
```

Código Java 10.21: ConversorDeRIC.java

Na pacote **br.com.k19.controle**, crie uma classe chamada **RICBean** e implemente o managed bean que dará suporte ao seu formulário.

```
1  package br.com.k19.controle;
2
3  import br.com.k19.modelo.RIC;
4  import javax.faces.bean.ManagedBean;
5
6  @ManagedBean
7  public class RICBean {
8    private RIC ric;
9
10   // GETTER E SETTER
11 }
```

Código Java 10.22: RICBean.java

Crie uma página chamada **ric.xhtml** e implemente um formulário que possua uma caixa de texto associada à propriedade **ric** do managed bean **RICBean**.

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
3
4  <html xmlns="http://www.w3.org/1999/xhtml"
5    xmlns:h="http://java.sun.com/jsf/html">
6  <h:head>
7    <title>K19 Treinamentos</title>
8  </h:head>
9
10 <h:body>
11   <h:form>
12     <h:outputLabel value="RIC: " for="campo-ric" />
13     <h:inputText id="campo-ric" value="#{rICBean.ric}" />
14     <h:commandButton value="Enviar" />
15   </h:form>
16
17   <h:message for="campo-ric" style="color: red" />
18 </h:body>
19 </html>
```

Código XHTML 10.26: ric.xhtml

Faça alguns testes acessando a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/ric.xhtml

Exercício Complementar 10.5

Primeiro, crie um managed bean chamado **ReservaBean** que armazenará a data escolhida pelo usuário. A classe deverá ser adicionada ao pacote **br.com.k19.controle**.

```java
package br.com.k19.controle;

import java.util.Date;
import javax.faces.bean.ManagedBean;

@ManagedBean
public class ReservaBean {

  private Date data;

  // GETTER E SETTER
}
```

Código Java 10.27: ReservaBean.java

Crie uma página chamada **reserva.xhtml** e implemente o formulário para o usuário digitar a data.

```xml
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:f="http://java.sun.com/jsf/core">
<h:head>
  <title>K19 Treinamentos</title>
</h:head>

<h:body>
  <h:outputText value="Escolha uma data entre 01/05/2014 e 30/08/2014." />
  <h:form>
    <h:outputLabel value="Data: (dd/MM/yyyy)" for="campo-data" />

    <h:inputText id="campo-data" value="#{reservaBean.data}" required="true">
      <f:convertDateTime pattern="dd/MM/yyyy" />

      <f:validator validatorId="validators.ValidadorDeData" />

      <f:attribute name="padraoDeData" value="dd/MM/yyyy" />
      <f:attribute name="inicio" value="01/05/2014" />
      <f:attribute name="fim" value="30/08/2014" />
    </h:inputText>

    <h:commandButton value="Enviar" />
  </h:form>

  <h:message for="campo-data" style="color: red" />
</h:body>
</html>
```

Código XHTML 10.30: reserva.xhtml

No pacote **br.com.k19.validators**, crie uma classe chamada **ValidadorDeData** para implementar o validador.

```java
package br.com.k19.validators;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import javax.faces.application.FacesMessage;
import javax.faces.component.UIComponent;
import javax.faces.context.FacesContext;
import javax.faces.validator.FacesValidator;
import javax.faces.validator.Validator;
```

```
11 import javax.faces.validator.ValidatorException;
12
13 @FacesValidator("validators.ValidadorDeData")
14 public class ValidadorDeData implements Validator {
15
16   @Override
17   public void validate(FacesContext context, UIComponent component,
18       Object value) throws ValidatorException {
19
20     String padraoDeData = (String) component.getAttributes().get("padraoDeData");
21     String inicio = (String) component.getAttributes().get("inicio");
22     String fim = (String) component.getAttributes().get("fim");
23
24     Date data = (Date) value;
25     Date dataInicio;
26     Date dataFim;
27
28     try {
29       dataInicio = (Date) new SimpleDateFormat(padraoDeData).parse(inicio);
30       dataFim = (Date) new SimpleDateFormat(padraoDeData).parse(fim);
31     } catch (ParseException e) {
32       FacesMessage mensagem = new FacesMessage(
33           "Erro ao criar as datas de início e fim do intervalo.");
34       mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
35       throw new ValidatorException(mensagem);
36     }
37
38     if (data.before(dataInicio) || data.after(dataFim)) {
39       FacesMessage mensagem = new FacesMessage(
40           "A data escolhida precisa estar entre " + inicio + " e " + fim);
41       mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
42       throw new ValidatorException(mensagem);
43     }
44   }
45 }
```

Código Java 10.28: ValidadorDeData.java

Acesse a aplicação no endereço:
http://localhost:8080/K19-Conversao-e-Validacao/reserva.xhtml

Teste o formulário submetendo-o com diversas datas diferentes, inclusive com datas nos extremos do intervalo permitido.

Agora, modifique o arquivo **reserva.xhtml** de forma que o conversor de datas considere o fuso horário da cidade de São Paulo, como no trecho de código abaixo.

```
1 ...
2 <f:convertDateTime pattern="dd/MM/yyyy" timeZone="GMT-3" />
3 ...
```

Código XHTML 10.31: reserva.xhtml

Teste o formulário novamente.